

ВВЕДЕНИЕ

В основании бесконечно развивающегося здания компьютерной графики краеугольными камнями лежат фундаментальные дисциплины – аналитическая геометрия и оптика, скрепленные раствором – искусством программирования [1]. Возникнув из потребностей рынка, развития информатики и вычислительной техники, компьютерная графика изучает методы построения изображений различных геометрических объектов и сцен. Задачи, связанные с применением компьютерной графики, возникают в самых различных сферах информационных технологий. Сюда можно отнести системы автоматизированного проектирования, в которых осуществляется интерактивное взаимодействие конструктора и синтезированного с помощью компьютера изображения создаваемого изделия или сооружения. Широко используется компьютерная графика в автоматизированных системах научных исследований, в которых осуществляется визуализация результатов экспериментов в виде трехмерных статических или динамических изображений, интерпретирующих огромные массивы первичных данных. Важную роль играют методы компьютерной графики для распознавания и обработки изображений в системах искусственного зрения, авиационной и космической картографии и других областях человеческой деятельности. Знание основ компьютерной графики в наше время необходимо любому ученому или инженеру. Для формирования навыков решения вышеупомянутых задач и служит учебное пособие «Основы компьютерной геометрии и графики».

Глава 1. ЦЕЛИ И ЗАДАЧИ КОМПЬЮТЕРНОЙ ГРАФИКИ

Понятие «компьютерная графика» объединяет довольно широкий круг операций по обработке графической информации с помощью компьютера. Причем наблюдается явная тенденция «компьютеризации» изображений, циркулирующих в обществе. Стали обыденностью термины «цифровое фото» и «видео». В виртуальных буднях грядущего компьютерной графике отводится огромная роль. Это связано с тем, что, по мнению ученых, исследующих проблемы мозга, зрительная система в иерархии мозговых структур человека занимает особое место. С восприятием и обработкой визуальной информации непосредственно связано примерно 20% мозга человека. Благодаря зрению мы получаем по разным оценкам от 70% до 90% сведений об окружающем мире. Следовательно, образный мир компьютерной графики является одним из глубинных проявлений человеческой природы.

В компьютерной графике можно выделить несколько основных направлений [2, 4].

1.1. Визуализация научных данных

Большинство современных математических программных пакетов (например, Maple, MatLab, MathCAD) имеют средства для отображения графиков, поверхностей и трехмерных тел, построенных на основе каких-либо расчетов. Кроме того, графическая информация может активно использоваться в самом процессе вычислений. Визуализация позволяет представить большой объем данных в удобной для анализа форме и широко используется при обработке результатов различных измерений и вычислений.

1.2. Геометрическое проектирование и моделирование

Это направление компьютерной графики связано с решением задач начертательной геометрии – построением чертежей, эскизов, объемных изображений с помощью программных систем, получивших название CAD-системы (от английского Computer-Aided Design), например AutoCAD.

Существует большое количество специализированных CAD-систем в машиностроении, архитектуре и т. д.

1.3. Распознавание образов

Способность распознавать абстрактные образы считают одним из важнейших факторов, определившим развитие мыслительных способностей человека,

выделив его из животного мира. Задача распознавания и классификации графической информации является одной из ключевых и при создании искусственного интеллекта. Уже в наши дни компьютеры распознают образы повсеместно (системы идентификации футбольных хулиганов у входа на стадион; анализ аэро-и космических фотоснимков; системы сортировки, наведения и т. д.). Возможно, самый известный пример распознавания образов – сканирование и перевод "фотографии" текста в набор отдельных символов, формирующих слова. Такую операцию позволяет выполнить программное обеспечение многих современных сканеров. Кроме того, существуют специализированные программы распознавания текста, например FineReader.

1.4. Изобразительное искусство

К этому направлению можно отнести разнообразную графическую рекламу: от текстовых транспарантов и фирменных знаков до компьютерных видеофильмов, обработку фотографий, создание рисунков, мультипликацию и т. д. В качестве примера популярных программ из этой области компьютерной графики можно назвать AdobePhotoshop (обработка растровых изображений), CorelDRAW (создание векторной графики), 3DS Max (трехмерное моделирование).

1.5. Виртуальная реальность

Реальность, даже виртуальная, подразумевает воздействия на всю совокупность органов чувств человека, в первую очередь на его зрение. К компьютерной графике можно отнести задачи моделирования внешнего мира в различных приложениях: от компьютерных игр до тренажеров.

1.6. Цифровое видео

Все более широкое распространение получают анимированные изображения, записанные в цифровом формате. Это, прежде всего, фильмы, передаваемые через компьютерные сети, а также видеодиски, цифровое, кабельное и спутниковое телевидение.

Приведенная классификация сфер применения компьютерной графики является во многом условной. Возможно, найдутся задачи, которые нельзя отнести ни к одному из обозначенных направлений.

Глава 2. ОСНОВНЫЕ ПОНЯТИЯ КОМПЬЮТЕРНОЙ ГРАФИКИ

2.1. Визуализация изображений

Наиболее известны два способа визуализации: **растровый** и **векторный**. Первый способ ассоциируется с такими графическими устройствами как дисплей, телевизор, принтер. Второй используется в векторных дисплеях, плоттерах.

Наиболее удобно, когда способ описания графического изображения соответствует способу визуализации. Иначе нужна конвертация. Например, изображение может храниться в растровом виде, а его необходимо вывести (визуализировать) на векторном устройстве. Для этого нужна предварительная **векторизация** – преобразование из растрового в векторное описание. Или, наоборот, описание изображения может быть в векторном виде, а нужно визуализировать на растровом устройстве – необходима **растеризация**.

Растровая визуализация основывается на представлении изображения на экране или бумаге в виде совокупности (массива) отдельных точек (пикселей). Вместе пиксели образуют **растр**.

Для обозначения массива пикселей часто используется термин **bitmap** (битовая карта). В **bitmap** каждому пикселу отводится определенное число битов (одинаковое для всех пикселей изображения). Это число называется **битовой глубиной** пикселя или **цветовой глубиной** изображения, т. к. от количества битов, отводимых на один пиксел, зависит количество цветов изображения.

Векторная визуализация основывается на формировании изображения на экране или бумаге рисованием линий (векторов) – прямых или кривых. Совокупность типов линий (графических примитивов), которые используются как базовые для векторной визуализации, зависит от определенного устройства. Типичная последовательность действий при векторной визуализации для плоттера или векторного дисплея такова: переместить перо в начальную точку (для дисплея – отклонить пучок электронов); опустить перо (увеличить яркость луча); переместить перо в конечную точку; поднять перо (уменьшить яркость луча).

Качество векторной визуализации для векторных устройств обуславливается точностью вывода и номенклатурой базовых графических примитивов – линий, дуг, кругов, эллипсов и других.

Доминирующим сейчас является растровый способ визуализации. Это обусловлено большей распространенностью растровых дисплеев и принтеров. Недостаток растровых устройств – дискретность изображения. Недостатки векторных устройств – проблемы при сплошном заполнении фигур, меньшее количество цветов, меньшая скорость (в сравнении с растровыми устройствами).

2.2. Растровые изображения и их основные характеристики

Растр— это матрица (массив) ячеек (пикселей). Каждый пиксел может иметь свой цвет. Совокупность пикселей различного цвета образует изображение. В зависимости от расположения пикселей в пространстве различают квадратный, прямоугольный, гексагональный или иные типы растра. Для описания расположения пикселей используют разнообразные системы координат. Общим для всех таких систем является то, что координаты пикселей образуют дискретный ряд значений (необязательно целые числа). Часто используется система целых координат — номеров пикселей с (0,0) в левом верхнем углу. Такую систему мы будем использовать и в дальнейшем, ибо она удобна для рассмотрения алгоритмов графического вывода.

2.3. Геометрические характеристики растра

Разрешающая способность. Она характеризует расстояние между соседними пикселями (рис. 2.1). Разрешающую способность измеряют количеством пикселей на единицу длины. Наиболее популярной единицей измерения является *dpi*(dotsperinch)— количество пикселей в одном дюйме длины (2.54см). Не следует отождествлять шаг с размерами пикселей— размер пикселей может быть равен шагу, а может быть, как меньше, так и больше, чем шаг.

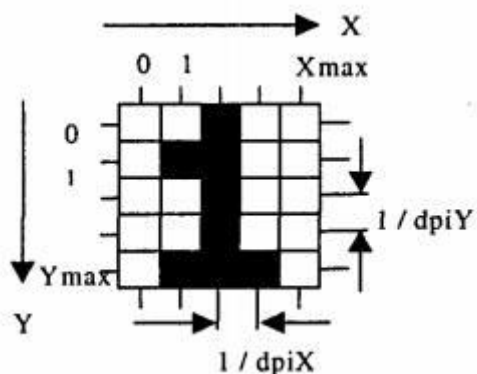


Рис. 2.1

Размер растра обычно измеряется количеством пикселей по горизонтали и вертикали. Можно сказать, что для компьютерной графики зачастую наиболее удобен растр с одинаковым шагом для обеих осей, то есть $dpiX = dpiY$. Это удобно для многих алгоритмов вывода графических объектов.

Форма пикселей растра определяется особенностями устройства графического вывода (рис. 2.2). Например, пиксели могут иметь форму прямоугольника или квадрата, которые по размерам равны шагу растра (дисплей на жидких кристаллах); пиксели круглой формы, которые по размерам могут и не равняться шагу растра (принтеры).

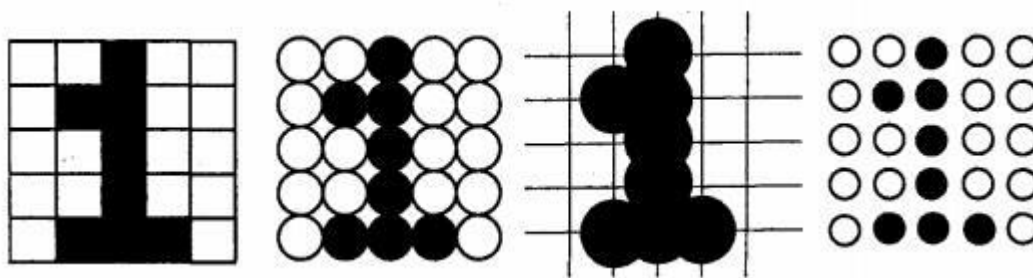


Рис. 2.2

2.4. Количество цветов

Количество цветов (глубина цвета) – также одна из важнейших характеристик раstra. Количество цветов является важной характеристикой для любого изображения, а не только растрового. Согласно психофизиологическим исследованиям глаз человека способен различать 350000 цветов.

Классифицируем изображения следующим образом:

Двухцветные (бинарные)– 1 бит на пиксел. Среди двухцветных чаще всего встречаются черно-белые изображения.

Полутонные– градации серого или иного цвета. Например, 256 градаций (1 байт на пиксел).

Цветные изображения. От 2 бит на пиксел и выше. Глубина цвета 16 бит на пиксел (65 536 цветов) получила название **HighColor**, 24 бит на пиксел (16,7 млн цветов) – **TrueColor**. В компьютерных графических системах используют и большую глубину цвета – 32, 48 и более бит на пиксел.

2.5. Кодирование цвета. Палитра

Для того чтобы компьютер имел возможность работать с цветными изображениями, необходимо представлять цвета в виде чисел– кодировать цвет. Способ кодирования зависит от цветовой модели и формата числовых данных в компьютере.

Чтобы оцифровать цвет, его необходимо измерить. Наука, которая изучает цвет и его измерения, называется **колориметрией**. Она описывает общие закономерности цветового восприятия света человеком. Одними из основных законов колориметрии являются законы смешивания цветов. Эти законы в наиболее полном виде были сформулированы в 1855 году немецким математиком Германом Грассманом:

- закон трехмерности – любой цвет может быть представлен комбинацией трех основных цветов;
- закон непрерывности – к любому цвету можно подобрать бесконечно близкий;

- закон аддитивности – цвет смеси зависит только от цвета составляющих.

Первый закон означает, что для любого заданного цвета (Color) можно записать такое цветовое уравнение, выражающее линейную зависимость цветов:

$$Color = k_1 Color1 + k_2 Color2 + k_3 Color3,$$

где $Color1$, $Color2$, $Color3$ – некоторые базисные, линейно независимые цвета, коэффициенты k_1 , k_2 , k_3 указывают количество соответствующего смешиваемого цвета. Линейная независимость цветов означает, что ни один из них не может быть выражен взвешенной суммой (линейной комбинацией) двух других.

Цветовая модель RGB. За основные три цвета приняты красный (**Red**), зеленый (**Green**), синий (**Blue**). В модели RGB любой цвет (Color) получается в результате сложения основных цветов.

Для модели RGB каждая из компонент может представляться числами, ограниченными некоторым диапазоном – например, дробными числами от 0 до 1, либо целыми числами от 0 до некоторого максимального значения. В настоящее время достаточно распространенным является формат TrueColor, в котором каждая компонента представлена в виде байта, что дает 256 градаций для каждой компоненты: $R=0\dots255$, $G=0\dots255$, $B=0\dots255$. Количество цветов составляет $256*256*256 = 16.7$ млн (2^{24}).

Такой способ кодирования цветов можно назвать **компонентным**. В компьютере коды изображений TrueColor представляются в виде троек байтов либо упаковываются в длинное целое (четырехбайтное) – 32 бита (так, например, сделано в API Windows):

C=00000000 bbbbbbbbggggggggrrrrrrrr.

Цветовая модель RGB применяется для создания графических образов в устройствах, *излучающих* свет – мониторах, телевизорах.

Цветовая модель СМΥК. В полиграфических системах напечатанный на бумаге графический объект сам не излучает световых волн. Изображение формируется на основе отраженной волны от окрашенных поверхностей. Окрашенные поверхности, на которые падает белый свет (т. е. сумма всех цветов), должны поглотить (т. е. вычесть) все составляющие цвета кроме того, цвет которой мы видим. Цвет поверхности можно получить красителями, которые *поглощают*, а не излучают. Например, если мы видим зеленое дерево, то это означает, что из падающего белого цвета, т. е. суммы красного, зеленого, синего, поглощены красный и синий, а зеленый отражен. Цвета красителей должны быть дополняющими:

- голубой (Cyan = B + G), дополняющий красного;
- пурпурный (Magenta = R + B), дополняющий зеленого;
- желтый (Yellow = R + G), дополняющий синего.

Но так как цветные красители по отражающим свойствам не одинаковы, то для повышения контрастности применяется еще черный (Black). Модель СМΥК

названа по первым буквам слов Cyan, Magenta, Yellow и последней букве слова Black. Так как цвета вычитаются, модель называется **субтрактивной**.

При работе с изображениями в системах компьютерной графики часто приходится искать компромисс между качеством изображения (требуется как можно больше цветов) и ресурсами, необходимыми для хранения и воспроизведения изображения, исчисляемыми, например, объемом памяти (надо уменьшать количество бит на пиксел).

Кроме того, некоторое изображение само по себе может использовать ограниченное количество цветов. Например, для черчения может быть достаточно двух цветов, для человеческого лица важны оттенки розового, желтого, пурпурного, красного, зеленого; а для неба – оттенки голубого и серого. В этих случаях использование полноцветного кодирования цвета является избыточным.

При ограничении количества цветов используют **палитру**, представляющую набор цветов, важных для данного изображения. Палитру можно воспринимать как таблицу цветов. Палитра устанавливает взаимосвязь между кодом цвета и его компонентами в выбранной цветовой модели.

2.6. Графический формат

Графическим форматом называют порядок (структуру), согласно которому данные, описывающие изображение, записаны в файле.

Типы форматов графических файлов определяются способом хранения и типом графических данных. Наиболее широко используются **растровый (битовый)**, **векторный** и **метафайловый** форматы.

Векторный формат наиболее удобен для хранения изображений, которые можно разложить на простые геометрические фигуры (например, чертежи или текст). Векторные файлы содержат математические описания элементов изображения. Наиболее распространенные векторные форматы: AutoCADDXF и MicrosoftSYLK.

Растровый формат используется для хранения растровых данных. Файлы такого типа особенно хорошо подходят для хранения изображений реального мира, например, оцифрованных фотографий. Растровые файлы содержат битовую карту изображения и ее спецификацию. Наиболее распространенные растровые форматы: BMP, TIFF, GIF, PCX, JPEG.

Каждый из этих форматов имеет свои преимущества и свои недостатки.

Битовые изображения, как правило, выводятся на экран быстрее, так как их внутренняя структура аналогична (до некоторой степени) структуре видеопамати. Изображения, получаемые при помощи сканеров и цифровых видеокамер, получаются именно как битовые изображения.

К недостаткам битовых изображений можно отнести большой объем памяти, требующийся для их хранения (около 1 Мбайт в режиме True Color), невозможность масштабирования без потери качества изображения, а также сложность выделения и изменения отдельных объектов изображения.

Векторные изображения состоят из описаний отдельных элементов, поэтому они легко масштабируются. Однако вывод векторных изображений выполняется, как правило, медленнее, чем битовых.

Следует отметить, что некоторые устройства вывода, такие как плоттер (графопостроитель), способны работать только с векторными изображениями, так как с помощью пера можно рисовать только линии.

Существует множество форматов файлов, предназначенных для хранения битовых и векторных изображений.

Метафайловый формат позволяет хранить в одном файле и векторные, и растровые данные. Примером такого формата являются файлы CorelDRAW–CDR.

Кроме того, существуют файловые форматы для хранения мультимедии (видеоинформации), мультимедиа-форматы (одновременно хранят звуковую, видео- и графическую информацию), гипертекстовые (позволяют хранить не только текст, но и связи-переходы внутри него) и гипермедиа (гипертекст плюс графическая и видеоинформация) форматы, форматы трехмерных сцен, форматы шрифтов и т. д.

Глава 3. РАБОТА С РАСТРОВЫМИ ИЗОБРАЖЕНИЯМИ

3.1. Виды растровых форматов

К настоящему времени известно много форматов файлов для растровых изображений. Здесь мы рассмотрим работу с одним из самых популярных форматов, который обязан своей распространенностью операционной системе Windows – формат **bmp**.

В операционной системе Windows используются два формата растровых (битовых) изображений – **аппаратно-зависимый DDB** (device-dependent bitmap) и **аппаратно-независимый DIB** (device-independent bitmap).

Согласно определению, данному в документации к SDK, **битовое изображение DDB** есть набор бит в оперативной памяти, который может быть отображен на устройстве вывода (например, выведен на экран видеомонитора или распечатан на принтере). Внутренняя структура изображения DDB жестко привязана к аппаратным особенностям устройства вывода. Поэтому представление изображения DDB в оперативной памяти полностью зависит от устройства вывода.

Для описания растровых изображений в ОС Windows используется структура **BITMAP**:

```
typedef struct tagBITMAP {
    LONG bmType; //Тип битового изображения (= 0)
    LONG bmWidth; //Ширина битового изображения в
                //пикселах (> 0)
    LONG bmHeight; //Высота битового изображения в
                //пикселах (>0)
    LONG bmWidthBytes; //Размер памяти, занимаемый одной
                //строкой раstra битового изображения.
    WORD bmPlanes; //Количество плоскостей в битовом
                //изображении
    WORD bmBitsPixel; //Количество бит, используемых
                //для представления цвета пиксела
    LPVOID bmBits; //Указатель на массив, содержащий
                //биты изображения
} BITMAP, *PBITMAP;
```

Если бы в Windows можно было работать только с изображениями DDB, было бы необходимо иметь отдельные наборы изображений для каждого типа видеоконтроллера и каждого видеорежима, что, очевидно, крайне неудобно.

Аппаратно-независимое битовое изображение DIB содержит описание цвета пикселей изображения, которое не зависит от особенностей устройства отображения. Операционная система Windows после соответствующего преобразования может отобразить такое изображение на любом устройстве вывода. Несмотря на некоторое замедление процесса вывода по сравнению с выводом изображений DDB, универсальность изображений DIB делает их весьма привлекательными для хранения изображений.

3.2. Битовые изображения в формате DDB

Как было сказано выше, битовые изображения в формате DDB являются **аппаратно-зависимыми**. Поэтому структура изображения в оперативной памяти зависит от особенностей аппаратуры.

Как правило, изображения DDB либо загружаются из ресурсов приложения, либо создаются непосредственно в оперативной памяти. Для вывода изображений DDB на экран используются такие функции, как **BitBlt** и **StretchBlt**.

Ниже приведено описание функций **BitBlt** и **StretchBlt**.

```
BOOL WINAPI BitBlt(
    HDC hdcDest, //Контекст для рисования
    int nXDest, //X-координата верхнего левого угла
                //области рисования
    int nYDest, //Y-координата верхнего левого угла
                //области рисования
```

```

int nWidth,      //Ширина изображения
int nHeight,     //Высота изображения
HDC hdcSrc,      //Идентификатор исходного контекста
int nXSrc,       //X-координата верхнего левого угла
                //исходной области
int nYSrc,       //Y-координата верхнего левого угла
                //исходной области
DWORD dwRop)     //Код растровой операции

```

Функция копирует битовое изображение из исходного контекста **hdcSrc** в контекст отображения **hdcDest**. Возвращаемое значение равно TRUE при успешном завершении или FALSE при ошибке.

В качестве кода растровой операции чаще всего используется константа **SRCCOPY**. При этом цвет пикселей копируемого изображения полностью заменяет цвет соответствующих пикселей контекста отображения. В этом случае цвет кисти, выбранной в контекст отображения, не имеет значения, так как ни цвет кисти, ни цвет фона не влияют на цвет нарисованного изображения.

Для рисования битовых изображений можно использовать вместо функции **BitBlt** функцию **StretchBlt**, с помощью которой можно выполнить масштабирование (сжатие или растяжение) битовых изображений:

```

BOOL WINAPI StretchBlt(
HDC hdcDest,      //контекст для рисования
int nXDest,       //X-координата верхнего левого
                //угла области рисования
int nYDest,       //Y-координата верхнего левого угла
                //области рисования
int nWidthDest,   //новая ширина изображения
int nHeightDest,  //новая высота изображения
HDC hdcSrc,       //идентификатор исходного контекста
int nXSrc,        //X-координата верхнего левого
                //угла исходной области
int nYSrc,        //Y-координата верхнего левого
                //угла исходной области
int nWidthSrc,    //ширина исходного изображения
int nHeightSrc,   //высота исходного изображения
DWORD dwRop)      //код растровой операции

```

Параметры этой функции аналогичны параметрам функции **BitBlt**, за исключением того, что ширина и высота исходного и полученного изображения должна определяться отдельно. Размеры исходного изображения (логические) задаются параметрами **nWidthSrc** и **nHeightSrc**, размеры нарисованного изображения задаются параметрами **nWidthDest** и **nHeightDest**.

Возвращаемое значение равно TRUE при успешном завершении или FALSE при ошибке.

3.3. Битовые изображения в формате DIB

Изображения DIB, в отличие от изображений DDB, являются аппаратно-независимыми, поэтому без дополнительного преобразования их нельзя отображать на экране с помощью функций **BitBlt** и **StretchBlt**. В операционной системе Windows битовые изображения хранятся в файлах с расширением имени **bmp**, при этом используется аппаратно-независимый формат **DIB**.

Формат **bmp**-файлов представлен на рис. 3.1.

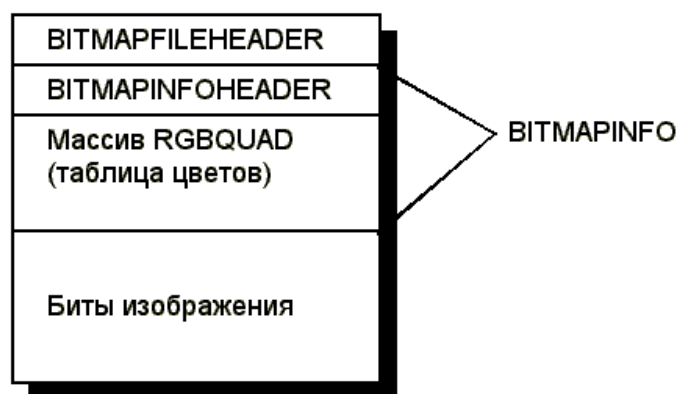


Рис. 3.1

Файл, содержащий битовое изображение, начинается со структуры **BITMAPFILEHEADER**. Эта структура описывает тип файла и его размер, а также смещение области битов изображения.

```

typedef struct tagBITMAPFILEHEADER {
    WORD b fType; //признак bmp-файла, 42 4D (коды букв BM)
    DWORD bfSize; //размер файла
    WORD bfReserved1; //первое резервное поле -
    //всегда ноль
    WORD bfReserved2; //второе резервное поле - тоже ноль
    DWORD bfOffBits; //смещение от начала файла до
    //первого байта графических данных
} BITMAPFILEHEADER, *PBITMAPFILEHEADER;
  
```

Сразу после структуры **BITMAPFILEHEADER** в файле следует структура **BITMAPINFO**, которая содержит описание изображения и таблицу цветов.

```

typedef struct tagBITMAPINFO {
    BITMAPINFOHEADER bmiHeader;
    RGBQUAD bmiColors[1];
} BITMAPINFO, *PBITMAPINFO;
  
```

Описание изображения (размеры изображения, метод компрессии, размер таблицы цветов и т. д.) находится в структуре **BITMAPINFOHEADER**.

```

typedef struct tagBITMAPINFOHEADER{
  
```

```

DWORD biSize;      //размер BITMAPINFOHEADER в байтах.
LONG biWidth;      //ширина картинки в пикселах
LONG biHeight;     //высота картинки в пикселах
WORD biPlanes;     //количество битовых плоскостей (=1)
WORD biBitCount;   //количество бит на пиксел
                //(определяет максимальное число цветов в bitmap)
DWORD biCompression; //способ сжатия (0 – данные
                //(не сжимаются).
DWORD biSizeImage;  //размер изображения в байтах
                //(без заголовков)
LONG biXPelsPerMeter; //число пикселей на метр
                //(по горизонтали)
LONG biYPelsPerMeter; //число пикселей на метр
                //(по вертикали)
DWORD biClrUsed;    //количество элементов палитры,
                //(хранящихся после заголовка)
DWORD biClrImportant; //количество разных цветов,
                //(действительноиспользуемыхврисунке)
} BITMAPINFOHEADER, *PBITMAPINFOHEADER;

```

В некоторых случаях (не всегда) в файле может присутствовать **палитра**—таблица цветов (как массив структур RGBQUAD), присутствующих в изображении.

```

typedef struct tagRGBQUAD
{
    BYTE rgbBlue;
    BYTE rgbGreen;
    BYTE rgbRed;
    BYTE rgbReserved;
} RGBQUAD;

```

Палитра отсутствует, если число бит на пиксел равно 24. Также палитра не нужна и для некоторых цветовых форматов 16 и 32 бит на пиксел.

После палитры (если она есть) в файле BMP записывается растр в виде битового (а точнее, байтового массива). В битовом массиве последовательно записываются байты строк растра. Количество байт в строке должно быть кратно четырем, поэтому если количество пикселей по горизонтали не соответствует такому условию, то справа в каждую строку дописывается некоторое число битов (выравнивание строк на границу двойного слова).

Точное значение смещения битов изображения находится в структуре BITMAPFILEHEADER.

3.4. Загрузка данных из bmp-файла

Чтобы изображение загрузить с диска в оперативную память и получить дескриптор изображения (типа HBITMAP), используются функции **LoadBitmap()**

и **LoadImage()**. Так, например, из файла с расширением **.bmp** изображение можно прочитать при помощи функции **LoadImage()**:

```
HBITMAP Bit=(HBITMAP)LoadImage(NULL, char* Filename,
IMAGE_BITMAP, 0, 0, LR_LOADFROMFILE | LR_CREATEDIBSECTION)
```

Значение **NULL** **первого параметра** указывает, что картинка вводится из файла. При получении изображения из ресурса в первом параметре указывается дескриптор приложения. Во **втором параметре** задается имя файла или идентификатор ресурса. **Третий параметр** задает тип изображения и может принимать значения **IMAGE_BITMAP**, **IMAGE_CURSOR**, **IMAGEICON**. **Четвертый и пятый параметры**, которые указывают размеры изображения, при вводе битовой карты задаются нулями, так как эта информация имеется в заголовке **bmp**-файла. **Шестой параметр** содержит сочетание флагов. В частности, флаг **LR_LOADFROMFILE** указывает, что изображение вводится из файла. При задании флага **LR_CREATEDIBSECTION** создается **аппаратно-независимое**, а при его отсутствии – **аппаратно-зависимое** растровое изображение. Ниже приводится код функции для загрузки битовой карты, которая получает путь к файлу **FileName** и тип раstra **Type**. Нулевое значение будет означать, что используется тип **DIB**, а единичное значение будет применяться для типа **DDB**.

```
HBITMAP LoadPict(char *FileName, int Type)
{
    if (Type)
        return LoadImage(NULL, FileName, IMAGE_BITMAP, 0, 0,
        LR_LOADFROMFILE);
    else
        return LoadImage(NULL, FileName, IMAGE_BITMAP, 0, 0,
        LR_LOADFROMFILE | LR_CREATEDIBSECTION);
}
```

Функция **LoadImage()** позволяет загружать из ресурса битовую карту, курсор или пиктограмму, но для этого более удобны простые специализированные функции **LoadBitmap()**, **LoadCursor()**, **LoadIcon()**.

3.5. Вывод растровых изображений на экран

Рассмотрим пример функции для отображения рисунка формата **bmp** в окне **Windows**.

```
Int ShowBitMap(HWND hWnd, HBITMAP hBit, int x, int y)
//функция отображает рисунок в заданной позиции окна
//hWnd - дескриптор окна, куда выводится изображение
//hBit - дескриптор рисунка
//(x, y) - координаты левого верхнего угла
//изображения в окне вывода
{
```

```

BITMAPBitMap;
//в структуру BitMap считываются параметры картинки
GetObject(hBit,sizeof(BITMAP),&BitMap);
//из ее полей bmHeight и bmWidth получаем размеры
//для копирования
int Height=BitMap.bmHeight;
int Width=BitMap.bmWidth;
//получаем контекст изображения
HDC hdc = GetDC(hWnd);
//создаем контекст памяти
HDC hdcMem=CreateCompatibleDC(hdc);
//в созданный контекст памяти заносим дескриптор
//битовой карты
HBITMAP OldBitmap=(HBITMAP)SelectObject(hdcMem,hBit);
//если в исходном тексте программы есть строка
//#include<windowsx.h>, то дескриптор hBit можно
//включить в контекст при помощи инструкции:
//HBITMAP OldBitmap=SelectBitmap(hdcMem,hBit);
//контекст hdcMem можно использовать для рисования
//средствами GDI - создаваемое изображение с учетом
//заданной растровой операции наносится на включенную
// в контекст картинку.
//в окно, с которым связан контекст изображения hdc,
// картинка переносится при помощи функции копирования:
BitBlt(hdc,x,y,Width,Height,hdcMem,0,0,
SRCCOPY);
//PATCOPY
//после копирования уничтожаются контексты памяти
//и изображения
SelectObject(hdcMem,OldBitmap);
//SelectBitmap(hdcMem,OldBitmap);
ReleaseDC(hWnd,hdc);
DeleteDC(hdcMem);
return 0;
}

```

Рассмотрим пример функции для сохранения в файле на диске рабочей области окна в формате **bmp**.

```

int ClientToBmp(HWND hWnd, char *Name)
//сохранение рабочей области окна в файле Name.bmp
//hWnd - дескриптор окна, рабочая область которого
//сохраняется
//Name - имя файла для сохранения
{
BITMAPFILEHEADER bmfHdr;
BITMAPINFOHEADER bi;
RECT r;
int BitToPixel=16; //устанавливаем цветовую
//глубину 16 бит

```

```

GetClientRect(hWnd,&r); //узнаем размер рабочей
                        //области

HDC hdc = GetDC(hWnd);
HDC hdcMem = CreateCompatibleDC(hdc);
//создаем битовую карту BitMap по размеру рабочей
//области окна
//битовая карта создается на основе контекста
//устройства hdc, поэтому она хранит изображение
//DDB-формата (а не DIB).
HBITMAP BitMap = CreateCompatibleBitmap(hdc, r.right,
r.bottom);
HBITMAP OldBitmap = (HBITMAP)SelectObject(hdcMem,
BitMap);
BitBlt(hdcMem, 0, 0, r.right, r.bottom, hdc,
0, 0, SRCCOPY);
BitMap = (HBITMAP)SelectObject(hdcMem, OldBitmap);
ZeroMemory(&bi, sizeof(BITMAPINFOHEADER)); //Это аналог
//функции memset(), который заполняет заголовок нулями.
bi.biSize = sizeof(BITMAPINFOHEADER);
bi.biWidth = r.right;
bi.biHeight = r.bottom;
bi.biPlanes = 1;
bi.biBitCount = BitToPixel; //в примерах режим 16
//бит тоже сохраняется,
//как 24- это DIB, но 8 бит на пиксел работать не
//будут - не записаны после заголовка элементы палитры
//Для вычисления размера изображения в байтах мы
//увеличиваем значение r.right * BitToPixel/8 байт
//на строку до значения, кратного четырем. Это
//вычисление может выполнить и функция GetDIBits()
bi.biSizeImage = (r.right * BitToPixel + 31) / 32 * 4 *
r.bottom;
DWORD dwWritten; //количество записанных файлов
//Открываем файл
HANDLE fh = CreateFile(Name, GENERIC_WRITE, 0, NULL,
CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL |
FILE_FLAG_SEQUENTIAL_SCAN, NULL);
if (fh == INVALID_HANDLE_VALUE) return 2;
bmfHdr.bfType = ('M' << 8) | 'B'; //заполняем дисковый
//заголовок
bmfHdr.bfSize = bi.biSizeImage + sizeof(bmfHdr) +
bi.biSize; //размер файла
bmfHdr.bfReserved1 = bmfHdr.bfReserved2 = 0;
bmfHdr.bfOffBits = sizeof(bmfHdr) + bi.biSize; //
//смещение до начала данных
//Запись заголовка в файл
WriteFile(fh, (LPSTR)&bmfHdr, sizeof(bmfHdr),
&dwWritten, NULL);
//Запись в файл загружаемого заголовка

```

```

WriteFile(fh, (LPSTR)&bi, sizeof(BITMAPINFOHEADER),
&dwWritten, NULL);
//Выделяем место в памяти для того, чтобы функция
//GetDIBits() перенесла туда коды цвета в DIB-формате
char *lp=(char *) GlobalAlloc(GMEM_FIXED,
bi.biSizeImage);
//Из карты BitMap строки с нулевой по bi.biHeight
//функция пересылает в массив lp по формату bi
// (беспалитровый формат)
interr=GetDIBits(hdc, BitMap, 0, (UINT)r.bottom, lp,
(LPBITMAPINFO) &b, DIB_RGB_COLORS);
//Запись изображения на диск
WriteFile(fh, lp, bi.biSizeImage, &dwWritten, NULL);
//Освобождение памяти и закрытие файла
GlobalFree(GlobalHandle(lp));
CloseHandle(fh);
ReleaseDC(hWnd, hdc);
DeleteDC(hdcMem);
if (dwWritten == 0) return 2;
return 0;
}

```

Глава 4. НЕКОТОРЫЕ АЛГОРИТМЫ КОМПЬЮТЕРНОЙ ГРАФИКИ

4.1. Классификация алгоритмов компьютерной графики

Алгоритмы компьютерной графики можно разделить на два уровня: **нижний** и **верхний**. Группа алгоритмов нижнего уровня предназначена для реализации графических примитивов (линий, окружностей, заливок и т.п.). Эти алгоритмы или подобные им воспроизведены в графических библиотеках языков высокого уровня или реализованы аппаратно в графических процессорах рабочих станций.

Среди алгоритмов **нижнего** уровня можно выделить следующие группы:

I группа –используют простые математические методы и отличаются простотой реализации. Как правило, такие алгоритмы не являются наилучшими по объему выполняемых вычислений или требуемым ресурсам памяти.

II группа– используют более сложные математические методы по сравнению с алгоритмами I группы (но часто и эвристические) и отличаются большей эффективностью.

III группа– алгоритмы, которые могут быть без больших затруднений реализованы аппаратно (допускающие распараллеливание, рекурсивные, реализуемые в простейших командах). В эту группу могут попасть и алгоритмы, представленные в первых двух группах.

К алгоритмам **верхнего** уровня относятся в первую очередь алгоритмы **удаления невидимых линий и поверхностей**. Задача удаления невидимых линий и поверхностей продолжает оставаться центральной в машинной графике. От эффективности алгоритмов, позволяющих решить эту задачу, зависят качество и скорость построения трехмерного изображения.

К задаче удаления невидимых линий и поверхностей примыкает **задача построения (закрашивания) полутоновых (реалистических) изображений**, т. е. учета явлений, связанных с количеством и характером источников света, учета свойств поверхности тела (прозрачность, преломление, отражение света).

Однако при этом не следует забывать, что вывод объектов в алгоритмах верхнего уровня обеспечивается примитивами, реализующими алгоритмы нижнего уровня, поэтому нельзя игнорировать проблему выбора и разработки эффективных алгоритмов нижнего уровня.

Для разных областей применения компьютерной графики на первый план могут выдвигаться разные свойства алгоритмов. Для научной графики большое значение имеет универсальность алгоритма, быстродействие может отходить на второй план. Для систем моделирования, воспроизводящих движущиеся объекты, быстродействие становится главным критерием, поскольку требуется генерировать изображение практически в реальном масштабе времени.

Особенности растровой графики связаны с тем, что обычные изображения, с которыми сталкивается человек в своей деятельности (чертежи, графики, карты, художественные картины и т.п.), реализованы на плоскости, состоящей из бесконечного набора точек. Экран же растрового дисплея представляется матрицей дискретных элементов, имеющих конкретные физические размеры. При этом число их существенно ограничено. Поэтому нельзя провести точную линию из одной точки в другую, а можно выполнить только аппроксимацию этой линии с отображением ее на дискретной матрице (плоскости). Такую плоскость также называют **целочисленной решеткой**, растровой плоскостью или растром. Эта решетка представляется квадратной сеткой с шагом 1. Отображение любого объекта на целочисленную решетку называется **разложением его в растр** или просто **растровым представлением**.

4.2. Базовые растровые алгоритмы

Сформировать растровое изображение можно по-разному. Для того чтобы создать изображение на растровом дисплее, можно просто скопировать готовый

растр в видеопамять. Этот растр может быть получен, например, с помощью сканера или цифрового фотоаппарата. А можно создавать изображение объекта путем последовательного рисования отдельных простых элементов.

Простые элементы, из которых складываются сложные объекты, будем называть **графическими примитивами**. Их можно встретить повсюду. Например, для построения изображения используется некоторый набор примитивов, которые поддерживаются определенными графическими устройствами вывода. Графические примитивы также можно применять для описания пространственных объектов в базе данных компьютерной системы – модели объектов. Могут использоваться различные множества примитивов для модели и для алгоритмов отображения. Удобно, когда эти множества совпадают, тогда упрощается процесс отображения.

Простейшим и, вместе с тем, наиболее универсальным растровым графическим примитивом является **пиксел**. Любое растровое изображение можно нарисовать по пикселям, но это сложно и долго. Необходимы более сложные элементы, для которых рисуются сразу несколько пикселей.

4.2.1. Инкрементные алгоритмы растеризации

Брезенхэм предложил подход, позволяющий разрабатывать так называемые **инкрементные** алгоритмы растеризации [2]. Основной целью для разработки таких алгоритмов было построение циклов вычисления координат на основе только целочисленных операций сложения/вычитания без использования умножения и деления. Инкрементные алгоритмы выполняются как последовательное вычисление координат соседних пикселей путем добавления приращений координат.

Разработаны инкрементные алгоритмы для вывода отрезка, окружности, эллипса.

4.2.2. Алгоритм вывода прямой линии

Пусть на растре заданы две точки $A_0(x_0, y_0)$ и $A_1(x_1, y_1)$ с целочисленными координатами (рис. 4.1).

Запишем уравнение прямой, проходящей через эти точки

$$\frac{x - x_0}{x_1 - x_0} = \frac{y - y_0}{y_1 - y_0}$$

или

$$y = y_0 + \frac{y_1 - y_0}{x_1 - x_0} (x - x_0) = y_0 + \frac{\Delta y}{\Delta x} (x - x_0) = kx + b \quad (4.1)$$

где

$$k = \frac{y_1 - y_0}{x_1 - x_0}, b = y_0 - kx_0, \Delta x = x_1 - x_0, \Delta y = y_1 - y_0$$

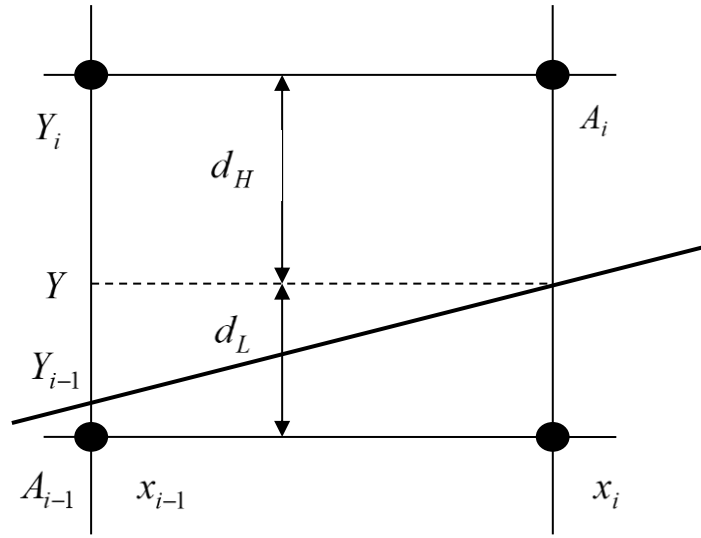


Рис. 4.1

Будем рассматривать случай для $k < 1$ и $x_1 > x_0$.

Пусть точка $A_{i-1}(x_{i-1}, y_{i-1})$ уже поставлена. Необходимо принять решение о том, какие координаты должна иметь точка $A_i(x_i, y_i)$: $(x_{i-1} + 1, y_{i-1})$ или $(x_{i-1} + 1, y_{i-1} + 1)$.

Запишем выражение для y -координаты математической прямой для $x = x_i$:

$$y = kx_i + b = k(x_{i-1} + 1) + b$$

Вычислим разности

$$\begin{aligned} d_L &= y - y_{i-1} = k(x_{i-1} + 1) + b - y_{i-1}, \\ d_H &= y_i - y = (y_{i-1} + 1) - y = y_{i-1} + 1 - k(x_{i-1} + 1) - b, \\ p_{i-1} &= d_L - d_H = [k(x_{i-1} + 1) + b - y_{i-1}] - [y_{i-1} + 1 - k(x_{i-1} + 1) - b] = \\ &= 2k(x_{i-1} + 1) - 2y_{i-1} + 2b - 1. \end{aligned}$$

Тогда

$$y_i = \begin{cases} y_{i-1}, & \text{при } p_{i-1} < 0 \\ y_{i-1} + 1, & \text{при } p_{i-1} \geq 0 \end{cases}$$

Выполним замену в p_{i-1}

$$k = \frac{\Delta y}{\Delta x}$$

$$p_{i-1} = 2 \frac{\Delta y}{\Delta x} (x_{i-1} + 1) - 2y_{i-1} + 2b - 1$$

или

$$\Delta x p_{i-1} = 2\Delta y(x_{i-1} + 1) - 2\Delta x y_{i-1} + (2b - 1)\Delta x$$

Полагая, что $\Delta x p_{i-1} = d_{i-1}$, получим

$$d_{i-1} = 2\Delta y x_{i-1} - 2\Delta x y_{i-1} + c, \quad (4.2)$$

где

$$c = 2\Delta y + (2b - 1)\Delta x \quad (4.3)$$

$$d_i = 2\Delta y x_i - 2\Delta x y_i + c \quad (4.4)$$

Так как $\Delta x > 0$, то

$$y_i = \begin{cases} y_{i-1}, & \text{при } d_{i-1} < 0 \\ y_{i-1} + 1, & \text{при } d_{i-1} \geq 0 \end{cases}$$

Вычитая (4.2) из (4.4), получим

$$d_i - d_{i-1} = 2\Delta y(x_i - x_{i-1}) - 2\Delta x(y_i - y_{i-1}).$$

Но $x_i - x_{i-1} = 1$, тогда $d_i = d_{i-1} + 2\Delta y - 2\Delta x(y_i - y_{i-1})$.

При этом

$$y_i = \begin{cases} y_{i-1}, & \text{при } d_{i-1} < 0 \\ y_{i-1} + 1, & \text{при } d_{i-1} \geq 0 \end{cases}.$$

Поэтому

$$d_i = \begin{cases} d_{i-1} + 2\Delta y, & \text{при } d_{i-1} < 0 \\ d_{i-1} + 2\Delta y - 2\Delta x, & \text{при } d_{i-1} \geq 0 \end{cases}.$$

Вычислим начальное значение d_0 .

Из (4.2) получаем

$$d_0 = 2\Delta y x_0 - 2\Delta x y_0 + c \quad (4.5)$$

Значение d_0 соответствует точке $A_0(x_0, y_0)$.

Точка $A_0(x_0, y_0)$ по условию удовлетворяет уравнению прямой (4.1)
 $y_0 = kx_0 + b$.

Отсюда

$$b = y_0 - kx_0 = y_0 - \frac{\Delta y}{\Delta x} x_0.$$

Подставляя полученное значение b в выражение (4.3) для c , получим

$$\begin{aligned} c &= 2\Delta y + (2b - 1)\Delta x = 2\Delta y + \left[2\left(y_0 - \frac{\Delta y}{\Delta x} x_0 \right) - 1 \right] \Delta x = \\ &= 2\Delta y + (2\Delta x y_0 - 2\Delta y x_0 - \Delta x). \end{aligned}$$

Подставляя полученное выражение для c в выражение (4.5) для d_0 , получим

$$d_0 = 2\Delta x y_0 - 2\Delta x y_0 + 2\Delta y + (2\Delta x y_0 - 2\Delta y x_0 - \Delta x) = 2\Delta y - \Delta x.$$

$d_i = \begin{cases} d_{i-1} + 2\Delta y, & \text{при } d_{i-1} < 0 \\ d_{i-1} + 2\Delta y - 2\Delta x, & \text{при } d_{i-1} \geq 0 \end{cases}.$
$y_i = \begin{cases} y_{i-1}, & \text{при } d_{i-1} < 0 \\ y_{i-1} + 1, & \text{при } d_{i-1} \geq 0 \end{cases}.$

Ниже приведен возможный вариант функции для рисования линии между точками (x_1, y_1) и (x_2, y_2) для $k < 1$ цветом $color$ для работы с использованием библиотеки классов MFC.

```
void Line(CDC& dc, int x1, int y1, int x2, int y2, COLORREF color)
//Алгоритм Брезенхема для прямой y=kx+b, при k<1!!!
//dc - ссылка на контекст устройства
//(x1,y1) - координаты начальной точки
//(x2,y2) - координаты конечной точки
//color - цвет линии
{
    int dx=x2-x1;
    int dy=y2-y1;
    int d=2*dy-dx;
    int d1=2*dy;
    int d2=2*(dy-dx);
    int y=y1;
    dc.SetPixel(x1, y1, color);
    for(int x=x1+1; x<=x2; x++)
    {
        if(d<0) d+=d1;
    }
}
```

```

    else
    {
        y++;
        d+=d2;
    }
    dc.SetPixel(x,y,color);
}
}

```

Приведенная ниже функция реализует алгоритм Брезенхема для любых k .

void MyLine(CDC& dc,int x1,int y1,int x2,int y2,COLORREF color)
 //Алгоритм Брезенхема для прямой $y=kx+b$ при любых k

```

{
    int dx=abs(x2-x1);
    int dy=abs(y2-y1);
    int incX=x2>=x1?1:-1;
    int incY=y2>=y1?1:-1;
    if(dy<= dx)
    {
        int d=2*dy-dx;
        int d1=2*dy;
        int d2=2*(dy-dx);
        dc.SetPixel(x1,y1,color);
        int x=x1;
        int y=y1;
        for(int i =1;i <=dx;i++)
        {
            x+=incX;
            if(d<0)d+=d1;
            else
            {
                y+=incY;
                d+=d2;
            }
            dc.SetPixel(x,y,color);
        }
    }
}

else
{
    int d=2*dx-dy;
    int d1=2*dx;
    int d2=2*(dx-dy);
    dc.SetPixel(x1,y1,color);
    int x=x1;
    int y=y1;
    for(int i =1; i <=dy;i++)
    {
        y+=incY;

```

```

        if (d<0) d+=d1;
        else
        {
            x+=incX;
            d+=d2;
        }
        dc.SetPixel(x,y, color);
    }
}

```

4.2.3. Алгоритм построения окружности методом средней точки

Рассмотрим уравнение окружности радиуса r с центром в начале координат

$$x^2 + y^2 = r^2 \quad (4.6)$$

Введем в рассмотрение функцию окружности

$$f(x, y) = x^2 + y^2 - r^2 \quad (4.7)$$

Любая точка $A(x, y)$, которая лежит на окружности удовлетворяет уравнению $f(x, y) = 0$.

Если точка находится внутри круга, то функция окружности будет иметь отрицательное значение. Если точка лежит за пределами круга, значение функции окружности будет положительным. Подытоживая, можно сказать, что относительное положение любой точки с координатами $A(x, y)$ определяется проверкой знака функции окружности: точка

$$f \rightarrow \begin{cases} < 0, & \text{если точка } A(x, y) \text{ лежит внутри окружности} \\ = 0, & \text{если точка } A(x, y) \text{ лежит на окружности} \\ > 0, & \text{если точка } A(x, y) \text{ лежит за пределами окружности} \end{cases} \quad 4.8)$$

Проверка выполняется на каждом этапе выборки для средних положений между пикселями вблизи заданной окружности. Таким образом, функция окружности – это параметр принятия решения в алгоритме средней точки, и для этой функции можно установить операции приращения, как это было сделано для алгоритма построения прямой линии.

На рис. 4.2 показана средняя точка между двумя возможными пикселями в точке выборки. Предположим, что мы только что поставили точку в пикселе с координатами $A_i(x_i, y_i)$. Теперь нужно определить, какой из двух пикселей ближе к заданной окружности – пиксел с координатами (x_i+1, y_i) или пиксел с координатами (x_i+1, y_i-1) . Параметром принятия решения будет функция окружности (4.7), которая рассчитывается для средней точки между этими двумя пикселями:

$$d_i = f\left(x_i + 1, y_i - \frac{1}{2}\right) = (x_i + 1)^2 + \left(y_i - \frac{1}{2}\right)^2 - r^2$$

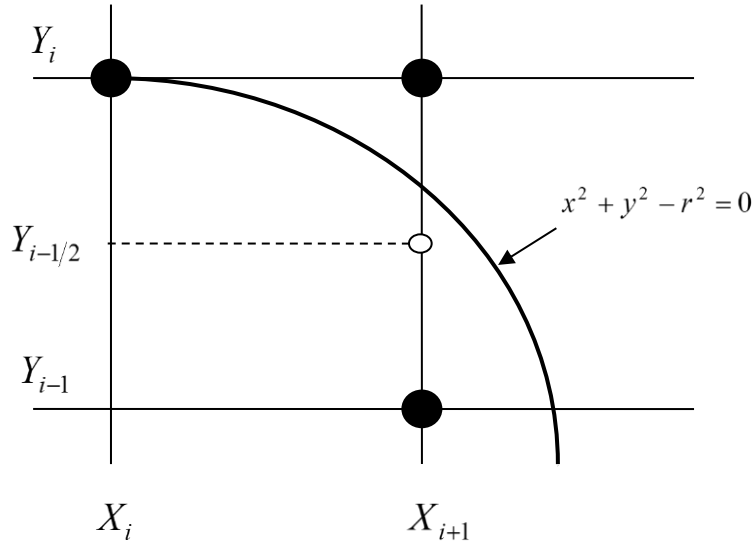


Рис. 4.2

В случае, когда $d_i < 0$, полагаем $y_{i+1} = y_i$ (если средняя точка **внутри** окружности, то она (окружность) ближе к верхнему пикселу). Тогда

$$d_{i+1} = f\left(x_i + 2, y_i - \frac{1}{2}\right) = (x_i + 2)^2 + \left(y_i - \frac{1}{2}\right)^2 - r^2$$

$$\Delta d_i = d_{i+1} - d_i = 2x_i + 3$$

$$d_{i+1} = d_i + \Delta d_i = d_i + 2x_i + 3 = d_i + p_i$$

где

$$p_i = 2x_i + 3 = 2(x_{i-1} + 1) + 3 = 2x_{i-1} + 3 + 2 = p_{i-1} + 2, \quad i = 1, 2, 3, \dots$$

При этом

$$p_0 = 2x_0 + 3$$

Тогда

$$d_{i+1} = d_i + p_i, \quad i = 0, 1, 2, \dots$$

$$p_i = p_{i-1} + 2, \quad i = 1, 2, 3, \dots, \text{ для } d_i < 0.$$

В случае, когда $d_i \geq 0$, делаем шаг **вниз** (если средняя точка **вне** окружности, то она (окружность) ближе к нижнему пикселу), полагая $y_{i+1} = y_i - 1$. Тогда

$$d_{i+1} = f\left(x_i + 2, y_i - \frac{3}{2}\right) = (x_i + 2)^2 + \left(y_i - \frac{3}{2}\right)^2 - r^2$$

$$\Delta d_i = d_{i+1} - d_i = 2x_i + 3 - 2y_i + 2 = 2(x_i - y_i) + 5$$

$$\begin{aligned} d_{i+1} &= d_i + 2(x_i - y_i) + 5 = d_i + 2x_i + 3 - 2y_i + 2 = \\ &= d_i + p_i - 2y_i + 2 = d_i + p_i + s_i = d_i + q_i, \end{aligned}$$

где

$$\begin{aligned} q_i &= p_i + s_i \\ s_i &= -2y_i + 2. \end{aligned}$$

Для рассматриваемого случая, когда $d_i \geq 0$, $y_i = y_{i-1} - 1$, получаем

$$s_i = -2y_i + 2 = -2(y_{i-1} - 1) + 2 = -2y_{i-1} + 2 + 2 = s_{i-1} + 2, \quad i = 1, 2, 3, \dots$$

При этом $s_0 = -2y_0 + 2$. Тогда

$$q_i = p_i + s_i = p_{i-1} + 2 + s_{i-1} + 2 = q_{i-1} + 4, \quad i = 1, 2, 3, \dots$$

$$q_0 = p_0 + s_0$$

Таким образом

$$\begin{aligned} d_{i+1} &= d_i + q_i, \quad i = 0, 1, 2, \dots \\ q_i &= q_{i-1} + 4, \quad i = 1, 2, 3, \dots \end{aligned}$$

для $d_i \geq 0$.

Рассмотрим алгоритмы вычисления s_i и q_i для случая, когда $d_i < 0$, $y_i = y_{i-1}$.

В этом случае

$$s_i = -2y_i + 2 = -2y_{i-1} + 2 = s_{i-1}.$$

И выражение для q_i принимает вид

$$q_i = p_i + s_i = p_{i-1} + 2 + s_{i-1} = q_{i-1} + 2, \quad i = 1, 2, 3, \dots$$

Таким образом, если $d_i < 0$, то

$$\begin{cases} x_{i+1} = x_i + 1, \\ y_{i+1} = y_i, \\ d_{i+1} = d_i + p_i, \end{cases} \quad i = 0, 1, 2, \dots$$

$$\begin{cases} p_i = p_{i-1} + 2, \\ q_i = q_{i-1} + 2, \quad i = 1, 2, 3, \dots \end{cases}$$

если $d_i \geq 0$, то

$$\begin{cases} x_{i+1} = x_i + 1, \\ y_{i+1} = y_i - 1, \\ d_{i+1} = d_i + q_i, \quad i = 0, 1, 2, \dots \end{cases}$$

$$\begin{cases} p_i = p_{i-1} + 2, \\ q_i = q_{i-1} + 4, \quad i = 1, 2, 3, \dots \end{cases}$$

Для удобства заменим в выражениях для p_i и q_i i на $i + 1$, тогда для $d_i < 0$

$$\begin{cases} x_{i+1} = x_i + 1, \\ y_{i+1} = y_i, \\ d_{i+1} = d_i + p_i, \\ p_{i+1} = p_i + 2, \\ q_{i+1} = q_i + 2, \quad i = 0, 1, 2, \dots \end{cases}$$

При $d_i \geq 0$ получаем

$$\begin{cases} x_{i+1} = x_i + 1, \\ y_{i+1} = y_i - 1, \\ d_{i+1} = d_i + q_i, \\ p_{i+1} = p_i + 2, \\ q_{i+1} = q_i + 4, \quad i = 0, 1, 2, \dots \end{cases}.$$

Начальные значения параметров d и q определим из условия $x_0 = 0$ и $y_0 = r$.

Тогда

$$d_0 = f(x_0, y_0) = f\left(0, r - \frac{1}{2}\right) = \frac{5}{4} - r.$$

Поскольку радиус r и приращение Δd_i целые числа, то значение d_0 можно округлить и принять

$$d_0 = 1 - r$$

$$p_0 = 2x_0 + 3 = 2 \cdot 0 + 3 = 3$$

$$s_0 = -2y_0 + 2 = -2r + 2$$

$$q_0 = p_0 + s_0 = 3 - 2r + 2 = -2r + 5$$

Подводя итог изложенному, запишем в собранном виде алгоритм построения окружности методом средней точки.

Установить начальные значения для x_0, y_0, d_0, p_0, q_0 .

Поставить начальные точки.

Выполнять в цикле пока $y > x$

если $d_i < 0$, то

$$\begin{cases} x_{i+1} = x_i + 1, \\ y_{i+1} = y_i, \\ d_{i+1} = d_i + p_i, \\ p_{i+1} = p_i + 2, \\ q_{i+1} = q_i + 2, \quad i = 0, 1, 2, \dots \end{cases}$$

если $d_i \geq 0$, то

$$\begin{cases} x_{i+1} = x_i + 1, \\ y_{i+1} = y_i - 1, \\ d_{i+1} = d_i + q_i, \\ p_{i+1} = p_i + 2, \\ q_{i+1} = q_i + 4, \quad i = 0, 1, 2, \dots \end{cases}.$$

Поставить точки, соответствующие текущему шагу (рис. 4.3)

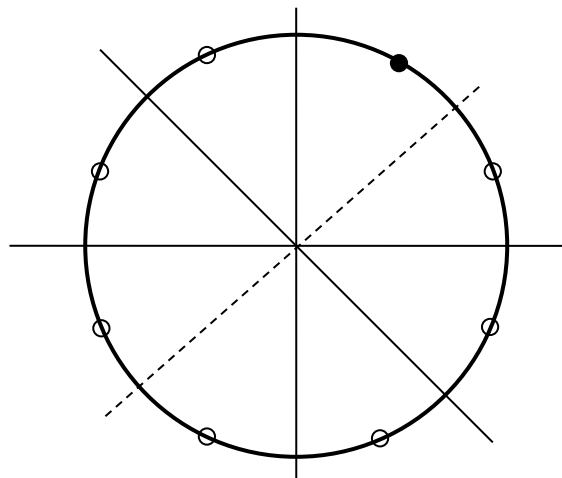


Рис. 4.3

Приводимая ниже функция расставляет точки окружности, соответствующие текущему шагу.

```
void CirclePoints(CDC& dc,int x,int y,CPoint &P,COLORREF color)
{
    dc.SetPixel(x+P.x,y+P.y,color);    //1
    dc.SetPixel(y+P.x,x+P.y,color);    //2
    dc.SetPixel(y+P.x,-x+P.y,color);   //3
    dc.SetPixel(x+P.x,-y+P.y,color);   //4
    dc.SetPixel(-x+P.x,-y+P.y,color);  //5
    dc.SetPixel(-y+P.x,-x+P.y,color);  //6
    dc.SetPixel(-y+P.x,x+P.y,color);   //7
    dc.SetPixel(-x+P.x,y+P.y,color);   //8
}
```

Приводимая ниже функция реализует алгоритм построения окружности методом средней точки.

```
void MyCircle(CDC& dc,CPoint &P,int r,COLORREF color)
{
    int x=0;
    int y=r;
    int d=1-r;
    int p=3;
    int q=-2*r+5;
    CirclePoints(dc,x,y,P,color);
    while(y>x)
    {
        if(d<0)
        {
            d+=p;
            p+=2;
            q+=2;
            x++;
        }
        else
        {
            d+=q;
            p+=2;
            q+=4;
            x++;
            y--;
        }
        CirclePoints(dc,x,y,P,color);
    }
}
```

4.3. Стиль линии. Перо

Для описания различных по виду изображений на основе линий используют термин **стиль линий** или **перо** [2]. Термин перо иногда делает более понятной суть алгоритма вывода линий для некоторых стилей – в особенности для толстых линий. Например, если для тонкой непрерывной линии перо соответствует одному пикселу, то для толстых линий перо можно представить себе как фигуру или отрезок линии, который скользит вдоль оси линии, оставляя за собой след (рис.4.4).

Форма пера	—		/	■	●
Результат					

Рис. 4.4

4.3.1. Алгоритмы вывода толстой линии

Взяв за основу любой алгоритм вывода обычных тонких линий (например, алгоритм Брезенхэма), запишем его в следующем обобщенном виде:

Вывод пиксела (x, y)

Можно представить себе такой алгоритм, как цикл, в котором определяются координаты (x, y) каждого пиксела. Этот алгоритм можно модифицировать для вывода толстой линии следующим образом:

Вывод фигуры (или линии) пера с центром (x, y)

Вместо вывода отдельного пиксела стоит вывод фигуры или линии, соответствующей перу – прямоугольник, круг, отрезок прямой.

Такой подход к разработке алгоритмов толстых линий имеет преимущества и недостатки. Преимущество – можно прямо использовать эффективные алгоритмы для вычисления координат точек линии оси, например, алгоритмы Брезенхэма. Недостаток – неэффективность для некоторых форм пера. Для перьев, которые соответствуют фигурам с заполнением, количество тактов работы алгоритма пропорционально квадрату толщины линии. При этом большинство пикселей многократно закрашивается в одних и тех же точках (рис. 4.5).

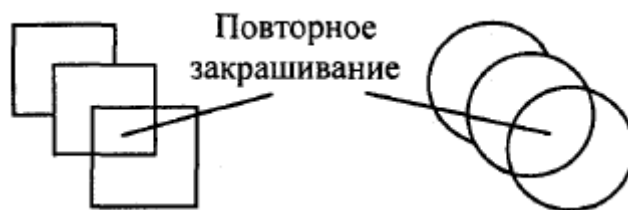


Рис. 4.5

Такие алгоритмы более эффективны для перьев в виде отрезков линий. В этом случае каждый пиксел рисуется только один раз. Но здесь важным является наклон изображаемой линии. Ширина пера зависит от наклона (рис. 4.6).

Очевидно, что горизонтальное перо не может рисовать толстую горизонтальную линию.

Для вывода толстых линий с помощью пера в качестве отрезка линии чаще всего используются отрезки горизонтальной или вертикальной линий, реже – диагональные отрезки под углом 45 градусов. Целесообразность такого способа определяется большой скоростью вывода горизонтальных и вертикальных отрезков прямой. Для того, чтобы достигнуть минимального количества тактов вывода, толстые линии, которые по наклону ближе к вертикальным, рисуют горизонтальным пером, а пологие линии – вертикальным пером.

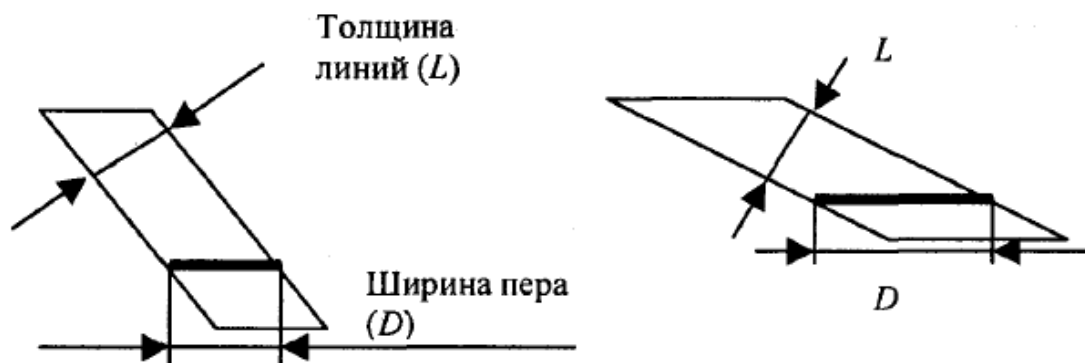


Рис. 4.6

4.3.2. Алгоритмы вывода пунктирной линии

Алгоритм для рисования тонкой пунктирной линии можно получить из алгоритма вывода тонкой непрерывной линии заменой процедуры вывода пиксела более сложной конструкцией:

Вывод пиксела (x , y)

Проверка значения счетчика C :

**Если C удовлетворяет некоторым условиям,
то вывод пиксела (x , y)**

Значение C увеличивается на единицу

При выводе полилиний, которые состоят из отрезков прямых, необходимо предотвратить обнуление значения счетчика в начале каждого отрезка и обеспечить продолжение непрерывного приращения вдоль всей сложной линии. Иначе будут нестыковки пунктира. Использование переменной-счетчика затруднено при генерации пунктирных линий в алгоритмах, которые используют симметрию, например, при выводе круга или эллипса. В этом случае будут нестыковки пунктира на границах октантов или квадрантов.

4.4. Стиль заполнения. Кисть. Текстура

При выводе фигур могут использоваться различные стили заполнения. Для обозначения стилей заполнения, отличных от сплошного стиля, используют такие понятия, как **кисть** и **текстура**. Их можно считать синонимами, однако понятие текстуры обычно используется применительно к трехмерным объектам, а кисть – для изображения двумерных объектов. Текстура – это стиль заполнения, закрашивание, которое имитирует сложную рельефную объемную поверхность, выполненную из какого-то материала.

Для описания алгоритмов заполнения фигур с определенным стилем можно использовать тот же способ, что и для описания алгоритмов рисования линий. Описание всех разновидностей подобных алгоритмов можно дать с помощью обобщенной схемы:

Вывод пиксела заполнения цвета C с координатами (x, y)

Например, в алгоритме вывода полигонов пиксела заполнения рисуются в теле цикла горизонталей, а все другие операции предназначены для подсчета координат этих пикселей. Сплошное заполнение означает, что цвет (C) всех пикселей одинаков, то есть $C = \text{const}$. Для получения определенного узора необходимо изменять цвет пикселей заполнения. Преобразуем алгоритм заполнения следующим образом:

$$C = f(x, y)$$

Вывод пиксела заполнения (x, y) цветом C

Функция $f(x, y)$ будет определять стиль заполнения. Аргументами функции цвета являются координаты текущего пиксела заполнения. Однако в отдельных случаях эти аргументы не нужны. Например, если цвет C вычислять как случайное значение в определенных границах: $C = \text{random}()$, то можно создать иллюзию шершавой матовой поверхности (рис.4.7, а).



Рис. 4.7

Другой стиль заполнения – штриховой (рис. 4.7, б). Для него функцию цвета также можно записать в аналитической форме:

$$f(x, y) = \begin{cases} C_{ш}, & \text{если } (x + y) \bmod S < T \\ C_{ф} & \text{в других случаях} \end{cases},$$

где S – период, T – толщина штрихов, $C_{ш}$ – цвет штрихов, $C_{ф}$ – цвет фона.

Подобную функцию можно записать и для других типов штриховки. Аналитическая форма описания стиля заполнения позволяет достаточно просто изменять размеры штрихов при изменениях масштаба показа, например, для обеспечения режима WYSIWYG.

Глава 5. МИРОВЫЕ И ЭКРАННЫЕ КООРДИНАТЫ

При отображении пространственных объектов на экране или на листе бумаги с помощью принтера необходимо знать координаты объектов. Мы рассмотрим две системы координат. Первая – **мировая система координат (МСК)**, которые описывают истинное положение объектов в пространстве с заданной точностью. Другая – **оконные координаты** или **система координат устройства изображения**, в котором осуществляется вывод изображения объектов в заданной проекции.

Мировые координаты объектов являются трехмерными. Положение объекта может быть описано, например, в прямоугольной или сферической системе координат. Где располагается центр системы координат и каковы единицы измерения вдоль каждой оси, не очень важно. Важно то, что для отображения должны быть известны какие-то числовые значения координат отображаемых объектов.

Первоначально рассмотрим работу с плоскими изображениями, для описания которых используются двухмерные мировые координаты. Положим, что изображение объекта сформировано в некоторой плоскости пространства. Рассмотрим задачу получения формул для пересчета мировых координат объекта в оконные.

Пусть в мировом пространстве задана плоскость P . Свяжем с этой плоскостью систему координат XOY и выделим на ней прямоугольную область D (рис. 5.1).

Рассмотрим оконную систему координат X^wOY^w с началом в левом верхнем углу окна и выделим в ней некоторую прямоугольную область D^w (рис. 5.2).

Пусть $M = M(x, y)$ – некоторая точка в мировой системе координат, а $M^w = M^w(x^w, y^w)$ ее образ в оконной системе координат.

Из приведенных рисунков можно получить пропорции

$$\begin{cases} \frac{x - x_L}{x_H - x_L} = \frac{x^w - x_L^w}{x_H^w - x_L^w} \\ \frac{y - y_L}{y_H - y_L} = \frac{y^w - y_L^w}{y_H^w - y_L^w} \end{cases} \quad (5.1)$$

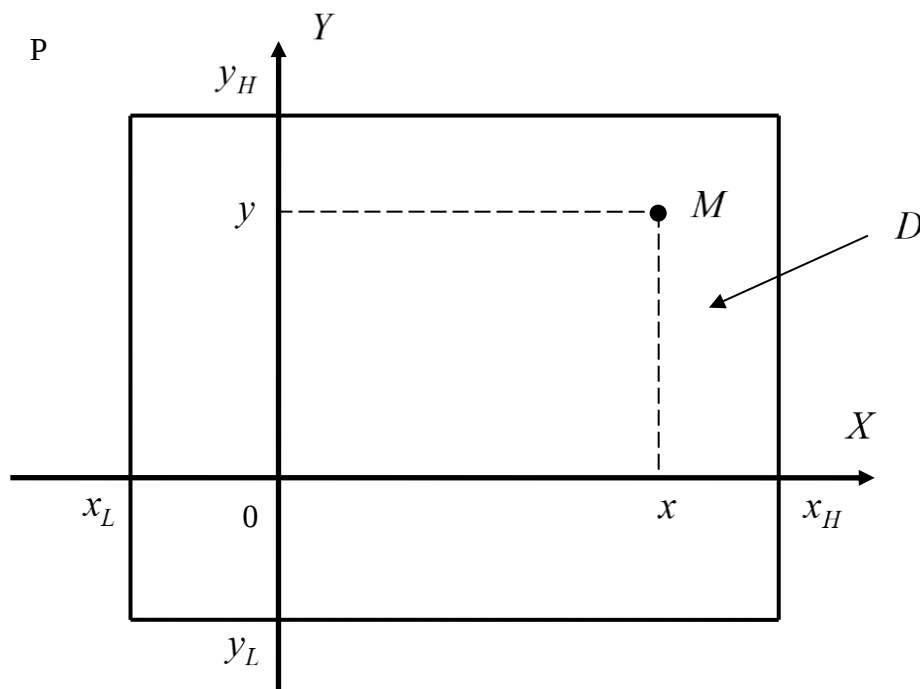


Рис. 5.1

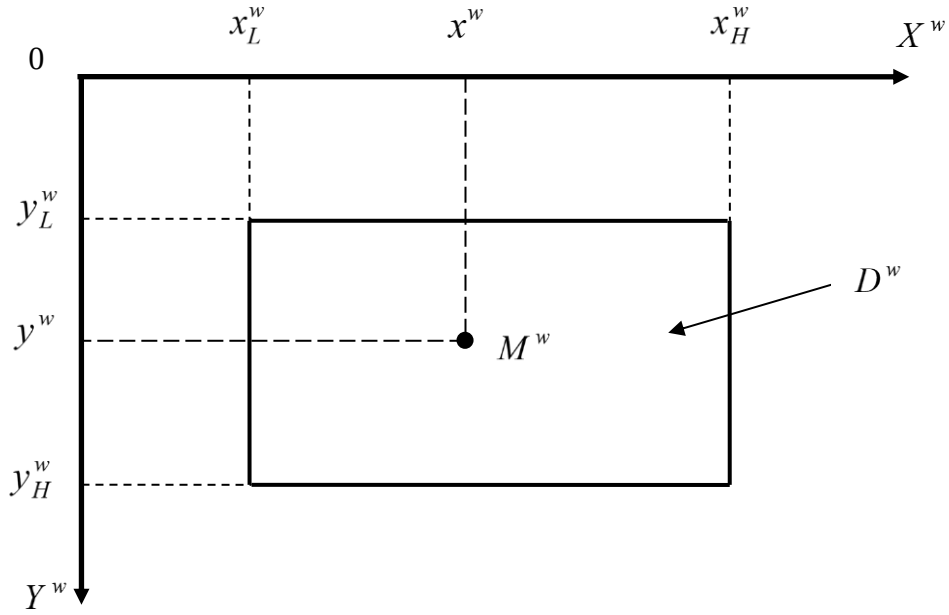


Рис. 5.2

$$\begin{cases} x^w = x_L^w + \frac{x_H^w - x_L^w}{x_H - x_L}(x - x_L) \\ y^w = y_H^w - \frac{y_H^w - y_L^w}{y_H - y_L}(y - y_L) \end{cases} \quad (5.2)$$

Введем обозначения:

- $x_H^w - x_L^w = \Delta x^w$ – ширина области отображения в оконных координатах,
- $x_H - x_L = \Delta x$ – ширина области отображения в мировых координатах,
- $y_H^w - y_L^w = \Delta y^w$ – высота области отображения в оконных координатах,
- $y_H - y_L = \Delta y$ – высота области отображения в мировых координатах.

Тогда выражение (5.2) можно переписать в виде

$$\begin{cases} x^w = x_L^w + \frac{\Delta x^w}{\Delta x}(x - x_L) \\ y^w = y_H^w - \frac{\Delta y^w}{\Delta y}(y - y_L) \end{cases} \quad (5.3)$$

или

$$\begin{cases} x^w = x_L^w + k_x(x - x_L) \\ y^w = y_H^w - k_y(y - y_L) \end{cases}, \quad (5.4)$$

где $k_x = \frac{\Delta x^w}{\Delta x}$, $k_y = \frac{\Delta y^w}{\Delta y}$.

Представим соотношения (5.4) в матричном виде

$$\begin{pmatrix} x^w \\ y^w \\ 1 \end{pmatrix} = \begin{pmatrix} k_x & 0 & x_L^w \\ 0 & -k_y & y_H^w \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x - x_L \\ y - y_L \\ 1 \end{pmatrix} \quad (5.5)$$

или

$$V^w = T_1 V_1 \quad (5.6)$$

где

$$V_1^w = \begin{pmatrix} x^w \\ y^w \\ 1 \end{pmatrix}, \quad T_1 = \begin{pmatrix} k_x & 0 & x_L^w \\ 0 & -k_y & y_H^w \\ 0 & 0 & 1 \end{pmatrix}, \quad V_1 = \begin{pmatrix} x - x_L \\ y - y_L \\ 1 \end{pmatrix}.$$

Перепишем пару рассматриваемых равенств в виде, удобном для дальнейшего использования.

$$\begin{cases} x^w = k_x x + (x_L^w - k_x x_L) \\ y^w = -k_y y + (y_H^w + k_y y_L) \end{cases} \quad (5.7)$$

Представим (5.7) также и в матричном виде

$$\begin{pmatrix} x^w \\ y^w \\ 1 \end{pmatrix} = \begin{pmatrix} k_x & 0 & x_L^w - k_x x_L \\ 0 & -k_y & y_H^w + k_y y_L \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}, \quad (5.8)$$

или

$$V^w = T V, \quad (5.9)$$

где

$$V^w = \begin{pmatrix} x^w \\ y^w \\ 1 \end{pmatrix}, \quad T = \begin{pmatrix} k_x & 0 & x_L^w - k_x x_L \\ 0 & -k_y & y_H^w + k_y y_L \\ 0 & 0 & 1 \end{pmatrix}, \quad V = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Глава 6. ФИЗИЧЕСКИЕ И ЛОГИЧЕСКИЕ КООРДИНАТЫ

6.1. Основные определения

Прежде всего, необходимо определить понятия «**физические координаты**» и «**логические координаты**».

Физические координаты, как это следует из названия, имеют непосредственное отношение к физическому устройству вывода. В качестве единицы измерения длины в системе физических координат всегда используется пиксел. Если устройством вывода является экран монитора, физические координаты обычно называют **экранными координатами**.

Логические координаты передаются функциям GDI, выполняющим рисование фигур или вывод текста. Используемые единицы измерения зависят от режима отображения.

При отображении GDI преобразует логические координаты в физические. Способ преобразования зависит от режима отображения и других атрибутов контекста отображения, таких как расположение начала системы координат для окна, расположение начала системы физических координат, масштаб осей для окна и масштаб осей физических координат.

6.2. Физическая система координат

На рис. 6.1 показана физическая система координат для экрана видеомонитора.

Начало этой системы координат располагается в левом верхнем углу экрана. Ось X направлена слева направо, ось Y – сверху вниз. В качестве единицы длины в данной системе координат используется пиксел.

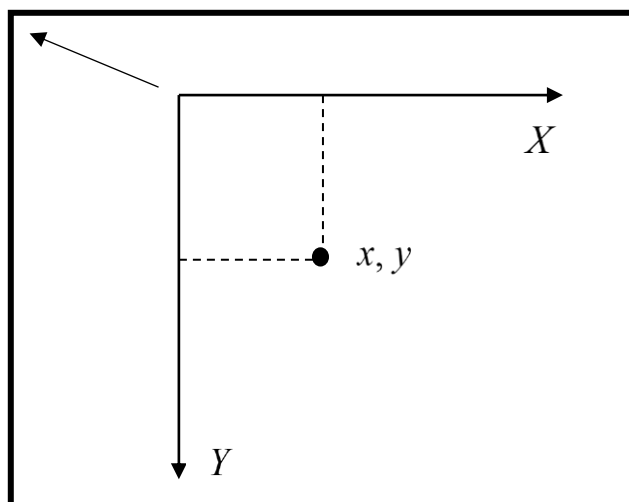


Рис. 6.1

6.3. Логическая система координат

Приложения Windows могут использовать одну из нескольких логических координат, устанавливая соответствующий режим отображения в контексте отображения. При этом можно использовать любое направление координатных осей и любое расположение начала координат. Например, возможна система координат, в которой задаются положительные и отрицательные координаты по любой оси (рис. 6.2).

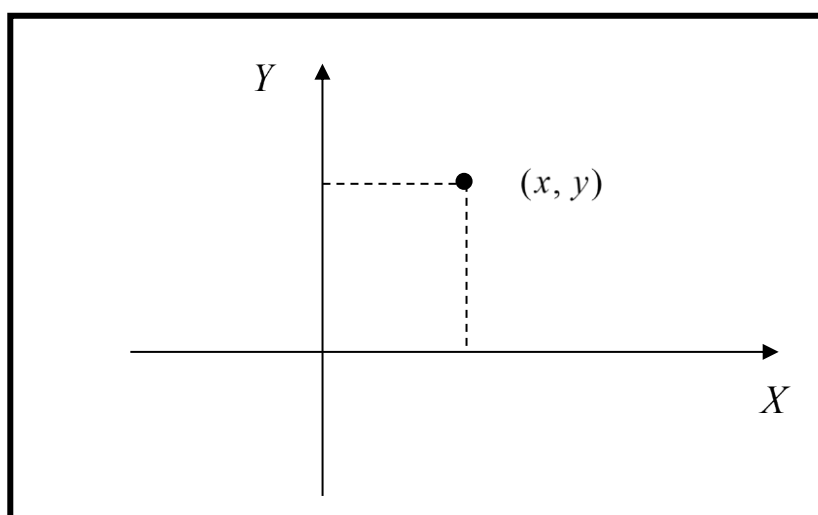


Рис.6.2

Для установки режима отображения, непосредственно определяющего

направление осей и размер логической единицы системы координат, используется функция **SetMapMode**:

```
int dc.SetMapMode(int nMapMode) // MFC
```

Параметр **nMapMode** может принимать одно из следующих значений, представленных в табл.6.1.

Таблица 6.1.

Значения параметра nMapMode

Режим отображения	Направление оси X	Направление оси Y	Размер одной логической единицы
MM_TEXT	Вправо	Вниз	1 пиксел
MM_LOMETRIC	Вправо	Вверх	0,1 мм
MM_HIMETRIC	Вправо	Вверх	0,01 мм
MM_LOENGLISH	Вправо	Вверх	0,01 дюйм
MM_HIENGLISH	Вправо	Вверх	0,001 дюйм
MM_TWIPS	Вправо	Вверх	1/1440 дюйма
MM_ISOTROPIC	Можно выбирать	Можно выбирать	Произвольный, одинаковый для осей X и Y
MM_ANISOTROPIC	Можно выбирать	Можно выбирать	Произвольный, может быть разный для осей X и Y

В любой момент времени приложение может определить номер режима отображения, выбранный в контекст отображения, используя функцию **GetMapMode**:

```
int dc.GetMapMode() // MFC
```

6.4. Преобразование координат

Приложение, вызывая для рисования функции GDI, указывает **логические** координаты. Перед выводом GDI преобразует их в **физические** с использованием следующих формул:

$$\begin{cases} xViewport = (xWindow - xWinOrg) \frac{xViewExt}{xWinExt} + xViewOrg \\ yViewport = (yWindow - yWinOrg) \frac{yViewExt}{yWinExt} + yViewOrg \end{cases}, \quad (6.1)$$

где

$xWindow$, $yWindow$ – логические координаты по оси X и Y соответственно,

$xViewport$, $yViewport$ – физические (экранные) координаты по оси X и Y соответственно,

$xWinOrg$, $yWinOrg$ – определяют расположение начала **логической** системы координат, по умолчанию $xWinOrg = 0$, $yWinOrg = 0$,

$xViewOrg$, $yViewOrg$ – определяют расположение начала **физической** системы координат, по умолчанию $xViewOrg = 0$, $yViewOrg = 0$,

$xViewExt$, $xWinExt$ – задают масштаб, который используется в процессе преобразования координат по оси x ,

$yViewExt$, $yWinExt$ – задают масштаб, который используется в процессе преобразования координат по оси y .

Эти масштабы зависят от установленного режима отображения. Приложения могут изменить его только в режимах MM_ISOTROPIC и MM_ANISOTROPIC, для остальных режимов отображения используются фиксированные значения.

Таким образом, логические координаты $(xWindow, yWindow)$ преобразуются в физические координаты $(xViewport, yViewport)$.

6.5. Режимы отображения

После рассмотрения физических и логических координат, а также их преобразований, займемся подробным описанием каждого режима отображения.

6.5.1. Режим MM_TEXT

Режим отображения MM_TEXT устанавливается в контексте отображения по умолчанию. Для этого режима формулы преобразования координат упрощаются:

$$\begin{cases} xViewport = xWindow - xWinOrg + xViewOrg \\ yViewport = yWindow - yWinOrg + yViewOrg \end{cases} \quad (6.2)$$

Так как по умолчанию $xViewOrg = 0$, $yViewOrg = 0$, $xWinOrg = 0$, $yWinOrg = 0$, то

$$\begin{cases} xViewport = xWindow \\ yViewport = yWindow \end{cases} \quad (6.3)$$

Соответствующая система координат представлена на рис. 6.3 (начало системы координат расположено точно в левом верхнем углу внутренней области окна, рисунок иллюстрирует только направление координатных осей).

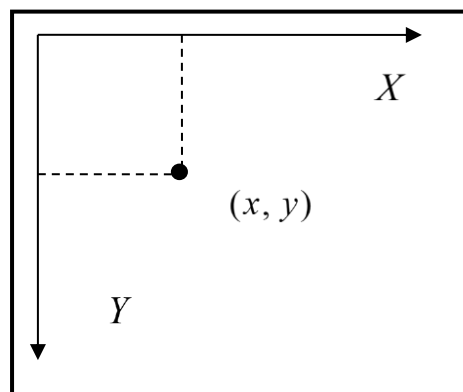


Рис. 6.3

Так как в формуле преобразования не присутствуют переменные $xViewExt$, $xWinExt$ и $yViewExt$, $yWinExt$, в данном режиме преобразования невозможно изменить масштаб осей координат. Поэтому логическая единица длины в режиме отображения `MM_TEXT` равна физической, т. е. одному пикселу.

Тем не менее, приложение может изменить смещение физической или логической системы координат, изменив, соответственно, значение пар переменных ($xViewOrg$, $yViewOrg$) и ($xWinOrg$, $yWinOrg$). Для установки смещения можно использовать функции **SetViewportOrg** и **SetWindowOrg**, которые являются методами класса `CDC` из библиотеки `MFC`:

```
virtual CPoint SetViewportOrg(int x, int y) или
CPoint SetViewportOrg(CPoint point),
virtual CPoint SetWindowOrg(int x, int y) или
CPoint SetWindowOrg(CPoint point),
```

где параметры x и y представляют собой новые значения соответствующих переменных.

6.5.2. Метрические режимы отображения

К метрическим режимам отображения относятся режимы `MM_LOMETRIC`, `MM_HIMETRIC`, `MM_LOENGLISH`, `MM_HIENGLISH` и `MM_TWIPS`. Эти режимы позволяют использовать привычные единицы измерения, такие как миллиметры и дюймы.

В метрических режимах отображения используются полные формулы преобразования координат, приведенные выше. В этих формулах приложение **может изменять** переменные, определяющие смещение начала физической или логической системы координат ($xViewOrg$, $yViewOrg$) и ($xWinOrg$, $yWinOrg$).

Приложение **не может изменить** значения переменных $xViewExt$, $xWinExt$ и $yViewExt$, $yWinExt$, от которых зависит масштаб по осям координат. Отношения

$xViewExt/xWinExt$ и $yViewExt/yWinExt$ имеют фиксированное значение для каждого из метрических режимов отображения.

Заметим, что для этих режимов отношение $yViewExt/yWinExt$ имеет **отрицательный знак**, в результате чего ось Y оказывается направленной **снизу вверх**.

Остановимся подробнее на описании режимов MM_ISOTROPIC и MM_ANISOTROPIC.

Режимы отображения MM_ISOTROPIC (изотропный) и MM_ANISOTROPIC (анизотропный) допускают изменение направления осей X и Y , а также изменение масштаба осей координат. В изотропном режиме отображения MM_ISOTROPIC масштаб вдоль осей X и Y всегда **одинаковый** (т. е. для обеих осей используются одинаковые логические единицы длины). Анизотропный режим MM_ANISOTROPIC предполагает использование разных масштабов для разных осей (хотя можно использовать и одинаковые масштабы).

Для изменения ориентации и масштаба осей можно воспользоваться функциями **SetViewportExt** и **SetWindowExt**, которые являются методами класса CDC из библиотеки MFC:

```
virtual CSize SetWindowExt(int cx, int cy),  
virtual CSize SetViewportExt(int cx, int cy).
```

Функция **SetWindowExt** устанавливает для формулы преобразования координат значения переменных $xWinExt$ и $yWinExt$.

Функция **SetViewportExt** устанавливает для формулы преобразования координат значения переменных $xViewExt$ и $yViewExt$. Функция **SetViewportExt** должна использоваться **после** функции **SetWindowExt**.

Изотропный режим отображения **удобно использовать** в тех случаях, когда надо сохранить установленное отношение масштабов осей X и Y при любом изменении размеров окна, в которое выводится изображение.

Анизотропный режим **удобен** в тех случаях, когда изображение должно занимать всю внутреннюю поверхность окна при любом изменении размеров окна. Соотношение масштабов при этом не сохраняется.

Сравним теперь выражения для пересчета координат из мировых в оконные (5.3), полученные нами ранее,

$$\begin{cases} x^w = x_L^w + \frac{\Delta x^w}{\Delta x}(x - x_L) \\ y^w = y_H^w - \frac{\Delta y^w}{\Delta y}(y - y_L) \end{cases}$$

и формулы (6.1), по которым пересчитывают координаты функции GDIWindows:

$$\begin{cases} xViewport = (xWindow - xWinOrg) \frac{xViewExt}{xWinExt} + xViewOrg \\ yViewport = (yWindow - yWinOrg) \frac{yViewExt}{yWinExt} + yViewOrg \end{cases}$$

Из сравнения следует, что если положить

$$\begin{aligned} xWinExt &= \Delta x, & xViewExt &= \Delta x^w, \\ yWinExt &= \Delta y, & yViewExt &= -\Delta y^w, \\ xWinOrg &= x_L, & xViewOrg &= x_L^w, \\ yWinOrg &= y_L, & yViewOrg &= y_H^w, \end{aligned}$$

то пересчет координат из мировых в оконные можно возложить на функции GDIWindows, установив предварительно режим отображения MM_ANISOTROPIC.

Замечание. При использовании режима MM_ANISOTROPIC следует иметь в виду особенности реализации в Win32 API объекта логического пера [6]. Дело в том, что при вызове функции создания пера, значение параметра, задающего его толщину, определяется отношениями $xViewExt/xWinExt$ и $yViewExt/yWinExt$. При таком определении, толщина пера может быть различной для рисования по вертикали и горизонтали. Исключением является нулевое значение толщины, определяющее перо толщиной в 1 пиксел.

Ниже приводится возможный вариант функции (при использовании библиотеки классов MFC) для установки режима отображения MM_ANISOTROPIC когда заданы области отображения в мировых и оконных координатах.

```
void Set MyMode(CDC&dc, CRect&RS, CRect&RW) //MFC
//dc - ссылка на класс CDCMFC
//RS - область в мировых координатах
//RW - область в оконных координатах
{
    int dsx=RS.right-RS.left;
    int dsy=RS.top-RS.bottom;
    int xsL=RS.left;
    int ysL=RS.bottom;

    int dwx=RW.right-RW.left;
    int dwy=RW.bottom-RW.top;
    int xwL=RW.left;
    int ywH=RW.bottom;

    dc.SetMapMode(MM_ANISOTROPIC);
    dc.SetWindowExt(dsx,dsy);
```

```

dc.SetViewportExt(dwX, -dwY);
dc.SetWindowOrg(xSL, ySL);
dc.SetViewportOrg(xwL, ywH);
}

```

Глава 7. ГЕОМЕТРИЧЕСКИЕ ОСНОВЫ КОМПЬЮТЕРНОЙ ГРАФИКИ

В настоящее время существует большое число прекрасных курсов высшей математики, которая и является тем фундаментом, на котором построены все методы и алгоритмы компьютерной графики. Необходимые разделы математики включаются также и в соответствующие учебные пособия по компьютерной графике [3].

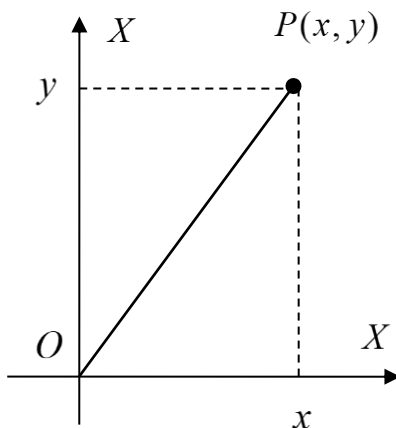
Тем не менее, авторы считают необходимым привести в качестве справочного материала некоторые сведения из курса аналитической геометрии и математического анализа, которые имеют отношение к тематике учебного пособия.

7.1. Системы координат и векторы

7.1.1 Декартова система координат

Декартовой прямоугольной системой координат на плоскости (в пространстве) называют две (три) взаимно перпендикулярные оси с общим началом (рис. 7.1.1, рис. 7.1.2). Первая ось OX называется осью абсцисс, вторая ось OY – осью ординат (третья ось OZ – осью аппликат).

Каждой точке плоскости (пространства) P ставится в соответствие упорядоченная пара (x, y) [тройка (x, y, z)] действительных чисел – координат данной точки (рис. 7.1.1, рис. 7.1.2).



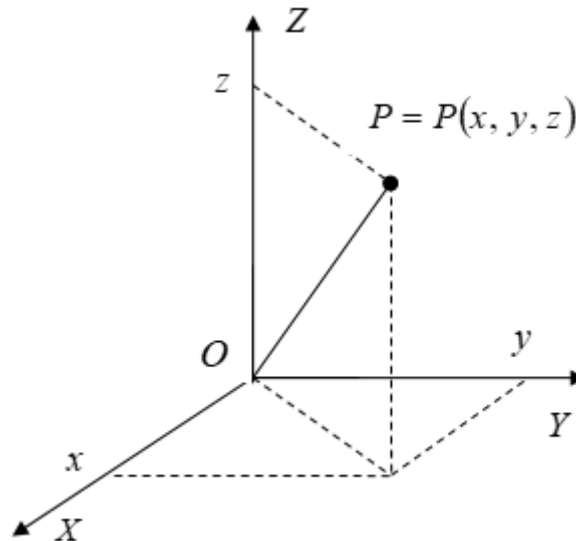


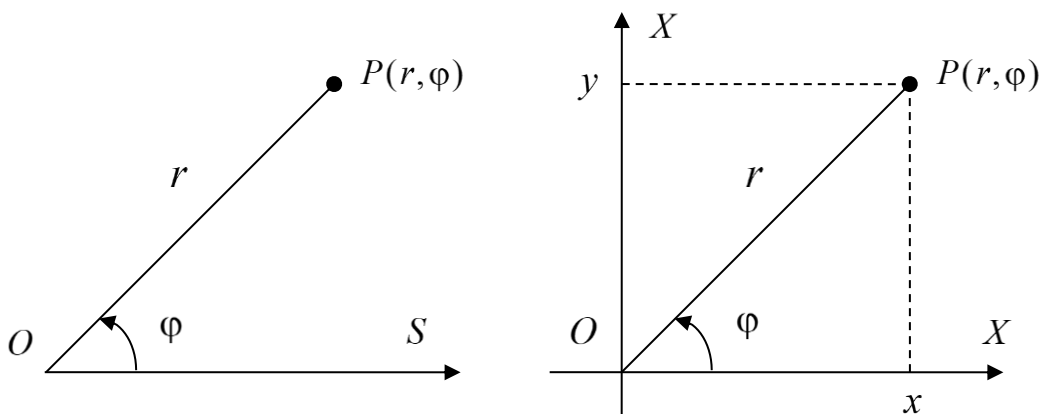
Рис. 7.1.1

Рис. 7.1.2

7.1.2 Полярная система координат

Полярная система координат — двухмерная система координат, в которой каждая точка на плоскости P определяется двумя числами — полярным углом (азимутом) φ и полярным радиусом r (рис. 7.1.3).

Ось OS , относительно которой отсчитывается угол φ , называется полярной осью. Угол φ считается положительным ($\varphi > 0$), если он отсчитывается против часовой стрелки и отрицательным ($\varphi < 0$) если против. Пределы изменения полярной координаты r составляют $r \in [0, \infty)$. Для координаты φ эти пределы выбираются как $\varphi \in (-\pi, \pi]$ или $\varphi \in [0, 2\pi)$.



Ниже приводятся соотношения, связывающие координаты точки P , в декартовой $P = P(x, y)$ и полярной $P = P(r, \varphi)$ системах координат (рис. 7.1.4).

$$\begin{cases} x = r \cos \varphi \\ y = r \sin \varphi \end{cases} \quad (7.1.1)$$

7.1.3 Сферическая система координат

Сферическая система координат – трёхмерная система координат (рис. 7.1.5), в которой каждая точка пространства P определяется тремя числами (r, φ, θ) , где r – расстояние от начала координат до точки P ($0 \leq r < \infty$), φ – угол, образованный положительным направлением оси OX и проекцией точки P на плоскость XOY (отсчитывается от оси OX против часовой стрелки, $(0 \leq \varphi < 2\pi)$), θ – угол, образованный положительным направлением оси OZ и отрезком OP (отсчитывается от оси OZ по часовой стрелке, $(0 \leq \theta \leq \pi)$).

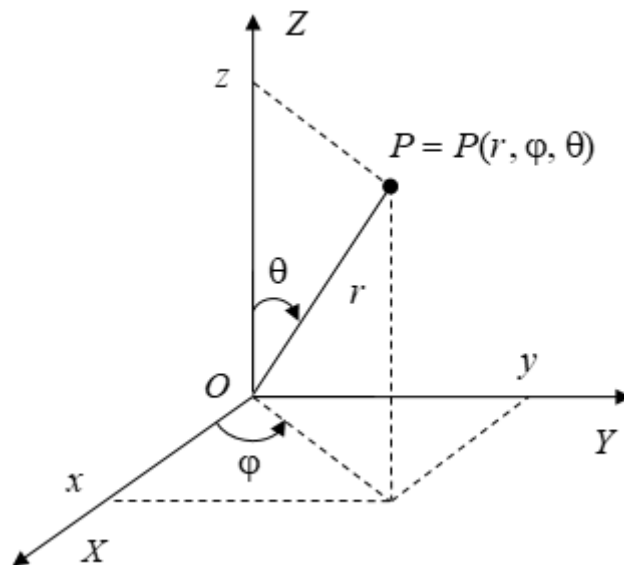


Рис. 7.1.5

Ниже приводятся соотношения, связывающие координаты точки P , в декартовой $P = P(x, y, z)$ и полярной $P = P(r, \varphi, \theta)$ системах координат (рис. 7.1.5).

$$\begin{cases} x = r \sin \theta \cos \varphi \\ y = r \sin \theta \sin \varphi \\ z = r \cos \theta \end{cases} \quad (7.1.2)$$

7.1.4 Векторы

Вектором называется направленный отрезок AB с начальной точкой A и конечной точкой B , который обозначается как $\overrightarrow{AB}$ или одной буквой, например $\vec{p}$ (рис. 7.1.6). *Длиной* или *нормой* вектора называется длина отрезка AB , которая обозначается как $|\overrightarrow{AB}|$ ($|\vec{p}|$) или $AB(p)$. Вектор, длина которого равна единице ($|\overrightarrow{AB}| = 1$ ($|\vec{p}| = 1$)) называется *единичным* вектором или *ортом*. Вектор, длина которого равна нулю ($|\overrightarrow{AB}| = 0$ ($|\vec{p}| = 0$)) называется *нулевым* или *нуль – вектором*. Для нуль-вектора понятие направления теряет смысл, так как начало и конец у него совпадают. Однако такому вектору удобно приписать произвольное направление, которое устанавливают в зависимости от конкретной ситуации.

Два ненулевых вектора $\vec{p}$ и $\vec{q}$ называются *коллинеарными* ($\vec{p} \parallel \vec{q}$), если существует некоторое число λ такое, что $\vec{p} = \lambda \vec{q}$. Коллинеарные вектора лежат на параллельных прямых, либо на одной прямой.

Векторы называются *компланарными*, если существует плоскость, которой они параллельны.

Два вектора $\vec{p}$ и $\vec{q}$ считаются равными ($\vec{p} = \vec{q}$), если они имеют одинаковое направление и длину ($|\vec{p}| = |\vec{q}|$) (рис. 7.1.7).



Рис. 7.1.6

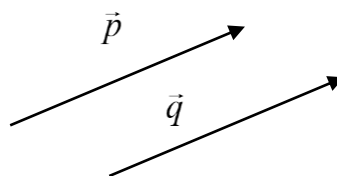


Рис. 7.1.7

Вектор, расположенный на прямой, параллельной данной прямой L (данной плоскости P), называется вектором, параллельным прямой L (плоскости P).

С введением понятия вектора любой точке на плоскости, заданной в декартовых $P = P(x, y)$ или полярных $P = P(r, \varphi)$ координатах можно сопоставить соответствующий радиус – вектор с координатами

$\vec{r} = \vec{r}(x, y)$ или $\vec{r} = \vec{r}(r, \varphi)$, где $r = |\vec{r}|$, соответственно (рис. . 7.1.8_1, рис.

7.1.8_1).

Аналогичным образом любой точке в пространстве, заданной в декартовых $P = P(x, y, z)$ или сферических $P = P(r, \varphi, \theta)$ координатах можно сопоставить соответствующий радиус – вектор с координатами $\vec{r} = \vec{r}(x, y, z)$ или $\vec{r} = \vec{r}(r, \varphi, \theta)$, где $r = |\vec{r}|$, соответственно (рис. 7.1.8_3, рис. 7.1.8_4).

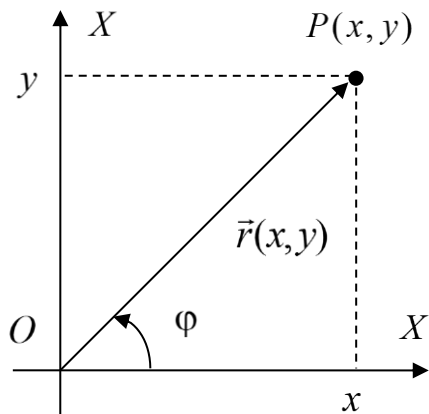


Рис. 7.1.8.1

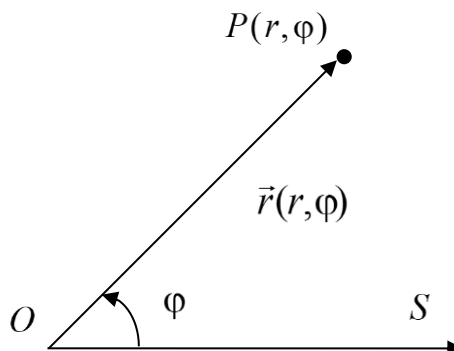


Рис. 7.1.8.2

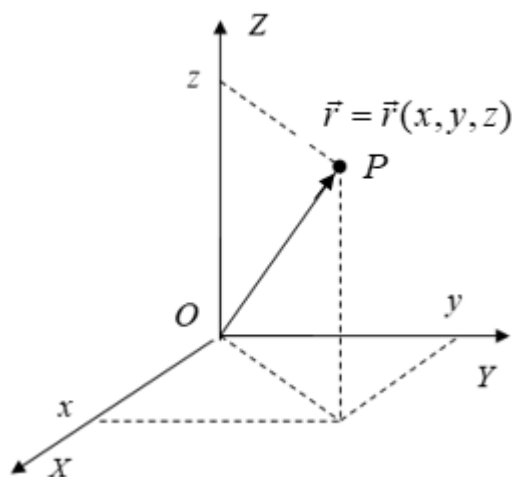


Рис. 7.1.8.3

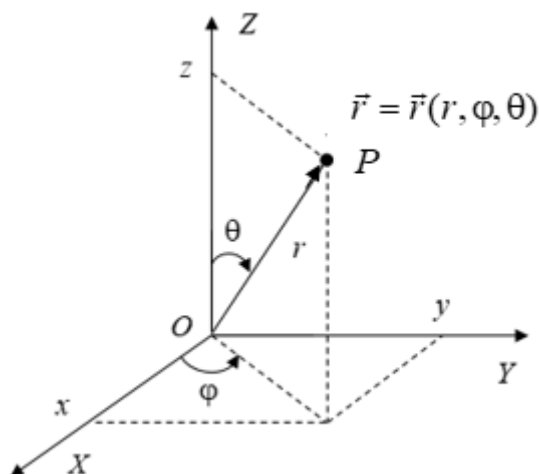


Рис. 7.1.8.4

Рассмотрим различные способы представления векторов на плоскости (в декартовой системе координат XOY) и в пространстве (в декартовой системе координат XYZ).

В прямоугольной системе координат XYZ направление осей задается тройкой взаимно перпендикулярных единичных векторов $\vec{i}, \vec{j}, \vec{k}$. При этом, система координат называется *правой*, если при повороте от вектора $\vec{i}$ к вектору $\vec{j}$ на $\pi/2$ направление вектора $\vec{k}$ совпадает с поступательным движением винта с правой резьбой (рис. 7.1.8). Если же направление вектора $\vec{k}$ имеет противоположное направление, то такая система координат называется *левой*.

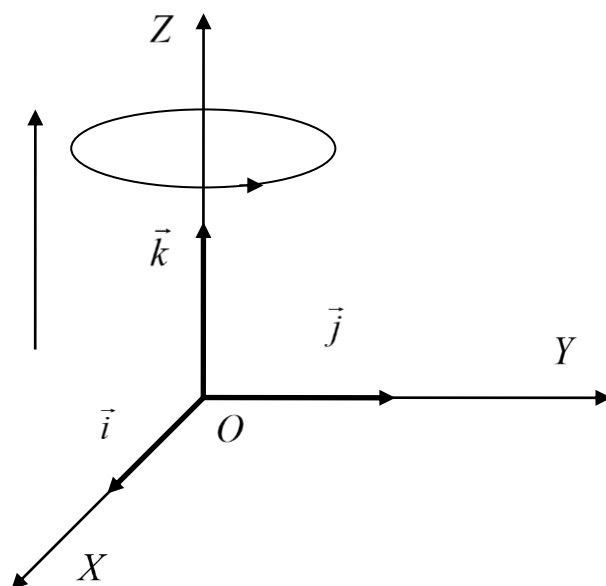
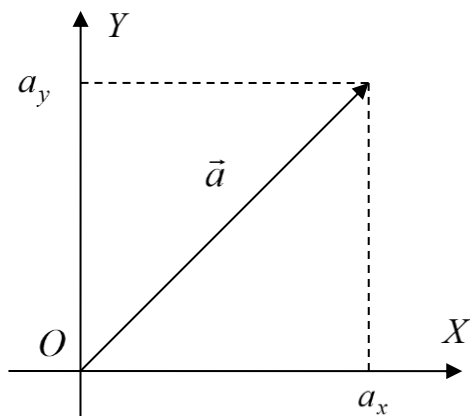
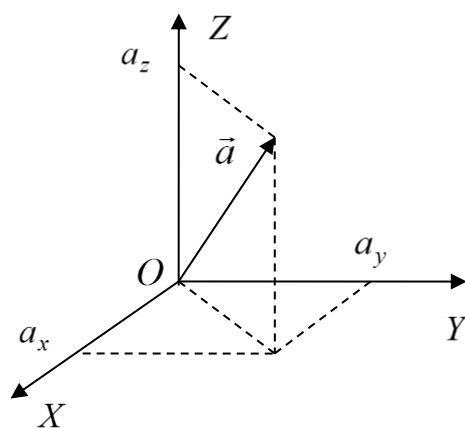


Рис. 7.1.8

Пусть $\vec{a}$ некоторый вектор, заданный в декартовой системе координат XOY на плоскости (рис.7.1.9, а) и декартовой системе координат XYZ в пространстве (рис. 7.1.9, б).



а)



б)

Рис. 7.1.9

Тогда

$$\vec{a} = a_x \vec{i} + a_y \vec{j} = \vec{a}(a_x, a_y) \quad (7.1)$$

$$\vec{a} = a_x \vec{i} + a_y \vec{j} + a_z \vec{k} = \vec{a}(a_x, a_y, a_z) \quad (7.2)$$

Другая форма представления векторов как пары (тройки) координат.

$$a = \begin{pmatrix} a_x \\ a_y \end{pmatrix} = (a_x, a_y)^T \quad (7.3)$$

$$a = \begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} = (a_x, a_y, a_z)^T \quad (7.4)$$

Модуль (длина) вектора

$$|a| = \sqrt{a_x^2 + a_y^2} = \sqrt{a^T a} \quad (7.5)$$

$$|a| = \sqrt{a_x^2 + a_y^2 + a_z^2} = \sqrt{a^T a} \quad (7.6)$$

Любой вектор $\vec{a}$, ($|\vec{a}| \neq 0$) можно представить в виде

$$\vec{a} = |\vec{a}| \frac{\vec{a}}{|\vec{a}|} = |\vec{a}| \vec{e}_a, \quad (7.6.1)$$

где $\vec{e}_a = \vec{a}/|\vec{a}|$ – единичный вектор вдоль направления вектора $\vec{a}$.

Пусть заданы два вектора $\vec{a}$ и $\vec{b}$ в форме (7.4)

$$a = (a_x, a_y, a_z)^T \text{ и } b = (b_x, b_y, b_z)^T,$$

Тогда

$$c = a + b = (a_x + b_x, a_y + b_y, a_z + b_z)^T \quad (7.7)$$

$$p = \lambda a = (\lambda a_x, \lambda a_y, \lambda a_z)^T \quad (7.8)$$

При этом справедливы равенства

$$\begin{aligned} \vec{a} + \vec{b} &= \vec{b} + \vec{a} \\ (\vec{a} + \vec{b}) + \vec{c} &= \vec{a} + (\vec{b} + \vec{c}) \\ \vec{a} + 0 &= \vec{a} \\ \vec{a} + (-\vec{a}) &= 0 \\ \lambda(\vec{a} + \vec{b}) &= \lambda\vec{a} + \lambda\vec{b} \\ 1\vec{a} &= \vec{a} \\ 0\vec{a} &= 0 \end{aligned} \quad (7.9)$$

7.1.5 Скалярное произведение векторов

Скалярное произведение векторов $\vec{a}$ и $\vec{b}$ определяется выражением

$$\vec{a} \cdot \vec{b} = (\vec{a}, \vec{b}) = |\vec{a}| |\vec{b}| \cos \varphi, \quad (7.10)$$

где φ - угол между векторами a и b , рис. 7.2.1

В координатной форме выражение (7.10) имеет вид

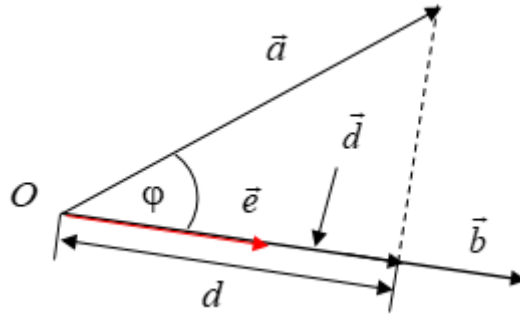


Рис. 7.2.1

$$\vec{a} \cdot \vec{b} = (a_x \vec{i} + a_y \vec{j} + a_z \vec{k})(b_x \vec{i} + b_y \vec{j} + b_z \vec{k}) = a_x b_x + a_y b_y + a_z b_z \quad (7.11)$$

или

$$\vec{a} \cdot \vec{b} = a^T b, \quad (7.12)$$

если вектора $\vec{a}$ и $\vec{b}$ представлены в форме (7.4).

Из (7.10) получаем

$$\cos \varphi = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| |\vec{b}|} \quad (7.13)$$

Как видно из (7.13), если $\cos \varphi > 0$, что эквивалентно $\vec{a} \cdot \vec{b} > 0$ (так как $|\vec{a}| |\vec{b}| > 0$), то угол между векторами $\vec{a}$ и $\vec{b}$ является острым ($|\varphi| < \pi/2$), при $\cos \varphi = 0$ ($\vec{a} \cdot \vec{b} = 0$) вектора $\vec{a}$ и $\vec{b}$ ортогональны ($|\varphi| = \pi/2$), а если $\cos \varphi < 0$ ($\vec{a} \cdot \vec{b} < 0$), то угол между векторами является тупым ($|\varphi| > \pi/2$).

Разделим обе части равенства (7.10) на $|\vec{b}|$. В результате получим

$$\vec{a} \cdot \frac{\vec{b}}{|\vec{b}|} = \vec{a} \cdot \vec{e}_b = |\vec{a}| |\vec{e}_b| \cos \varphi = d = |\vec{d}|, \quad (7.13.1)$$

где (рис. 7.2.1) $\vec{e}_b = \frac{\vec{b}}{|\vec{b}|}$, $|\vec{e}_b| = 1$ – единичный вектор, построенный на векторе $\vec{b}$,

$$d = \vec{a} \cdot \vec{e}_b = |\vec{a}| \cos \varphi – \quad (7.13.2)$$

проекция модуля вектора $\vec{a}$ на вектор $\vec{b}$,

$$\vec{d} = d \vec{e}_b = (\vec{a}, \vec{e}_b) \vec{e}_b – \quad (7.13.3)$$

проекция вектора $\vec{a}$ на направление, определяемое единичным вектором $\vec{e}_b$.

Отметим некоторые свойства скалярного произведения. Пусть p, q, c – некоторые числа. Тогда справедливы равенства:

$$p(q\vec{a} \cdot \vec{b}) = pq(\vec{a} \cdot \vec{b}) \quad (7.14.1)$$

$$(p\vec{a} + q\vec{b}) \cdot c = p\vec{a} \cdot c + q\vec{b} \cdot c \quad (7.14.2)$$

$$\vec{a} \cdot \vec{b} = \vec{b} \cdot \vec{a} \quad (7.14.3)$$

$$\vec{a} \cdot \vec{a} = 0, \quad \text{при } \vec{a} = 0 \quad (7.14.4)$$

Свойство (7.14.3) говорит о том, что скалярное произведение векторов коммутативно.

7.1.6 Векторное произведение векторов

Векторное произведение векторов $\vec{a}$ и $\vec{b}$ определяется выражением

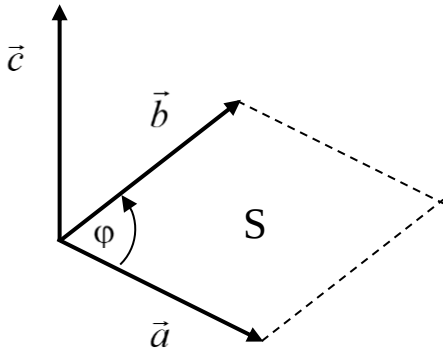
$$\vec{c} = \vec{a} \times \vec{b} = [\vec{a}, \vec{b}], \quad (7.15)$$

где φ – угол между векторами $\vec{a}$ и $\vec{b}$. Направление вектора $\vec{c}$ перпендикулярно векторам $\vec{a}$ и $\vec{b}$ и совпадает с направлением поступательного движения винта с правой резьбой при повороте вектора $\vec{a}$ к вектору $\vec{b}$ кратчайшим путем (рис. 7.2.2, рис. 7.2.3).

Отметим некоторые свойства векторного произведения.

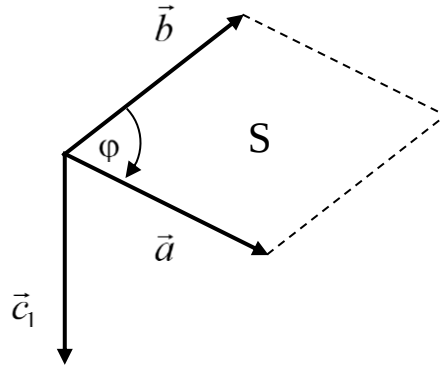
Длина вектора $\vec{c}$ равна

$$|\vec{c}| = |\vec{a}||\vec{b}|\sin\varphi \quad (7.16)$$



$$\vec{c} = \vec{a} \times \vec{b}$$

Рис. 7.2.2



$$\vec{c}_1 = \vec{b} \times \vec{a} = -(\vec{a} \times \vec{b}) = -\vec{c}$$

Рис. 7.2.3

Пусть S – площадь параллелограмма, построенного на векторах $\vec{a}$ и $\vec{b}$, (рис. 7.2.2, рис. 7.2.3). Тогда из (7.15) и (7.16) получаем

$$S = |\vec{c}| = |\vec{a}||\vec{b}|\sin\varphi = |\vec{a} \times \vec{b}| = |\vec{b} \times \vec{a}| \quad (7.16.1)$$

Если вектора $\vec{a}$ и $\vec{b}$ коллинеарны, т.е. $\vec{b} = \lambda\vec{a}$, где λ скаляр, то (см. ниже)

$$\vec{c} = \vec{a} \times \vec{b} = \lambda(\vec{a} \times \vec{a}) = 0, \quad (7.17)$$

так как $\vec{a} \times \vec{a} = 0$.

Пусть p – некоторая константа, тогда справедливы следующие свойства векторного произведения векторов.

$$(p\vec{a}) \times \vec{b} = p(\vec{a} \times \vec{b}) \quad (7.18.1)$$

$$\vec{a} \times (\vec{b} + \vec{c}) = \vec{a} \times \vec{b} + \vec{a} \times \vec{c} \quad (7.18.2)$$

$$\vec{a} \times \vec{b} = -(\vec{b} \times \vec{a}) \quad (7.18.3)$$

Свойство (7.18.3) говорит о том, что *векторное* произведение векторов не коммутативно (антикоммутативно).

Рассмотрим еще одну форму записи векторного произведения

$$\begin{aligned}
\vec{c} &= \vec{a} \times \vec{b} = (a_x \vec{i} + a_y \vec{j} + a_z \vec{k}) \times (b_x \vec{i} + b_y \vec{j} + b_z \vec{k}) = \\
&= (a_y b_z - a_z b_y) \vec{i} - (a_x b_z - a_z b_x) \vec{j} + (a_x b_y - a_y b_x) \vec{k} = \\
&= \begin{vmatrix} a_y & a_z \\ b_y & b_z \end{vmatrix} \vec{i} - \begin{vmatrix} a_x & a_z \\ b_x & b_z \end{vmatrix} \vec{j} + \begin{vmatrix} a_x & a_y \\ b_x & b_y \end{vmatrix} \vec{k} = \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix}
\end{aligned} \tag{7.19}$$

7.1.7 Смешанное произведение векторов

Смешанным произведением трех векторов $\vec{a}, \vec{b}, \vec{c}$ называют число

$$V = (\vec{a} \times \vec{b}) \cdot \vec{c} = ([\vec{a}, \vec{b}], \vec{c}) = (\vec{a}, \vec{b}, \vec{c}) \tag{7.19.1}$$

равное скалярному произведению векторного произведения $\vec{a} \times \vec{b} = [\vec{a}, \vec{b}]$ первых двух векторов и третьего вектора $\vec{c}$.

Пусть

$$\vec{p} = \vec{a} \times \vec{b} = |\vec{a} \times \vec{b}| \vec{e}_p = S \vec{e}_p, \tag{7.19.2}$$

где $S = |\vec{a} \times \vec{b}|$ – площадь параллелограмма, построенного на векторах $\vec{a}$ и $\vec{b}$ и $\vec{e}_p = \vec{p}/|\vec{p}|$ – единичный вектор (рис. 7.2.3)

Тогда

$$V = S(\vec{c}, \vec{e}_p) = S|\vec{c}||\vec{e}_p| \cos \theta = Sh, \tag{7.19.3}$$

где θ – угол между векторами $\vec{p}$ и $\vec{c}$, $h = |\vec{c}| \cos \theta$ – высота параллелепипеда, построенного на векторах $\vec{a}, \vec{b}$ и $\vec{c}$ (рис. 7.2.3).

Из (7.19.3) следует что при $\theta < \pi/2$ смешанное произведение трех векторов, не лежащих в одной плоскости (*некомпланарных*), равно объему параллелепипеда V , построенного на этих векторах как на ребрах, выходящих из одной вершины (рис. 7.2.3). При $\theta > \pi/2$ смешанное произведение имеет значение $-V$.

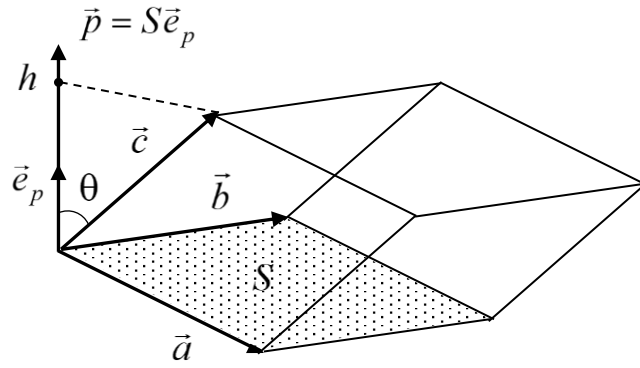


Рис. 7.2.3

Если вектора $\vec{a}$, $\vec{b}$ и $\vec{c}$ лежат в одной плоскости (являются *компланарными*), то угол φ между векторами $\vec{p}$ и $\vec{c}$ (рис. 7.2.3) равен $\pi/2$ и

$$(\vec{a} \times \vec{b}) \cdot \vec{c} = 0. \quad (7.19.4)$$

Условие (7.19.4) называется *условием компланарности* векторов $\vec{a}$, $\vec{b}$ и $\vec{c}$.

Если вектора $\vec{a}$, $\vec{b}$ и $\vec{c}$ представлены в координатной форме

$\vec{a} = \vec{a}(a_x, a_y, a_z)$, $\vec{b} = \vec{b}(b_x, b_y, b_z)$ и $\vec{c} = \vec{c}(c_x, c_y, c_z)$, то, учитывая (7.19), их смешанное произведение можно представить в виде

$$\begin{aligned} (\vec{a} \times \vec{b}) \cdot \vec{c} &= (a_y b_z - a_z b_y) c_x - (a_x b_z - a_z b_x) c_y + (a_x b_y - a_y b_x) c_z = \\ &= \begin{vmatrix} a_y & a_z \\ b_y & b_z \end{vmatrix} c_x - \begin{vmatrix} a_x & a_z \\ b_x & b_z \end{vmatrix} c_y + \begin{vmatrix} a_x & a_y \\ b_x & b_y \end{vmatrix} c_z = \begin{vmatrix} a_x & a_y & a_z \\ b_x & b_y & b_z \\ c_x & c_y & c_z \end{vmatrix} \end{aligned} \quad (7.19.5)$$

Также справедливо свойство

$$(\vec{a} \times \vec{b}) \cdot \vec{c} = \vec{a} \cdot (\vec{b} \times \vec{c}). \quad (7.19.6)$$

В силу коммутативности скалярного произведения и антикоммутативности векторного произведения из свойства 7.19.6) получаем цепочку равенств

$$\begin{aligned} (\vec{a} \times \vec{b}) \cdot \vec{c} &= (\vec{c} \times \vec{a}) \cdot \vec{b} = (\vec{b} \times \vec{c}) \cdot \vec{a} = -(\vec{b} \times \vec{a}) \cdot \vec{c} = -(\vec{c} \times \vec{b}) \cdot \vec{a} = \\ &= -(\vec{a} \times \vec{c}) \cdot \vec{b} \end{aligned} \quad (7.19.7)$$

7.2 Графические элементы на плоскости

7.2.1 Модели прямой линии на плоскости

7.2.1.1 Нормальное уравнение прямой на плоскости

Нормальное уравнение прямой – это уравнение прямой, проходящей через заданную точку и имеющей заданный вектор нормали.

Пусть в декартовой системе координат XOY на плоскости (рис. 7.2.4) задана прямая L , точка P_0 , имеющая радиус – вектор $\vec{r}_0 = \vec{r}_0(x_0, y_0)$ и вектор нормали $\vec{n} = \vec{n}(a, b)$ к прямой L . Далее, пусть P – текущая точка прямой L , имеющая радиус – вектор $\vec{r} = \vec{r}(x, y)$. Тогда из рис. 7.2.4 получаем значение $\Delta\vec{r} = \vec{r} - \vec{r}_0$. В силу ортогональности вектора $\vec{n}$ и прямой L скалярное произведение векторов $\vec{n}$ и $\Delta\vec{r}$ равно нулю.

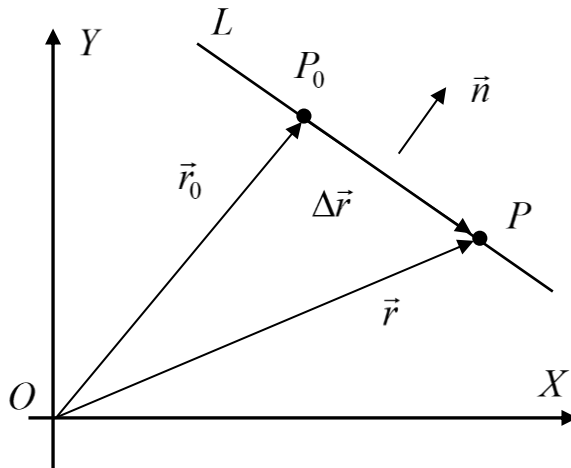


Рис. 7.2.4

$$\Delta\vec{r} \cdot \vec{n} = (\vec{r} - \vec{r}_0) \cdot \vec{n} = 0 \quad (7.19.8)$$

Уравнение (7.19.8) представляет собой *векторное* нормальное уравнение прямой

Переписывая (7.19.8) в координатной форме, получаем

$$a(x - x_0) + b(y - y_0) = 0 \quad (7.19.9)$$

Уравнения (7.19.9) представляет собой *нормальное* уравнение прямой,

проходящей через заданную точку $P_0 = P_0(x_0, y_0)$ и имеющей заданный вектор нормали $\vec{n} = \vec{n}(a, b)$, записанное в координатной форме.

7.2.1.2 Уравнение прямой на плоскости в неявном виде

Раскрывая скобки в (7.19.9), получим

$$ax + by - (ax_0 + by_0) = ax + by + c = 0, \quad (7.19.10)$$

где

$$c = -(ax_0 + by_0). \quad (7.19.11)$$

Уравнение (7.19.10) можно переписать в виде

$$\vec{r} \cdot \vec{n} + c = 0, \quad (7.19.12)$$

Уравнения (7.19.10) представляет собой уравнения прямой в *неявном* виде, записанное в координатной форме, а (7.19.12) есть *векторное* уравнения прямой в *неявном* виде.

7.2.1.3 Уравнение прямой на плоскости в явном виде

Уравнение прямой в явном виде (уравнение прямой с угловым коэффициентом) получим из (7.19.10). Пусть $b \neq 0$, тогда

$$y = -\frac{a}{b}x - \frac{c}{b} = kx + p, \quad (7.19.13)$$

где $k = -a/b = \operatorname{tg} \varphi$, φ – угол наклона прямой по отношению к оси OX (отсчитывается от оси OX против часовой стрелки), $p = -c/b$.

Если прямая (7.19.13) проходит через точку $A(x_0, y_0)$, то

$$y_0 = kx_0 + p, \quad (7.19.14)$$

Тогда вычитая (7.19.14) из (7.19.13) и получаем

$$y = y_0 + k(x - x_0), \quad (7.19.15)$$

7.2.1.4 Уравнение прямой на плоскости в отрезках

Перепишем уравнение (7.19.10) в виде

$$\frac{x}{-(c/a)} + \frac{y}{-(c/b)} = 1, \quad a \neq 0, b \neq 0, c \neq 0. \quad (7.19.16)$$

Вводя обозначения $-(c/a) = p$ и $-(c/b) = q$, получим

$$\frac{x}{p} + \frac{y}{q} = 1 \quad (7.19.17)$$

уравнение прямой в отрезках, где p и q – отрезки, которые прямая отсекает на координатных осях OX и OY соответственно.

7.2.1.5 Параметрическое уравнение прямой на плоскости

Пусть в декартовой системе координат XOY на плоскости (рис. 7.2.5) задана прямая L и вектор $\vec{q}$, параллельный этой прямой.

Определение. Любой ненулевой вектор, параллельный прямой L , называется её *направляющим вектором*.

Пусть на прямой L задана точка P_0 , имеющая радиус – вектор $\vec{r}_0 = \vec{r}_0(x_0, y_0)$, текущая точка P прямой L , имеющая радиус – вектор $\vec{r} = \vec{r}(x, y)$ и направляющий вектор этой прямой $\vec{q} = \vec{q}(m, n)$.

Так как направляющий вектор $\vec{q}$ параллелен прямой L , то он коллинеарен вектору $\Delta\vec{r} = \vec{r} - \vec{r}_0$ ($\Delta\vec{r} \parallel \vec{q}$). Последнее означает, что

$$\vec{r} - \vec{r}_0 = t\vec{q}, \quad (7.19.18)$$

или

$$\vec{r} = \vec{r}_0 + t\vec{q}, \quad (7.19.19)$$

где t – некоторый параметр.

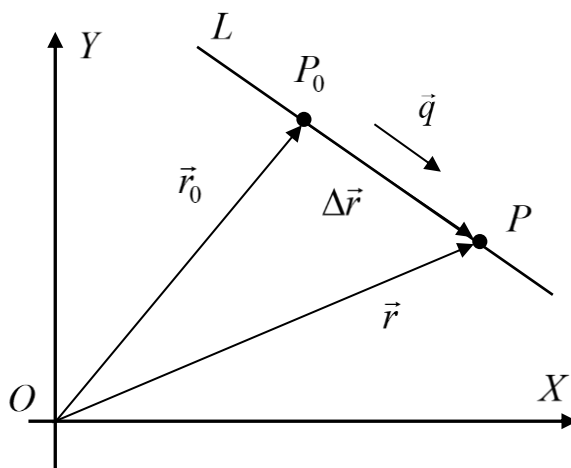


Рис. 7.2.5

Уравнение (7.19.19) есть *параметрическое уравнение* прямой L в векторном виде.

Каждой точке прямой L соответствует некоторое значение *параметра* t и наоборот, каждому значению $t \in (-\infty, \infty)$ соответствует некоторая точка прямой L , причём различным значениям параметра t соответствуют различные точки прямой L .

Если $t \in (a, \infty)$, получим полупрямую; если $t \in [a, b]$ – отрезок прямой L .

Параметрическому уравнению прямой можно дать механическую интерпретацию. Если t представляет собой время, то данное уравнение описывает закон движения материальной точки с постоянной скоростью $\vec{q}$. Такая точка будет двигаться прямолинейно.

Запишем теперь *параметрические уравнения* прямой на плоскости в координатной форме:

$$\begin{cases} x = x_0 + mt \\ y = y_0 + nt \end{cases}, \quad (7.19.20)$$

7.2.1.6 Каноническое уравнение прямой на плоскости

Исключив из параметрических уравнений (7.19.20) параметр t , получим

$$\frac{x - x_0}{m} = \frac{y - y_0}{n}, \quad (7.19.21)$$

Это каноническое уравнение прямой L . В знаменателях допускаются нули; в этом случае соотношение нужно перемножить «крест-накрест», как пропорцию.

7.2.1.7 Уравнение прямой на плоскости, проходящей через две заданные точки

Пусть в декартовой системе координат XOY на плоскости (рис. 7.2.6) заданы две точки: точка P_1 , имеющая радиус – вектор $\vec{r}_1 = \vec{r}_1(x_1, y_1)$ и точка P_2 , имеющая радиус – вектор $\vec{r}_2 = \vec{r}_2(x_2, y_2)$. Пусть P – текущая точка прямой L , проходящей через точки P_1 и P_2 и имеющая радиус – вектор $\vec{r} = \vec{r}(x, y)$.

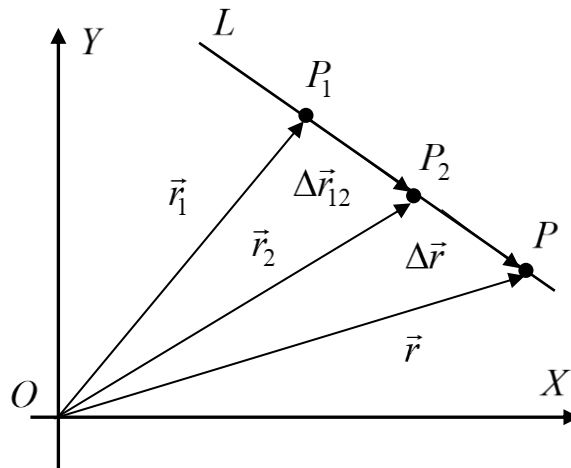


Рис. 7.2.6

Как видно из рисунка вектор $\overrightarrow{P_1P_2} = \vec{r}_2 - \vec{r}_1 = \Delta\vec{r}_{12}(x_2 - x_1, y_2 - y_1)$ и вектор $\overrightarrow{P_1P} = \vec{r} - \vec{r}_1 = \Delta\vec{r}(x - x_1, y - y_1)$ лежат на одной прямой и, следовательно, коллинеарны ($\Delta\vec{r}_{12} \parallel \Delta\vec{r}$). Выбирая вектор $\Delta\vec{r}_{12}$ в качестве направляющего для прямой L , получим (7.19.19)

$$\vec{r} = \vec{r}_1 + t\Delta\vec{r}_{12} = \vec{r}_1 + t(\vec{r}_2 - \vec{r}_1), \quad (7.19.22)$$

где t – некоторый параметр.

Уравнение (7.19.22) есть *уравнение прямой проходящей через две точки в векторной форме*.

Переписывая (7.19.22) в координатной форме, получаем

$$\begin{cases} x = x_1 + t(x_2 - x_1) \\ y = y_1 + t(y_2 - y_1) \end{cases} \quad (7.19.23)$$

Уравнения 7.19.23 можно представить в матричной форме

$$P = P_1 + t(P_2 - P_1), \quad (7.19.24)$$

где $P = (x, y)^T$, $P_1 = (x_1, y_1)^T$, $P_2 = (x_2, y_2)^T$

Исключив из (7.19.23) параметр t , получим уравнение прямой проходящей через две точки в канонической форме.

$$\frac{x - x_1}{x_2 - x_1} = \frac{y - y_1}{y_2 - y_1}. \quad (7.19.25)$$

Перепишем теперь (7.19.23) в виде

$$\begin{cases} x = x_1(1 - t) + tx_2 \\ y = y_1(1 - t) + ty_2 \end{cases} \quad (7.19.26)$$

или в матричной форме

$$P = (1 - t)P_1 + tP_2, \quad (7.19.27)$$

Из (7.19.26) следует, что при $t = 0$ получаем точку с координатами (x_1, y_1) , а при $t = 1$ точку с координатами (x_2, y_2) . Таким образом, выражения (7.19.26) и (7.19.27) при $t \in [0; 1]$ можно рассматривать как *уравнение отрезка прямой* соединяющего точки с номерами 1 и 2 (например, отрезок P_1P_2 на рис. 7.2.6).

7.2.2 Взаимное расположение графических элементов на плоскости

7.2.2.1 Расстояние от точки до прямой на плоскости

Пусть на плоскости заданы прямая L и точка $P_0 = P_0(x_0, y_0)$ (рис. 7.2.7). Необходимо найти расстояние $\rho(P_0, L)$ от точки P_0 до прямой L .

Для описания прямой воспользуемся уравнением прямой в неявном виде.

$$ax + by + c = 0, \quad (7.19.28)$$

где $\vec{n} = \vec{n}(a, b)$ – вектор нормали, а

$$\vec{e}_n = \vec{e}_n \left(\frac{a}{\sqrt{a^2 + b^2}}, \frac{b}{\sqrt{a^2 + b^2}} \right) - \quad (7.19.29)$$

единичный вектор нормали к прямой L .

Пусть $P = P(x, y)$ – текущая точка прямой L , имеющая радиус – вектор

$$\vec{r}_P = \vec{r}_P(x - x_0, y - y_0), \quad (7.19.30)$$

проведенный из точки P_0 .

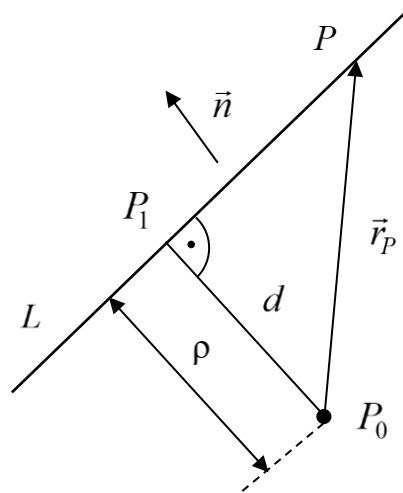


Рис. 7.2.7

Тогда $\rho(P_0, L) = |d|$, где d – проекция вектора $\vec{r}_P$ на ось P_1P_0 , определяемую вектором $\vec{n}$ (рис. 7.2.7).

Согласно (7.13.2) с учетом выражений (7.19.28 – 7.19.30) получаем

$$\begin{aligned}
 d = \vec{r}_P \cdot \vec{e}_n &= \frac{a}{\sqrt{a^2 + b^2}}(x - x_0) + \frac{b}{\sqrt{a^2 + b^2}}(y - y_0) = \\
 &= \frac{ax + by - ax_0 - by_0}{\sqrt{a^2 + b^2}} = -\frac{ax_0 + by_0 + c}{\sqrt{a^2 + b^2}}
 \end{aligned}
 \tag{7.19.31}$$

Таким образом, расстояние от точки P_0 до прямой L определяется выражением

$$\rho(P_0, L) = |d| = \frac{|ax_0 + by_0 + c|}{\sqrt{a^2 + b^2}}, \tag{7.19.32}$$

7.2.2.2 Угол между двумя прямыми на плоскости

Пусть прямые L_1 и L_2 пересекаются в одной точке. Углом φ между прямыми L_1 и L_2 называется тот из углов, образованных этими прямыми, который лежит в пределах от 0 до $\pi/2$ (рис. 7.2.7.1, рис. 7.2.7.2).

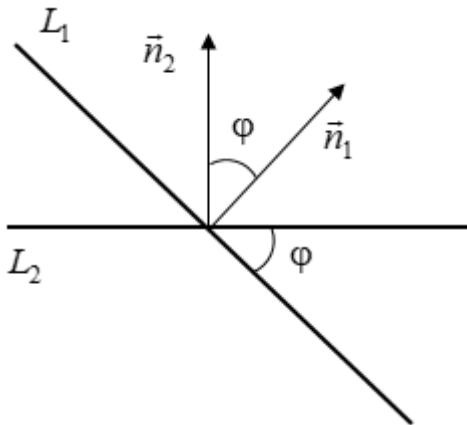


Рис. 7.2.7.1

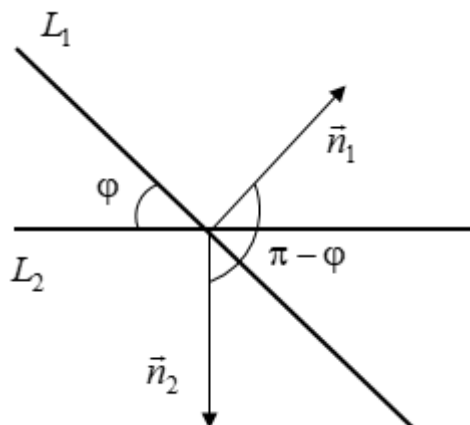


Рис. 7.2.7.2

Если векторы нормалей к прямым L_1 и L_2 составляют острые или прямой угол, то этот угол равен углу φ между прямыми. Если же векторы нормалей составляют тупой угол, то он равен $\pi - \varphi$. Таким образом, либо

$$\vec{n}_1 \cdot \vec{n}_2 = |\vec{n}_1| |\vec{n}_2| \cos \varphi$$

либо

$$\vec{n}_1 \cdot \vec{n}_2 = |\vec{n}_1| |\vec{n}_2| \cos(\pi - \varphi) = -|\vec{n}_1| |\vec{n}_2| \cos \varphi.$$

Отсюда получаем, что

$$\varphi = \arccos \frac{|\vec{n}_1 \cdot \vec{n}_2|}{|\vec{n}_1| |\vec{n}_2|} \quad (7.19.33)$$

Аналогично угол между прямыми L_1 и L_2 выражается через угол между их направляющими векторами $\vec{q}_1$ и $\vec{q}_2$:

$$\varphi = \arccos \frac{|\vec{q}_1 \cdot \vec{q}_2|}{|\vec{q}_1| |\vec{q}_2|} \quad (7.19.34)$$

7.2.2.3 Расположение точек относительно прямой на плоскости

Рассмотрим прямую L , заданную уравнением $ax + by + c = 0$, для которой вектор нормали имеет вид $\vec{n} = \vec{n}(a, b)$, рис. 7.2.7.3. Прямая L делит плоскость на две полуплоскости (1 и 2).

Справедливо утверждение: *прямая $ax + by + c = 0$ делит плоскость на две полуплоскости, в одной из которых $ax + by + c > 0$, а в другой $ax + by + c < 0$.*

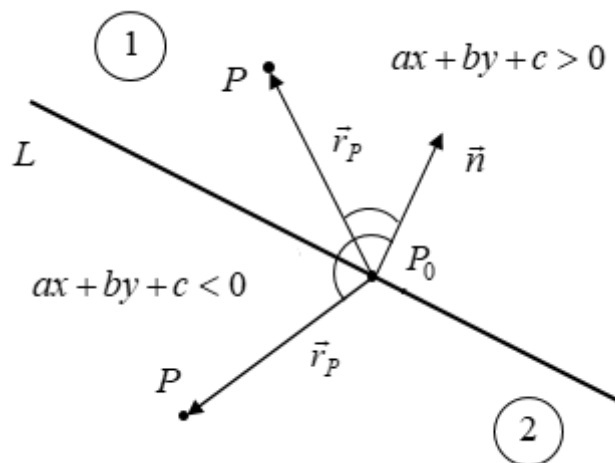


Рис. 7.2.7.3

Пусть $P_0 = P(x_0, y_0)$ – произвольная точка на рассматриваемой прямой L . Рассмотрим произвольную точку $P = P(x, y)$ плоскости с радиус – вектором

$\vec{r}_P = \vec{r}_P(x - x_0, y - y_0)$ относительно точки P_0 , которая может находиться в одной из полуплоскостей (1 или 2), рис. 7.2.7.3.

Если точка P находится в полуплоскости 1, то вектор $\vec{r}_P$ образует с вектором нормали $\vec{n}$ острый угол и для их скалярного произведения справедливо неравенство $\vec{r}_P \cdot \vec{n} > 0$. Если же точка P находится в полуплоскости 2, то угол, образованный вектором $\vec{r}_P$ с вектором нормали $\vec{n}$ будет тупым и $\vec{r}_P \cdot \vec{n} < 0$.

С другой стороны

$$\vec{r}_P \cdot \vec{n} = a(x - x_0) + b(y - y_0) = ax + by - (ax_0 + by_0) \quad (7.19.35)$$

Так как точка P_0 принадлежит прямой L , то ее координаты удовлетворяют уравнению этой прямой и

$$ax_0 + by_0 + c = 0. \quad (7.19.36)$$

С учетом (7.19.36) выражение (7.19.35) для скалярного произведения примет вид

$$\vec{r}_P \cdot \vec{n} = ax + by + c. \quad (7.19.37)$$

Тогда, как показано выше, в одной полуплоскости $ax + by + c > 0$, а в другой полуплоскости $ax + by + c < 0$.

Следствие. Точки $P_1 = P_1(x_1, y_1)$ и $P_2 = P_2(x_2, y_2)$ лежат по одну сторону от прямой $ax + by + c = 0$, если знаки величин $ax_1 + by_1 + c$ и $ax_2 + by_2 + c$ одинаковы, и по разные стороны от прямой, если знаки этих величин противоположны.

7.2.2.4 Взаимное положение двух прямых на плоскости

Пусть на плоскости заданы прямые: $a_1x + b_1y + c_1 = 0 (L_1)$ и $a_2x + b_2y + c_2 = 0 (L_2)$. Заданные прямые могут пересекаться, быть параллельными или совпадать.

Предположим, что прямые L_1 и L_2 пересекаются и найдем координаты точки их пересечения P , которые могут быть определены из системы линейных уравнений

$$\begin{cases} a_1x + b_1y + c_1 = 0 \\ a_2x + b_2y + c_2 = 0 \end{cases}, \quad (7.19.37)$$

которую представим в виде

$$\begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} -c_1 \\ -c_2 \end{pmatrix} \quad (7.19.38)$$

или

$$AP = B, \quad (7.19.39)$$

где

$$A = \begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \end{pmatrix}, \quad P = (x \ y)^T, \quad B = (-c_1 \ -c_2)^T$$

Если определитель системы (7.19.37) $|A| \neq 0$, то она имеет единственное решение

$$P = P_0 = A^{-1}B, \quad (7.19.40)$$

где $P_0 = (x_0 \ y_0)^T$ – координаты точки пересечения прямых L_1 и L_2 .

Если же $|A| = 0$, то система уравнений (7.19.37) решения не имеет (прямые L_1 и L_2 параллельны) или имеет бесконечное множество решений (прямые L_1 и L_2 совпадают).

7.2.2.5 Взаимное положение двух отрезков прямых на плоскости

Рассмотрим на плоскости два отрезка прямой A_1A_2 и B_1B_2 (рис. 7.2.7.4 – рис. 7.2.7.7). Координаты границ отрезков представим в виде

$$A_1 = (x_1^a, y_1^a)^T, \quad A_2 = (x_2^a, y_2^a)^T, \\ B_1 = (x_1^b, y_1^b)^T, \quad B_2 = (x_2^b, y_2^b)^T.$$

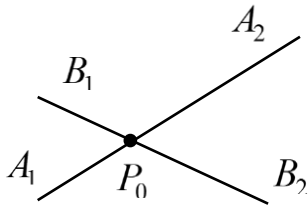


Рис. 7.2.7.4

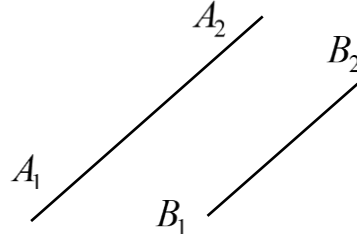


Рис. 7.2.7.5

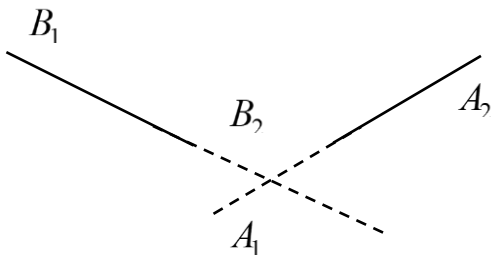


Рис. 7.2.7.6

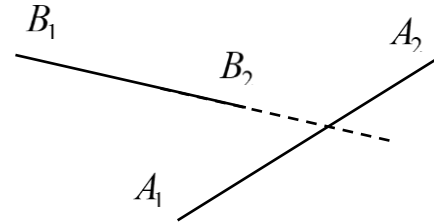


Рис. 7.2.7.7

Запишем уравнения отрезков A_1A_2 и B_1B_2 в матричной форме (7.19.27)

$$\begin{aligned} A &= A_1 + (A_2 - A_1)t, \\ B &= B_1 + (B_2 - B_1)s \end{aligned} \quad (7.19.41)$$

Предположим, что отрезки пересекаются, и P_0 - точка их пересечения. В этой точке справедливо равенство

$$A = B.$$

или

$$A_1 + (A_2 - A_1)t = B_1 + (B_2 - B_1)s. \quad (7.19.42)$$

Отсюда

$$(A_2 - A_1)t - (B_2 - B_1)s = B_1 - A_1. \quad (7.19.43)$$

Переписывая (7.19.43) в координатной форме, получаем

$$\begin{pmatrix} x_2^a - x_1^a \\ y_2^a - y_1^a \end{pmatrix} t - \begin{pmatrix} x_2^b - x_1^b \\ y_2^b - y_1^b \end{pmatrix} s = \begin{pmatrix} x_1^b - x_1^a \\ y_1^b - y_1^a \end{pmatrix} \quad (7.19.44)$$

или

$$\begin{cases} (x_2^a - x_1^a) t - (x_2^b - x_1^b) s = x_1^b - x_1^a \\ (y_2^a - y_1^a) t - (y_2^b - y_1^b) s = y_1^b - y_1^a \end{cases} \quad (7.19.45)$$

Представим систему уравнений (7.19.45) в матричной форме

$$MX = D, \quad (7.19.46)$$

где

$$M = \begin{pmatrix} x_2^a - x_1^a & -(x_2^b - x_1^b) \\ y_2^a - y_1^a & -(y_2^b - y_1^b) \end{pmatrix}, \quad (7.19.47)$$

$$X = (t \ s)^T, \quad D = (x_1^b - x_1^a \ y_1^b - y_1^a)^T$$

Определитель системы (7.19.45) равен

$$|M| = \begin{vmatrix} x_2^a - x_1^a & -(x_2^b - x_1^b) \\ y_2^a - y_1^a & -(y_2^b - y_1^b) \end{vmatrix}, \quad (7.19.48)$$

Если $|M| \neq 0$, то система уравнений (7.19.45) имеет единственное решение относительно параметров t и s

$$X = X_0 = M^{-1}D, \quad (7.19.49)$$

где

$$X = (t_0 \ s_0)^T, \quad (7.19.50)$$

При этом, если $t_0 \in [0, 1]$ и $s_0 \in [0, 1]$, то отрезки пересекаются (рис. 7.2.7.4). Если же $t_0 \notin [0, 1]$ и $s_0 \notin [0, 1]$ пересекаются их продолжения (рис. 7.2.7.6). В случае, когда $t_0 \in [0, 1]$, а $s_0 \notin [0, 1]$ отрезок A_1A_2 пересекается с продолжением отрезка B_1B_2 (рис. 7.2.7.7). При $t_0 \notin [0, 1]$, а $s_0 \in [0, 1]$ отрезок B_1B_2 пересекается с продолжением отрезка A_1A_2 .

Подставляя найденные значения t_0 и s_0 в одно из уравнений (7.19.50), можно определить координаты точки P_0 , которая является точкой пересечения отрезков или их продолжений или точкой пересечения одного отрезка с продолжением другого.

$$P_0 = A_0 = A_1 + (A_2 - A_1)t_0 \quad (7.19.51)$$

или

$$P_0 = B_0 = B_1 + (B_2 - B_1)s_0, \quad (7.19.52)$$

В случае, когда $|M| = 0$, то система уравнений (7.19.45) не имеет решений относительно параметров t и s . В этом случае отрезки A_1A_2 и B_1B_2 не пересекаются (отрезки параллельны) (рис. 7.2.7.5).

7.2.2.6 Взаимное положение точки и треугольника на плоскости

Пусть на плоскости задан ΔABC , координаты вершин которого имеют значения

$$A = A(x_A, y_A), \quad B = B(x_B, y_B) \text{ и } C = C(x_C, y_C).$$

Также на плоскости задана точка $P = P(x_P, y_P)$.

Необходимо определить положение точки P относительно ΔABC , т.е. установить находится точка P внутри треугольника или снаружи.

Предположим, что точка P находится внутри ΔABC (рис. 7.2.7.8).

Образует вектора

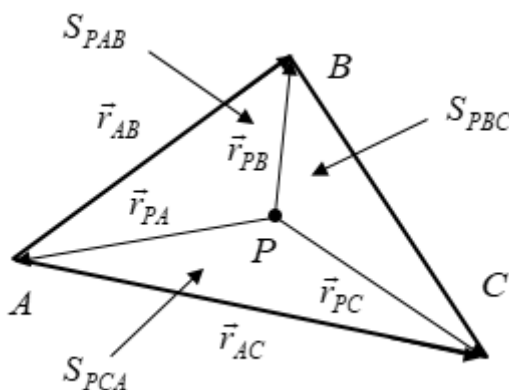


Рис. 7.2.7.8

$$\vec{r}_{AB} = \vec{r}_{AB}(x_B - x_A, y_B - y_A).$$

$$\begin{aligned}\vec{r}_{AC} &= \vec{r}_{AC}(x_C - x_A, y_C - y_A) \\ \vec{r}_{PA} &= \vec{r}_{PA}(x_A - x_P, y_A - y_P) \\ \vec{r}_{PB} &= \vec{r}_{PB}(x_B - x_P, y_B - y_P) \\ \vec{r}_{PC} &= \vec{r}_{PC}(x_C - x_P, y_C - y_P)\end{aligned}$$

Вычислим площадь ΔABC

$$S_{ABC} = \frac{1}{2} |\vec{r}_{AC} \times \vec{r}_{AB}|.$$

Вычислим площади треугольников внутри ΔABC

$$S_{PAB} = \frac{1}{2} |\vec{r}_{PB} \times \vec{r}_{PA}|.$$

$$S_{PBC} = \frac{1}{2} |\vec{r}_{BC} \times \vec{r}_{PB}|.$$

$$S_{PCA} = \frac{1}{2} |\vec{r}_{AC} \times \vec{r}_{PC}|.$$

Вычислим разность

$$\Delta S = S_{ABC} - (S_{PAB} + S_{PBC} + S_{PCA}).$$

Образует отношение

$$q = \frac{|\Delta S|}{S_{ABC}} = \left| 1 - \frac{(S_{PAB} + S_{PBC} + S_{PCA})}{S_{ABC}} \right|.$$

Тогда, если $q = 0$, то точка P лежит внутри ΔABC или на одной из его сторон. Если же $q > 0$, то точка P расположена вне ΔABC .

Следует иметь в виду, что вычисления выполняются, как правило, с граничной точностью. Поэтому, для проверки условия на принадлежность точки треугольнику вместо равенства $q = 0$ следует использовать неравенство $q < \delta$, где δ некоторое малое заданное число.

Если точка P лежит на одной из сторон ΔABC , например на стороне AB , тогда угол между векторами $\vec{r}_{PA}$ и $\vec{r}_{PB}$ $\varphi = \pi$ и

$$S_{PAB} = \frac{1}{2} |r_{PB} r_{PA} \sin \varphi| = 0,$$

где $r_{PB} = |\vec{r}_{PB}|$ и $r_{PA} = |\vec{r}_{PA}|$.

7.3 Графические элементы в пространстве

7.3.1 Модели прямой в пространстве

7.3.1.1 Каноническое уравнение прямой в пространстве

Пусть в декартовой системе координат XYZ задана точка $P_0 = P_0(x_0, y_0, z_0)$ и направляющий вектор $\vec{q} = \vec{q}(m, n, p)$. Тогда уравнение прямой, проходящей через точку P_0 с направляющим вектором $\vec{q}$ будет иметь вид

$$\frac{x - x_0}{m} = \frac{y - y_0}{n} = \frac{z - z_0}{p}, \quad (7.3.1.1.1)$$

Записанное уравнение и есть каноническое уравнение прямой в пространстве.

7.3.1.2 Уравнение прямой в пространстве, проходящей через две заданные точки

Пусть в декартовой системе координат XYZ заданы точки $P_1 = P_1(x_1, y_1, z_1)$ и $P_2 = P_2(x_2, y_2, z_2)$.

Тогда уравнение прямой, проходящей через точки P_1 и P_2 будет иметь вид

$$\frac{x - x_1}{x_2 - x_1} = \frac{y - y_1}{y_2 - y_1} = \frac{z - z_1}{z_2 - z_1}. \quad (7.3.1.1.2)$$

7.3.1.3 Параметрическое уравнение прямой в пространстве

Пусть в декартовой системе координат XYZ задана прямая L , точка $P_0 = P_0(x_0, y_0, z_0)$ на этой прямой, имеющая радиус – вектор $\vec{r}_0 = \vec{r}_0(x_0, y_0, z_0)$, и направляющий вектор $\vec{q} = \vec{q}(m, n, p)$ этой прямой.

Пусть $P = P(x, y, z)$ – текущая точка прямой L , имеющая радиус – вектор

$$\vec{r} = \vec{r}(x, y, z).$$

Тогда *векторное параметрическое уравнение* прямой L в пространстве будет иметь вид

$$\vec{r} = \vec{r}_0 + t\vec{q}, \quad (7.3.1.1.3)$$

где t – некоторый параметр.

Переписывая (7.3.1.1.3) в координатной форме, получаем *параметрическое уравнение прямой в пространстве в координатной форме*.

$$\begin{cases} x = x_0 + mt \\ y = y_0 + nt, \\ z = z_0 + pt \end{cases} \quad (7.3.1.1.4)$$

Если направляющий вектор $\vec{q} = \vec{q}(m, n, p)$ определен координатами двух точек $P_1 = P_1(x_1, y_1, z_1)$ и $P_2 = P_2(x_2, y_2, z_2)$. Тогда

$\vec{q} = \vec{q}(m, n, p) = \vec{q}(x_2 - x_1, y_2 - y_1, z_2 - z_1)$ и уравнения (7.3.1.1.4) примут вид

$$\begin{cases} x = x_0 + (x_2 - x_1)t \\ y = y_0 + (y_2 - y_1)t, \\ z = z_0 + (z_2 - z_1)t \end{cases} \quad (7.3.1.1.5)$$

Уравнения (7.3.1.1.5), как и ранее, можно представить в компактной форме

$$P = P_1 + (P_2 - P_1)t, \quad (7.3.1.1.6)$$

где P_1 и P_2 представлены в виде $P_1 = (x_1, y_1, z_1)^T$ и $P_2 = (x_2, y_2, z_2)^T$.

7.3.1.4 Общее уравнение плоскости

Пусть в декартовой системе координат XYZ задана плоскость S , которая содержит точку $P_0 = P_0(x_0, y_0, z_0)$, имеющую радиус – вектор $\vec{r}_0 = \vec{r}_0(x_0, y_0, z_0)$, рис. 7.3.1.4.1. Задан также вектор нормали $\vec{N} = \vec{N}(A, B, C)$ к этой плоскости. Необходимо получить уравнение плоскости S .

Пусть $P = P(x, y, z)$ текущая точка плоскости S имеющая радиус – вектор $\vec{r} = \vec{r}(x, y, z)$.

Тогда вектор

$$\Delta\vec{r} = \vec{r} - \vec{r}_0 = \Delta\vec{r}(x - x_0, y - y_0, z - z_0),$$

лежит в плоскости S и ортогонален вектору $\vec{N}$. Следовательно

$$\Delta\vec{r} \cdot \vec{N} = (\vec{r} - \vec{r}_0) \cdot \vec{N} = 0, \quad (7.3.1.1.7)$$

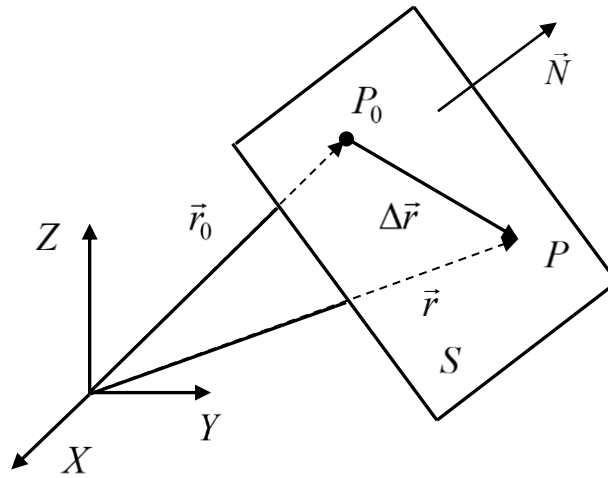


Рис. 7.3.1.4.1

Уравнение (7.3.1.1.7) называется *уравнением плоскости в векторной форме*. Раскрывая скобки в (7.3.1.1.7), получаем другую форму записи векторного уравнения плоскости.

$$\vec{r} \cdot \vec{N} + D = 0, \quad (7.3.1.1.8)$$

где

$$D = -\vec{r}_0 \cdot \vec{N} = -(Ax_0 + By_0 + Cz_0), \quad (7.3.1.1.9)$$

Перепишем уравнение плоскости (7.3.1.1.8) в координатной форме. Тогда получим

$$Ax + By + Cz + D = 0, \quad (7.3.1.1.10)$$

Уравнение (7.3.1.1.10) называется *общим уравнением плоскости*.

7.3.1.5 Уравнение плоскости, проходящей через три точки

Пусть на плоскости S (рис. 7.3.1.4.2) заданы три точки $P_0 = P_0(x_0, y_0, z_0)$,

$P_1 = P_1(x_1, y_1, z_1)$, $P_2 = P_2(x_2, y_2, z_2)$ и пусть $P = P(x, y, z)$ – текущая точка плоскости. Образуем вектора $\overrightarrow{P_0P_1} = \vec{r}_1(x_1 - x_0, y_1 - y_0, z_1 - z_0)$, $\overrightarrow{P_0P_2} = \vec{r}_2(x_2 - x_0, y_2 - y_0, z_2 - z_0)$ и $\overrightarrow{P_0P} = \vec{r}(x - x_0, y - y_0, z - z_0)$.

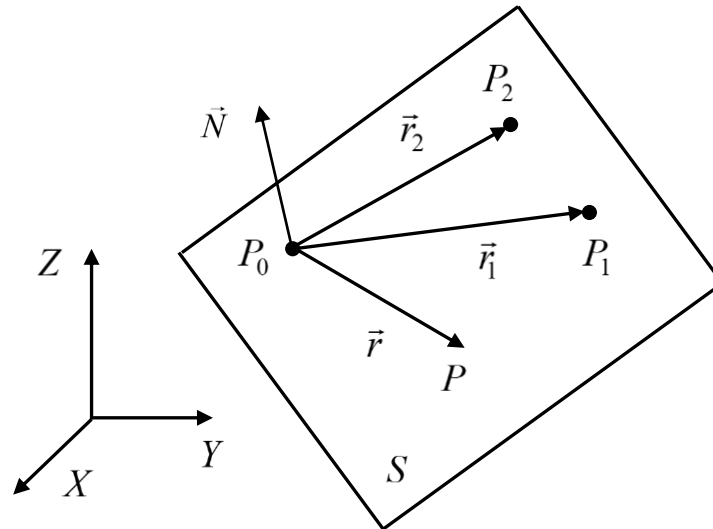


Рис. 7.3.1.5.1

Вычислим вектор нормали к плоскости S

$$\vec{N} = \vec{r}_1 \times \vec{r}_2 = \vec{N}(A, B, C). \quad (7.3.1.1.11)$$

Тогда вектор $\vec{N} \perp \vec{r}$ и их скалярное

$$\vec{N} \cdot \vec{r} = (\vec{r}_1 \times \vec{r}_2) \cdot \vec{r} = A(x - x_0) + B(y - y_0) + C(z - z_0) = 0. \quad (7.3.1.1.12)$$

Уравнение (7.3.1.1.11) есть смешанное произведение векторов $\vec{r}_0, \vec{r}_1, \vec{r}_2$ и является уравнением плоскости, проходящей через три точки P_0, P_1, P_2 этой плоскости.

7.3.1.6 Уравнение плоскости в отрезках

Пусть задано общее уравнение плоскости

$$Ax + By + Cz + D = 0,$$

Перепишем его в виде

$$\frac{x}{-(D/A)} + \frac{y}{-(D/B)} + \frac{z}{-(D/C)} = 1,$$

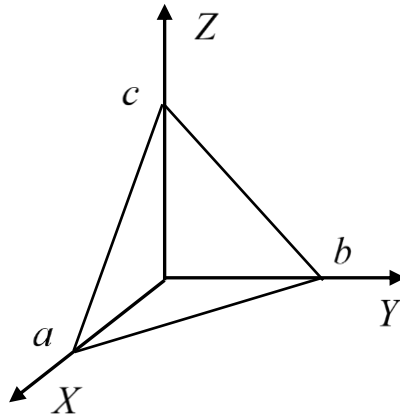


Рис. 7.3.1.6.1

Вводя обозначения $-(D/A) = a$, $-(D/B) = b$ и $-(D/C) = c$, получаем уравнение плоскости в отрезках.

$$\frac{x}{a} + \frac{y}{b} + \frac{z}{c} = 1, \quad (7.3.1.1.13)$$

где a, b, c – величины отрезков, которые плоскость отсекает на осях координат (рис. 7.3.1.6.1).

7.3.1.7 Нормальное уравнение плоскости

Рассмотрим некоторую плоскость S , заданную в декартовой системе координат XYZ (рис. 7.3.1.7.1). Пусть $\vec{e}_N$ – единичный вектор нормали к плоскости S , проведенный к плоскости из начала координат. В этом случае его координатами будут направляющие косинусы $\cos\alpha$, $\cos\beta$ и $\cos\gamma$, где α, β, γ – углы между вектором $\vec{e}_N$ и координатными осями OX , OY и OZ соответственно.

Тогда $\vec{e}_N = \vec{e}_N(\cos\alpha, \cos\beta, \cos\gamma)$, $\cos^2\alpha + \cos^2\beta + \cos^2\gamma = 1$.

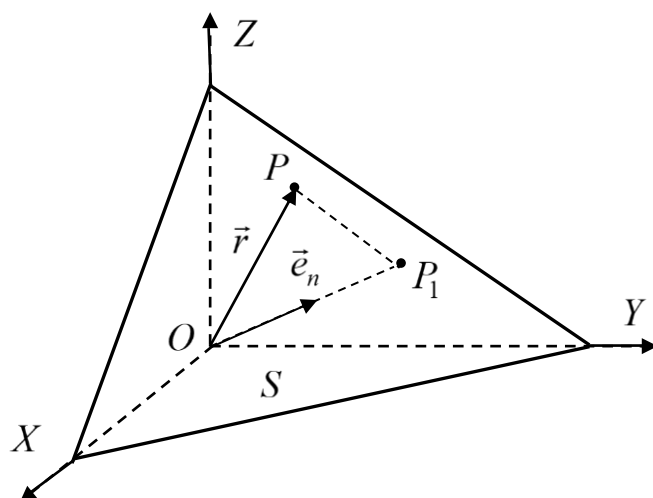


Рис. 7.3.1.7.1

Пусть $\vec{r} = \vec{r}(x, y, z)$ – радиус - вектор произвольной точки P плоскости S и $OP_1 = p$ – перпендикуляр длиной p , $p \geq 0$, проведенный из начала координат к рассматриваемой плоскости. Тогда, если $\vec{r} \cdot \vec{e}_N = p$, то уравнение

$$\vec{r} \cdot \vec{e}_N - p = 0, \quad p \geq 0, \quad (7.3.1.1.14)$$

называется *нормальным векторным уравнением плоскости*. Очевидно, что при $p = 0$ плоскость проходит через начало координат. Переписывая (7.3.1.1.14) в координатной форме, получим

$$x \cos\alpha + y \cos\beta + z \cos\gamma - p = 0, \quad p \geq 0. \quad (7.3.1.1.15)$$

Уравнение (7.3.1.1.15) называется *нормальным уравнением плоскости* или *уравнением плоскости в нормальном виде*.

Заметим, что если в уравнении вида (7.3.1.1.15) – $p > 0$, т.е. $p < 0$, то уравнение (7.3.1.1.15) *не является* нормальным уравнением плоскости.

Рассмотрим приведение общего уравнения плоскости к нормальному виду. Пусть одна и та же плоскость задана общим уравнением (7.3.1.1.10)

$$Ax + By + Cz + D = 0$$

и нормальным уравнением (7.3.1.1.15).

Поскольку оба уравнения задают одну плоскость, их коэффициенты пропорциональны:

$$\mu A = \cos \alpha, \quad \mu B = \cos \beta, \quad \mu C = \cos \gamma, \quad \mu D = -p, \quad (7.3.1.1.16)$$

где μ – некоторый нормирующий множитель.

Первые три равенства из (7.3.1.1.16) возведем в квадрат и сложим:

$$\mu^2 (A^2 + B^2 + C^2) = \cos^2 \alpha + \cos^2 \beta + \cos^2 \gamma = 1$$

Отсюда найдем нормирующий множитель μ :

$$\eta = \pm \frac{1}{\sqrt{A^2 + B^2 + C^2}} \quad (7.3.1.1.17)$$

Из последнего равенства (7.3.1.1.16) видно, что при $p > 0$ произведение μD имеет отрицательный знак. Отсюда следует, что знак нормирующего множителя μ необходимо выбирать противоположным знаком коэффициента D .

Далее, умножив общее уравнение плоскости на нормирующий множитель μ , получим уравнение плоскости в нормальном виде.

$$\mu Ax + \mu By + \mu Cz + \mu D = 0$$

7.3.1.8 Параметрическое уравнение плоскости

Пусть в декартовой системе координат XYZ плоскость S определена двумя неколлинеарными векторами (рис. 7.3.1.7.1) $\vec{a}_1 = \vec{a}_1(a_x^1, a_y^1, a_z^1)$, $\vec{a}_2 = \vec{a}_2(a_x^2, a_y^2, a_z^2)$ и точкой $P_0 = P_0(x_0, y_0, z_0) \in S$, имеющей радиус – вектор $\vec{r}_0 = \vec{r}_0(x_0, y_0, z_0)$. Пусть $\vec{r} = \vec{r}(x, y, z)$ – радиус – вектор произвольной точки P плоскости S . Тогда вектор $\Delta \vec{r} = \vec{r} - \vec{r}_0$ лежит в рассматриваемой плоскости. Следовательно, найдутся такие числа t и s , что

$$\Delta \vec{r} = \vec{r} - \vec{r}_0 = t\vec{a}_1 + s\vec{a}_2 \quad (7.3.1.1.18)$$

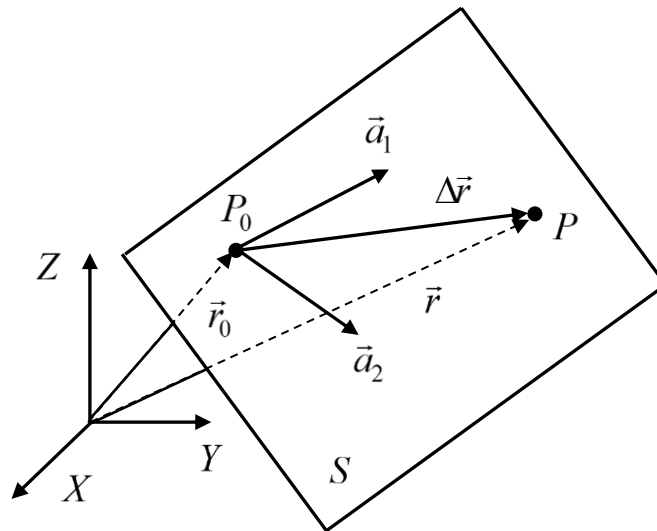


Рис. 7.3.1.7.1

Полученное соотношение называется *векторным параметрическим уравнением плоскости*.

Переписывая (7.3.1.1.14) в координатной форме, получим

$$\begin{aligned} x - x_0 &= ta_x^1 + sa_x^2 \\ y - y_0 &= ta_y^1 + sa_y^2 \\ z - z_0 &= ta_z^1 + sa_z^2 \end{aligned} \quad (7.3.1.1.19)$$

Уравнения (7.3.1.1.19) называются *параметрическими уравнениями плоскости*.

7.3.2 Взаимное расположение графических элементов в пространстве

7.3.2.1 Расстояние от точки до плоскости

Пусть в декартовой системе координат XYZ задана плоскость S , которая определена общим уравнением плоскости (7.3.1.1.10).

$$Ax + By + Cz + D = 0 \quad (7.3.2.1.1)$$

и точка $P_0 = P_0(x_0, y_0, z_0)$, рис. 7.3.2.1.1.

Требуется найти расстояние от точки P_0 до плоскости S . Искомое расстояние есть длина перпендикуляра, опущенного из точки P_0 на плоскость S .

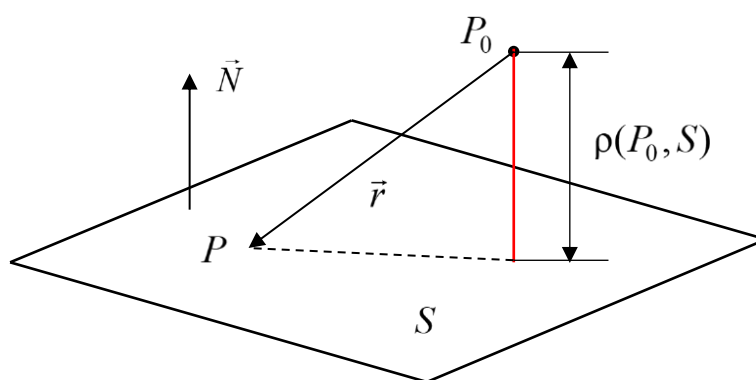


Рис. 7.3.2.1.1

Пусть $P = P(x, y, z)$ — текущая точка плоскости S и $\overrightarrow{PP_0} = \vec{r}(x - x_0, y - y_0, z - z_0)$ — радиус - вектор точки P относительно точки P_0 .

Вычислим проекцию d модуля вектора $\vec{r}$ на направление, определяемое единичным вектором нормали $\vec{e}_N$ к плоскости S .

Так как

$$\vec{e}_N = \vec{e}_N \left(A/|\vec{N}|, B/|\vec{N}|, C/|\vec{N}| \right),$$

где

$$|\vec{N}| = \sqrt{A^2 + B^2 + C^2},$$

то

$$\begin{aligned}
d = \vec{r} \cdot \vec{e}_N &= (x - x_0) \frac{A}{|\vec{N}|} + (y - y_0) \frac{B}{|\vec{N}|} + (z - z_0) \frac{C}{|\vec{N}|} = \\
&= \frac{Ax + By + Cz - (Ax_0 + By_0 + Cz_0)}{|\vec{N}|} = - \frac{Ax_0 + By_0 + Cz_0 + D}{|\vec{N}|}.
\end{aligned} \tag{7.3.2.1.1}$$

Здесь учтено, что координаты точки P удовлетворяют уравнению плоскости (7.3.1.1.10) и поэтому

$$Ax + By + Cz = -D.$$

Если $d \geq 0$, то его значение и будет расстоянием от точки P_0 до плоскости, если же $d < 0$, то за искомое расстояние следует взять значение $-d$.

Таким образом, расстояние от точки P_0 до плоскости S вычисляется по формуле

$$\rho(P_0, S) = |d| = \frac{|Ax_0 + By_0 + Cz_0 + D|}{\sqrt{A^2 + B^2 + C^2}}.$$

7.3.2.2 Точка пересечения прямой и плоскости

Пусть задана плоскость S , которая определяется уравнением

$$Ax + By + Cz + D = 0,$$

прямая L , проходящая через точку $P_0 = P_0(x_0, y_0, z_0)$ и направляющий вектор этой прямой $\vec{q} = \vec{q}(m, n, p)$, рис. . 7.3.2.2.1. Необходимо найти координаты точки $P_S = P_S(x_S, y_S, z_S)$ пересечения прямой L с плоскостью S .

Заданная прямая L может как пересекать плоскость S так и не пересекать ее, т.е. быть параллельной этой плоскости.

Вычислим скалярное произведение вектора нормали $\vec{N} = \vec{N}(A, B, C)$ к рассматриваемой плоскости и направляющего вектора прямой $\vec{q}$

$$b = \vec{N} \cdot \vec{q} = Am + Bn + Cp.$$

Тогда, если $b = 0$, то $\vec{q} \perp \vec{N}$ и прямая L параллельна плоскости S . Последнее означает, что точка пересечения отсутствует.

Пусть теперь $b \neq 0$. Запишем уравнение заданной прямой L в параметрическом виде

$$\begin{cases} x = x_0 + mt \\ y = y_0 + nt, \\ z = z_0 + pt \end{cases}$$

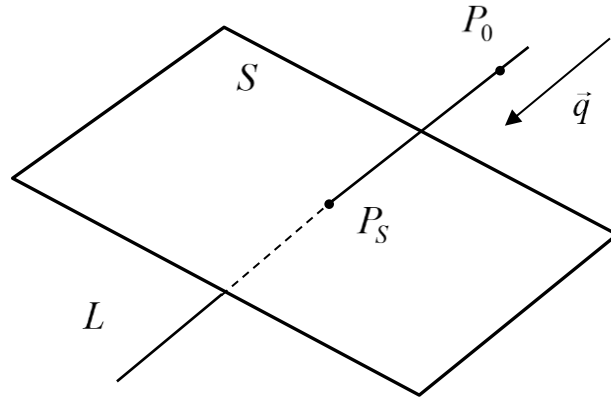


Рис. 7.3.2.2.1

и подставим значения x, y, z в заданное уравнение плоскости

$$A(x_0 + mt) + B(y_0 + nt) + C(z_0 + pt) + D = 0.$$

Из полученного уравнение определим значение параметра $t = t_S$, определяющего координаты x_S, y_S, z_S точки пересечения P_S прямой L и плоскости S .

$$t = t_S = -\frac{Ax_0 + By_0 + Cz_0 + D}{Am + Bn + Cp}. \quad (7.3.2.2.1)$$

или

$$t = t_S = -\frac{\vec{N} \cdot \vec{r}_0 + D}{\vec{N} \cdot \vec{q}}, \quad (7.3.2.2.2)$$

где $\vec{r}_0 = \vec{r}_0(x_0, y_0, z_0)$ – радиус – вектор точки P_0 .

Подставляя найденное значение параметра t_S в параметрические уравнения заданной прямой, получим значения искомых координат x_S, y_S, z_S точки пересечения P_S прямой и плоскости.

$$\begin{cases} x_S = x_0 + mt_S \\ y_S = y_0 + nt_S \\ z_S = z_0 + pt_S \end{cases} \quad (7.3.2.2.3)$$

Пусть теперь прямая L определена двумя точками $P_0 = P_0(x_0, y_0, z_0)$ и $P_1 = P_1(x_1, y_1, z_1)$. Тогда в качестве направляющего вектора примем вектор $\vec{q} = \vec{q}(m, n, p) = \vec{q}(x_1 - x_0, y_1 - y_0, z_1 - z_0)$ и выполним анализ, приведенный выше. В результате получаем значение параметра

$$t_S = -\frac{Ax_0 + By_0 + Cz_0 + D}{A(x_1 - x_0) + B(y_1 - y_0) + C(z_1 - z_0)} \quad (7.3.2.2.4)$$

и координаты точки пересечения прямой и плоскости.

$$\begin{cases} x_S = x_0 + (x_1 - x_0)t_S \\ y_S = y_0 + (y_1 - y_0)t_S \\ z_S = z_0 + (z_1 - z_0)t_S \end{cases} \quad (7.3.2.2.5)$$

7.3.2.3 Положение точек относительно плоскости

Пусть дана плоскость в пространстве. Две точки P_1 и P_2 лежат по одну сторону от плоскости, если отрезок P_1P_2 не пересекается с данной плоскостью.

Полупространством называют множество всех точек, которые лежат по одну сторону от плоскости.

Рассмотрим плоскость S , заданную уравнением $Ax + By + Cz + D = 0$, для которой вектор нормали имеет вид $\vec{N} = \vec{N}(A, B, C)$, рис. 7.3.2.3.1.

Плоскость S делит пространство на два полупространства (1 и 2).

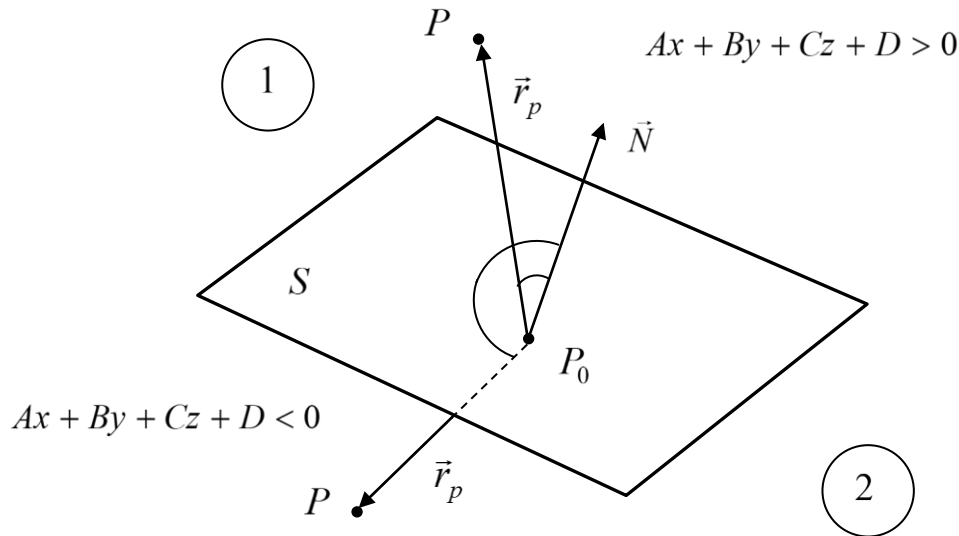


Рис. 7.3.2.3.1

Справедливо утверждение: плоскость $Ax + By + Cz + D = 0$ делит пространство на два полупространства, в одном из которых $Ax + By + Cz + D > 0$ (положительное полупространство), а в другом $Ax + By + Cz + D < 0$ (отрицательное полупространство).

Пусть $P_0 = P(x_0, y_0, z_0)$ – произвольная точка на рассматриваемой плоскости S . Рассмотрим произвольную точку $P = P(x, y, z)$ пространства с радиус – вектором $\vec{r}_p = \vec{r}_p(x - x_0, y - y_0, z - z_0)$ относительно точки P_0 , которая может находиться в одном из полупространств (1 или 2), рис. 7.3.2.3. 1. Если точка P находится в полупространстве 1 (рис. 7.3.2.3.1), то вектор $\vec{r}_p$ образует с вектором нормали $\vec{N}$ острый угол и для их скалярного произведения справедливо неравенство $\vec{r}_p \cdot \vec{N} > 0$. Если же точка P находится в полупространстве 2, то угол, образованный вектором $\vec{r}_p$ с вектором нормали $\vec{N}$ будет тупым и $\vec{r}_p \cdot \vec{N} < 0$.

С другой стороны

$$\begin{aligned} \vec{r}_p \cdot \vec{N} &= A(x - x_0) + B(y - y_0) + C(z - z_0) = \\ &= Ax + By + Cz - (Ax_0 + By_0 + Cz_0) \end{aligned} \quad (7.3.2.3.1)$$

Так как точка P_0 принадлежит плоскости S , то ее координаты удовлетворяют уравнению этой плоскости и

$$Ax_0 + By_0 + Cz_0 + D = 0. \quad (7.3.2.3.2)$$

Из уравнения (7.3.2.3.2) находим значение D и подставляем его в выражение (7.3.2.3.1) для скалярного произведения. В результате получим

$$\vec{r}_p \cdot \vec{N} = Ax + By + Cz + D. \quad (7.19.37)$$

Тогда, как показано выше, в одном полупространстве $Ax + By + Cz + D > 0$, а в другом $Ax + By + Cz + D < 0$.

Следствие. Точки $P_1 = P_1(x_1, y_1, z_1)$ и $P_2 = P_2(x_2, y_2, z_2)$ лежат по одну сторону от плоскости $Ax + By + Cz + D = 0$, если знаки величин $Ax_1 + By_1 + Cz_1 + D$ и $Ax_2 + By_2 + Cz_2 + D$ одинаковы, и по разные стороны от плоскости, если знаки этих величин противоположны.

7.3.2.4 Взаимное положение отрезка прямой и плоскости.

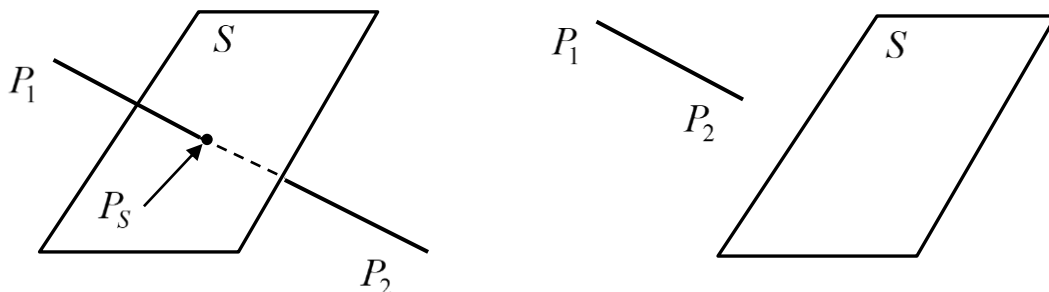
Пусть в декартовой системе координат задана плоскость S , определяемая уравнением

$$F(x, y, z) = 0,$$

где

$$F(x, y, z) = Ax + By + Cz + D, \quad (7.3.2.4.1)$$

и отрезок прямой P_1P_2 , где $P_1 = P_1(x_1, y_1, z_1)$ и $P_2 = P_2(x_2, y_2, z_2)$, рис. 7.3.2.4.1 и рис. 7.3.2.4.2.



Вычислим произведение $w = F(x_1, y_1, z_1)F(x_2, y_2, z_2)$. Если $w > 0$ то сомножители $F(x_1, y_1, z_1)$ и $F(x_2, y_2, z_2)$ имеют одинаковые знаки. В этом случае, как было показано выше, точки P_1 и P_2 лежат в одном полупространстве относительно рассматриваемой плоскости и, следовательно, отрезок прямой P_1P_2 *не пересекает* плоскость S . При этом, если вектор $\vec{V} = \overrightarrow{P_1P_2} = \vec{V}(x_2 - x_1, y_2 - y_1, z_2 - z_1)$ перпендикулярен вектору нормали $\vec{N} = \vec{N}(A, B, C)$ к плоскости S ($\vec{V} \cdot \vec{N} = 0$), то отрезок прямой P_1P_2 параллелен плоскости S .

Если же $w < 0$, то сомножители $F(x_1, y_1, z_1)$ и $F(x_2, y_2, z_2)$ имеют разные знаки. В этом случае точки P_1 и P_2 лежат в разных полупространствах относительно рассматриваемой плоскости и отрезок прямой P_1P_2 *пересекает* плоскость S . Координаты точки пересечения $P_S = P_S(x_S, y_S, z_S)$ будут определяться выражениями

$$\begin{cases} x_S = x_1 + (x_2 - x_1)t_S \\ y_S = y_1 + (y_2 - y_1)t_S \\ z_S = z_1 + (z_2 - z_1)t_S \end{cases} \quad (7.3.2.4.2)$$

где

$$t_S = -\frac{Ax_1 + By_1 + Cz_1 + D}{A(x_2 - x_1) + B(y_2 - y_1) + C(z_2 - z_1)} \quad (7.3.2.4.3)$$

Рассмотрим один частный случай, когда $w < 0$ и плоскость S параллельна одной из координатных плоскостей, например, плоскости XOY . В этом случае вектор нормали к плоскости $\vec{N} = \vec{N}(0, 0, C)$ и уравнение плоскости принимает вид

$$Cz + D = 0. \quad (7.3.2.4.4)$$

Пусть рассматриваемая плоскость находится на расстоянии z_0 от координатной плоскости XOY . Тогда точка $z = z_0$ удовлетворяет уравнению плоскости, которое для рассматриваемого случая примет вид

$$Cz_0 + D = 0. \quad (7.3.2.4.5)$$

Вычтем уравнение (7.3.2.4.5) из уравнения (7.3.2.4.4) и разделим полученное уравнение на $C \neq 0$. В результате получаем уравнение картинной плоскости для рассматриваемого случая

$$z - z_0 = 0, \quad (7.3.2.4.6)$$

где $C = 1$ и $D = -z_0$.

Из (7.3.2.4.3) с учетом (7.3.2.4.6) находим значение параметра t_s

$$t_s = -\frac{z_1 - z_0}{z_2 - z_1} \quad (7.3.2.4.7)$$

и координаты точки пересечения отрезка прямой с плоскостью.

$$\begin{cases} x_s = x_1 - \frac{x_2 - x_1}{z_2 - z_1}(z_1 - z_0) = x_1 - \frac{\Delta x}{\Delta z}(z_1 - z_0) \\ y_s = y_1 - \frac{y_2 - y_1}{z_2 - z_1}(z_1 - z_0) = y_1 - \frac{\Delta y}{\Delta z}(z_1 - z_0), \\ z_s = z_0 \end{cases} \quad (7.3.2.4.8)$$

где $\Delta x = x_2 - x_1$, $\Delta y = y_2 - y_1$, $\Delta z = z_2 - z_1$.

Рассмотренная задача возникает при построении линий уровня (см. ниже).

7.3.2.5 Вычисление вектора нормали к плоскости

Пусть на плоскости S заданы три точки $P_1 = P_1(x_1, y_1, z_1)$, $P_2 = P_2(x_2, y_2, z_2)$ и $P_3 = P_3(x_3, y_3, z_3)$, рис. 7.3.2.5.1.

Образуем векторы $\vec{V}_1 = \vec{V}_1(x_3 - x_1, y_3 - y_1, z_3 - z_1)$ и $\vec{V}_2 = \vec{V}_2(x_2 - x_1, y_2 - y_1, z_2 - z_1)$, рис. 7.3.2.5.2.

Тогда вектор нормали $\vec{N}$ к плоскости S можно вычислить как векторное произведение векторов $\vec{V}_1$ и $\vec{V}_2$

$$\vec{N} = \vec{V}_1 \times \vec{V}_2 = \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ x_3 - x_1 & y_3 - y_1 & z_3 - z_1 \\ x_2 - x_1 & y_2 - y_1 & z_2 - z_1 \end{vmatrix} \quad (7.3.2.5.1)$$

Выражение (7.3.2.5.1) используется для вычисления координат вектора нормали в случае описания поверхности векторно-полигональной моделью (см. ниже).

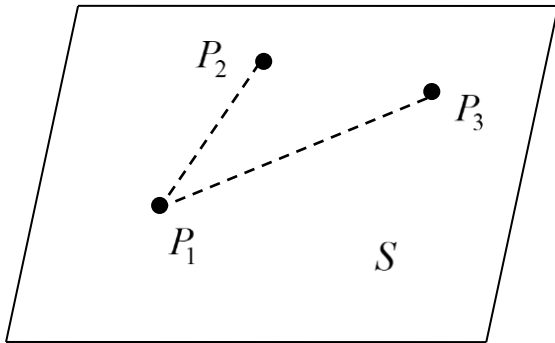


Рис. 7.3.2.5.1

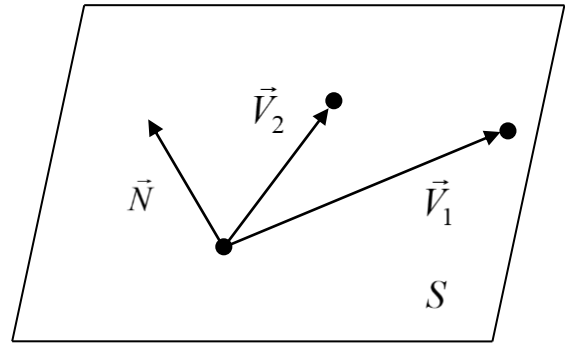


Рис. 7.3.2.5.2

8. АФФИННЫЕ ПРЕОБРАЗОВАНИЯ НА ПЛОСКОСТИ

8.1. Аффинные преобразования системы координат

Пусть на плоскости задана система координат (СК) XOY и точка $M(x, y)$ принадлежащая некоторому объекту. Система координат XOY трансформируется в СК $X'OY'$ путем ряда последовательных смещений и поворотов относительно своего исходного состояния. При этом точка M (объект) остается **неподвижной**. Необходимо определить координаты точки $M(x', y')$ (объекта) в системе координат $X'OY'$ (рис. 8.1, а, б).

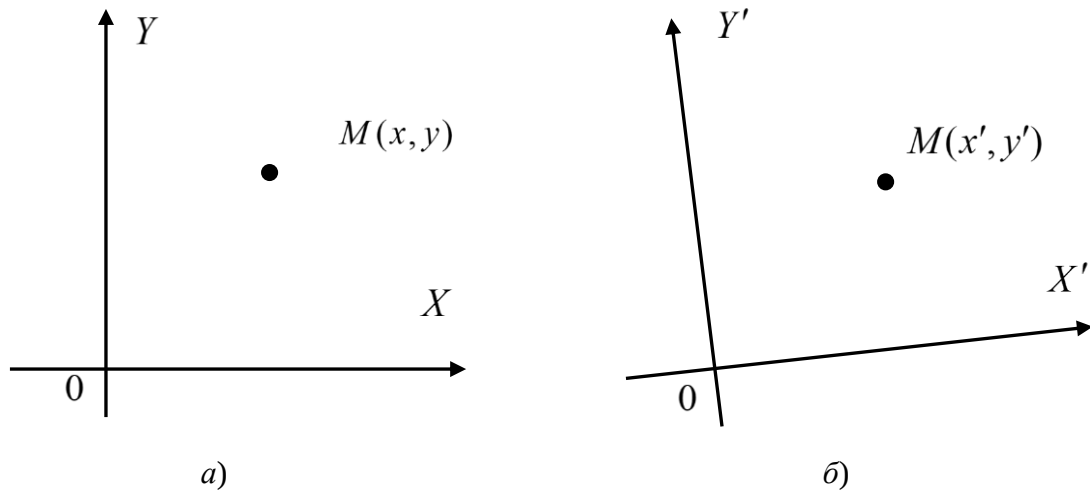


Рис. 8.1

В общем случае преобразование координат точки M при переходе от системы координат XOY к системе координат $X'OY'$ определяется системой линейных уравнений:

$$\begin{cases} x' = a_{11}x + a_{12}y + a_{13} \\ y' = a_{21}x + a_{22}y + a_{23} \end{cases}, \quad (8.1)$$

где $\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} \neq 0$.

Выражение (8.1) представляет собой аффинное преобразование координат при переходе от системы координат XOY к системе координат $X'OY'$.

Обратный переход от СК $X'OY'$ к СК XOY определяется как

$$\begin{cases} x = a'_{11}x' + a'_{12}y' + a'_{13} \\ y = a'_{21}x' + a'_{22}y' + a'_{23} \end{cases}, \quad (8.2)$$

Аффинное преобразование (8.1) удобно представить в матричном виде:

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} x' \\ y' \end{pmatrix} + \begin{pmatrix} a_{13} \\ a_{23} \end{pmatrix} \quad (8.3)$$

В компьютерной графике принято использовать **однородные** координаты, которые вводятся следующим образом. Точке $M(x, y)$ ставится в соответствие точка $M(x, y, 1)$, а точке $M(x', y')$ точка $M(x', y', 1)$.

Тогда переход от системы координат XOY к системе координат $X'OY'$ в матричном виде можно записать как

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad (8.4)$$

или

$$X' = AX, \quad (8.5)$$

где $X' = (x', y', 1)^T$, $X = (x, y, 1)^T$,

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{pmatrix} \quad (8.6)$$

Обратное преобразование

$$X = A^{-1} X' = A' X', \quad (8.7)$$

где

$$A' = A^{-1} = \begin{pmatrix} a'_{11} & a'_{12} & a'_{13} \\ a'_{21} & a'_{22} & a'_{23} \\ 0 & 0 & 1 \end{pmatrix}. \quad (8.8)$$

Рассмотрим теперь различные виды аффинных преобразований на плоскости и соответствующие им матрицы A и A' .

Параллельный сдвиг системы координат (рис. 8.2)

$$\begin{cases} x' = x - \Delta x \\ y' = y - \Delta y \end{cases}, \quad (8.9)$$

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & -\Delta x \\ 0 & 1 & -\Delta y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad (8.10)$$

или

$$X' = AX = T^s X, \quad (8.11)$$

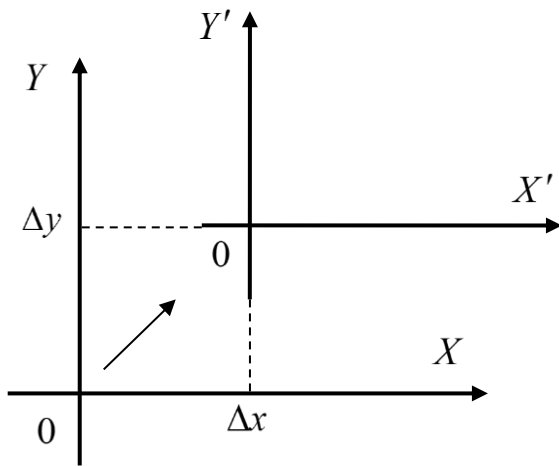


Рис. 8.2

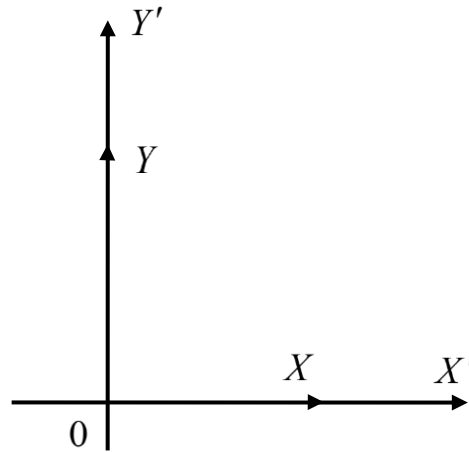


Рис. 8.3

где

$$A = T^s = T^s(\Delta x, \Delta y) = \begin{pmatrix} 1 & 0 & -\Delta x \\ 0 & 1 & -\Delta y \\ 0 & 0 & 1 \end{pmatrix}. \quad (8.12)$$

Здесь и далее индекс «s» означает, что преобразованию подвергается система координат.

Обратное преобразование

Обратное преобразование соответствует перемещению системы координат в противоположном направлении. Соответствующие выражения для преобразования координат могут быть получены из (8.9) – (8.12) путем соответствующей замены в них $(x', y') \rightarrow (x, y)$ и $(\Delta x, \Delta y) \rightarrow (-\Delta x, -\Delta y)$.

Матрица преобразования будет иметь вид

$$A' = (T^s)^{-1} = [T^s(\Delta x, \Delta y)]^{-1} = \begin{pmatrix} 1 & 0 & \Delta x \\ 0 & 1 & \Delta y \\ 0 & 0 & 1 \end{pmatrix} = T^s(-\Delta x, -\Delta y) \quad (8.13)$$

Растяжение – сжатие системы координат (рис. 8.3)

$$\begin{cases} x' = x/k_x \\ y' = y/k_y \end{cases} \quad (8.14)$$

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1/k_x & 0 & 0 \\ 0 & 1/k_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad (8.15)$$

или

$$X' = AX = K^s X, \quad (8.16)$$

где

$$A = K^s = K^s(k_x, k_y) = \begin{pmatrix} 1/k_x & 0 & 0 \\ 0 & 1/k_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (8.17)$$

Обратное преобразование

Соответствующие выражения для преобразования координат могут быть получены из (8.14) – (8.17) путем соответствующей замены в них $(x', y') \rightarrow (x, y)$ и $(k_x, k_y) \rightarrow (1/k_x, 1/k_y)$.

Приведем выражение для матрицы преобразования.

$$A' = (K^s)^{-1} = [K^s(k_x, k_y)]^{-1} = \begin{pmatrix} k_x & 0 & 0 \\ 0 & k_y & 0 \\ 0 & 0 & 1 \end{pmatrix} = K^s(1/k_x, 1/k_y), \quad (8.18)$$

Поворот системы координат на угол φ (рис. 8.4а, б)

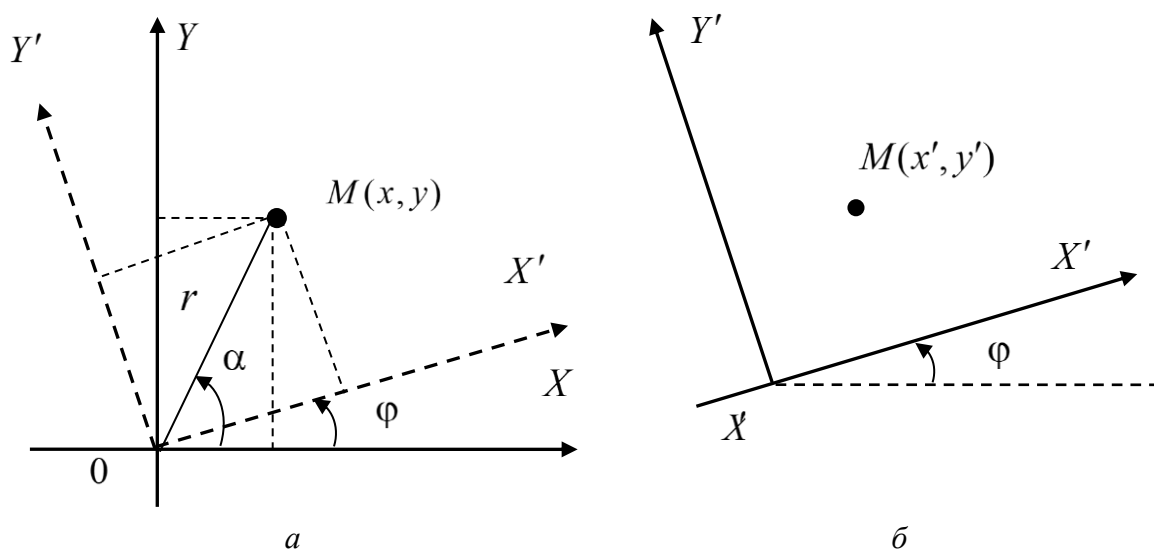


Рис. 8.4

Координаты точки M в системе координат XOY :

$$\begin{cases} x = r \cos \alpha \\ y = r \sin \alpha \end{cases} \quad (8.19)$$

В системе координат $X'OY'$:

$$\begin{cases} x' = r \cos(\alpha - \varphi) = r \cos \alpha \cos \varphi + r \sin \alpha \sin \varphi \\ y' = r \sin(\alpha - \varphi) = r \sin \alpha \cos \varphi - r \cos \alpha \sin \varphi \end{cases}$$

С учетом (8.19), получаем:

$$\begin{cases} x' = x \cos \varphi + y \sin \varphi \\ y' = -x \sin \varphi + y \cos \varphi \end{cases} \quad (8.20)$$

или

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \varphi & \sin \varphi & 0 \\ -\sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad (8.21)$$

$$X' = AX = R^s X, \quad (8.22)$$

где

$$A = R^s = R^s(\varphi) = \begin{pmatrix} \cos \varphi & \sin \varphi & 0 \\ -\sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{pmatrix}. \quad (8.23)$$

Обратное преобразование

Обратное преобразование соответствует повороту системы координат $X'OY'$ на угол $-r$. Соответствующие выражения для преобразования координат могут быть получены из (8.19) – (8.23) путем замены в них $(x', y') \rightarrow (x, y)$ и $r \rightarrow -r$.

Матрица преобразования будет иметь вид

$$A' = (R^s)^{-1} = [R^s(\varphi)]^{-1} = \begin{pmatrix} \cos \varphi & -\sin \varphi & 0 \\ \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{pmatrix} = R^s(-\varphi) \quad (8.24)$$

8.2.Аффинные преобразования объектов на плоскости

Под преобразованием объектов будем понимать изменение координат точек, принадлежащих этому объекту при изменении его положения в некоторой системе координат.

Пусть в системе координат XOY некоторая точка $M(x, y)$ перемещается из положения $M(x, y)$ в положение $M(x', y')$ (рис. 8.4, а, б) или обратно из положения $M(x', y')$ в положение $M(x, y)$.

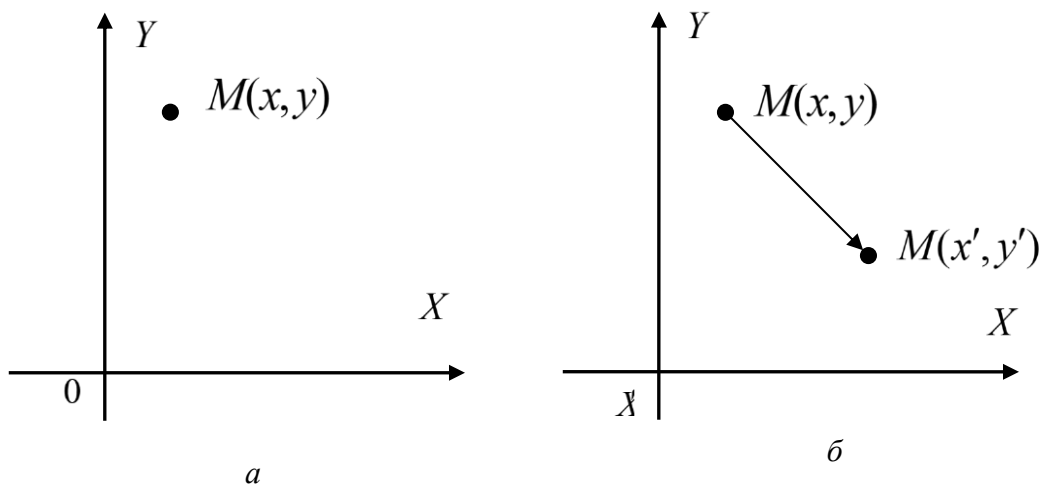


Рис. 8.5

Тогда старые координаты точки (x, y) и новые (x', y') связаны соотношениями (8.1) – (8.8). Подчеркнем лишь, что в данном случае выражения (8.1) – (8.8) описывают взаимосвязь между старыми и новыми координатами точки M при изменении ее положения **в одной и той же** системе координат.

Рассмотрим частные случаи аффинных преобразований объектов.

Сдвиг объекта (рис. 8.6)

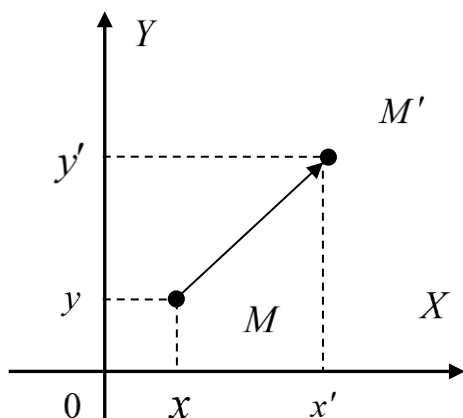


Рис. 8.6

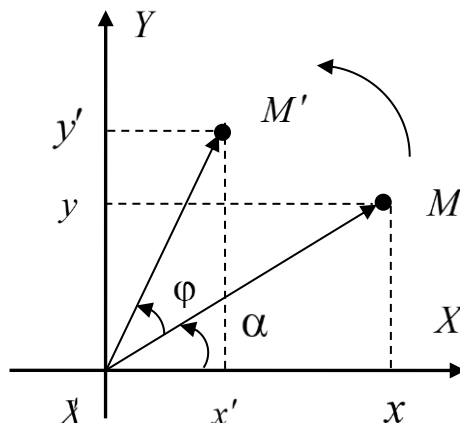


Рис. 8.7

Из рис. 8.6 получаем

$$\begin{cases} x' = x + \Delta x \\ y' = y + \Delta y \end{cases} \quad (8.25)$$

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & \Delta x \\ 0 & 1 & \Delta y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad (8.26)$$

или

$$X' = AX = T^o X, \quad (8.27)$$

где

$$A = T^o = T^o(\Delta x, \Delta y) = \begin{pmatrix} 1 & 0 & \Delta x \\ 0 & 1 & \Delta y \\ 0 & 0 & 1 \end{pmatrix}. \quad (8.28)$$

Здесь и далее индекс «o» означает, что преобразованию подвергаются координаты объекта.

Обратное преобразование

Обратное преобразование соответствует перемещению объекта в противоположном направлении. Соответствующие выражения для преобразования координат могут быть получены из (8.25) – (8.28) путем соответствующей замены в них $(x', y') \rightarrow (x, y)$ и $(\Delta x, \Delta y) \rightarrow (-\Delta x, -\Delta y)$.

Матрица преобразования будет иметь вид

$$A' = (T^o)^{-1} = [T^o(\Delta x, \Delta y)]^{-1} = \begin{pmatrix} 1 & 0 & -\Delta x \\ 0 & 1 & -\Delta y \\ 0 & 0 & 1 \end{pmatrix} = T^o(-\Delta x, -\Delta y) \quad (8.29)$$

Растяжение – сжатие объекта

$$\begin{cases} x' = k_x x \\ y' = k_y y \end{cases} \quad (8.30)$$

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} k_x & 0 & 0 \\ 0 & k_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad (8.31)$$

или

$$X' = AX = K^o X, \quad (8.32)$$

где

$$A = K^o = K^o(k_x, k_y) = \begin{pmatrix} k_x & 0 & 0 \\ 0 & k_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (8.33)$$

Обратное преобразование

Соответствующие выражения для преобразования координат могут быть получены из (8.30) – (8.33) путем соответствующей замены в них $(x', y') \rightarrow (x, y)$ и $(k_x, k_y) \rightarrow (1/k_x, 1/k_y)$.

Приведем выражение для матрицы преобразования.

$$A' = (K^o)^{-1} = [K^o(k_x, k_y)]^{-1} = \begin{pmatrix} 1/k_x & 0 & 0 \\ 0 & 1/k_y & 0 \\ 0 & 0 & 1 \end{pmatrix} = K^o(1/k_x, 1/k_y), \quad (8.34)$$

Поворот объекта вокруг центра координат (рис. 8. 7)

Координаты точки M в системе координат XOY (рис. 8.7):

$$\begin{cases} x = r \cos \alpha \\ y = r \sin \alpha \end{cases} \quad (8.35)$$

Координаты точки M' в системе координат XOY :

$$\begin{cases} x' = r \cos(\alpha + \varphi) = r \cos \alpha \cos \varphi - r \sin \alpha \sin \varphi \\ y' = r \sin(\alpha + \varphi) = r \sin \alpha \cos \varphi + r \cos \alpha \sin \varphi \end{cases}$$

С учетом (8.35), получаем:

$$\begin{cases} x' = x \cos \varphi - y \sin \varphi \\ y' = x \sin \varphi + y \cos \varphi \end{cases} \quad (8.36)$$

или

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \varphi & -\sin \varphi & 0 \\ \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad (8.37)$$

$$X' = AX = R^o X, \quad (8.38)$$

где

$$A = R^o = R^o(\varphi) = \begin{pmatrix} \cos \varphi & -\sin \varphi & 0 \\ \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (8.39)$$

Обратное преобразование

Обратное преобразование соответствует повороту объекта $X'OY'$ на угол $-\varphi$. Соответствующие выражения для преобразования координат могут быть получены из (8.35) – (8.39) путем замены в них $(x', y') \rightarrow (x, y)$ и $\varphi \rightarrow -\varphi$.

Приведем выражение для матрицы преобразования.

$$A' = (R^o)^{-1} = [R^o(\varphi)]^{-1} = \begin{pmatrix} \cos \varphi & \sin \varphi & 0 \\ -\sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{pmatrix} = R^o(-\varphi) \quad (8.40)$$

Связь преобразований объектов с преобразованиями координат

Преобразование объектов и преобразование систем координат тесно связаны между собой. Движение объектов можно рассматривать как движение в обратном направлении соответствующей системы координат. Такая относительность движения дает дополнительные возможности для моделирования и визуализации различных объектов.

Запишем соответствующие соотношения, основываясь на сравнении выражений (8.12) и (8.28) для сдвига

$$T^o(\Delta x, \Delta y) = T^s(-\Delta x, -\Delta y), \quad (8.41)$$

выражений (8.18) и (8.33) для растяжения – сжатия

$$K^o(k_x, k_y) = K^s(1/k_x, 1/k_y), \quad (8.42)$$

выражений (8.23) и (8.39) для поворота

$$R^o(\varphi) = R^s(-\varphi) \quad (8.43)$$

В Приложении 1 приведен пример приложения Windows, где используются аффинные преобразования на плоскости.

Глава 9. АФФИННЫЕ ПРЕОБРАЗОВАНИЯ В ПРОСТРАНСТВЕ

9.1. Аффинные преобразования системы координат

Пусть в пространстве задана система координат (СК) XYZ и точка $M(x, y, z)$ принадлежащая некоторому объекту. Система координат XYZ трансформируется в СК $X'Y'Z'$ путем ряда последовательных смещений и поворотов относительно своего исходного состояния. При этом точка M (объект) остается **неподвижной**. Необходимо определить координаты точки $M(x', y', z')$ (объекта) в системе координат $X'Y'Z'$ (рис. 9.1 а, б).

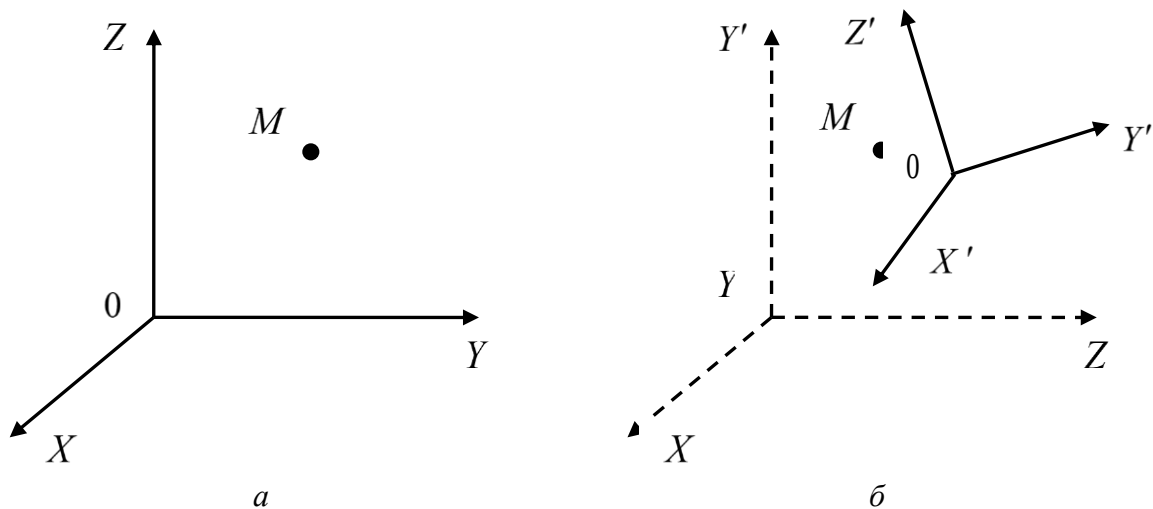


Рис. 9.1

В общем случае преобразование координат точки M при переходе от системы координат XYZ к системе координат $X'Y'Z'$ определяется системой линейных уравнений:

$$\begin{cases} x' = a_{11}x + a_{12}y + a_{13}z + a_{14} \\ y' = a_{21}x + a_{22}y + a_{23}z + a_{24} \\ z' = a_{31}x + a_{32}y + a_{33}z + a_{34} \end{cases}, \quad (9.1)$$

$$\text{где } \begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} \neq 0.$$

Выражение (9.1) представляет собой аффинное преобразование координат при переходе от системы координат XYZ к системе координат $X'Y'Z'$.

Обратный переход от СК $X'Y'Z'$ к СК XYZ определяется как

$$\begin{cases} x = a'_{11}x' + a'_{12}y' + a'_{13}z' + a'_{14} \\ y = a'_{21}x' + a'_{22}y' + a'_{23}z' + a'_{24} \\ z = a'_{31}x' + a'_{32}y' + a'_{33}z' + a'_{34} \end{cases}, \quad (9.2)$$

Аффинные преобразования (9.1) удобно представить в матричном виде:

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} + \begin{pmatrix} a_{14} \\ a_{24} \\ a_{34} \end{pmatrix} \quad (9.3)$$

Как и ранее, введем в рассмотрение однородные координаты, когда точке $M(x, y, z)$ ставится в соответствие точка $M(x, y, z, 1)$, а точке $M(x', y', z')$ – точка $M(x', y', z', 1)$.

Тогда переход от системы координат XYZ к системе координат $X'Y'Z'$ в матричном виде можно записать как

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}, \quad (9.4)$$

или

$$X' = AX, \quad (9.5)$$

где

$$X' = (x', y', z', 1)^T, \quad X = (x, y, z, 1)^T,$$

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (9.6)$$

Обратное преобразование имеет вид

$$X = A^{-1} X' = A' X', \quad (9.7)$$

где

$$A' = A^{-1} = \begin{pmatrix} a'_{11} & a'_{12} & a'_{13} & a'_{14} \\ a'_{21} & a'_{22} & a'_{23} & a'_{24} \\ a'_{31} & a'_{32} & a'_{33} & a'_{34} \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (9.8)$$

Рассмотрим теперь различные виды аффинных преобразований в пространстве и соответствующие им матрицы A и A' .

Параллельный сдвиг системы координат (рис. 9.2)

$$\begin{cases} x' = x - \Delta x \\ y' = y - \Delta y, \\ z' = z - \Delta z \end{cases} \quad (9.9)$$

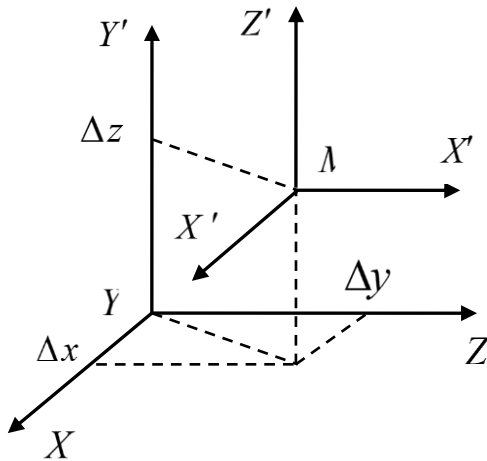


Рис. 9.2

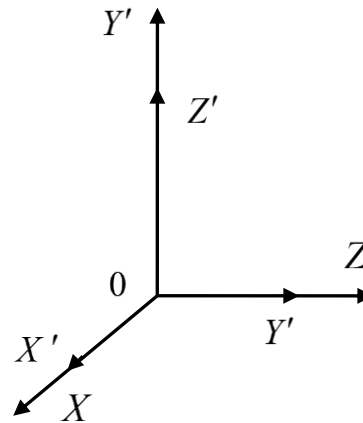


Рис. 9.3

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & -\Delta x \\ 0 & 1 & 0 & -\Delta y \\ 0 & 0 & 1 & -\Delta z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}, \quad (9.10)$$

или

$$X' = AX = T^s X, \quad (9.11)$$

где

$$A = T^s = T^s(\Delta x, \Delta y, \Delta z) = \begin{pmatrix} 1 & 0 & 0 & -\Delta x \\ 0 & 1 & 0 & -\Delta y \\ 0 & 0 & 1 & -\Delta z \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (9.12)$$

Здесь и далее индекс «s» означает, преобразованию подвергается система координат.

Обратное преобразование

Обратное преобразование соответствует перемещению системы в противоположном направлении. Соответствующие выражения для преобразования координат могут быть получены из (9.10) – (9.13) путем соответствующей замены в них $(x', y', z') \rightarrow (x, y, z)$ и $(\Delta x, \Delta y, \Delta z) \rightarrow (-\Delta x, -\Delta y, -\Delta z)$.

Матрица преобразования будет иметь вид

$$A' = (T^s)^{-1} = [T^s(\Delta x, \Delta y, \Delta z)]^{-1} = \begin{pmatrix} 1 & 0 & 0 & \Delta x \\ 0 & 1 & 0 & \Delta y \\ 0 & 0 & 1 & \Delta z \\ 0 & 0 & 0 & 1 \end{pmatrix} = T^s(-\Delta x, -\Delta y, -\Delta z) \quad (9.13)$$

Растяжение – сжатие системы координат (рис. 9.3)

$$\begin{cases} x' = x/k_x \\ y' = y/k_y \\ z' = z/k_z \end{cases} \quad (9.14)$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1/k_x & 0 & 0 & 0 \\ 0 & 1/k_y & 0 & 0 \\ 0 & 0 & 1/k_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}, \quad (9.15)$$

или

$$X' = AX = K^s X, \quad (9.16)$$

где

$$A = K^s = K^s(k_x, k_y, k_z) = \begin{pmatrix} 1/k_x & 0 & 0 & 0 \\ 0 & 1/k_y & 0 & 0 \\ 0 & 0 & 1/k_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (9.17)$$

Обратное преобразование

Соответствующие выражения для преобразования координат могут быть получены из (9.15) – (9.18) путем соответствующей замены в них $(x', y', z') \rightarrow (x, y, z)$ и $(k_x, k_y, k_z) \rightarrow (1/k_x, 1/k_y, 1/k_z)$.

Приведем выражение для матрицы преобразования.

$$A' = (K^s)^{-1} = [K^s(k_x, k_y, k_z)]^{-1} = \begin{pmatrix} k_x & 0 & 0 & 0 \\ 0 & k_y & 0 & 0 \\ 0 & 0 & k_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = K^s(1/k_x, 1/k_y, 1/k_z) \quad (9.18)$$

Поворот системы координат вокруг оси X на угол φ (рис. 9.4)

$$\begin{cases} x' = x \\ y' = y \cos \varphi + z \sin \varphi \\ z' = -y \sin \varphi + z \cos \varphi \end{cases} \quad (9.19)$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi & 0 \\ 0 & -\sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad (9.20)$$

или

$$X' = AX = R_X^s X, \quad (9.21)$$

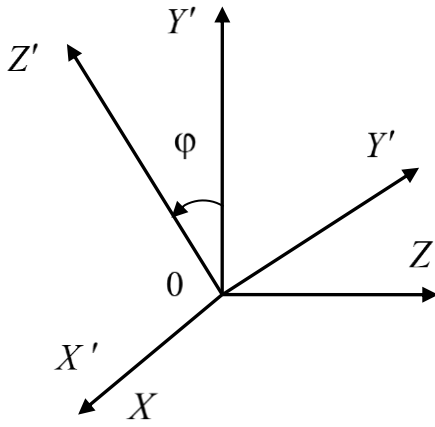


Рис. 9.4

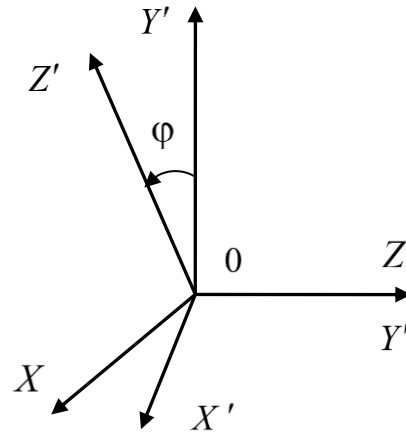


Рис. 9.5

где

$$A = R_X^s = R_X^s(\varphi) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi & 0 \\ 0 & -\sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (9.22)$$

Обратное преобразование

Обратное преобразование соответствует повороту системы координат вокруг оси X в противоположном направлении. Соответствующие выражения для преобразования координат могут быть получены из (9.20) – (9.23) путем замены в них $(x', y', z') \rightarrow (x, y, z)$ и $\varphi \rightarrow -\varphi$.

Приведем выражение для матрицы преобразования.

$$A' = (R_X^s)^{-1} = [R_X^s(\varphi)]^{-1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \varphi & -\sin \varphi & 0 \\ 0 & \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = R_X^s(-\varphi) \quad (9.23)$$

Поворот системы координат вокруг оси Y на угол φ (рис.9. 5)

$$\begin{cases} x' = x \cos \varphi - z \sin \varphi \\ y' = y \\ z' = x \sin \varphi + z \cos \varphi \end{cases} \quad (9.24)$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \varphi & 0 & -\sin \varphi & 0 \\ 0 & 1 & 0 & 0 \\ \sin \varphi & 0 & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad (9.25)$$

или

$$X' = AX = R_Y^s X, \quad (9.26)$$

где

$$A = R_Y^s = R_Y^s(\varphi) = \begin{pmatrix} \cos \varphi & 0 & -\sin \varphi & 0 \\ 0 & 1 & 0 & 0 \\ \sin \varphi & 0 & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (9.27)$$

Обратное преобразование

Обратное преобразование соответствует повороту системы координат вокруг оси Y в противоположном направлении. Соответствующие выражения для преобразования координат могут быть получены из (9.25) – (9.28) путем замены в них $(x', y', z') \rightarrow (x, y, z)$ и $\varphi \rightarrow -\varphi$.

Приведем выражение для матрицы преобразования.

$$A' = (R_Y^s)^{-1} = [R_Y^s(\varphi)]^{-1} = \begin{pmatrix} \cos \varphi & 0 & \sin \varphi & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \varphi & 0 & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = R_Y^s(-\varphi) \quad (9.28)$$

Поворот системы координат вокруг оси Z на угол φ (рис. 9.6)

$$\begin{cases} x' = x \cos \varphi + y \sin \varphi \\ y' = -x \sin \varphi + y \cos \varphi \\ z' = z \end{cases} \quad (9.29)$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \varphi & \sin \varphi & 0 & 0 \\ -\sin \varphi & \cos \varphi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}, \quad (9.30)$$

или

$$X' = AX = R_Z^s X, \quad (9.31)$$

где

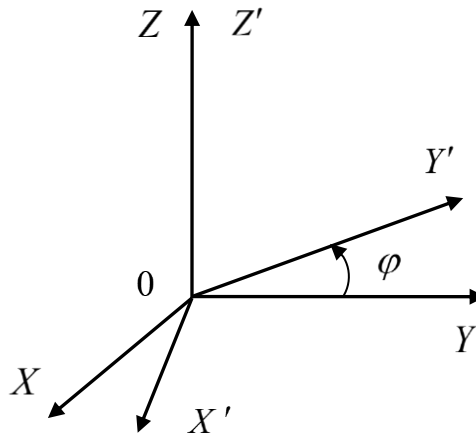


Рис. 9.6

$$A = R_Z^s = R_Z^s(\varphi) = \begin{pmatrix} \cos \varphi & \sin \varphi & 0 & 0 \\ -\sin \varphi & \cos \varphi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (9.32)$$

Обратное преобразование

Обратное преобразование соответствует повороту системы координат вокруг оси Z в противоположном направлении. Соответствующие выражения для

преобразования координат могут быть получены из (9.30) – (9.33) путем замены в них $(x', y', z') \rightarrow (x, y, z)$ и $\varphi \rightarrow -\varphi$.

Матрица преобразования для этого случая будет иметь вид

$$A' = (R_Z^s)^{-1} = [R_Z(\varphi)]^{-1} = \begin{pmatrix} \cos \varphi & -\sin \varphi & 0 & 0 \\ \sin \varphi & \cos \varphi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = R_Z^s(-\varphi) \quad (9.33)$$

9.2. Аффинные преобразования координат объектов

Как и ранее под преобразованием объекта будем понимать изменение координат точек, принадлежащих этому объекту при изменении его положения в некоторой системе координат.

Пусть в пространстве задана система координат XYZ и точка $M(x, y, z)$ принадлежащая некоторому объекту. Объект перемещается из одной точки пространства в другую путем ряда последовательных смещений и поворотов относительно своего исходного положения. При этом система координат остается **неподвижной**. Необходимо определить новые координаты точки $M(x', y', z')$ (объекта) в СК XYZ . В общем случае старые координаты точки (x, y, z) и новые (x', y', z) связаны соотношениями (9.1) – (9.9). Подчеркнём лишь, что в данном случае выражения (9.1) – (9.9) описывают взаимосвязь между старыми и новыми координатами точки M при изменении ее положения **в одной и той же** системе координат.

Рассмотрим частные случаи аффинных преобразований координат объектов.

Смещение объекта вдоль координатных осей (рис.9.7)

$$\begin{cases} x' = x + \Delta x \\ y' = y + \Delta y, \\ z' = z + \Delta z \end{cases} \quad (9.34)$$

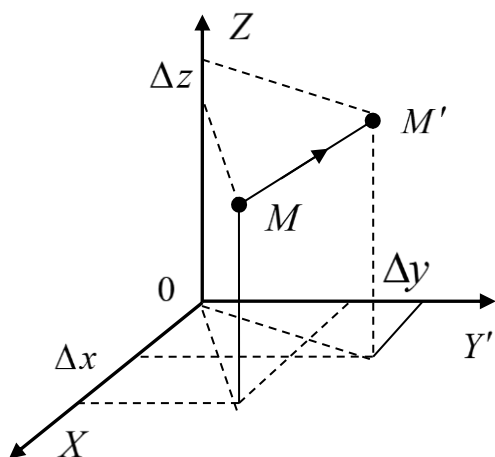


Рис. 9.7

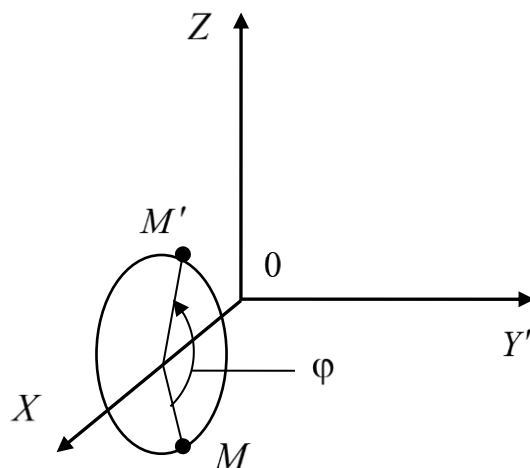


Рис. 9.8

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & \Delta x \\ 0 & 1 & 0 & \Delta y \\ 0 & 0 & 1 & \Delta z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad (9.35)$$

или

$$X' = AX = T^o X, \quad (9.36)$$

где

$$A = T^o = T^o(\Delta x, \Delta y, \Delta z) = \begin{pmatrix} 1 & 0 & 0 & \Delta x \\ 0 & 1 & 0 & \Delta y \\ 0 & 0 & 1 & \Delta z \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (9.37)$$

Здесь и далее индекс «O» означает, что преобразованию подвергаются координаты объекта.

Обратное преобразование

Обратное преобразование соответствует перемещению объекта в противоположном направлении. Соответствующие выражения для преобразования координат могут быть получены из (9.35) – (9.38) путем соответствующей замены в них $(x', y', z') \rightarrow (x, y, z)$ и $(\Delta x, \Delta y, \Delta z) \rightarrow (-\Delta x, -\Delta y, -\Delta z)$.

Приведем выражение для матрицы преобразования.

$$A' = (T^o)^{-1} = [T^o(\Delta x, \Delta y, \Delta z)]^{-1} = \begin{pmatrix} 1 & 0 & 0 & -\Delta x \\ 0 & 1 & 0 & -\Delta y \\ 0 & 0 & 1 & -\Delta z \\ 0 & 0 & 0 & 1 \end{pmatrix} = T^o(-\Delta x, -\Delta y, -\Delta z) \quad (9.38)$$

Растяжение – сжатие объекта

$$\begin{cases} x' = k_x x \\ y' = k_y y, \\ z' = k_z z \end{cases} \quad (9.39)$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} k_x & 0 & 0 & 0 \\ 0 & k_y & 0 & 0 \\ 0 & 0 & k_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}, \quad (9.40)$$

или

$$X' = AX = K^o X, \quad (9.41)$$

где

$$A = K^o = K^o(k_x, k_y, k_z) = \begin{pmatrix} k_x & 0 & 0 & 0 \\ 0 & k_y & 0 & 0 \\ 0 & 0 & k_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (9.42)$$

Обратное преобразование.

Соответствующие выражения для преобразования координат могут быть получены из (9.40) – (9.43) путем соответствующей замены в них $(x', y', z') \rightarrow (x, y, z)$ и $(k_x, k_y, k_z) \rightarrow (1/k_x, 1/k_y, 1/k_z)$.

Приведем выражение для матрицы преобразования.

$$A' = (K^o)^{-1} = [K^o(k_x, k_y, k_z)]^{-1} = \begin{pmatrix} 1/k_x & 0 & 0 & 0 \\ 0 & 1/k_y & 0 & 0 \\ 0 & 0 & 1/k_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = K^o(1/k_x, 1/k_y, 1/k_z) \quad (9.43)$$

Поворот объекта вокруг оси X на угол φ (рис. 9.8).

$$\begin{cases} x' = x \\ y' = y \cos \varphi - z \sin \varphi \\ z' = y \sin \varphi + z \cos \varphi \end{cases} \quad (9.44)$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \varphi & -\sin \varphi & 0 \\ 0 & \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}, \quad (9.45)$$

или

$$X' = AX = R_X^o X, \quad (9.46)$$

где

$$A = R_X^o = R_X^o(\varphi) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \varphi & -\sin \varphi & 0 \\ 0 & \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (9.47)$$

Обратное преобразование

Обратное преобразование соответствует повороту объекта вокруг оси X в противоположном направлении. Соответствующие выражения для преобразования координат могут быть получены из (9.45) – (9.48) путем соответствующей замены в них $(x', y', z') \rightarrow (x, y, z)$ и $\varphi \rightarrow -\varphi$.

Приведем выражение для матрицы преобразования.

$$A' = (R_X^o)^{-1} = (R_X^o(\varphi))^{-1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi & 0 \\ 0 & -\sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = R_X^o(-\varphi) \quad (9.48)$$

Поворот объекта вокруг оси Y на угол φ (рис. 9.9).

$$\begin{cases} x' = x \cos \varphi + z \sin \varphi \\ y' = y \\ z' = -x \sin \varphi + z \cos \varphi \end{cases} \quad (9.49)$$

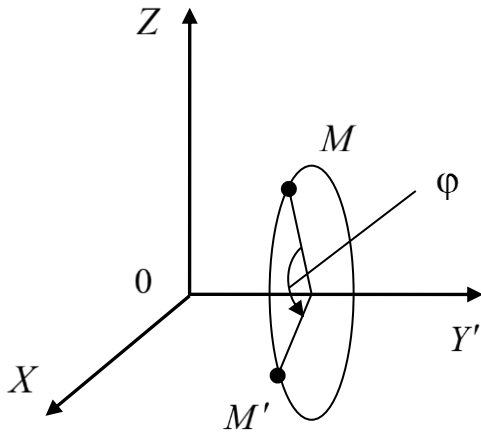


Рис. 9.9

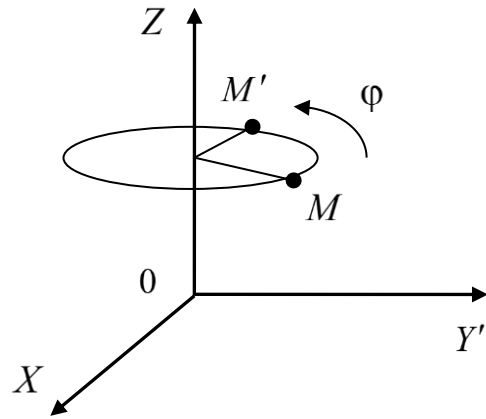


Рис. 9.10

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \varphi & 0 & \sin \varphi & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \varphi & 0 & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad (9.50)$$

или

$$X' = AX = R_Y^o X, \quad (9.51)$$

где

$$A = R_Y^o = R_Y^o(\varphi) = \begin{pmatrix} \cos \varphi & 0 & \sin \varphi & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \varphi & 0 & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (9.52)$$

Обратное преобразование

Соответствующие выражения для преобразования координат могут быть получены из (9.50) – (9.53) путем соответствующей замены в них $(x', y', z') \rightarrow$

(x, y, z) и $\varphi \rightarrow -\varphi$.

Приведем выражение для матрицы преобразования.

$$A' = (R_Y^o)^{-1} = (R_Y^o(\varphi))^{-1} = \begin{pmatrix} \cos \varphi & 0 & \sin \varphi & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \varphi & 0 & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = R_Y^o(-\varphi) \quad (9.53)$$

Поворот объекта вокруг оси Z на угол φ (рис. 9.10)

$$\begin{cases} x' = x \cos \varphi - y \sin \varphi \\ y' = x \sin \varphi + y \cos \varphi \\ z' = z \end{cases} \quad (9.54)$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \varphi & -\sin \varphi & 0 & 0 \\ \sin \varphi & \cos \varphi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad (9.55)$$

или

$$X' = AX = R_Z^o X, \quad (9.56)$$

где

$$A = R_Z^o = R_Z^o(\varphi) = \begin{pmatrix} \cos \varphi & -\sin \varphi & 0 & 0 \\ \sin \varphi & \cos \varphi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (9.57)$$

Обратное преобразование

Соответствующие выражения для преобразования координат могут быть получены из (9.55) – (9.58) путем соответствующей замены в них $(x', y', z') \rightarrow (x, y, z)$ и $\varphi \rightarrow -\varphi$.

Приведем выражение для матрицы преобразования.

$$A' = (R_Z^o)^{-1} = (R_Z^o(\varphi))^{-1} = \begin{pmatrix} \cos \varphi & \sin \varphi & 0 & 0 \\ -\sin \varphi & \cos \varphi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = R_Z^o(-\varphi) \quad (9.58)$$

9.3. Связь преобразований объектов с преобразованиями координат

Как было отмечено при рассмотрении аффинных преобразований на плоскости, движение объектов можно рассматривать как движение в обратном направлении соответствующей системы координат.

Запишем соответствующие соотношения, связывающие матрицы аффинных преобразований для перемещений объектов и системы координат, основываясь на сравнении выражений (9.13) и (9.38) для сдвига

$$T^o(\Delta x, \Delta y, \Delta z) = T^s(-\Delta x, -\Delta y, \Delta z), \quad (9.59)$$

выражений (9.18) и (9.43) для растяжения – сжатия

$$K^o(k_x, k_y, k_z) = K^s(1/k_x, 1/k_y, 1/k_z), \quad (9.60)$$

выражений (9.23), (9.28), (9.33) и (9.48), (9.53), (9.58) для поворотов

$$R_X^o(\varphi) = R_X^s(-\varphi) \quad (9.61)$$

$$R_Y^o(\varphi) = R_Y^s(-\varphi) \quad (9.62)$$

$$R_Z^o(\varphi) = R_Z^s(-\varphi) \quad (9.63)$$

Глава 10. ПРОЕКЦИИ

10.1. Основные типы проекций

В настоящее время наиболее распространены устройства отображения, которые формируют изображение на плоскости – экране дисплея или бумаге. Для того, чтобы построить на плоскости монитора трехмерное изображение некоторого объекта, заданного в трехмерном пространстве (пространстве R^3), необходимо задать способ отображения точек этого объекта на некоторую плоскость – плоскость проектирования). Требуемый способ отображения определяется с помощью операции проектирования.

Проектирование – это преобразование, которое ставит в соответствие точкам из пространства R^3 точки некоторой плоскости. Плоскость, на которую выполняется проекция объекта, называется *картинной плоскостью*.

Мы будем рассматривать только проекции на плоскость.

В компьютерной графике используются два основных вида проектирования – *параллельное* и *перспективное*.

Параллельное проектирование.

Пусть в декартовой системе координат XYZ (в пространстве R^3) задана некоторая точка P и плоскость S – плоскость проектирования (рис. 10.1a). Пусть также задан вектор $\vec{q}$ вдоль которого выполняется проектирование, такой что $\vec{N} \cdot \vec{q} \neq 0$, где $\vec{N}$ – вектор нормали к плоскости S . Последнее условие означает, что вектор $\vec{q}$ не является ортогональным вектору нормали $\vec{N}$ и, следовательно, не параллелен плоскости проектирования S .

Проведем через точку P прямую L , с направляющим вектором $\vec{q}$.

Тогда, *параллельной проекцией* произвольной точки P на плоскость проектирования S называется точка пересечения P_S прямой L с плоскостью S (рис. 10.1a).

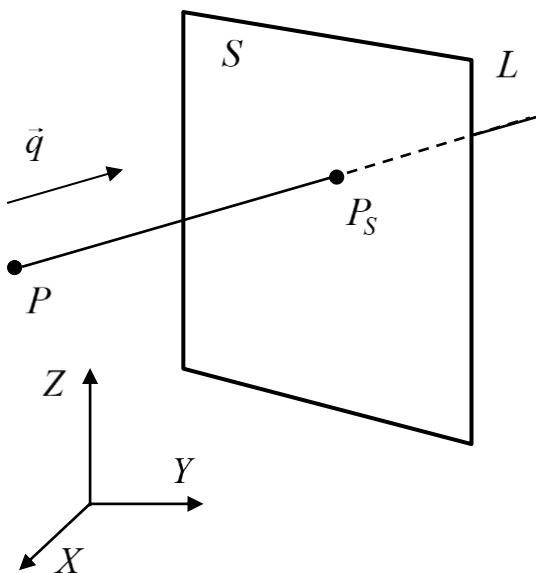


Рис.10.1a

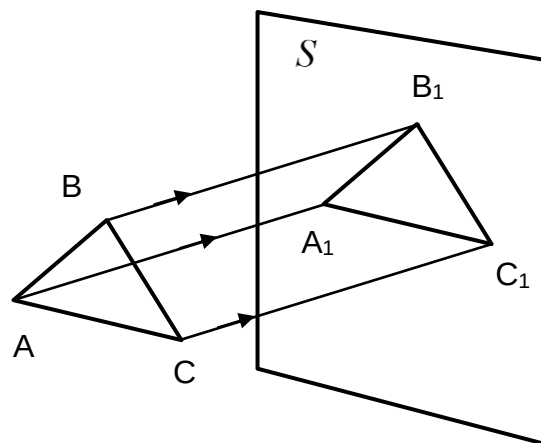


Рис.10.1б

На рис 10.16 показана параллельная проекция фигуры в виде треугольника на плоскость.

Для параллельной проекции проектирующие лучи параллельны. Частным случаем параллельной проекции является *аксонометрическая* проекция. Для аксонометрической проекции $\vec{q} \perp S$. Другими словами, все проектирующие лучи располагаются под прямым углом к плоскости проектирования.

Пусть $P = P(x_P, y_P, z_P)$, $P_S = P_S(x_S, y_S, z_S)$, $\vec{q} = \vec{q}(m, n, p)$ и картинная плоскость определена уравнением $Ax + By + Cz + D = 0$. Тогда координаты точки P_S вычисляются по методике, изложенной в разделе 7.3.2.2 («Точка пересечения прямой и плоскости»).

$$\begin{cases} x_S = x_P + mt_S \\ y_S = y_P + nt_S \\ z_S = z_P + pt_S \end{cases}, \quad (10.1.1)$$

$$t_S = -\frac{Ax_P + By_P + Cz_P + D}{Am + Bn + Cp}. \quad (10.1.2)$$

Рассмотрим простой случай, когда картинная плоскость S параллельна координатной плоскости ZX системы координат XYZ , рис. 10.1а.

В этом случае вектор нормали к плоскости есть $\vec{N} = \vec{N}(A, B, C) = \vec{N}(0, B, 0)$ и уравнение плоскости S принимает вид

$$By + D = 0. \quad (10.1.2_1)$$

Пусть рассматриваемая картинная плоскость S находится на расстоянии $d = |y_0|$ от координатной плоскости XZ (рис. 10.1а). Тогда точка $y = y_0$ удовлетворяет уравнению плоскости (10.1.2_1) и

$$By_0 + D = 0. \quad (10.1.2_2)$$

Вычтем уравнение (10.1.2_2) из уравнения (10.1.2_1) и разделим полученное уравнение на значение $B \neq 0$. В результате получаем уравнение картинной плоскости для рассматриваемого случая

$$y - y_0 = 0, \quad (10.1.2_3)$$

где $B=1$ и $D = -y_0$

Для уравнения плоскости вида (10.1.2_3) выражение (10.1.2) для параметра t_S и (10.1.1) для координат проекции примут вид

$$t_S = -\frac{y_P - y_0}{n}. \quad (10.1.2_4)$$

и

$$\begin{cases} x_S = x_P + mt_S \\ y_S = y_P + nt_S = y_P - n \frac{y_P - y_0}{n} = y_0, \\ z_S = z_P + pt_S \end{cases} \quad (10.1.2_5)$$

При этом для аксанометрической проекции, когда

$$\vec{q} = \vec{q}(m, n, p) = \vec{q}(0, n, 0), \quad (10.1.2_6)$$

выражения (10.1.2_5) для координат проекции примут вид

$$\begin{cases} x_S = x_P \\ y_S = y_0, \\ z_S = z_P \end{cases} \quad (10.1.2_7)$$

В качестве примера рассмотрим параллельную проекцию куба на картинную плоскость S , определенную уравнением $Ax + By + Cz + D = 0$, вдоль направляющего вектора $\vec{q}$ (рис. 11). Пусть $P_i = P_i(x_i, y_i, z_i)$, $i = 0, 1, \dots, 7$ – вершина куба с номером i .

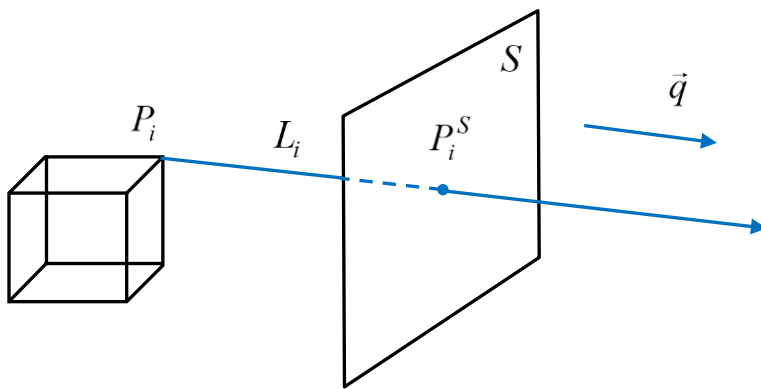


Рис. 11

Будем рассматривать случай, когда картинная плоскость S параллельна координатной плоскости ZX системы координат XYZ , рис. 12.

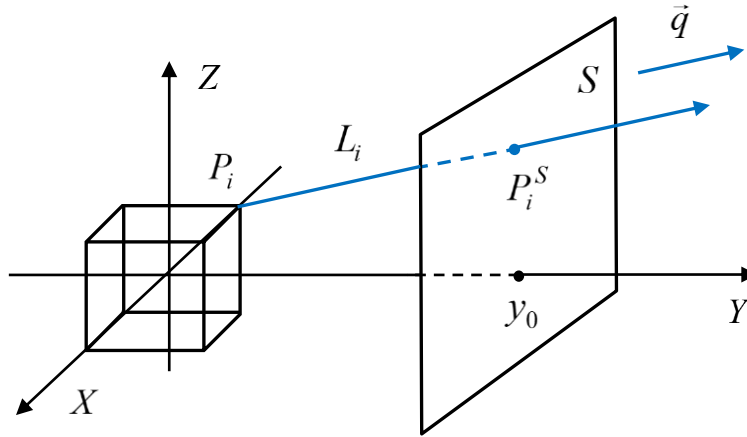


Рис. 12

Из (10.1.2_4) и (10.1.2_5) находим координаты проекций вершин куба на картинную плоскость S .

$$t_S^i = -\frac{y_P^i - y_0}{n}, \quad (10.1.2_8)$$

$$\begin{cases} x_S^i = x_P^i + m t_S^i \\ y_S^i = y_0 \\ z_S^i = z_P^i + p t_S^i \end{cases}, \quad (10.1.2_9)$$

При этом для аксанометрической проекции выражения (10.1.2_9) для координат примут вид

$$\begin{cases} x_S^i = x_P^i \\ y_S^i = y_0 \\ z_S^i = z_P^i \end{cases}. \quad (10.1.2_10)$$

Перспективное проектирование.

Перспектива - способ изображать на плоскости трехмерные объекты в соответствии с тем *кажущимся* изменением их величины, которое обусловлено степенью отдаленности их от точки наблюдения.

Рассмотрим, как и ранее, декартовую систему координат XYZ (в пространстве R^3), в которой задана некоторая точка $P = P(x_P, y_P, z_P)$ и плоскость S – плоскость проектирования. Пусть также задана точка $C = C(x_C, y_C, z_C)$ (center), которую будем называть центром проектирования или точкой схода лучей (рис. 10.1a).

Тогда, *перспективной проекцией* точки P на плоскость проектирования S называется точка пересечения P_S плоскости S с лучом L , выходящим из точки C и проходящим через точку P , при условии, что точки C и P лежат в различных полупространствах относительно плоскости S (рис. 10.1a).

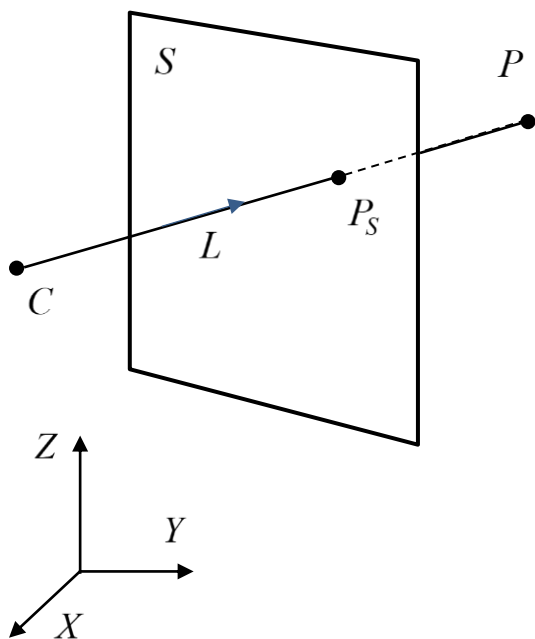


Рис.11.1a

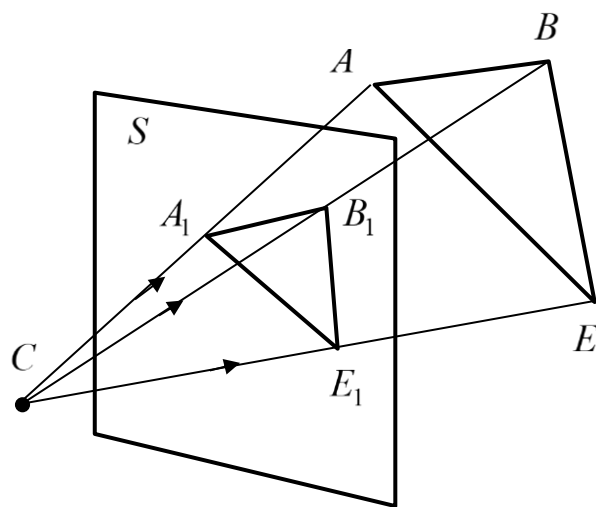


Рис.11.1б

На рис 11.1б показана перспективная проекция фигуры в виде треугольника на плоскость.

Для перспективной проекции проектирующие лучи выходят из одной точки (центра проектирования), расположенной на конечном расстоянии от объектов и плоскости проектирования.

Координаты точки $P_S = P_S(x_S, y_S, z_S)$ вычисляются по методике, изложенной в разделе 7.3.2.2 («Точка пересечения прямой и плоскости»), где в качестве направляющего вектора для проектирующего луча можно взять вектор $\overrightarrow{CP} = \vec{q}$, где $\vec{q} = \vec{q}(x_P - x_C, y_P - y_C, z_P - z_C)$. В результате будем иметь

$$\begin{cases} x_S = x_C + (x_P - x_C)t_S \\ y_S = y_C + (y_P - y_C)t_S, \\ z_S = z_C + (z_P - z_C)t_S \end{cases} \quad (10.1.9_1)$$

где

$$t_S = -\frac{Ax_C + By_C + Cz_C + D}{A(x_P - x_C) + B(y_P - y_C) + C(z_P - z_C)} \quad (10.1.9_2)$$

Рассмотрим случай, когда картинная плоскость S параллельна плоскости XZ и находится от нее на расстоянии $d = |y_0|$.

Как было показано выше, в рассматриваемом случае вектор нормали к плоскости $\vec{N} = \vec{N}(A, B, C) = \vec{N}(0, B, 0)$ и уравнение плоскости S имеет вид (10.1.2_3)

$$y - y_0 = 0 \quad (B = 1 \text{ и } D = -y_0)$$

Тогда из (10.1.9_2) следует, что

$$t_S = -\frac{y_C - y_0}{y_P - y_C} \quad (10.1.9_3)$$

и выражения (10.1.9_1) для координат проекции приобретают вид

$$\begin{cases} x_S = x_C + (x_P - x_C)t_S, \\ y_S = y_C - (y_P - y_C) \frac{y_C - y_0}{y_P - y_C} = y_0, \\ z_S = z_C + (z_P - z_C)t_S. \end{cases} \quad (10.1.9_4)$$

Пусть для рассматриваемой картинной плоскости центр проектирования находится на оси Y . Тогда $C = C(0, y_C, 0)$ и выражения (10.1.9_4) для координат

проекций примут простой вид

$$\begin{cases} x_S = x_P t_S, \\ y_S = y_0, \\ z_S = z_P t_S. \end{cases} \quad (10.1.9_5)$$

Представим выражение (10.1.9_3) для t_S в виде

$$t_S = -\frac{y_C - y_0}{y_P - y_C} = \frac{1 - \frac{y_0}{y_C}}{1 - \frac{y_P}{y_C}} \quad (10.1.9_6)$$

Из (10.1.9_6) видно, что при $y_C \rightarrow \infty$ значение $t_S \rightarrow 1$. Тогда из (10.1.9_5) следует, что $x_S \rightarrow x_P$ и $z_S \rightarrow z_P$. Это означает, что проектирующий луч становится параллельным оси Y и перспективная проекция переходит в параллельную.

В качестве примера перспективной проекции рассмотрим перспективную проекцию куба на картинную плоскость S , определенную уравнением $Ax + By + Cz + D = 0$ (рис. 12).

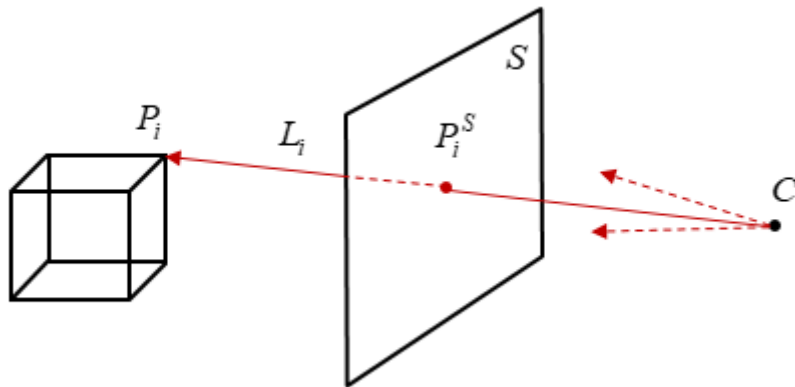


Рис. 12

Пусть, как и ранее, $P_i = P_i(x_i, y_i, z_i)$, $i = 0, 1, \dots, 7$ – вершина куба с номером i . Тогда направляющий вектор проектирующего луча для i -ой вершины будет иметь вид $\vec{q}_i = \vec{q}_i(x_i - x_C, y_i - y_C, z_i - z_C)$.

Координаты проекций P_S^i вычисляются согласно (10.1.9_1)

$$\begin{cases} x_S^i = x_C + (x_i - x_C)t_S^i \\ y_S^i = y_C + (y_i - y_C)t_S^i, \\ z_S^i = z_C + (z_i - z_C)t_S^i \end{cases} \quad (10.1.10)$$

где

$$t_S^i = -\frac{Ax_C + By_C + Cz_C + D}{A(x_i - x_C) + B(y_i - y_C) + C(z_i - z_C)} \quad (10.1.11)$$

Если картинная плоскость S параллельна плоскости XZ и находится от нее на расстоянии $d = |y_0|$ (рис. 13), то из (10.1.9_3) и (10.1.9_4) получаем

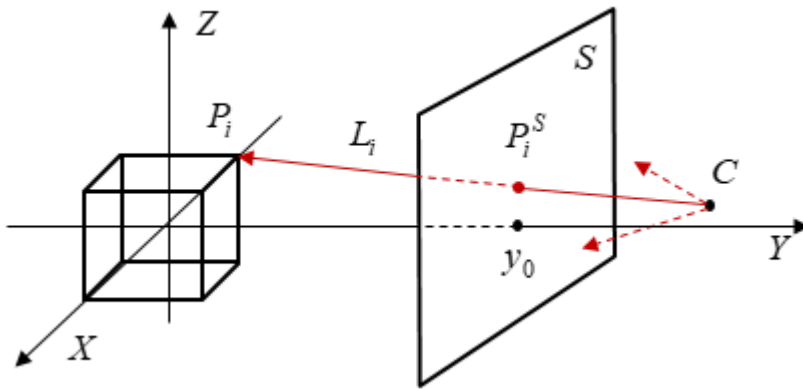


Рис.13

$$t_S^i = -\frac{y_C - y_0}{y_i - y_C} \quad (10.1.12)$$

и

$$\begin{cases} x_S^i = x_C + (x_i - x_C)t_S^i, \\ y_S^i = y_0, \\ z_S^i = z_C + (z_i - z_C)t_S^i. \end{cases} \quad (10.1.13)$$

Если в рассматриваемом случае центр проектирования находится на оси Y

(рис. 14). Тогда $C = C(0, y_C, 0)$ и выражения (10.1.13) для координат проекций примут простой вид

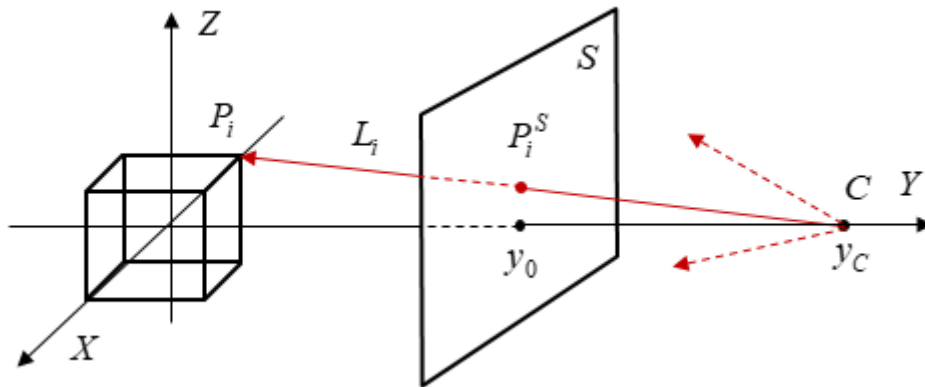


Рис.14

$$\begin{cases} x_S^i = x_i t_S^i, \\ y_S^i = y_0, \\ z_S^i = z_i t_S^i. \end{cases} \quad (10.1.14)$$

10.2. Видовая система координат

Для изображения объекта на экране его мировые координаты необходимо преобразовать (пересчитать) в другую систему координат, которая связана с точкой наблюдения. Эта система координат называется *видовой системой координат* и является левосторонней.

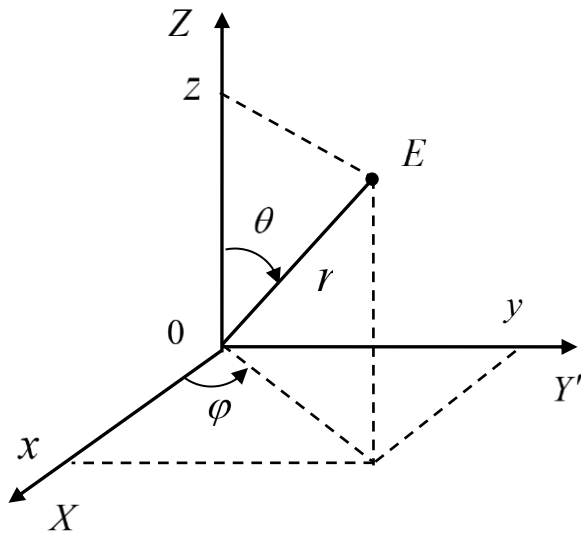


Рис. 10.2

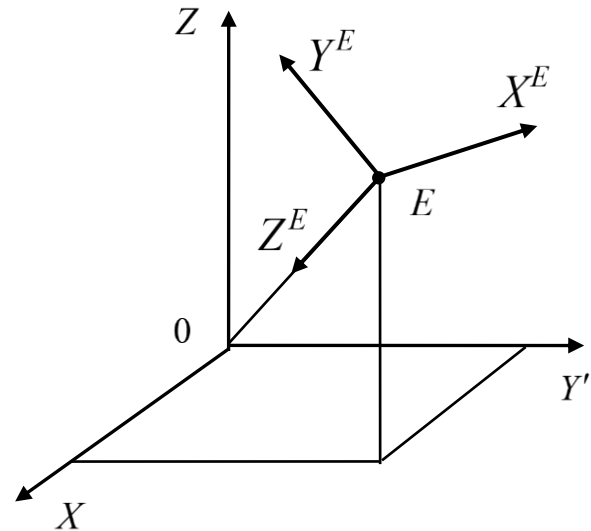


Рис. 10.3

Рассмотрим правую систему мировых координат XYZ (рис. 10.2) и зададим в ней точку наблюдения $E(x, y, z)$, где

$$\begin{cases} x = r \sin \theta \cos \varphi \\ y = r \sin \theta \sin \varphi \\ z = r \cos \theta \end{cases} \quad (10.1)$$

Система видовых координат $X^E Y^E Z^E$ показана на рис. 10.3.

Преобразование мировых координат в видовые можно представить в виде

$$V^E = AV, \quad (10.2)$$

где $V = (x, y, z, 1)^T$ – координаты некоторой точки в мировой системе координат, $V^E = (x^E, y^E, z^E, 1)^T$ – координаты этой же точки в видовой системе координат, A – матрица преобразования.

Получим выражение для матрицы A , выполняя переход от мировой системы координат к видовой путем последовательности элементарных преобразований[4].

Сместим начало системы координат XYZ (рис.10.2) в точку E (рис. 10.4), переход $XYZ \rightarrow X^1 Y^1 Z^1$.

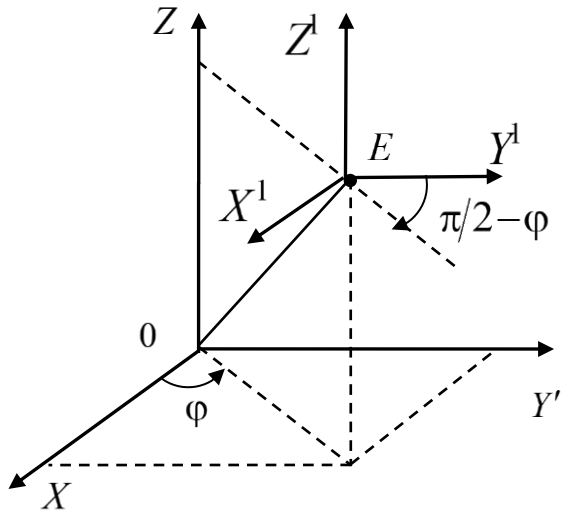


Рис. 10.4

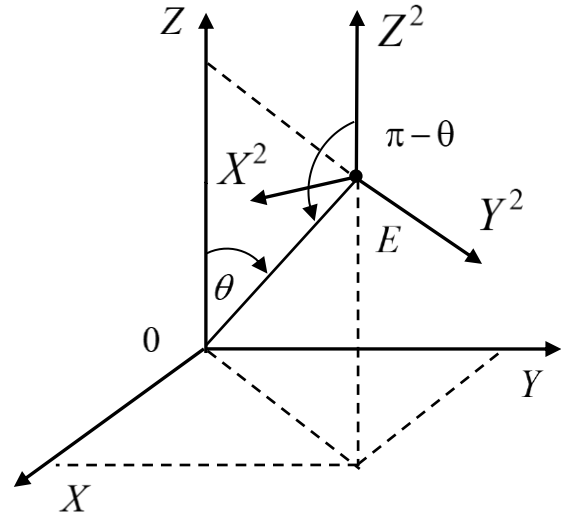


Рис. 10.5

Преобразование координат имеет вид

$$V^1 = T^s V, \quad (10.3)$$

где

$$T^s = \begin{pmatrix} 1 & 0 & 0 & -x \\ 0 & 1 & 0 & -y \\ 0 & 0 & 1 & -z \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad (10.4)$$

где $V^1 = (x^1, y^1, z^1, 1)^T$.

Повернем систему координат $\pi/2 - \varphi$ (рис. 10.4) вокруг оси Z^1 на угол $\frac{\pi}{2} - \varphi$, переход $\pi/2 - \varphi \rightarrow X^2 Y^2 Z^2$ (рис. 10.5).

Преобразование координат имеет вид

$$V^2 = R_{Z^1}^s V^1, \quad (10.5)$$

где $V^2 = (x^2, y^2, z^2, 1)^T$.

Поскольку поворот выполняется по часовой стрелке (в отрицательном

направлении), то в матрице преобразования $R_{Z_1}^s$ (9.33) следует использовать аргумент $-\left(\frac{\pi}{2} - \varphi\right)$. В результате матрица преобразования примет вид

$$R_{Z_1}^s = \begin{pmatrix} \cos\left(\frac{\pi}{2} - \varphi\right) & -\sin\left(\frac{\pi}{2} - \varphi\right) & 0 & 0 \\ \sin\left(\frac{\pi}{2} - \varphi\right) & \cos\left(\frac{\pi}{2} - \varphi\right) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} \sin\varphi & -\cos\varphi & 0 & 0 \\ \cos\varphi & \sin\varphi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (10.6)$$

Повернем систему координат $X^2Y^2Z^2$ (рис. 10.5) вокруг координатной оси X^2 на угол $\pi - \theta$, переход $X^2Y^2Z^2 \rightarrow X^3Y^3Z^3$ (рис. 10.6)

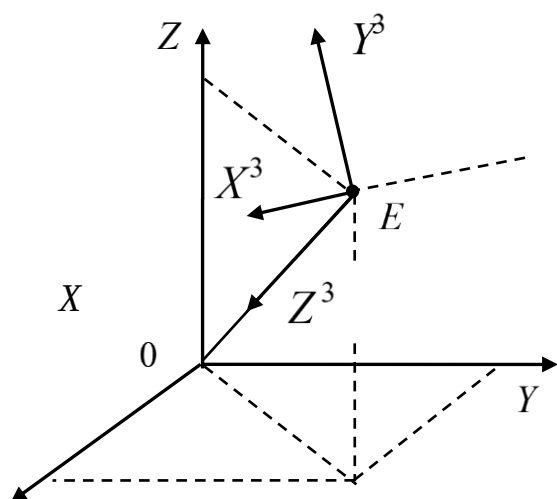


Рис. 10.6

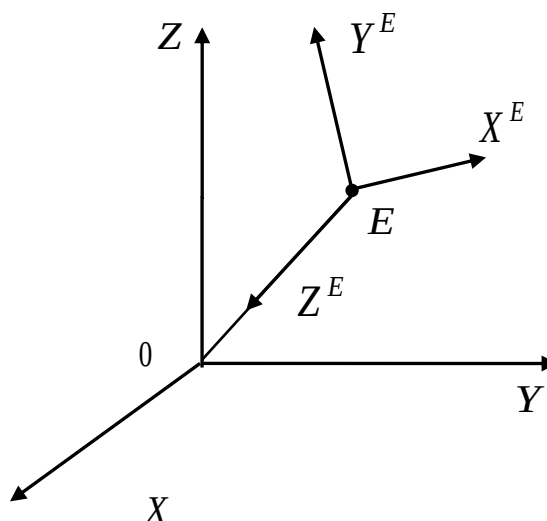


Рис. 10.7

Преобразование координат имеет вид

$$V^3 = R_{X^2}^s V^2, \quad (10.7)$$

где $V^3 = (x^3, y^3, z^3, 1)^T$.

В результате матрица преобразования примет вид

$$R_{X^2}^s = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\pi - \theta) & \sin(\pi - \theta) & 0 \\ 0 & -\sin(\pi - \theta) & \cos(\pi - \theta) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -\cos\theta & \sin\theta & 0 \\ 0 & -\sin\theta & -\cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (10.8)$$

Изменим направление оси X^3 (рис. 10.6) на противоположное ($X^3 \rightarrow -X^3$), переход $X^3 Y^3 Z^3 \rightarrow X^E Y^E Z^E$ (рис. 10.7).

Преобразование координат имеет вид

$$V^E = MV^3, \quad (10.9)$$

где

$$M = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (10.10)$$

– матрица преобразования для перехода ($X \rightarrow -X$).

Выражение (10.9) с учетом (10.3), (10.5), (10.7) и (10.10) можно представить в виде

$$V^E = MR_{X^2}^s R_{Z^1}^s T^s V = A^E V = A^E(r, \varphi, \theta) V, \quad (10.11)$$

где

$$A^E = A^E(r, \varphi, \theta) = MR_{X^2}^s R_{Z^1}^s T^s - \quad (10.12)$$

матрица преобразования координат объекта при переходе от мировой системы координат XYZ к видовой системе координат $X^E Y^E Z^E$.

Подставляя в (10.12) выражения для матриц (10.4), (10.6), (10.8) и (10.10), получим

$$A^E = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -\cos\theta & \sin\theta & 0 \\ 0 & -\sin\theta & -\cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \sin\varphi & -\cos\varphi & 0 & 0 \\ \cos\varphi & \sin\varphi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \times \\ \times \begin{pmatrix} 1 & 0 & 0 & -x \\ 0 & 1 & 0 & -y \\ 0 & 0 & 1 & -z \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} -\sin\varphi & \cos\varphi & 0 & 0 \\ -\cos\theta\cos\varphi & -\cos\theta\sin\varphi & \sin\theta & 0 \\ -\sin\theta\cos\varphi & -\sin\theta\sin\varphi & -\cos\theta & r \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (10.13)$$

При получении (10.13) учтено, что элементы матрицы T^s x , y и z определяются выражениями (10.1).

Таким образом, преобразование (10.2), связывающее координаты мировой и

видовой систем координат можно представить в виде

$$\begin{pmatrix} x^E \\ y^E \\ z^E \\ 1 \end{pmatrix} = \begin{pmatrix} -\sin \varphi & \cos \varphi & 0 & 0 \\ -\cos \theta \cos \varphi & -\cos \theta \sin \varphi & \sin \theta & 0 \\ -\sin \theta \cos \varphi & -\sin \theta \sin \varphi & -\cos \theta & r \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad (10.14)$$

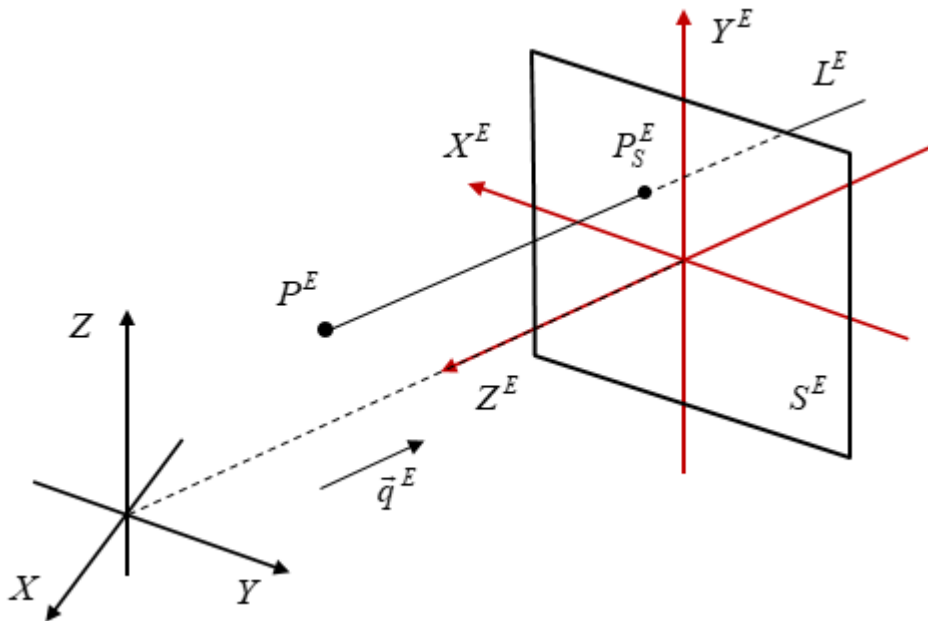
или

$$\begin{cases} x^E = -x \sin \varphi + y \cos \varphi \\ y^E = -x \cos \theta \cos \varphi - y \cos \theta \sin \varphi + z \sin \theta \\ z^E = -x \sin \theta \cos \varphi - y \sin \theta \sin \varphi - z \cos \theta + r \end{cases} \quad (10.15)$$

Параллельное и перспективное проектирование в видовой системе координат.

В видовой системе координат $X^E Y^E Z^E$ картинной плоскостью для построения проекций является плоскость $X^E Y^E$, которую обозначим как S^E .

Пусть в мировой СК XYZ задана точка $P = P(x_p, y_p, z_p)$. В этой же системе координат располагается камера и связанная с ней видовая СК $X^E Y^E Z^E$ (рис. 11), положение которой в СК XYZ определяется радиус – вектором $\vec{r}_V = \vec{r}_V(r_V, \varphi_V, \theta_V)$



Необходимо получить *параллельную* проекцию $P_S^E = P_S^E(x_S^E, y_S^E, z_S^E)$ точки P на картинную плоскость S^E в направлении, которое в видовой СК $X^E Y^E Z^E$ определяется вектором $\vec{q}^E = \vec{q}^E(m^E, n^E, p^E)$ (рис. 11).

Для решения сформулированной задачи перейдем от мировой СК XYZ к видовой СК $X^E Y^E Z^E$. Координаты точки P в видовой системе координат (точка P^E) будут иметь вид

$$P^E = A^E P, \quad (10.16)$$

где $P^E = P^E(x_P^E, y_P^E, z_P^E)$ и $A^E = A^E(r_V, \phi_V, \theta_V)$.

Тогда, воспользовавшись выражениями (10.1.1) и (10.1.2), получаем для координат проекции в СК $X^E Y^E Z^E$

$$t_S^E = -\frac{A^E x_P^E + B^E y_P^E + C^E z_P^E + D^E}{A^E m^E + B^E n^E + C^E p^E} \quad (10.17)$$

и

$$\begin{cases} x_S^E = x_P^E + m^E t_S^E \\ y_S^E = y_P^E + n^E t_S^E \\ z_S^E = z_P^E + p^E t_S^E \end{cases} \quad (10.18)$$

Так как в СК $X^E Y^E Z^E$ уравнение картинной плоскости S^E имеет вид $z^E = 0$ ($A^E = 0, B^E = 0, C^E = 1, D^E = 0$), то выражения (10.17) и (10.18) приобретают вид

$$t_S^E = -\frac{z_P^E}{p^E} \quad (10.19)$$

и

$$\begin{cases} x_S^E = x_P^E - m^E \frac{z_P^E}{p^E} \\ y_S^E = y_P^E - n^E \frac{z_P^E}{p^E} \\ z_S^E = z_P^E - p^E \frac{z_P^E}{p^E} = 0 \end{cases} . \quad (10.20)$$

Для аксанометрической проекции на картинную плоскость S^E ($\vec{q}^E \perp S^E$), когда $\vec{q}^E = \vec{q}^E(m^E, n^E, p^E) = \vec{q}^E(0, 0, p^E)$, из (10.20) получаем

$$\begin{cases} x_S^E = x_P^E \\ y_S^E = y_P^E \\ z_S^E = 0 \end{cases} , \quad (10.21)$$

Таким образом, как следует из (10.21), для получения аксанометрической проекции точки P , заданной в мировой СК XYZ , на картинную плоскость S^E видовой СК $X^E Y^E Z^E$, необходимо пересчитать ее координаты из мировой СК в видовую $(x_P, y_P, z_P) \rightarrow (x_P^E, y_P^E, z_P^E)$. Пара координат (x_P^E, y_P^E) и будет *аксанометрической проекцией* точки P на картинную плоскость S^E .

Рассмотрим теперь построение *перспективной* проекции точки P^E на картинную плоскость S^E в видовой системе координат $X^E Y^E Z^E$.

Будем рассматривать случай, когда точка схода лучей находится на координатной оси Z^E (рис. 12), т.е. $C^E = C^E(x_C^E, y_C^E, z_C^E) = C^E(0, 0, z_C^E)$. При этом надо иметь в виду следующее. Как было указано выше, для перспективной проекции точки P^E и C^E должны находиться в разных полупространствах относительно картинной плоскости S^E (рис. 12). Начало видовой СК $X^E Y^E Z^E$ в мировой СК XYZ выбирается таким образом, что для наблюдаемой точки P^E ее координата $z_P^E > 0$. Отсюда следует, что точка схода лучей C^E выбирается так, что $z_C^E < 0$.

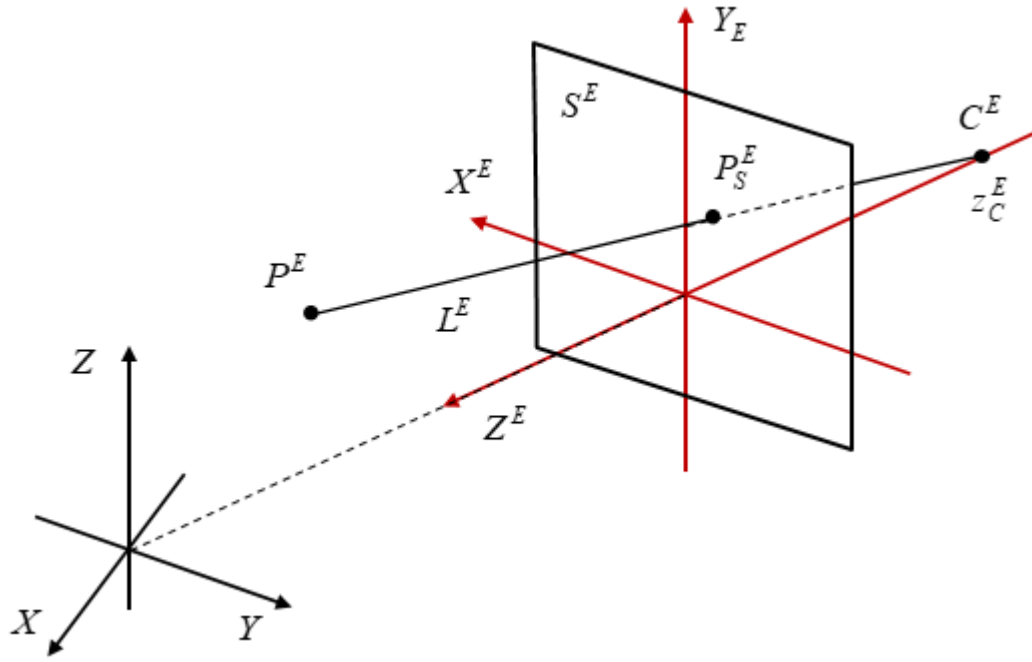


Рис. 12

Поскольку уравнение картинной плоскости в рассматриваемом случае имеет вид $z^E = 0$ ($A^E = 0, B^E = 0, C^E = 1, D^E = 0$), то из (10.1.9_2) и (10.1.9_1) находим координаты проекции

$$t_S^E = -\frac{z_C^E}{z_P^E - z_C^E} \quad (10.22)$$

и

$$\begin{cases} x_S^E = x_C^E + (x_P^E - x_C^E)t_S^E = x_P^E t_S^E \\ y_S^E = y_C^E + (y_P^E - y_C^E)t_S^E = y_P^E t_S^E \\ z_S^E = z_C^E - (z_P^E - z_C^E)\frac{z_C^E}{z_P^E - z_C^E} = 0 \end{cases}, \quad (10.23)$$

Глава 11. МОДЕЛИ ОПИСАНИЯ ПОВЕРХНОСТЕЙ

Рассмотрим возможные способы представления трехмерных объектов в системах компьютерной графики. Для описания формы поверхностей могут использоваться разнообразные методы. Рассмотрим некоторые из них.

11.1. Аналитическая модель

Под аналитической моделью будем понимать описание поверхности в виде математических формул. В компьютерной графике можно использовать много разновидностей такого описания. Например, в виде функции двух аргументов $z = f(x, y)$ или в неявной форме $F(x, y, z)$. Приведем примеры:

$z(x, y) = x^2 + y^2$ – параболоид вращения, $x^2 + y^2 + z^2 - R^2 = 0$ – сферическая поверхность радиуса R .

Часто используется параметрическая форма описания поверхности, для которой выражения, описывающие эту поверхность в трехмерной декартовой системе координат, имеют вид

$$\begin{cases} x = F_x(s, t) \\ y = F_y(s, t), \\ z = F_z(s, t) \end{cases} \quad (11.1)$$

где t и s – параметры, которые изменяются в определенных пределах, а F_x , F_y и F_z – функции, определяющие форму поверхности.

Например, параметрическое описание поверхности сферы радиуса R будет иметь вид

$$\begin{cases} x = R \sin \theta \cos \varphi \\ y = R \sin \theta \sin \varphi, \quad \theta \in [0, \pi], \quad \varphi \in [0, 2\pi], \\ z = R \cos \theta \end{cases} \quad (11.2)$$

где в качестве параметров традиционно используются переменные θ и φ .

Преимущества параметрического описания – легко описывать поверхности, которые отвечают неоднозначным функциям, замкнутые поверхности.

Для описания сложных поверхностей часто используют их приближения, составленные из элементарных фрагментов. В случае, когда эти элементарные

фрагменты строятся по единой сравнительно простой схеме, такие составные поверхности принято называть **сплайновыми поверхностями**. Упомянутые элементарные фрагменты поверхности называются сплайнами. Таким образом, сплайн – это специальная функция, более всего пригодная для аппроксимации отдельных фрагментов поверхности. Несколько сплайнов образуют модель сложной поверхности. Другими словами, сплайн – это тоже поверхность, но такая, для которой можно достаточно просто вычислять координаты ее точек. Обычно используют кубические сплайны, так как третья степень – наименьшая из степеней, позволяющих описывать любую форму, и при стыковке сплайнов можно обеспечить непрерывную первую производную – такая поверхность будет без изломов в местах стыка.

Рассмотрим одну из разновидностей сплайнов – сплайн Безье. Приведем его сначала в обобщенной форме – степени $n \times m$ [2].

$$P(s,t) = \sum_{i=0}^m \sum_{j=0}^n C_m^i s^i (1-s)^{m-i} C_n^j t^j (1-t)^{n-j} P_{ij}, \quad (11.3)$$

где P_{ij} – опорные точки-ориентиры, $0 \leq s \leq 1$, $0 \leq t \leq 1$, C_m^i и C_n^j – коэффициенты бинома Ньютона, которые вычисляются по формуле

$$C_n^k = \frac{n!}{k!(n-k)!}. \quad (11.4)$$

Кубический сплайн Безье соответствует значениям $m = 3$, $n = 3$. Для его определения нужны 16 точек – ориентиров P_{ij} (рис. 11.1). Коэффициенты C_m^i и C_n^j равны соответственно 1, 3, 3, 1 при $i, j = 0, 1, 2, 3$.

Характеризуя аналитическую модель в целом, можно сказать, что эта модель наиболее пригодна для многих операций анализа поверхностей. С позиций компьютерной графики можно указать такие положительные черты модели: легкая процедура расчета координат каждой точки поверхности, нормали; небольшой объем информации для описания достаточно сложных форм.

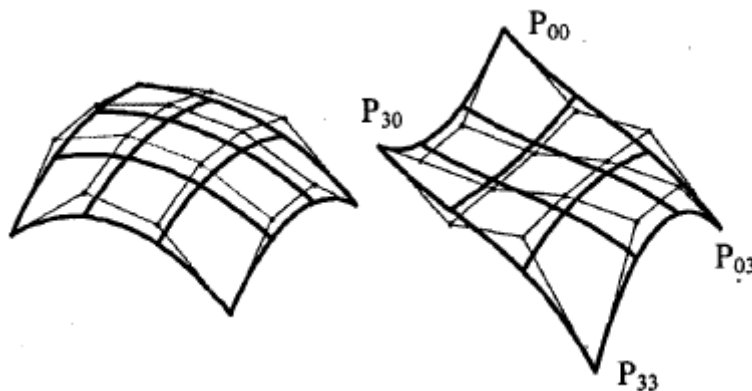


Рис. 11.1

К недостаткам относятся следующие: сложные формулы описания с использованием функций, которые медленно вычисляются на компьютере, снижают скорость выполнения операций отображения; невозможность в большинстве случаев применения данной формы описания непосредственно для построения изображения поверхности. В таких случаях поверхность отображают как многогранник, используя формулы аналитического описания для расчета координат вершин граней в процессе отображения, что уменьшает скорость сравнительно с полигональной моделью описания.

11.2. Векторная полигональная модель

Для описания пространственных объектов здесь используются следующие элементы: вершины; отрезки прямых (векторы); полилинии, полигоны, полигональные поверхности (рис. 11.2).

Элемент «вершина» (vertex) – главный элемент описания, все другие являются производными. При использовании трехмерной декартовой системы координаты вершин определяются как (x_i, y_i, z_i) . Каждый объект однозначно определяется координатами собственных вершин.

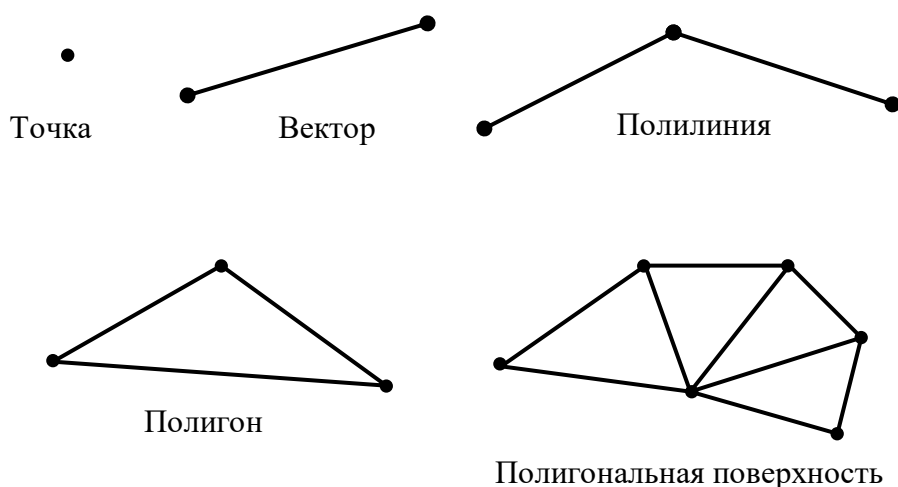


Рис. 11.2

Вершина может моделировать отдельный точечный объект, размер которого не имеет значения, а также может использоваться в качестве конечных точек для линейных объектов и полигонов. Двумя вершинами задается вектор. Несколько векторов составляют полилинию. Полилиния может моделировать отдельный линейный объект, толщина которого не учитывается, а также может представлять контур полигона. Полигон моделирует площадный объект. Один полигон

может описывать плоскую грань объемного объекта. Несколько граней составляют объемный объект в виде полигональной поверхности – многогранник или незамкнутую поверхность (в литературе часто употребляется название «полигональная сетка»).

Векторную полигональную модель можно считать наиболее распространенной в современных системах трехмерной компьютерной графики. Ее используют в системах автоматизированного проектирования, в компьютерных играх и тренажерах, в САПР, геоинформационных системах и тому подобное.

Векторная полигональная модель, как и всякая другая, имеет свои достоинства и недостатки.

Положительные черты векторной полигональной модели:

- удобство масштабирования объектов. При увеличении или уменьшении объекты выглядят более качественно, чем при растровых моделях описания. Диапазон масштабирования определяется точностью аппроксимации и разрядностью чисел для представления координат вершин;
- небольшой объем данных для описания простых поверхностей, которые адекватно аппроксимируются плоскими гранями;
- необходимость вычислять только координаты вершин при преобразованиях систем координат или перемещении объектов;
- аппаратная поддержка многих операций в современных графических видеосистемах, которая обуславливает достаточную скорость для анимации.

Недостатки полигональной модели:

- сложные алгоритмы визуализации для создания реалистичных изображений; сложные алгоритмы выполнения топологических операций, таких, например, как разрезы;
- аппроксимация плоскими гранями приводит к погрешности моделирования. При моделировании поверхностей, которые имеют сложную форму, обычно невозможно увеличивать количество граней из-за ограничений по быстродействию и объему памяти компьютера.

11.3.Равномерная сетка

Эта модель описывает координаты отдельных точек поверхности следующим способом (рис. 11.3). Каждому узлу сетки с индексами (i, j) приписывается значение высоты z_{ij} . Индексам (i, j) отвечают определенные значения координат (x_i, y_j) . Расстояние между узлами одинаково и равно $\Delta x = x_i - x_{i-1}$ по оси x и $\Delta y = y_j - y_{j-1}$ по оси y .

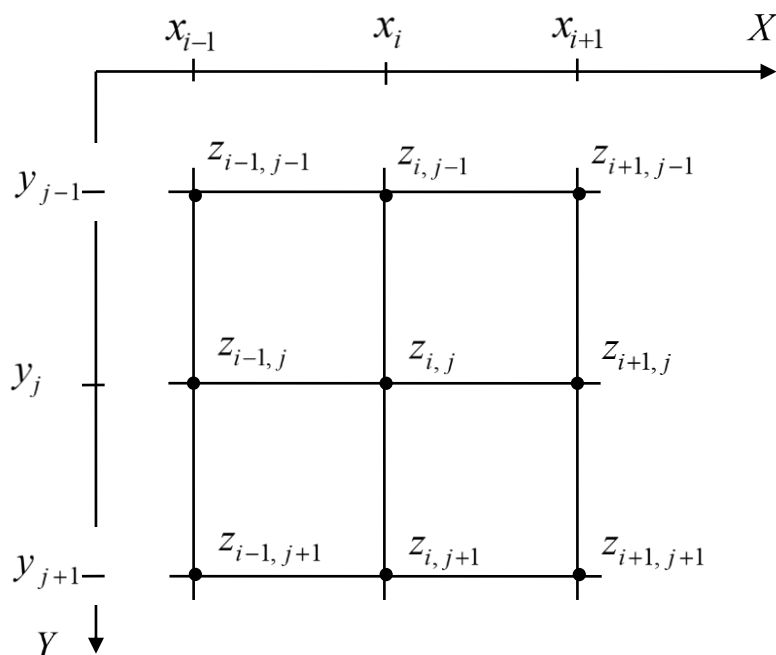
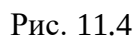


Рис. 11.3

Фактически, такая модель – двумерный массив, растр, матрица, каждый элемент которой сохраняет значение высоты.

Не каждая поверхность может быть представлена этой моделью. Если в каждом узле записывается только одно значение высоты, то это означает, что поверхность описывается однозначной функцией $z = f(x, y)$. Иначе говоря, это такая поверхность, которую любая вертикаль пересекает только один раз. Не могут моделироваться также вертикальные грани. Необходимо заметить, что для сетки не обязательно использовать только декартовы координаты. Например, для того чтобы описать поверхность шара однозначной функцией, можно использовать полярные координаты. Равномерная сетка часто используется для описания рельефа земной поверхности.

Рассмотрим, как можно вычислить значения высоты для любой точки внутри границ сетки. Пусть ее координаты равны $(\tilde{x}, \tilde{y})$. Надо найти соответствующее значение $\tilde{z}$. Решением такой задачи является интерполяция значений координат z ближайших узлов (рис. 11.4).


$$i = \left[\frac{\tilde{x} - x_0}{\Delta x} \right], \quad (11.5)$$

$$j = \left[\frac{\tilde{y} - y_0}{\Delta y} \right], \quad (11.6)$$

Используя линейную интерполяцию по оси X вдоль прямой $Y = y_i$, вычислим значение Z в точке A (значение Z_a). Для этой цели запишем уравнение прямой, проходящей через точки (x_i, z_{ij}) и

$$\frac{X - X_i}{X_{i+1} - X_i} = \frac{Z - Z_{ij}}{Z_{i+1,j} - Z_{ij}} \quad (11.7)$$

$$z = z_{ij} + \frac{z_{i+1,j} - z_{ij}}{\Delta x} (x - x_i). \quad (11.8)$$
$$z_a = z_{ij} + \frac{z_{i+1,j} - z_{ij}}{\Delta x} (\tilde{x} - x_i). \quad (11.9)$$

Совершенно аналогично, используя линейную интерполяцию по оси X вдоль прямой $y = y_{i+1}$, вычислим значение z в точке b (значение (z_b))

$$z_b = z_{i,j+1} + \frac{z_{i+1,j+1} - z_{i,j+1}}{\Delta x} (\tilde{x} - x_i) \quad (11.10)$$

Используя теперь линейную интерполяцию по оси Y вдоль прямой $x = \tilde{x}$, вычислим значение z в точке $(\tilde{x}, \tilde{y})$. Для этой цели запишем уравнение прямой, проходящей через точки (y_j, z_a) и (y_{j+1}, z_b)

$$\frac{y - y_j}{y_{j+1} - y_j} = \frac{z - z_a}{z_b - z_a} \quad (11.11)$$

или

$$z = z_a + \frac{z_b - z_a}{\Delta y} (y - y_j). \quad (11.12)$$

Подставляя в (11.12) значение $y = \tilde{y}$, получаем

$$\tilde{z} = z_a + \frac{z_b - z_a}{\Delta y} (\tilde{y} - y_j). \quad (11.13)$$

Отметим некоторые положительные черты и недостатки равномерной сетки. Положительные черты равномерной сетки:

- простота описания поверхностей;
- возможность быстро узнать высоту любой точки поверхности простой интерполяцией.

Недостатки равномерной сетки:

- поверхности, которые соответствуют неоднозначной функции высоты в узлах сетки, не могут моделироваться;
- для описания сложных поверхностей необходимо большое количество узлов, которое может быть ограничено объемом памяти компьютера;
- описание отдельных типов поверхностей может быть сложнее, чем в других моделях. Например, многогранная поверхность требует избыточный объем данных для описания по сравнению с полигональной моделью.

11.4. Линии уровня

При визуализации данных, полученных экспериментальным или расчетным путем, часто встречается задача построения линий уровня на поверхностях.

Пусть функция $z = f(x, y)$ определяет некоторую поверхность в декартовой

системе координат XYZ . Тогда, геометрическое место точек (x, y) , удовлетворяющее условию $z_0 = f(x, y)$, где $z_0 = \text{const}$, называется линией уровня. Линия уровня представляет собой результат сечения поверхности $z = f(x, y)$ плоскостью $z_0 = \text{const}$.

Рассмотрим пример. Пусть поверхность определяется уравнением $z(x, y) = x^2 + y^2$ (рис. 11.5).

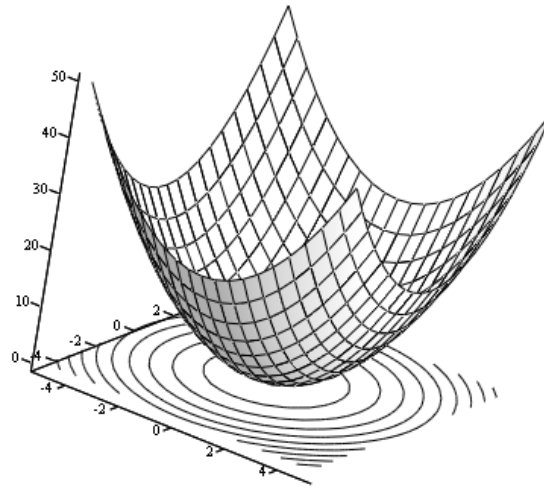


Рис. 11.5

Зададимся множеством плоскостей $z_i = \text{const}$, $i = 1, 2, \dots$, $z_i \geq 0$. Теперь можно записать уравнение для линий уровня

$$x^2 + y^2 = z_i. \quad (11.14)$$

Как видно из (11.14), линии уровня для рассматриваемой поверхности представляют собой окружности радиуса $\sqrt{z_i}$ с центром в начале координат (рис. 11.5).

В частности, линии уровня знакомы нам по географическим картам, на которых они служат для обозначения высоты местности над уровнем моря.

Рассмотрим общий подход к построению линий уровня.

Пусть поверхность $z = f(x, y)$ задана массивом своих значений $z_{ij} = f(x_i, y_j)$, рассчитанных на сетке $x = \{x_1, x_2, \dots, x_n\}$, $y = \{y_1, y_2, \dots, y_m\}$ (рис. 11.6).

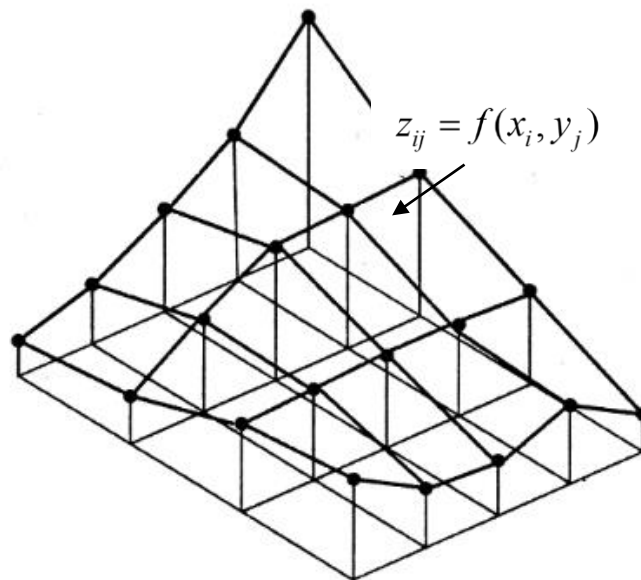


Рис. 11.6

Тогда процедура построения линий уровня заключается в следующем.

1. Выполняется триангуляция поверхности, каждая ячейка сетки разбивается на два треугольника (рис. 11.7).
2. Для каждого треугольника находится пересечение с плоскостью $z_0 = \text{const}$ (рис. 11.8).

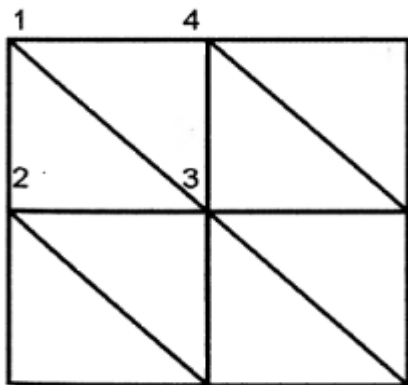


Рис. 11.7

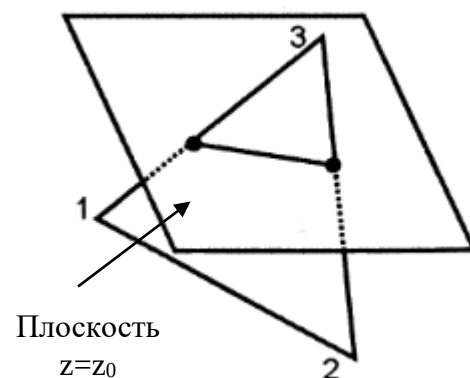


Рис.11.8

Этот метод достаточно прост и позволяет получить хорошее изображение линии уровня. Надо отметить, что линия уровня может быть с разрывами.

Глава 12. ВИЗУАЛИЗАЦИЯ ОБЪЕМНЫХ ИЗОБРАЖЕНИЙ

Любой объект, в том числе и объемный, может быть изображен различными способами. В одном случае необходимо показать внутреннюю структуру объектов, в другом – внешнюю форму объекта, в третьем – имитировать реальную действительность. Условно способы визуализации объектов можно разделить по характеру изображений и по степени сложности соответствующих алгоритмов.

Рассмотрим следующие уровни визуализации:

1. каркасная («проволочная») модель;
2. показ поверхностей в виде многогранников с плоскими гранями или сплайнов с удалением невидимых точек;
3. то же, что и для второго уровня, плюс сложное закрашивание объектов для имитации отражения света, затенения, прозрачности, использование текстур.

12.1. Каркасная визуализация

Каркас обычно состоит из отрезков прямых линий (соответствует многограннику), хотя можно строить каркас и на основе кривых, в частности сплайновых кривых Безье. Все ребра, показанные в окне вывода, видны – как ближние, так и дальние (рис. 12.1–12.2).

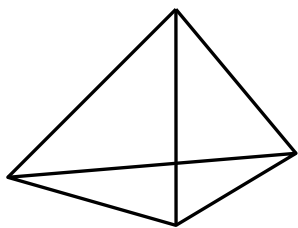


Рис. 12.1

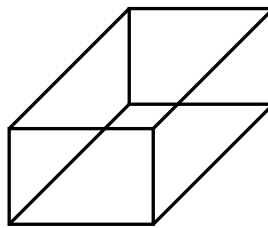


Рис. 12.2

Для построения каркасного изображения надо знать координаты всех вершин в мировой системе координат. Потом преобразовать координаты каждой вершины в экранные координаты в соответствии с выбранной проекцией. Затем выполнить цикл вывода в плоскости экрана всех ребер как отрезков прямых (или кривых), соединяющих вершины.

12.2. Показ с удалением невидимых граней

Для построения правильного изображения трехмерных объектов необходимо уметь определять, какие части объектов (ребра, грани) будут видны при заданном проектировании, а какие будут закрыты другими гранями объектов. В качестве возможных видов проектирования традиционно рассматриваются параллельное и центральное (перспективное) проектирования.

Далее будем считать, что все объекты представлены набором выпуклых плоских граней, которые пересекаются только вдоль своих ребер.

В качестве примера на рис. 12.3 и рис. 12.4 показаны ранее изображенные фигуры, у которых удалены невидимые грани.

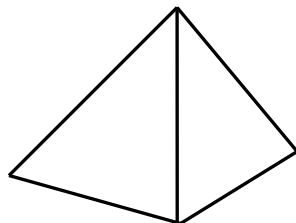


Рис. 12.3

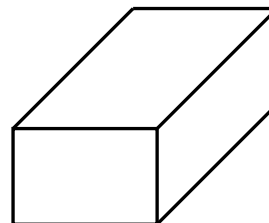


Рис. 12.4

12.2.1. Отсечение не лицевых граней

Рассмотрим многогранник, для каждой грани которого задан вектор внешней нормали (рис. 12.5). Несложно заметить, что если вектор нормали грани $\vec{n}_i$ ($i=1,2,3$) составляет с вектором $\vec{r}$, задающим направление проектирования, тупой угол, то эта грань заведомо не может быть видна. Такие грани называются **не лицевыми**. В случае, когда соответствующий угол является острым, грань называется **лицевой**.

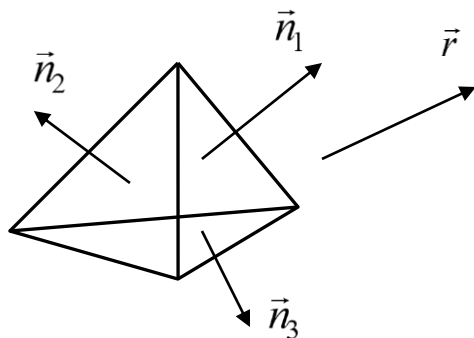


Рис. 12.5

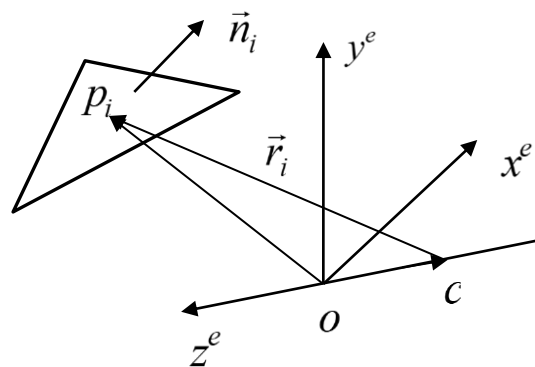


Рис. 12.6

Пусть φ – угол между вектором нормали $\vec{n}_i$ к некоторой грани и направлением проектирования $\vec{r}$, которые заданы в мировой системе координат.

Так как для случая **параллельного** проектирования направление проектирования не зависит от грани (значение вектора $\vec{r}$ не зависит от номера грани i), то условие отбора лицевой грани с номером i можно записать в виде

$$\cos \varphi_i = \frac{\vec{n}_i \cdot \vec{r}}{|\vec{n}_i| |\vec{r}|} \geq 0, \quad (12.1)$$

что говорит о том, что угол φ является острым и его значение принадлежит отрезку $[0, \pi/2]$. Поскольку знаменатель в (12.1) всегда положителен, то условие (12.1) можно заменить на более простое

$$\vec{n}_i \cdot \vec{r} \geq 0. \quad (12.2)$$

Рассмотрим случай **центрального** проектирования (рис. 12.6). Пусть для этого случая в видовой системе координат заданы: p_i – произвольная точка на некоторой грани с номером i , $\vec{n}_i$ – вектор внешней нормали к этой грани, c – точка, определяющая центр проектирования, $\vec{r}_i$ – направление проектирования для точки p_i , φ_i – угол между вектором нормали $\vec{n}_i$ и направлением проектирования $\vec{r}_i$. В отличие от случая параллельного проектирования значение вектора $\vec{r}_i$ зависит от положения точки p_i на выбранной грани.

$$\vec{r}_i = \overrightarrow{op_i} - \overrightarrow{oc} \quad (12.3)$$

Теперь можно записать условие отбора лицевой грани

$$\cos \varphi_i = \frac{\vec{n}_i \cdot (-\vec{r}_i)}{|\vec{n}_i| |\vec{r}_i|} \geq 0 \quad (12.4)$$

или более просто

$$\vec{n}_i \cdot (-\vec{r}_i) \geq 0 \quad (\vec{n}_i \cdot \vec{r}_i \leq 0) \quad (12.5)$$

Знак этого скалярного произведения в (12.5) не зависит от выбора точки на грани, а определяется тем, в каком полупространстве относительно плоскости, содержащей данную грань, лежит центр проектирования. Поэтому для определения того, является заданная грань лицевой или нет, достаточно взять произвольную точку на этой грани и проверить выполнение условия (12.5).

В случае, когда фигура представляет собой один выпуклый многогранник, удаление не лицевых граней полностью решает задачу удаления невидимых граней.

В общем случае предложенный подход хотя и не решает задачу полностью, но позволяет примерно вдвое сократить количество рассматриваемых граней.

12.2.2. Сортировка граней по глубине

Это означает рисование полигонов граней в порядке от самых дальних к самым близким. Этот метод не является универсальным, ибо иногда нельзя четко

различить, какая грань ближе (рис. 12.7).

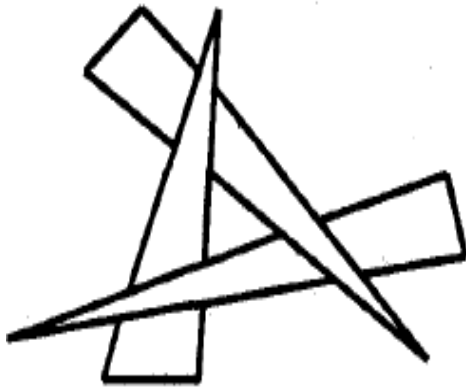


Рис. 12.7

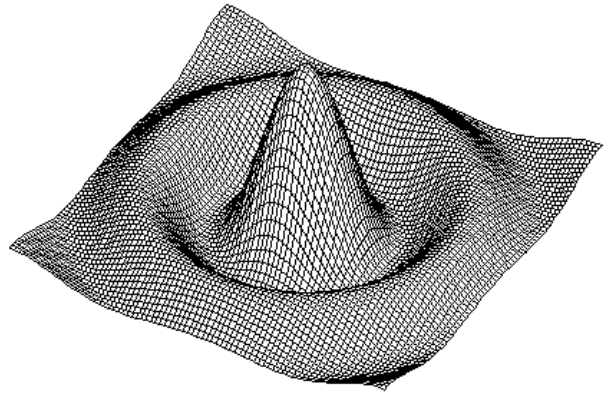


Рис. 12.8

Известны модификации этого метода, которые позволяют корректно рисовать такие грани. Метод сортировки по глубине эффективен для показа поверхностей (рис.12.8), заданных функциями $z = f(x, y)$.

В качестве примера рассмотрим возможный вариант определения дальней грани в зависимости от положения наблюдателя для случая построения графика поверхности, определяемой функцией $z = f(x, y)$.

Пусть рассчитана матрица значений функции $f(x, y)$.

$$M_{ij} = z_{ij} = f(x_i, y_j), \quad (12.6)$$

где

$$x_i = x_L + i\Delta x, \quad i = 0, 1, 2, \dots, N_x, \quad N_x = \frac{x_H - x_L}{\Delta x}, \quad (12.7)$$

$$y_j = y_L + j\Delta y, \quad j = 0, 1, 2, \dots, N_y, \quad N_y = \frac{y_H - y_L}{\Delta y}, \quad (12.8)$$

x_L, x_H – соответственно нижняя и верхняя границы для изменения аргумента x , а y_L, y_H – нижняя и верхняя границы для изменения аргумента y , Δx и Δy – интервалы дискретизации для аргументов x и y .

Далее, пусть $P(x_p, y_p, z_p)$ – точка в системе координат XYZ , где расположен наблюдатель, а $P_{xy}(x_p, y_p)$ – проекция точки P на плоскость XY (рис. 1–2).

На приведенных рисунках матрица M представлена прямоугольником $ACBD$.

Через точки A и B проведем прямую AB , уравнение которой будет иметь вид

$$\frac{x - x_L}{x_H - x_L} = \frac{y - y_L}{y_H - y_L}$$

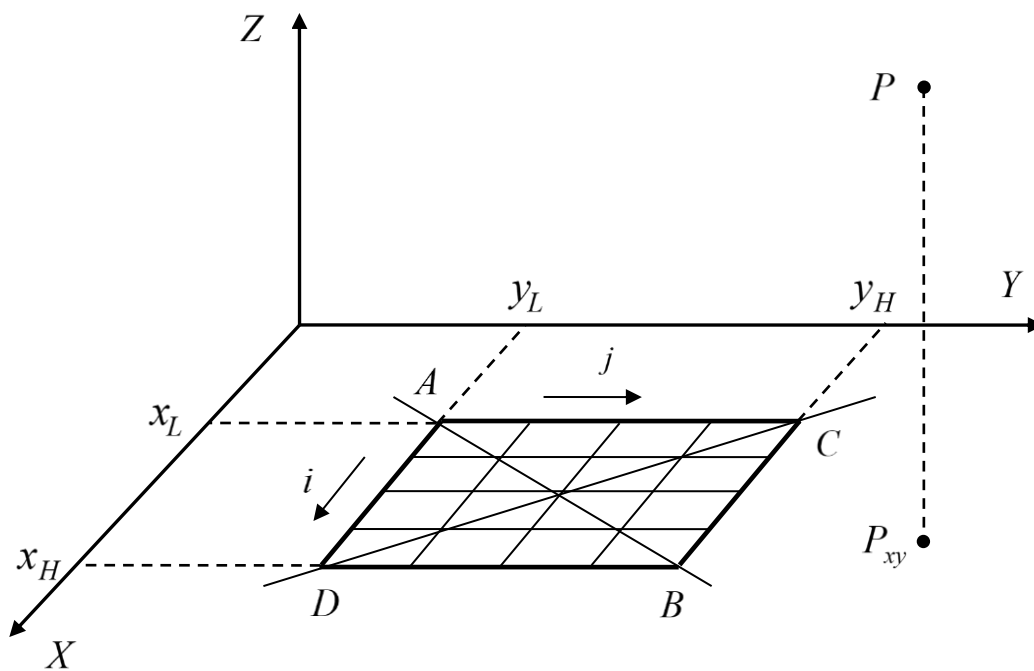


Рис. 12.9

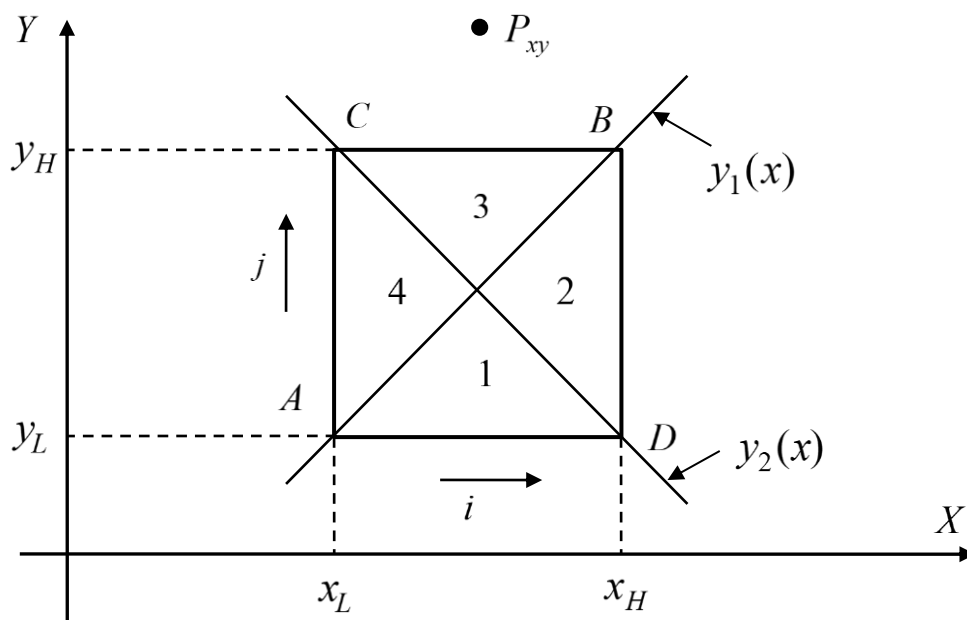


Рис. 12.10

ИЛИ

$$y = y_L + \frac{y_H - y_L}{x_H - x_L} (x - x_L) = y_1(x) \quad (12.9)$$

Аналогично через точки C и D проведем прямую CD , уравнение которой будет иметь вид

$$\frac{x - x_L}{x_H - x_L} = \frac{y - y_L}{y_L - y_H}$$

или

$$y = y_L - \frac{y_H - y_L}{x_H - x_L} (x - x_L) = y_2(x). \quad (12.10)$$

Прямые AB и CD делят плоскость XU на четыре области: 1, 2, 3, 4 (рис.12.10). Из рис. 12.10 видно, что дальняя грань определяется по отношению к расположению точки $P_{xy}(x_p, y_p)$ на плоскости XU .

Если $y_p \geq y_1(x_p)$ и $y_p \geq y_2(x_p)$, то точка P_{xy} находится в области 3 и дальней является грань AD .

Если $y_2(x_p) < y_p < y_1(x_p)$, то точка P_{xy} находится в области 2 и дальней является грань AC .

Если $y_p \leq y_1(x_p)$ и $y_p \leq y_2(x_p)$, то точка P_{xy} находится в области 1 и дальней является грань CB .

Если $y_1(x_p) < y_p < y_2(x_p)$, то точка P_{xy} находится в области 4 и дальней является грань BD .

12.2.3.Метод плавающего горизонта

В отличие от предыдущего метода при методе плавающего горизонта грани выводятся в последовательности от ближайших к самым дальним. На каждом шаге границы граней образуют две ломаные линии – верхний горизонт и нижний горизонт. Во время вывода каждой новой грани рисуется только то, что выше верхнего горизонта, и то, что ниже нижнего горизонта. Соответственно, каждая новая грань поднимает верхний и опускает нижний горизонты. Этот метод также используют для показа поверхностей, которые описываются функциями $z = f(x, y)$.

12.2.4.Метод Z-буфера

Метод основывается на использовании дополнительного массива, буфера в памяти, в котором сохраняются координаты z для каждого пиксела раstra. Координата z отвечает расстоянию точек пространственных объектов до плоскости проецирования.

Рассмотрим алгоритм рисования объектов согласно этому методу. Пусть, чем ближе точка в пространстве к плоскости проецирования, тем больше значение Z . Тогда сначала Z -буфер заполняется минимальными значениями. Потом начинается вывод всех объектов. Причем не имеет значение порядок вывода объектов. Для каждого объекта выводятся все его пикселы в любом порядке. Во время вывода каждого пиксела по его координатам (X, Y) находится текущее значение Z в Z -буфере. Если рисуемый пиксел имеет большее значение Z , чем значение в Z -буфере, то это означает, что эта точка ближе к объекту. В этом случае пиксел действительно рисуется, а его Z -координата записывается в Z -буфер. Таким образом, после рисования всех пикселов всех объектов растровое изображение будет состоять из пикселов, которые соответствуют точкам объектов с самыми большими значениями координат Z , то есть видимые точки – ближе всех к нам.

Этот метод прост и эффективен благодаря тому, что не нужно ни сортировать объекты, ни сортировать их точки. При рисовании объектов, которые описываются многогранниками или полигональными сетками, манипуляции со значениями Z -буфера легко совместить с выводом пикселов заполнения полигонов плоских граней.

В настоящее время метод Z -буфера используется во многих графических 3d-акселераторах, которые аппаратно реализуют этот метод. Наиболее целесообразно, когда акселератор имеет собственную память для Z -буфера, доступ к которой осуществляется быстрее, чем к оперативной памяти компьютера. Возможности аппаратной реализации используются разработчиками и пользователями компьютерной анимации, позволяя достичь большой скорости прорисовки кадров.

Глава 13. ЗАКРАШИВАНИЕ ПОВЕРХНОСТЕЙ

13.1. Модели отражения света

Рассмотрим, как можно определить цвет пикселов изображения поверхности согласно интенсивности отраженного света при учете взаимного расположения поверхности, источника света и наблюдателя.

13.1.1. Зеркальное отражение света

Угол между нормалью и падающим лучом φ равен углу между нормалью и отраженным лучом. Падающий луч, отраженный, и нормаль располагаются в одной плоскости (рис. 13.1).

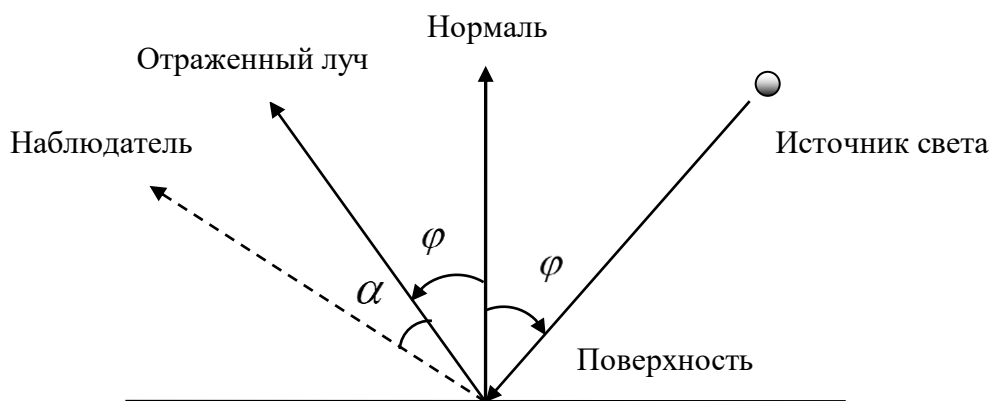


Рис. 13.1

Поверхность считается **идеально зеркальной**, если на ней отсутствуют какие-либо неровности, шероховатости. Собственный цвет у такой поверхности не наблюдается. Световая энергия падающего луча отражается только по линии отраженного луча. Какое-либо рассеяние в стороны от этой линии отсутствует. В природе, вероятно, нет идеально гладких поверхностей, поэтому полагают, что если глубина шероховатостей существенно меньше длины волны излучения, то рассеивания не наблюдается. Для видимого спектра можно принять, что глубина шероховатостей поверхности зеркала должна быть существенно меньше 0.5 мкм [2].

Если поверхность зеркала отполирована неидеально, то наблюдается зависимость интенсивности отраженного света от длины волны – чем больше длина волны, тем лучше отражение. Например, красные лучи отражаются сильнее, чем синие.

Падающий луч, попадая на слегка шероховатую поверхность реального зеркала, порождает не один отраженный луч, а несколько лучей, рассеиваемых по различным направлениям. Зона рассеивания зависит от качества полировки и может быть описана некоторым законом распределения. Как правило, форма зоны рассеивания симметрична относительно линии идеально зеркально отраженного луча. К числу простейших, но достаточно часто используемых, относится эмпирическая модель распределения Фонга, согласно которой интенсивность зеркально отраженного излучения определяется выражением

$$I_s = I_0 K_s \cos^p \alpha, \quad (13.1)$$

где I_0 – интенсивность излучения источника, K_s – коэффициент пропорциональности, α – угол отклонения от линии идеально отраженного луча (рис. 13.1), показатель p находится в диапазоне от 1 до 200 и зависит от качества полировки.

13.1.2. Диффузное отражение света

Этот вид отражения присущ **матовым** поверхностям. Матовой можно считать такую поверхность, размер шероховатостей которой уже настолько велик, что падающий луч рассеивается равномерно во все стороны. Такой тип отражения характерен, например, для гипса, песка, бумаги.

Для матовой поверхности законы отражения установлены Ламбертом. Их суть иллюстрируется рис. 13.2 и 13.3.

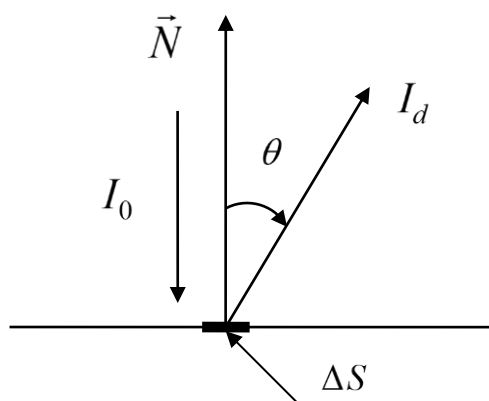


Рис. 13.2

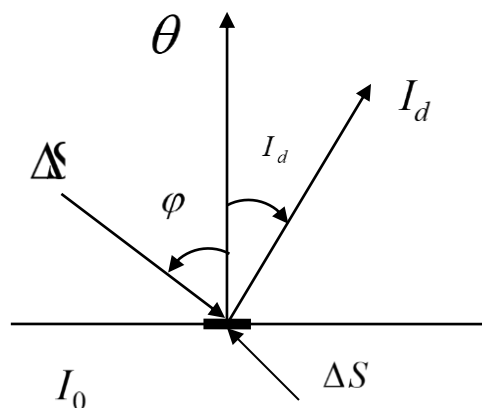


Рис. 13.3

Если световой поток интенсивностью (мощностью) I_0 падает нормально к матовой поверхности (рис. 13.2), то интенсивность вторичного излучения I_d под углом θ к нормали, проведенной к элементарной площадке ΔS , пропорционально $\cos \theta$ [2].

$$I_d = I_0 K_d \cos \theta, \quad (13.2)$$

где K_d – коэффициент, который учитывает свойства материала поверхности. Значение K_d находится в диапазоне от 0 до 1.

Физический смысл формулы (13.2) заключается в том, что в направлении, определяемом углом θ , площадь ΔS проектируется как $\Delta S \cos \theta$, при этом пропорционально $\cos \theta$ уменьшается площадь излучающей площадки и интенсивность излучения.

Если поток излучения падает под углом φ по отношению к нормали, проведенной к поверхности в месте падения (рис. 13.3), то

$$I_d = I_1 K_d \cos \theta, \text{ где } I_1 = I_0 \cos \varphi.$$

Следовательно

$$I_d = I_0 K_d \cos \varphi \cos \theta. \quad (13.3)$$

Здесь I_0 – интенсивность светового потока, которую фиксировал бы приемник, находящийся в «зените», если бы площадка облучалась бы то же с зенита.

Физический смысл формулы (13.3) состоит в том, что с увеличением φ уменьшается перехватываемый поверхностью падающий световой поток, отчего уменьшается ее освещенность и, как следствие, яркость.

Матовая поверхность имеет свой цвет. Наблюдаемый цвет матовой поверхности определяется комбинацией собственного цвета поверхности и цвета излучения источника света.

При создании реалистичных изображений следует учитывать то, что в природе, вероятно, не существует идеально зеркальных или полностью матовых поверхностей. При изображении объектов средствами компьютерной графики обычно моделируют сочетание зеркальности и диффузного рассеивания в пропорции, характерной для конкретного материала. В этом случае модель отражения записывают в виде суммы диффузной и зеркальной компонент:

$$I_{omp} = I_0[K_d \cos \varphi \cos(\varphi + \alpha) + K_s \cos^p \alpha], \quad (13.4)$$

где константы K_d и K_s , определяют отражательные свойства материала. Согласно этой формуле интенсивность отраженного света равна нулю для некоторых углов φ и α . Однако в реальных сценах обычно нет полностью затемненных объектов, следует учитывать фоновую подсветку, освещение рассеянным светом, отраженным от других объектов. В таком случае интенсивность может быть эмпирически выражена следующей формулой:

$$I_{omp} = I_a K_a + I_0[K_d \cos \varphi \cos(\varphi + \alpha) + K_s \cos^p \alpha], \quad (13.5)$$

где I_a – интенсивность рассеянного света, K_a константа.

Можно еще усовершенствовать модель отражения, если учесть то, что энергия от точечного источника света уменьшается пропорционально квадрату расстояния. Использование такого правила вызывает сложности, поэтому на практике часто реализуют модель, выражаемую эмпирической формулой[2]:

$$I_{omp} = I_a K_a + \frac{I_0}{R + k}[K_d \cos \varphi \cos(\varphi + \alpha) + K_s \cos^p \alpha], \quad (13.6)$$

где R – расстояние от центра проекции до поверхности, k константа.

Распределение световой энергии по возможным направлениям световых лучей можно отобразить с помощью векторных диаграмм, в которых длина векторов соответствует интенсивности (рис. 13.4). На приведенном рисунке показаны векторные диаграммы для различных видов отражений: а) – идеальное зеркальное, б) – неидеальное зеркальное, в) – диффузионное, г) – сумма диффузионного и зеркального.

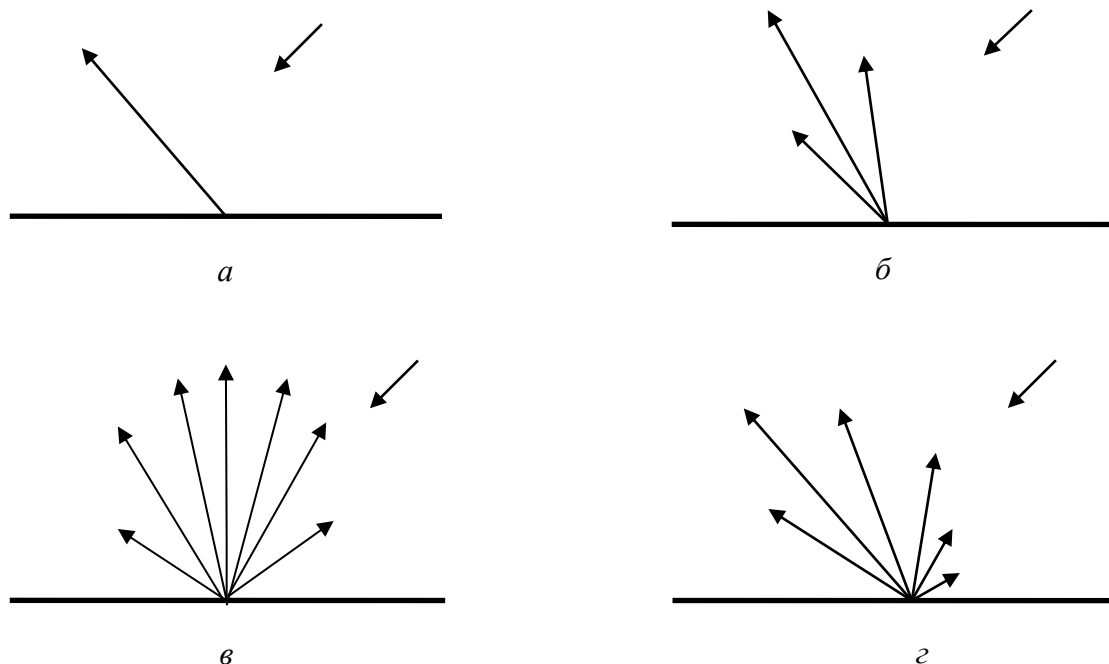


Рис. 13.4

Рассмотрим, как определить цвет закрашивания точек объектов в соответствии с выбранной моделью. Наиболее просто выполняется расчет в градациях серого цвета (например, для белого источника света и серых объектов). В данном случае интенсивность отраженного света соответствует яркости. Сложнее обстоит дело с цветными источниками света, освещающими цветные поверхности. Например, для модели RGB составляются три формулы расчета интенсивности отраженного света для различных цветовых компонент. Коэффициенты K_a и K_d различны для разных компонент – они выражают собственный цвет поверхности. Поскольку цвет отраженного зеркального луча равен цвету источника, то коэффициент K_s , будет одинаковым для всех компонент цветовой модели. Цвет источника света выражается значениями интенсивности I для соответствующих цветовых компонент.

13.2.Вычисление углов

В рассматриваемых нами моделях отражения света присутствуют такие параметры как угол падения и отражения φ , угол α – угол отклонения от линии идеально отраженного луча (рис. 13.1) и угол θ – угол под которым наблюдается результат диффузионного отражения света (рис. 13.2, рис.13.3).

Рассмотрим алгоритмы вычисления этих углов в выбранной системе координат.

Диффузионное отражение. Определим косинус угла ϑ между вектором нормали $\vec{N}$ к поверхности и некоторым направлением, определяемым вектором $\vec{S}$. Вектор $\vec{S}$ может указывать как на положение источника света, так и на положение точки наблюдения. Таким образом, угол ϑ – это угол, соответствующий углам ϕ или Θ на рис. 13.1–13.3.

Сначала рассмотрим случай, когда источник света или наблюдатель находятся на бесконечности по отношению к некоторому элементу поверхности (рис. 13.5).

Пусть заданы:

$\vec{N} = \vec{N}(N_x, N_y, N_z)$ – вектор нормали к элементу поверхности и

$\vec{S} = \vec{S}(S_x, S_y, S_z)$ – вектор, определяющий некоторое направление в пространстве.

Тогда

$$\cos \vartheta = \frac{\vec{S} \cdot \vec{N}}{|\vec{S}| \cdot |\vec{N}|} = \frac{S_x N_x + S_y N_y + S_z N_z}{\sqrt{S_x^2 + S_y^2 + S_z^2} \sqrt{N_x^2 + N_y^2 + N_z^2}}. \quad (13.7)$$

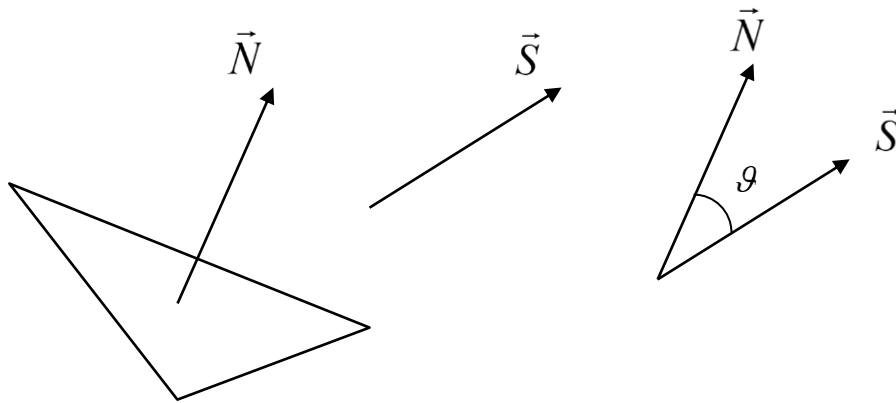


Рис. 13.5

Перейдем к рассмотрению случая, когда источник света или наблюдатель находятся на конечном расстоянии от поверхности (рис. 13.6)

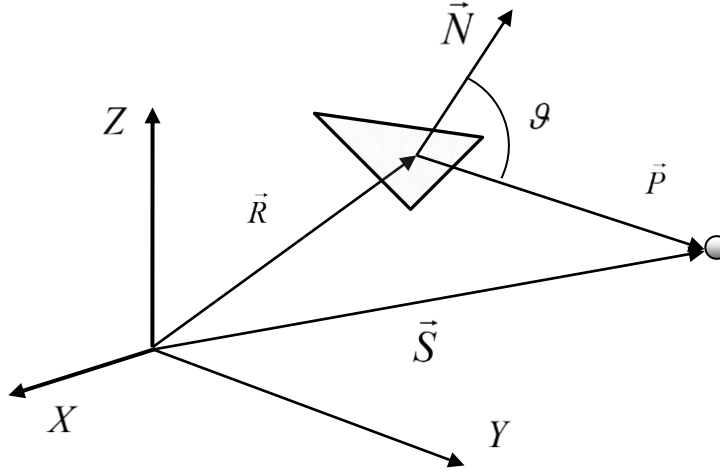


Рис. 13.6

Пусть заданы в системе координат XYZ (рис. 13.6):

$\vec{N} = \vec{N}(N_x, N_y, N_z)$ – вектор нормали к элементу поверхности,

$\vec{S} = \vec{S}(S_x, S_y, S_z)$ – радиус-вектор, определяющий положение источника света или точки наблюдения,

$\vec{R} = \vec{R}(R_x, R_y, R_z)$ – радиус-вектор, определяющий положение элемента поверхности.

Тогда

$$\cos \vartheta = \frac{\vec{P} \cdot \vec{N}}{|\vec{P}| \cdot |\vec{N}|} = \frac{P_x N_x + P_y N_y + P_z N_z}{\sqrt{P_x^2 + P_y^2 + P_z^2} \sqrt{N_x^2 + N_y^2 + N_z^2}} \quad (13.8)$$

Координаты вектора $\vec{P} = \vec{P}(P_x, P_y, P_z)$ определяются из рис. 13.6

$$\vec{P} = \vec{S} - \vec{R} \quad (13.9)$$

Таким образом

$$\vec{P} = \vec{P}(P_x, P_y, P_z) = \vec{P}(S_x - R_x, S_y - R_y, S_z - R_z) \quad (13.10)$$

С учетом (13.9) и (13.10) выражение (13.8) можно представить в виде

$$\cos \vartheta = \frac{(\vec{S} - \vec{R}) \cdot \vec{N}}{|\vec{S} - \vec{R}| \cdot |\vec{N}|} = \frac{(S_x - R_x)N_x + (S_y - R_y)N_y + (S_z - R_z)N_z}{\sqrt{(S_x - R_x)^2 + (S_y - R_y)^2 + (S_z - R_z)^2} \sqrt{N_x^2 + N_y^2 + N_z^2}} \quad (13.11)$$

Зеркальное отражение

Пусть заданы (рис. 13.7)

$\vec{N} = \vec{N}(N_x, N_y, N_z)$ – вектор нормали к элементу поверхности,
 $\vec{S} = \vec{S}(S_x, S_y, S_z)$ – вектор, определяющий направление на источник света,
 $\vec{R} = \vec{R}(R_x, R_y, R_z)$ – вектор, направление отраженного луча,
 $\vec{K} = \vec{K}(K_x, K_y, K_z)$ – вектор, определяющий направление на камеру.
 Будем полагать, что

$$|\vec{R}| = |\vec{S}|. \quad (13.12)$$

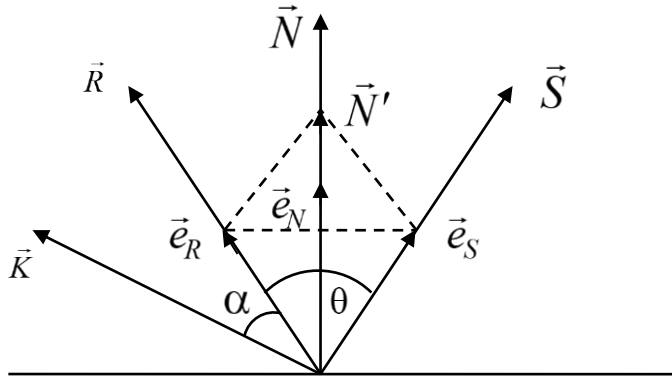


Рис. 13.7

Требуется определить косинус угла α .

На векторах $\vec{R}$, $\vec{N}$ и $\vec{S}$ построим соответствующие им единичные вектора $\vec{e}_R$, $\vec{e}_N$, $\vec{e}_S$ и выполним построения, показанные на рис. 13.7.

$$\vec{e}_R = \frac{\vec{R}}{|\vec{R}|}, \quad \vec{e}_N = \frac{\vec{N}}{|\vec{N}|}, \quad \vec{e}_S = \frac{\vec{S}}{|\vec{S}|} \quad (13.13)$$

Из рис. 13.7 получаем

$$\vec{e}_R + \vec{e}_S = \vec{N}' = 2\vec{e}_N |\vec{e}_S| \cos \theta = 2\vec{e}_N \cos \theta, \quad (13.14)$$

$$\cos \theta = \frac{\vec{e}_N \cdot \vec{e}_S}{|\vec{e}_N| |\vec{e}_S|} = \vec{e}_N \cdot \vec{e}_S, \quad (13.15)$$

где символ « $\cdot$ » – знак скалярного произведения.

Подставляя (13.15) в (13.14), получим

$$\vec{e}_R + \vec{e}_S = 2\vec{e}_N \cos \theta = 2\vec{e}_N (\vec{e}_N \cdot \vec{e}_S). \quad (13.16)$$

Отсюда

$$\vec{e}_R = 2\vec{e}_N(\vec{e}_N \cdot \vec{e}_S) - \vec{e}_S. \quad (13.17)$$

Подставляя в (13.17) выражения для единичных векторов из (13.13), получим

$$\frac{\vec{R}}{|\vec{R}|} = 2 \frac{\vec{N}}{|\vec{N}|} \left(\frac{\vec{N}}{|\vec{N}|} \cdot \frac{\vec{S}}{|\vec{S}|} \right) - \frac{\vec{S}}{|\vec{S}|} = 2 \frac{\vec{N}(\vec{N} \cdot \vec{S})}{|\vec{N}|^2 |\vec{S}|} - \frac{\vec{S}}{|\vec{S}|},$$

откуда с учётом (13.12)

$$\vec{R} = 2 \frac{\vec{N}(\vec{N} \cdot \vec{S})}{|\vec{N}|^2} - \vec{S} \quad (13.18)$$

И окончательно

$$\cos \alpha = \frac{\vec{K} \cdot \vec{R}}{|\vec{K}| |\vec{R}|}. \quad (13.19)$$

13.3. Метод Гуро

Этот метод предназначен для создания иллюзии гладкой криволинейной поверхности, описанной в виде многогранников или полигональной сетки с плоскими гранями. Если каждая плоская грань имеет один постоянный цвет, определенный с учетом отражения, то различные цвета соседних граней очень заметны, и поверхность выглядит именно как многогранник. Казалось бы, этот дефект можно замаскировать за счет увеличения количества граней при аппроксимации поверхности. Но зрение человека имеет способность подчеркивать перепады яркости на границах смежных граней – такой эффект называется **эффектом полос Маха**. Поэтому для создания иллюзии гладкости нужно намного увеличить количество граней, что приводит к существенному замедлению визуализации – чем больше граней, тем меньше скорость рисования объектов.

Метод Гуро основывается на идее закрашивания каждой плоской грани не одним цветом, а плавно изменяющимися оттенками, вычисляемыми путем интерполяции цветов примыкающих граней. Закрашивание граней по методу Гуро осуществляется в четыре этапа:

- вычисляются нормали к каждой грани;
- определяются нормали в вершинах, нормаль в вершине определяется усреднением нормалей примыкающих граней (рис. 13.8);
- на основе нормалей в вершинах вычисляются значения интенсивностей в вершинах согласно выбранной модели отражения света;
- закрашиваются полигоны граней цветом, соответствующим линейной интерполяции значений интенсивности в вершинах.

Вектор нормали в вершине а (рис. 13.8) равен

$$\vec{N}_a = \frac{\vec{N}_1 + \vec{N}_2 + \vec{N}_3}{3} \quad (13.20)$$

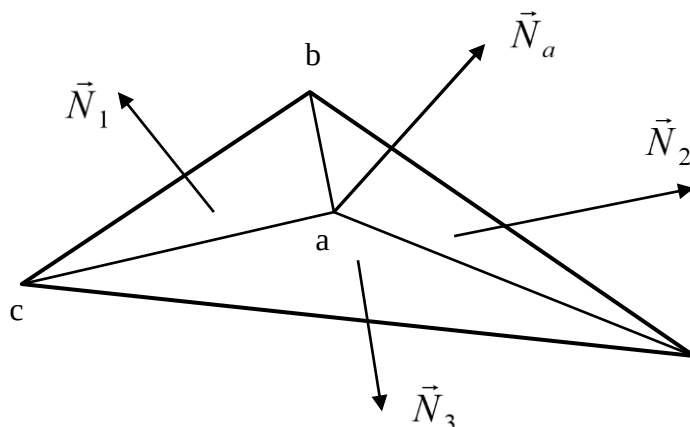


Рис. 13.8

Определение интерполированных значений интенсивности отраженного света в каждой точке грани (и, следовательно, цвет каждого пиксела) удобно выполнять во время цикла заполнения полигона. Рассмотрим заполнение контура грани горизонталями в экранных координатах (рис. 13.9).

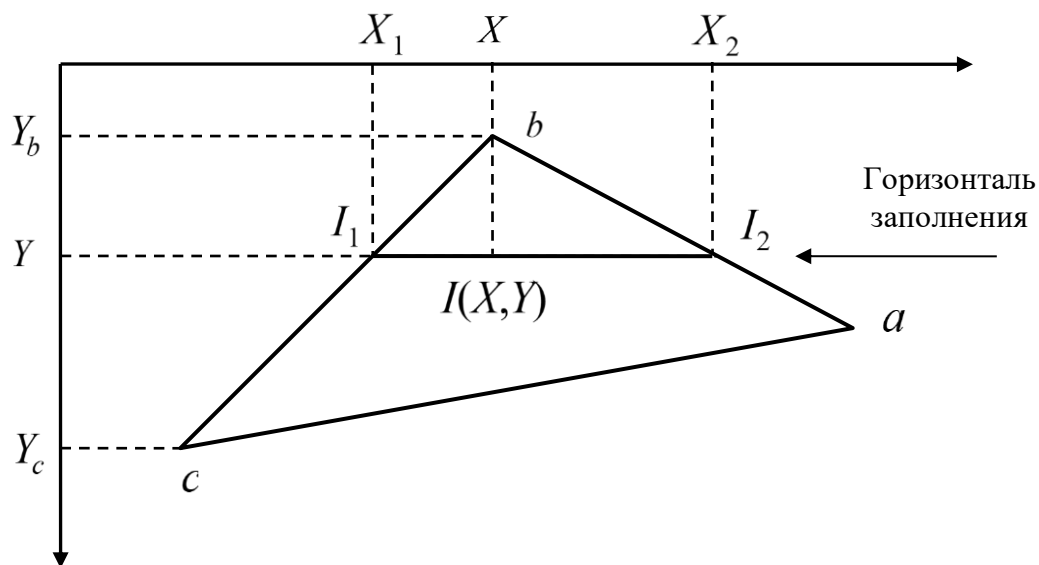


Рис. 13.9

Интерполированная интенсивность I в точке (X, Y) определяется исходя из пропорции

$$\frac{I - I_1}{I_2 - I_1} = \frac{X - X_1}{X_2 - X_1}. \quad (13.21)$$

Отсюда находим

$$I = I_1 + (I_2 - I_1) \frac{X - X_1}{X_2 - X_1}. \quad (13.22)$$

Значения интенсивностей I_1 и I_2 на концах горизонтального отрезка представляют собой интерполяцию интенсивности в вершинах.

Для интенсивности I_1 получаем пропорцию

$$\frac{I_1 - I_b}{I_c - I_b} = \frac{Y - Y_b}{Y_c - Y_b}$$

Откуда получаем

$$I_1 = I_b + (I_c - I_b) \frac{Y - Y_b}{Y_c - Y_b} \quad (13.23)$$

Аналогично для I_2

$$\frac{I_2 - I_b}{I_a - I_b} = \frac{Y - Y_b}{Y_a - Y_b}$$

Откуда

$$I_2 = I_b + (I_a - I_b) \frac{Y - Y_b}{Y_a - Y_b}. \quad (13.24)$$

Глава 14. ПОСТРОЕНИЕ КРИВЫХ

Одним из важнейших применений компьютерной графики является визуализация научных данных [5]. Визуализацию научных данных принято относить к той области компьютерной графики, которая получила название научная графика. Научная графика – это представление результатов научных расчетов или экспериментов в графической форме. Такая форма представления значительно упрощает анализ полученных результатов и позволяет взглянуть на них по-новому. Первые графики на машине получали в режиме символьной печати. Затем

появились специальные устройства – графопостроители (плоттеры) для вычерчивания чертежей и графиков чернильным пером на бумаге. Современная научная компьютерная графика дает возможность проводить вычислительные эксперименты с наглядным представлением их результатов в виде кривых на плоскости и поверхностей в пространстве.

Данный раздел учебного пособия посвящен некоторым методам построения плоских кривых по дискретному набору данных – точек на плоскости, заданных своими координатами.

14.1. Интерполяция функций

Интерполяция – в вычислительной математике способ нахождения промежуточных значений величины по имеющемуся дискретному набору известных значений. Многим из тех, кто сталкивается с научными и инженерными расчётами часто приходится оперировать наборами значений, полученных экспериментальным путём или методом случайной выборки. Как правило, на основании этих наборов требуется построить функцию, на которую могли бы с высокой точностью попадать другие получаемые значения. Такая задача называется **аппроксимацией**. **Интерполяцией** называют такую разновидность аппроксимации, при которой кривая построенной функции проходит точно через имеющиеся точки данных.

14.2. Постановка задачи интерполяции

Пусть задано множество несовпадающих точек (x_i, y_i) , $i = 0, 1, 2, \dots, n$ на плоскости.

Требуется найти такую функцию $f(x)$, принадлежащую заданному классу функций, что для всех точек x_i , $i = 0, 1, 2, \dots, n$ выполняется условие $f(x_i) = y_i$.

Геометрически это означает, что нужно найти кривую $y = f(x)$, которая на плоскости XOY проходит через заданную систему точек (x_i, y_i) , (рис. 14.1)

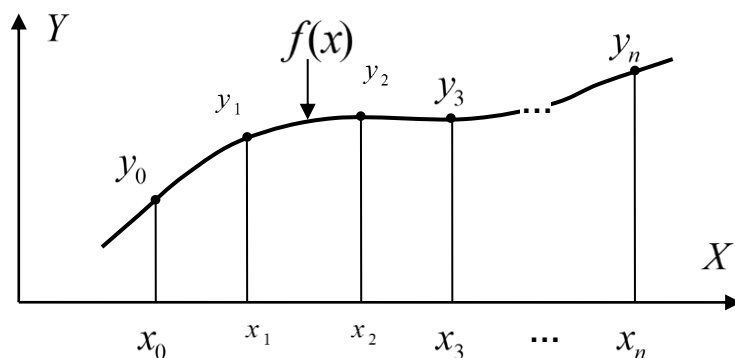


Рис. 14.1

Сформулированная задача называется **задачей интерполяции**. Функция $f(x)$ называется **интерполирующей** функцией. Точки $x_0, x_1, x_2, \dots, x_n$ называются **узлами интерполяции**.

Разумеется, задача интерполяции может иметь решение (и при том не единственное) или не иметь его вовсе. Все зависит от заданного класса функций.

Множество точек (x_i, y_i) , можно рассматривать как набор дискретных значений некоторой неизвестной функции $y = \varphi(x)$. В этом случае функцию $\varphi(x)$ называют **интерполируемой** функцией.

Полученную функцию $f(x)$ обычно используют в случае приближенного вычисления значений функции $\varphi(x)$ для значений аргумента x , отличных от узлов интерполяции. Процесс вычисления значений функции $\varphi(x)$ в точках, отличных от узлов интерполяции, называется **интерполированием** функции $\varphi(x)$.

При этом различают **интерполирование в узком смысле**, если $x \in [x_0, x_n]$, и **экстраполирование**, когда $x \notin [x_0, x_n]$.

Замена функции $\varphi(x)$ ее интерполирующей функцией $f(x)$ может потребоваться не только тогда, когда известна лишь таблица ее значений (например, полученная в результате эксперимента), но и в том случае, когда аналитическое выражение для функции $\varphi(x)$ известно, однако является слишком сложным и неудобным для дальнейших математических преобразований (интегрирования, дифференцирования и т. д.).

14.3. Интерполяционный полином Лагранжа

Будем искать функцию $f(x)$ в виде полинома заданной степени n – **интерполяционного полинома**, т. е. положим что

$$f(x) = P_n(x),$$

где

$$P_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n \quad (14.1)$$

и в соответствии с определением интерполирующей функции

$$P_n(x_i) = y_i, \quad i = 0, 1, 2, \dots, n \quad (14.2)$$

Поскольку степень полинома известна, то поиск полинома сводится к нахождению набора его коэффициентов $a_0, a_1, \dots, a_n$.

14.3.1. Определение коэффициентов интерполяционного полинома

Воспользовавшись выражением (14.1) для искомого полинома $P_n(x)$ и условием (14.2), получим систему линейных уравнений относительно неизвестных коэффициентов $a_0, a_1, \dots, a_n$.

$$\begin{cases} a_0 + a_1x_0 + a_2x_0^2 + \dots + a_nx_0^n = y_0, \\ a_0 + a_1x_1 + a_2x_1^2 + \dots + a_nx_1^n = y_1, \\ \dots \\ a_0 + a_1x_n + a_2x_n^2 + \dots + a_nx_n^n = y_n. \end{cases} \quad (14.3)$$

Систему уравнений (14.3) представим в матричном виде, введя соответствующие обозначения.

$$S = \begin{pmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ \dots & & & & \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{pmatrix} Y = \begin{pmatrix} y_0 \\ y_1 \\ \dots \\ y_n \end{pmatrix} A = \begin{pmatrix} a_0 \\ a_1 \\ \dots \\ a_n \end{pmatrix}.$$

В результате система уравнений (14.3) приобретает вид

$$SA = Y. \quad (14.4)$$

Откуда

$$A = S^{-1}Y. \quad (14.5)$$

Отметим, что задача интерполяции в описанной выше постановке имеет единственное решение. Это означает, что не существует двух различных наборов коэффициентов $a_0, a_1, \dots, a_n$, удовлетворяющих условию (14.2). Это следует из того, что искомые коэффициенты полинома определяются как решение системы

линейных уравнений, имеющей единственное решение.

Французский математик **Лагранж** предложил способ построения интерполяционного полинома без предварительного вычисления коэффициентов $a_0, a_1, \dots, a_n$, т. е. без решения системы уравнений (14.3).

Будем искать интерполяционный полином, который в данном случае обозначим через $L_n(x)$, в виде

$$\begin{aligned} L_n(x) = & A_0(x-x_1)(x-x_2)\dots(x-x_n) + A_1(x-x_0)(x-x_2)\dots(x-x_n) + \dots \\ & + A_i(x-x_0)(x-x_1)\dots(x-x_{i-1})(x-x_{i+1})\dots(x-x_n) + \dots \\ & + A_n(x-x_0)(x-x_1)\dots(x-x_{n-1}) = \sum_{i=0}^n A_i \prod_{\substack{j=0 \\ j \neq i}}^n (x-x_j). \end{aligned} \quad (14.6)$$

Неизвестные коэффициенты A_i ($i=0,1,2,\dots,n$) определим из условия $L_n(x_i) = f(x_i) = y_i$.

Последовательно полагая $x = x_i$, $i=0,1,2,\dots,n$, получим

$$\begin{aligned} A_i(x_i-x_0)(x_i-x_1)\dots(x_i-x_{i-1})(x_i-x_{i+1})\dots(x_i-x_n) &= y_i, \\ A_i &= \frac{y_i}{(x_i-x_0)(x_i-x_1)\dots(x_i-x_{i-1})(x_i-x_{i+1})\dots(x_i-x_n)}. \end{aligned}$$

Подставляя найденные значения коэффициентов A_i ($i=0,1,2,\dots,n$) в выражение (14.6) для многочлена $L_n(x)$, получим

$$L_n(x) = \sum_{i=0}^n y_i \frac{(x-x_0)(x-x_1)\dots(x-x_{i-1})(x-x_{i+1})\dots(x-x_n)}{(x_i-x_0)(x_i-x_1)\dots(x_i-x_{i-1})(x_i-x_{i+1})\dots(x_i-x_n)} \quad (14.7)$$

Полученный таким образом полином называется интерполяционным полиномом Лагранжа.

Следует отметить, что полином Лагранжа $L_n(x)$ просто представляет собой другую форму записи рассмотренного ранее полинома $P_n(x)$, что следует из единственности решения задачи интерполяции.

Выражение для полинома Лагранжа $L_n(x)$ может быть легко преобразовано к виду $P_n(x)$ путем группировки коэффициентов при соответствующих степенях аргумента x .

Примеры.

Пусть имеем две точки (x_0, y_0) и (x_1, y_1) , что соответствует значению $n=1$

. Тогда

$$L_1(x) = y_0 \frac{(x - x_1)}{(x_0 - x_1)} + y_1 \frac{(x - x_0)}{(x_1 - x_0)} = \frac{y_0 x_1 + y_1 x_0}{x_0 - x_1} + \frac{y_0 - y_1}{x_0 - x_1} x.$$

Обозначая $a_0 = \frac{y_0 x_1 + y_1 x_0}{x_0 - x_1}$ и $a_1 = \frac{y_0 - y_1}{x_0 - x_1} x$, получаем

$$L_1(x) = a_0 + a_1 x = P_1(x).$$

Пусть имеем три точки $(x_0, y_0) = (1, 2)$, $(x_1, y_1) = (2, 8)$ и $(x_2, y_2) = (3, 6)$, что соответствует значению $n = 2$. Тогда

$$\begin{aligned} L_2(x) &= y_0 \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} + y_1 \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} + y_2 \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} = \\ &= 2 \frac{(x - 2)(x - 3)}{(1 - 2)(1 - 3)} + 8 \frac{(x - 1)(x - 3)}{(2 - 1)(2 - 3)} + 6 \frac{(x - 1)(x - 2)}{(3 - 1)(3 - 2)} = \\ &= (x - 2)(x - 3) - 8(x - 1)(x - 3) + 3(x - 1)(x - 2) = -4x^2 + 18x - 12. \end{aligned}$$

Достоинства полинома Лагранжа:

- график интерполяционного многочлена Лагранжа проходит через каждую точку массива;
- конструируемая функция легко описывается (число подлежащих определению коэффициентов интерполяционного многочлена Лагранжа на сетке n равно $n + 1$);
- построенная функция имеет непрерывные производные любого порядка;
- заданным массивом интерполяционный многочлен определен однозначно.

Недостатки полинома Лагранжа:

- степень интерполяционного многочлена Лагранжа зависит от числа узлов сетки, и чем больше это число, тем выше степень интерполяционного многочлена и, значит, тем больше требуется вычислений;
- изменение хотя бы одной точки в массиве требует полного пересчета коэффициентов интерполяционного многочлена Лагранжа.

14.4. Интерполяция сплайнами

Сплайн (англ. spline – планка, рейка) – функция, область определения которой разбита на конечное число отрезков, на каждом из которых сплайн совпадает с некоторым алгебраическим полиномом. Сплайны имеют многочисленные применения как в математической теории, так и в разнообразных вычислительных приложениях. В частности, сплайны двух переменных интенсивно используются для задания поверхностей в различных системах компьютерного моделирования.

14.4.1. Определение сплайна

Пусть на отрезке $[a, b]$ задана упорядоченная система (массив) несовпадающих точек x_i ($i = 0, 1, \dots, n$).

Определение. СплAINOM $S_m(x)$ называется определенная на $[a, b]$ функция, принадлежащая классу $C^l[a, b]$ l раз непрерывно дифференцируемых функций, такая, что на каждом промежутке $[x_{i-1}, x_i]$ $i = 0, 1, \dots, n$ – это многочлен m -й степени. Разность $d = m - l$ между степенью сплайна m и показателем его гладкости l называется **дефектом** сплайна.

Если сплайн $S_m(x)$ строится по некоторой функции $f(x)$ так, чтобы выполнялись условия $S_m(x_i) = f(x_i)$ то такой сплайн называется **интерполяционным сплайном** для функции $f(x)$; при этом узлы сплайна, вообще говоря, могут не совпадать с узлами интерполяции x_k .

14.4.2. Интерполяционный кубический сплайн

Рассмотрим наиболее известный и широко применяемый интерполяционный сплайн степени 3 дефекта 1. При этом будем исходить из предположения, что узлы сплайна

$$a = x_0, x_1, \dots, x_n = b \quad (14.8)$$

одновременно служат узлами интерполяции, т.е. в них известны значения функции $f_i = f(x_i)$, $i = 0, 1, \dots, n$.

Определение. Кубическим сплайном дефекта 1, интерполирующим на отрезке $[a, b]$ данную функцию $f(x)$, называется функция

$$g(x) = \{g_i(x), \text{ при } x \in [x_{i-1}, x_i]\}_{i=1}^n, \quad (14.9)$$

где

$$g_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad (14.10)$$

удовлетворяет совокупности условий:

$$g(x_i) = f(x_i) - \quad (14.11)$$

условие интерполяции в узлах сплайна,

$$g(x) \in C_{[a, b]}^2 - \quad (14.12)$$

двойная непрерывная дифференцируемость,

$$g''(a) = g''(b) = 0 - \quad (14.13)$$

краевые (граничные) условия.

Заметим, что граничные условия вида (14.13) называются естественными граничными условиями.

Определенный таким образом сплайн называют еще **естественными** или **чертежным сплайном** и связано это со следующим обстоятельством. Желая провести плавную линию через заданные точки плоскости, чертежники фиксировали в этих точках гибкую упругую рейку, тогда под влиянием упругих сил она принимала нужную форму, обеспечивающую минимум потенциальной энергии.

Для построения по данной функции $f(x)$ интерполирующего ее сплайна (14.9) нужно найти $4n$ его коэффициентов a_i, b_i, c_i, d_i ($i = 1, 2, 3, \dots, n$)

Имеем:

из условий интерполяции (14.11) для функции

$$g_1(x_0) = f_0, \quad g_i(x_i) = f_i = a_i \quad \text{при } i = 1, 2, \dots, n \quad (14.14)$$

из условий гладкой стыковки звеньев сплайна (14.12)

$$\begin{cases} g_{i-1}(x_{i-1}) = g_i(x_{i-1}) \\ g'_{i-1}(x_{i-1}) = g'_i(x_{i-1}), \quad \text{при } i = 2, 3, \dots, n \\ g''_{i-1}(x_{i-1}) = g''_i(x_{i-1}) \end{cases} \quad (14.15)$$

из краевых условий (14.13)

$$g''(x_0) = g''(x_n) = 0, \quad (14.16)$$

Подставляя сюда выражения (14.9) для функций $g_i(x)$,

$$g_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3,$$

их производных

$$g'_i(x) = b_i + 2c_i(x - x_i) + 3d_i(x - x_i)^2 \quad (14.17)$$

и

$$g''_i(x) = 2c_i + 6d_i(x - x_i) \quad (14.18)$$

через коэффициенты a_i, b_i, c_i, d_i при указанных значениях i и, полагая для краткости

$$h_i = x_i - x_{i-1}, \quad (14.19)$$

можно получить систему уравнений для определения неизвестных коэффициентов [7].

Полученная система решается методом прогонки [7].

Рассмотрим случай равноотстоящих узлов.

Тогда $h_k = h$ и расчетные формулы для коэффициентов полинома [7] принимают вид:

$$\delta_1 = -\frac{1}{4}, \quad \lambda_1 = \frac{3}{4h^2}(f_2 - 2f_1 + f_0)$$

Для $i = 3, 4, \dots, n$ вычисляем

$$\delta_{i-1} = -\frac{1}{4 + \delta_{i-2}}, \quad \lambda_{i-1} = \frac{\frac{3}{h^2}(f_i - 2f_{i-1} + f_{i-2}) - \lambda_{i-2}}{4 + \delta_{i-2}}.$$

Полагаем $c_n = 0$ и для $i = n, n-1, \dots, 2$ вычисляем

$$c_{i-1} = \delta_{i-1}c_i + \lambda_{i-1}.$$

Полагаем $i = 1, 2, \dots, n$ и с учетом $c_n = 0$ получаем

$$b_i = \frac{f_i - f_{i-1}}{h} + \frac{h}{3}(2c_i + c_{i-1}), \quad d_i = \frac{c_i - c_{i-1}}{3h}.$$

В результате при $x \in [x_{i-1}, x_i]$ значение $f(x)$ можно заменить значением

$$g_i(x) = f_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3.$$

с найденными значениями коэффициентов b_i, c_i, d_i .

Достоинства кубической сплайн-интерполяции:

- график построенной функции проходит через каждую точку массива;
- конструируемая функция сравнительно легко описывается (число подлежащих определению коэффициентов равно $4n$);
- заданным массивом построенная функция определена однозначно;
- степень многочленов не зависит от числа узлов сетки и, следовательно, не изменяется при его увеличении;
- построенная функция имеет непрерывные первые и вторые производные;
- построенная функция обладает хорошими аппроксимационными свойствами.

К недостаткам кубических сплайнов относится то, что они склонны осциллировать в окрестностях точки, существенно отличающейся от своих соседей.

14.5. Геометрические сплайны

Во многих задачах требование того, чтобы конструируемая кривая однозначно проектировалась соответственно на прямую, является слишком жестким. Расширяя допустимые классы кривых, естественно обратиться и к более общему способу описания их частичных фрагментов[5]. В качестве нового способа задания кривых удобно использовать параметрический способ.

Формулировка задачи: по заданному множеству вершин $P = \{P_0, P_1, \dots, P_{m-1}, P_m\}$ с учетом их нумерации построить гладкую кривую, которая, плавно изменяясь, последовательно проходила бы вблизи этих вершин и удовлетворяла некоторым дополнительным условиям. Эти условия могут иметь различный характер. Например, можно потребовать, чтобы искомая кривая проходила через все заданные вершины или, проходя через заданные вершины, касалась заданных направлений, являлась замкнутой или имела заданную регулярность и т. п.

При отыскании подходящего решения задачи приближения важную роль играет ломаная, звенья которой соединяют соседние вершины заданного набора. Эту ломаную называют **контрольной** или **опорной**, а ее вершины – контрольными или опорными.

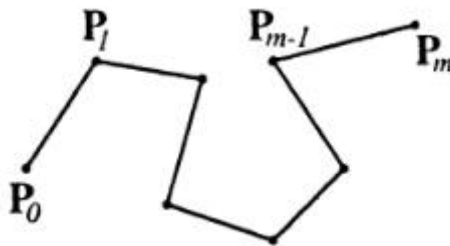


Рис. 14.2

Во многих случаях она довольно точно показывает, как будет проходить искомая кривая, что особенно полезно при решении задачи сглаживания. Каждая вершина заданного массива является либо **внутренней** либо **граничной** (концевой). В массиве P вершины $P_1 \dots, P_{m-1}$ внутренние, а вершины P_0 и P_m – граничные (концевые).

Никаких ограничений на множество вершин не накладывается – они могут быть заданы как на плоскости, так и в пространстве, их взаимное расположение может быть совершенно произвольным, некоторые из вершин могут совпадать и т. д. Поэтому описание нужной кривой ищут в следующем виде:

$$R(t) = \sum a_i(t) P_i \quad (14.20)$$

где $a_i(t)$ – некоторые функциональные коэффициенты, подлежащие определению.

Если количество вершин в заданном множестве P достаточно велико, то найти универсальные функциональные коэффициенты, как правило, довольно затруднительно. Если универсальные коэффициенты $a_i(t)$ все же найдены, то часто оказывается, что они наряду с нужными свойствами обладают и такими, которые не всегда удовлетворительно согласуются с ожидаемым поведением соответствующей кривой (например, кривая, описываемая уравнением (14.20) с этими коэффициентами, может осциллировать или заметно отклоняться от заданного множества).

Для успешного решения поставленной задачи приближения, весьма удобно привлечь кривые, составленные из элементарных фрагментов. В случае, когда эти элементарные фрагменты строятся по единой сравнительно простой схеме, такие составные кривые принято называть сплайновыми кривыми.

Параметрические уравнения каждого элементарного фрагмента ищутся в виде (14.20) с той лишь разницей, что всякий раз привлекается только часть заданных вершин множества P . Для описания элементарных кривых и вычисления их геометрических характеристик (информация о которых необходима при состыковке) в качестве функциональных коэффициентов обычно используются многочлены невысоких степеней 2-й или 3-й, в первую очередь потому, что они сравнительно просто вычисляются. Наибольшее распространение получили методы конструирования составных кривых, в которых используются кубические многочлены.

14.5.1. Кривая Безье

Разработана математиком **Пьером Безье**. Кривые и поверхности Безье были использованы в 60-х годах компанией «Рено» для компьютерного проектирования формы кузовов автомобилей. В настоящее время они широко используются в компьютерной графике.

Кривые Безье описываются в параметрической форме [2,5]:

$$\begin{cases} x = P_x(t) \\ y = P_y(t) \end{cases} \quad (14.21)$$

Значение t выступает как параметр, которому отвечают координаты отдельной точки линии. Параметрическая форма описания может быть более удобной для некоторых кривых, чем задание в виде функции $y = f(x)$. Это потому, что функция $f(x)$ может быть намного сложнее, чем $P_x(t)$ и $P_y(t)$, кроме того, $f(x)$ может быть неоднозначной.

Многочлены Безье для $P_x(t)$ и $P_y(t)$ имеют такой вид:

$$\begin{cases} P_x(t) = \sum_{i=0}^m C_m^i t^i (1-t)^{m-i} x_i \\ P_y(t) = \sum_{i=0}^m C_m^i t^i (1-t)^{m-i} y_i \end{cases}, \quad (14.22)$$

где $C_m^i = \frac{m!}{i!(m-i)!}$ – сочетание m по i , x_i и y_i – координаты точек-ориентиров P_i .

Значение m можно рассматривать и как степень полинома, и как значение, которое на единицу меньше количества точек-ориентиров.

Рассмотрим кривые Безье, классифицируя их по значениям m .

$m=1$ (по двум точкам).

Кривая вырождается в отрезок прямой линии, определяемой концевыми точками (рис.14.3)

$$P(t) = (1-t)P_0 + tP_1, \quad (14.23)$$

где $P = (P_x, P_y)^T$.

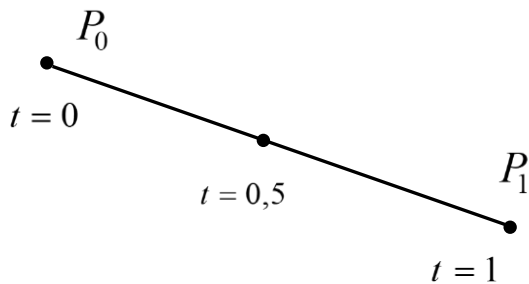


Рис. 14.3

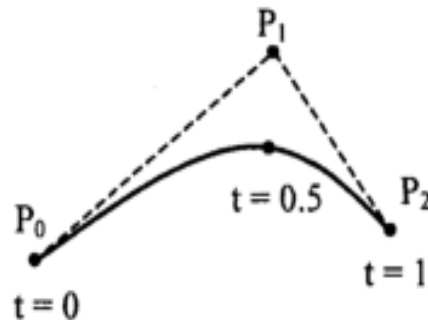


Рис. 14.4

$m=2$ (по трем точкам, рис. 14.3).

$$P(t) = (1-t)^2 P_0 + 2t(1-t)P_1 + t^2 P_2 \quad (14.24)$$

$m=3$ (по четырем точкам, рис. 14.4 а,б).

$$P(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t)P_2 + t^3 P_3 \quad (14.25)$$

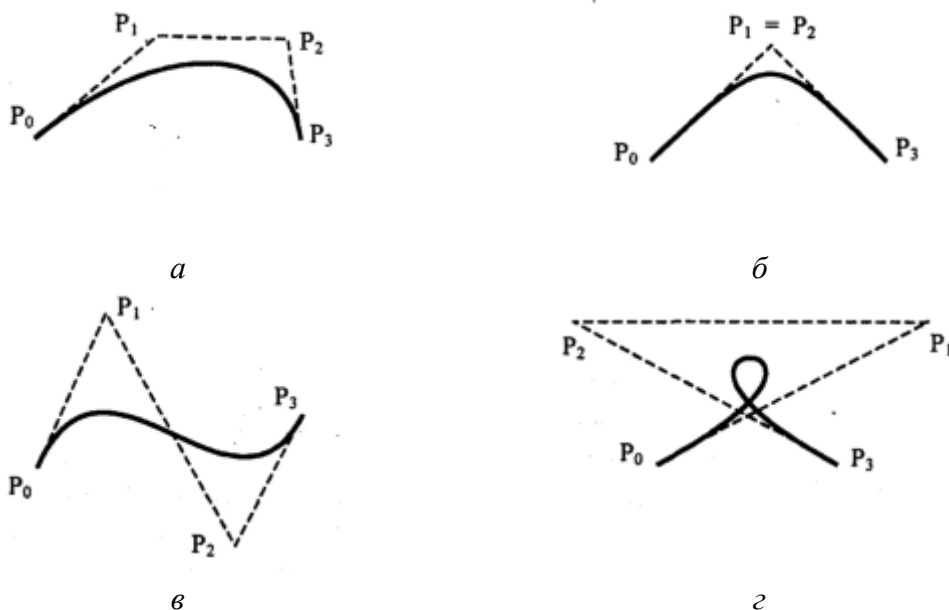


Рис. 14.4

14.5.2. Геометрический алгоритм для кривой Безье

Этот алгоритм позволяет вычислить координаты (x, y) точки кривой Безье по значению параметра t .

Суть алгоритма заключается в следующем:

- каждая сторона контура многоугольника, проходящего по точкам-ориентирам, делится пропорционально значению t ;
- точки деления соединяются отрезками прямых и образуют новый многоугольник. Количество узлов нового контура на единицу меньше, чем количество узлов предыдущего контура;
- стороны нового контура снова делятся пропорционально значению t . И так далее.

Это продолжается до тех пор, пока не будет получена единственная точка деления. Эта точка и будет точкой кривой Безье (рис. 14.5а, б).

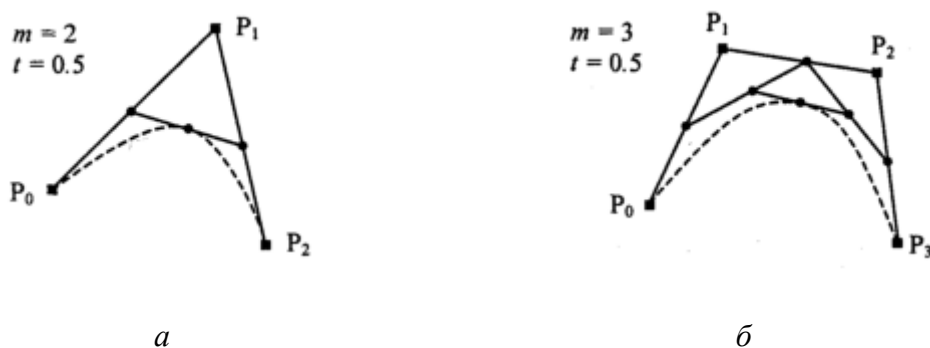


Рис. 14.5

Приведем запись геометрического алгоритма на языке C++:

```
for(i=0; i<=m; i++)  
R[i]=P[i]; // Формируем вспомогательный массив R  
for(j=m; j>0; j--)  
for(i=0; i<j; i++)  
    R[i]=R[i]+t*(R[i+1]-R[i]);
```

Результат работы алгоритма – координаты одной точки кривой Безье – записываются в R[0].

Глава 15. ГРАФИЧЕСКАЯ БИБЛИОТЕКА OPENGL

15.1. Общие сведения

OpenGL является одним из самых популярных прикладных программных интерфейсов (API – Application Programming Interface) для разработки приложений в области двумерной и трехмерной графики [9 – 11]. Стандарт OpenGL (**O**pen **G**raphics **L**ibrary – открытая графическая библиотека) был разработан и утвержден в 1992 году ведущими фирмами в области разработки программного обеспечения как эффективный аппаратно-независимый интерфейс, пригодный для реализации на различных платформах. Основой стандарта стала библиотека IRISGL, разработанная фирмой Silicon Graphics Inc. Библиотека насчитывает около 120 различных команд, которые программист использует для задания объектов и операций, необходимых для написания интерактивных графических приложений. На сегодняшний день графическая система OpenGL поддерживается большинством производителей аппаратных и программных платформ. Эта система доступна тем, кто работает в среде Windows, пользователям компьютеров Apple. Характерными особенностями OpenGL, которые обеспечили распространение и развитие этого графического стандарта, являются:

- **Стабильность.** Дополнения и изменения в стандарте реализуются таким образом, чтобы сохранить совместимость с разработанным ранее программным обеспечением.
- **Надежность и переносимость.** Приложения, использующие OpenGL, гарантируют одинаковый визуальный результат вне зависимости от типа используемой операционной системы и организации отображения информации. Кроме того, эти приложения могут выполняться как на персональных компьютерах, так и на рабочих станциях и суперкомпьютерах.
- **Легкость применения.** Стандарт OpenGL имеет продуманную структуру и интуитивно понятный интерфейс, что позволяет с меньшими затратами созда-

вать эффективные приложения, содержащие меньше строк кода, чем с использованием других графических библиотек. Необходимые функции для обеспечения совместимости с различным оборудованием реализованы на уровне библиотеки и значительно упрощают разработку приложений.

15.2. Основные возможности

Описать возможности OpenGL можно через функции его библиотеки. Все функции можно разделить на пять категорий:

1. **Функции описания примитивов** определяют объекты нижнего уровня иерархии (примитивы), которые способна отображать графическая подсистема. В OpenGL в качестве примитивов выступают точки, линии, многоугольники и т.д.

2. **Функции описания источников света** служат для описания положения и параметров источников света, расположенных в трехмерной сцене.

3. **Функции задания атрибутов.** С помощью задания атрибутов программист определяет, как будут выглядеть на экране отображаемые объекты. Другими словами, если с помощью примитивов определяется, **что** появится на экране, то атрибуты определяют **способ** вывода на экран. В качестве атрибутов OpenGL позволяет задавать цвет, характеристики материала, текстуры, параметры освещения.

4. **Функции визуализации** позволяет задать положение наблюдателя в виртуальном пространстве, параметры объектива камеры. Зная эти параметры, система сможет не только правильно построить изображение, но и отсеять объекты, оказавшиеся вне поля зрения.

5. **Функции геометрических преобразований** позволяют программисту выполнять различные преобразования объектов – поворот, перенос, масштабирование. При этом OpenGL может выполнять дополнительные операции, такие как использование сплайнов для построения линий и поверхностей, удаление невидимых фрагментов изображениями на уровне пикселей и т.д.

15.3. Интерфейс OpenGL

OpenGL состоит из набора библиотек. Все базовые функции хранятся в основной библиотеке, для обозначения которой используется аббревиатура **GL**. Помимо основной, OpenGL включает в себя несколько дополнительных библиотек.

Первая из них – библиотека утилит **GL(GLU – GL Utility)**. Все функции этой библиотеки определены через базовые функции **GL**. В состав **GLU** вошла реализация более сложных функций, таких как набор популярных геометрических

примитивов (куб, шар, цилиндр, диск), функции построения сплайнов, реализация дополнительных операций над матрицами и т.п.

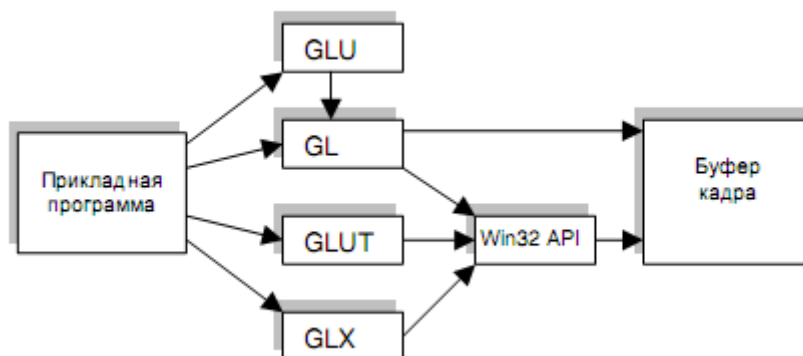


Рис. 15.1

OpenGL не включает в себя никаких специальных команд для работы с окнами или ввода информации от пользователя. Поэтому были созданы специальные переносимые библиотеки для обеспечения часто используемых функций взаимодействия с пользователем и для отображения информации с помощью оконной подсистемы. Наиболее популярной является библиотека **GLUT** (**GL Utility Toolkit**). Формально GLUT не входит в OpenGL, но фактически включается почти во все его дистрибутивы и имеет реализации для различных платформ. GLUT предоставляет только минимально необходимый набор функций для создания OpenGL-приложения. Существует также функционально аналогичная GLUT библиотека GLX. Однако она менее популярна.

Кроме того, функции, специфичные для конкретной оконной подсистемы, обычно входят в ее прикладной программный интерфейс. Так, функции, поддерживающие выполнение OpenGL, есть в составе Win32 API и Window. На рис. 15.1 схематически представлена организация системы библиотек в версии, работающей под управлением ОС Windows. Аналогичная организация используется и в других версиях OpenGL.

15.4. Архитектура OpenGL

Функции OpenGL реализованы в модели клиент–сервер. Приложение выступает в роли клиента – оно вырабатывает команды, а сервер OpenGL интерпретирует и выполняет их. Сам сервер может находиться как на том же компьютере, на котором находится клиент (например, в виде динамически загружаемой библиотеки – DLL), так и на другом (при этом может быть использован специальный протокол передачи данных между машинами).

GL обрабатывает и рисует в буфере кадра графические примитивы с учетом

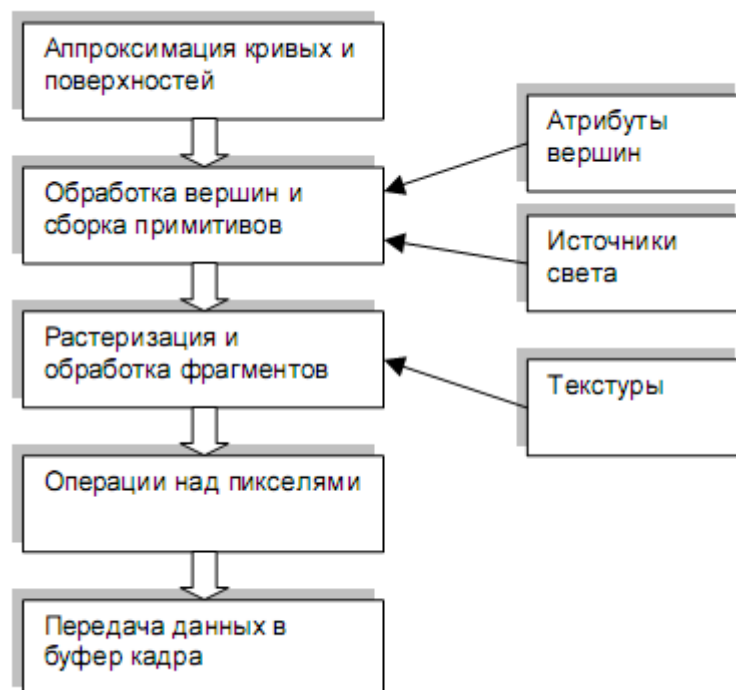
некоторого числа выбранных режимов. Каждый примитив – это точка, отрезок, многоугольник и т. д. Каждый режим может быть изменен независимо от других. Определение примитивов, выбор режимов и другие операции описываются с помощью команд в форме вызовов функций прикладной библиотеки.

Примитивы определяются набором из одной или более вершин (**vertex**). Вершина определяет точку, конец отрезка или угол многоугольника. С каждой вершиной ассоциируются некоторые данные (координаты, цвет, нормаль, текстурные координаты и т. д.), называемые атрибутами. В подавляющем большинстве случаев каждая вершина обрабатывается независимо от других.

С точки зрения архитектуры графическая система OpenGL является конвейером (рис. 15.2), состоящим из нескольких последовательных этапов обработки графических данных.

Команды OpenGL всегда обрабатываются в том порядке, в котором они поступают, хотя могут происходить задержки перед тем, как проявится эффект от их выполнения. В большинстве случаев OpenGL предоставляет непосредственный интерфейс, т. е. определение объекта вызывает его визуализацию в буфере кадра.

С точки зрения разработчиков, OpenGL – это **набор команд, которые управляют использованием графической аппаратуры**. Если аппаратура состоит только из адресуемого буфера кадра, тогда OpenGL должен быть реализован полностью с использованием ресурсов центрального процессора. Обычно графическая аппаратура предоставляет различные уровни ускорения: от аппаратной реализации вывода линий и многоугольников до изоэшеленных графических процессоров с поддержкой различных операций над геометрическими данными.



OpenGL является прослойкой между аппаратурой и пользовательским уровнем, что позволяет предоставлять единый интерфейс на разных платформах, используя возможности аппаратной поддержки.

Кроме того, OpenGL можно рассматривать как конечный автомат, состояние которого определяется множеством значений специальных переменных и значениями текущей нормали, цвета, координат текстуры и других атрибутов и признаков. Вся эта информация будет использована при поступлении в графическую систему координат вершины для построения фигуры, в которую она входит. Смена состояний происходит с помощью команд, которые оформляются как вызовы функций.

15.5. Синтаксис команд

Определения команд GL находятся в файле `gl.h`, для включения которого нужно написать (для Windows 7)

```
#include<gl.h>
```

Для работы с библиотекой GLU нужно аналогично включить файл `glu.h`. Версии этих библиотек, как правило, включаются в дистрибутивы систем программирования, например Microsoft Visual Studio, Borland Delphi.

В отличие от стандартных библиотек, пакет GLUT нужно устанавливать и подключать отдельно.

Все команды (процедуры и функции) библиотеки GL начинаются с префикса **gl**, все константы – с префикса **GL_**. Соответствующие команды и константы библиотек GLU и GLUT аналогично имеют префиксы **glu** (GLU_) и **glut** (GLUT_).

Кроме того, в имена команд входят суффиксы, несущие информацию о числе и типе передаваемых параметров. В OpenGL полное имя команды имеет вид:

```
type LibNameCommandName[1 2 3 4][b s i f d ub us ui][v]  
(type1 arg1,...,typeNargN),
```

где

type– тип возвращаемого функцией значения,

LibNameCommandName– имя команды, которое состоит из двух частей: **LibName** и **CommandName**,

LibName – имя библиотеки, в которой описана эта функция

Для базовых функций OpenGL, функций из библиотек GL, GLU, GLUT, GLAUX это **gl**, **glu**, **glut**, **aux** соответственно.

CommandName – имя команды (процедуры или функции),

[1 2 3 4] – число аргументов команды,

[b s i f d ub us ui] – тип аргумента, символ.

Соответствие типов OpenGL и языка C(C++) приведено в табл. 15.1.

Таблица 15.1

Типы OpenGL и языка C(C++)

Символ	Описание	Тип OpenGL	Тип C(C++)
b	8 – битовое целое	GLbyte	signed char
s	16 – битовое целое	GLshort	short
i	32 – битовое целое	GLint Gsizei	long
f	32 – битовое число с плавающей точкой	GLfloat GLclampf	float
d	64 – битовое число с плавающей точкой	GLdouble GLclampd	double
ub	8 – битовое беззнаковое целое	GLubyte GLboolean	unsigned char
us	16 – битовое беззнаковое целое	GLushort	unsigned short
ui	32 – битовое беззнаковое целое	GLuint GLenum GLbitfield	unsigned long
		GLvoid	void

h [v] – наличие этого символа показывает, что в качестве параметров функции используется указатель на массив значений. Символы в квадратных скобках в некоторых названиях не используются. Например, команда **glVertex2i()**, описанная в библиотеке GL, использует в качестве параметров два целых числа, а команда **glColor3fv()** использует в качестве параметра указатель на массив из трех вещественных чисел. Использование нескольких вариантов каждой команды можно частично избежать, применяя перегрузку функций языка C++. Но интерфейс OpenGL не рассчитан на конкретный язык программирования, и, следовательно, должен быть максимально универсален.

15.6. Структура GLUT-приложения

Далее будем рассматривать построение консольного приложения при помощи библиотеки GLUT. Эта библиотека обеспечивает единый интерфейс для работы с окнами вне зависимости от платформы, поэтому описываемая ниже структура приложения остается неизменной для операционных систем Windows, Linux и других.

Функции GLUT могут быть классифицированы на несколько групп по своему назначению:

- Инициализация.
- Начало обработки событий.
- Управление окнами.
- Управление меню.
- Регистрация функций с обратным вызовом.
- Управление индексированной палитрой цветов.
- Отображение шрифтов.
- Отображение дополнительных геометрических фигур (тор, конус и др.).

Инициализация проводится с помощью функции:

void glutInit (int *argc, char **argv), где переменная **argc** – указатель на стандартную переменную **argc** описываемую в функции **main()**, а **argv** – указатель на параметры, передаваемые программе при запуске, который описывается там же. Эта функция проводит необходимые начальные действия для построения окна приложения, и только несколько функций GLUT могут быть вызваны до нее. К ним относятся:

void glutInitWindowPosition (int x, int y) – служит для установки положения окна OpenGL на экране, здесь **x** есть X-координата **левого верхнего** угла окна, **y** есть Y – координата **левого верхнего** угла окна. Координаты **x** и **y** определяются в пикселах.

void glutInitWindowSize(int width, int height), – служит для установки размеров окна на экране, здесь **width** – ширина окна, **height** – высота окна. Значения **width** и **height** определяются в пикселах.

Если вызов этих функций опустить, то система присвоит параметрам окна значения по умолчанию.

void glutInitDisplayMode(unsigned int mode) – задает режим отображения, где **mode** – определяет различные режимы отображения информации, которые могут совместно задаваться с использованием операции побитового «ИЛИ» («|»):

Таблица 15.2

Значения

Значение	Комментарий
GLUT_RGBA	Выбор режима RGBA
GLUT_RGB	То же, что и GLUT_RGBA
GLUT_INDEX	Палитровый видеорежим
GLUT_SINGLE	Использование одинарного видеобуфера
GLUT_DOUBLE	Использование двойной буферизации, применяется для создания анимации

GLUT_ACCUM	Создавать аккумулярующий буфер
GLUT_ALPHA	Создавать окно с α - каналом
GLUT_DEPTH	Создавать окно с буфером глубины
GLUT_STENCIL	Создавать окно с буфером трафарета

После инициализации можно создать окно, в которое будет происходить вывод при помощи функции

int glutCreateWindow (char * name), где параметр **name** задает заголовок окна. Функция возвращает целочисленный идентификатор окна.

Функции библиотеки GLUT реализуют так называемый событийно-управляемый механизм. Это означает, что есть некоторый внутренний цикл, который запускается после соответствующей инициализации и обрабатывает одно за другим все события, объявленные во время инициализации. К событиям относятся: **щелчок мыши, закрытие окна, изменение свойств окна, передвижение курсора, нажатие клавиши** «пустое» (**idle**) событие, когда ничего не происходит. Для проведения периодической проверки совершения того или иного события надо зарегистрировать функцию, которая будет его обрабатывать. Для этого используются следующие функции:

void glutDisplayFunc(void (*func) (void))— служит для установки обработчика события, связанного с необходимостью перерисовки содержимого окна. Параметр **func** задает функцию рисования для окна приложения, которая вызывается при необходимости создания или восстановления изображения.

void glutPostRedisplay (void)— служит для явного указания, что окно надо обновить.

void glutReshapeFunc (void (*func) (int width, int height))—служит для установки обработчика события, связанного с изменением размера окна. Параметр **func** задает функцию которая будет вызываться при изменении размеров окна перед вызовом функции отрисовки содержимого окна. Параметры **width** и **height** определяют *новые* значения ширины и высоты окна в пикселах.

void glutKeyboardFunc(void (*func)(unsigned char key, int x, int y))—служит для установки обработчика сообщений от клавиатуры. Параметр **func** задает функцию которая будет вызываться в ответ на сообщения клавиатуры, параметр **key** содержит ASCII – код символа, а параметры **x** и **y** – координаты курсора мыши в пикселах по отношению к верхнему левому углу окна.

Обработчик клавиш, не генерирующих ASCII – кода (такие как <Shift>), обрабатываются с помощью функции

void glutSpecialFunc(void(*func)(int key, int x, int y)), где параметр **key** несет в себе информацию о нажатой клавише и может принимать одно из значений, указанных в табл. 15.3.

Таблица 15.3

Значения параметра **key**

Значение <i>key</i>	Клавиша
GLUT_KEY_F1	Клавиша <F1>
GLUT_KEY_F2	Клавиша <F2>
GLUT_KEY_F3	Клавиша <F3>
GLUT_KEY_F4	Клавиша <F4>
GLUT_KEY_F5	Клавиша <F5>
GLUT_KEY_F6	Клавиша <F6>
GLUT_KEY_F7	Клавиша <F7>
GLUT_KEY_F8	Клавиша <F8>
GLUT_KEY_F9	Клавиша <F9>
GLUT_KEY_F10	Клавиша <F10>
GLUT_KEY_F11	Клавиша <F11>
GLUT_KEY_F12	Клавиша <F12>
GLUT_KEY_LEFT	Клавиша <←>
GLUT_KEY_RIGHT	Клавиша <→>
GLUT_KEY_UP	Клавиша <↑>
GLUT_KEY_DOWN	Клавиша <↓>
GLUT_KEY_PAGE_UP	Клавиша <Page UP>
GLUT_KEY_PAGE_DOWN	Клавиша <Page Down>
GLUT_KEY_HOME	Клавиша <Home>
GLUT_KEY_END	Клавиша <End>
GLUT_KEY_INSERT	Клавиша <Insert>

void glutMouseFunc(void (*func) (int button, int state, int x, int y))—служит для установки обработчика сообщений от мыши. Параметр **func** задает функцию которая будет вызываться в ответ на сообщения от мыши. Параметр **button** принимает одно из значений GLUT_LEFT_BUTTON, GLUT_MIDDLE_BUTTON, GLUT_RIGHT_BUTTON и несет в себе информацию о том, какая клавиша мыши была нажата (или отпущена). Параметр **state** принимает одно из значений GLUT_DOWN (клавиша нажата) или GLUT_UP (клавиша отпущена). Параметры **x** и **y** содержат координаты курсора мыши.

Вызов обработчика, установленного **glutMouseFunc**, происходит лишь при нажатии или отпуске клавиши мыши. Если необходимо установить обработчик на перемещение мыши, то для этого следует воспользоваться функциями

void glutMotionFunc (void (*func)(int x, int y)),

void glutPassiveMotionFunc (void (*func)(int x, int y)).

Первая из них устанавливает обработчик, который будет вызываться при перемещении мыши при **нажатой** клавише (любой из имеющихся) мыши, а вторая – при передвижении мыши в том случае, когда ни одна из клавиш мыши **не нажата**.

int glutCreateMenu(void (*func)(int ItemNumber))— служит для создания меню. В качестве аргумента используется **имя** функции, которая будет вызы-

ваться при выборе пунктов меню. Параметр **ItemNumber** этой функции определяет пункт меню, который будет выбран при ее вызове.

glutAddMenuEntry(char* ItemName, int Number)—служит для добавления пунктов в уже созданное меню. Параметр **ItemName** определяет название пункта меню, а параметр **Number** – идентификатор пункта, который через параметр **ItemNumber** передается функции, вызываемой при выборе этого пункта меню.

glutAttachMenu(int button)—задает кнопку мыши, которая будет использоваться для активизации меню. Параметр **button** принимает одно из значений GLUT_LEFT_BUTTON, GLUT_MIDDLE_BUTTON, GLUT_RIGHT_BUTTON.

Ниже приводится последовательность команд для создания меню, добавления в него пунктов и активизации с помощью правой клавиши мыши.

```
glutCreateMenu(NaneFunction);
glutAddMenuEntry("Первый пункт меню", 1);
glutAddMenuEntry("Второй пункт меню", 2);
glutAttachMenu(GLUT_RIGHT_BUTTON);
```

В приведенном примере параметр **NaneFunction** – имя функции которая будет вызвана при выборе пункта меню.

void glutIdleFunc (void (*func) (void))—служит для установки обработчика, который будет постоянно вызываться в «фоновом» режиме. Наиболее часто этот обработчик используется для проведения некоторых вспомогательных действий, которые можно осуществить в моменты времени, когда приложение «ничего не делает».

void glutMainLoop (void)— служит для запуска бесконечного цикла обработки сообщений. Функция вызывается в конце любой программы, использующей GLUT.

void glutSwapBuffers()—при использовании двойной буферизации служит для смены буферов местами (буфера, содержимое которого видно сейчас на экране, и буфера в который производится вывод).

int glutGet(GLenum state)— служит для получения информации о внутренних переменных glut, параметр **state** может принимать значения, указанные в табл. 15.4.

Таблица 15.4

Значения параметра **state**

Значение state	Комментарий
GLUT_WINDOW_X	– для возврата x- координаты окна
GLUT_WINDOW_Y	– для возврата y- координаты окна
GLUT_WINDOW_WIDTH	– для возврата ширины окна
GLUT_WINDOW_HEIGHT	– для возврата высоты окна
GLUT_ELAPSED_TIME	– для возврата времени в миллисекундах с момента инициализации GLUT

Библиотека `Opengl` реализована по модели клиент-сервер, приложение выступает в роли клиента – вырабатывает команды, а `OpenGL` (сервер) интерпретирует и обрабатывает их. При этом сервер может располагаться как на том же компьютере, что и клиент, так и на другом. Сервер может поддерживать несколько контекстов `OpenGL`, каждый из которых инкапсулирует состояние `OpenGL`. Клиент может подключиться к любому из них. По этой причине желательно гарантировать, что выполнение всех вызванных команд завершится либо за определенное, либо за конечное время. Для обеспечения этого в `OpenGL` реализованы две команды.

Первая из них

`void glFinish()` блокирует дальнейшее выполнение программы, пока не будут завершены все вызванные перед ней команды `OpenGL`, включая изменения состояния и содержимого буфера кадра.

Вторая команда

`void glFlush()` служит для освобождения буферов `OpenGL`.

Различные реализации команд `OpenGL` для работы с буферами размещаются в различных местах, включая сетевые буферы и графические акселераторы. Команда **`glFlush`** освобождает все эти буферы, заставляя все вызванные команды выполняться так быстро, как это требуется фактической реализацией воспроизведения. И хотя выполнение не может быть закончено за некоторый заданный отрезок времени, оно завершается за конечный период.

Поскольку любая программа `OpenGL` может выполняться в сети или на графическом акселераторе, который накапливает команды, необходимо убедиться, что функция **`glFlush`** вызвана во всех программах, которые требуют, чтобы все их предварительно вызванные команды были закончены. Например, необходимо вызвать эту команду перед ожиданием пользовательского ввода, который зависит от сформированного образа.

Команда **`glFlush`**, в отличие от **`glFinish`**, не ожидает завершения выполнения всех предварительно вызванных команд.

Мы рассмотрели только малую часть команд, составляющих библиотеку `GLUT`. Полное описание `GLUT`, а также сами файлы для работы с ней можно найти на официальном сайте библиотеки `OpenGL` www.opengl.org.

15.7. Рисование геометрических объектов

15.7.1. Процесс обновления изображения

Как правило, задачей программы, использующей `OpenGL`, является обработка трехмерной сцены и интерактивное отображение в буфере кадра. Сцена состоит из набора трехмерных объектов, источников света и виртуальной камеры, определяющей текущее положение наблюдателя. Обычно приложение

OpenGL в бесконечном цикле вызывает функцию обновления изображения в окне. В этой функции и сосредоточены вызовы основных команд OpenGL. Если используется библиотека GLUT, то это будет функция с обратным вызовом, зарегистрированная с помощью вызова **glut DisplayFunc()**. GLUT вызывает эту функцию, когда операционная система информирует приложение о том, что содержимое окна необходимо перерисовать (например, если окно было перекрыто другим). Создаваемое изображение может быть как статичным, так и анимированным, т. е. зависеть от каких-либо параметров, изменяющихся со временем. В этом случае лучше вызывать функцию обновления самостоятельно. Например, с помощью команды **glutPostRedisplay()**. Типичная функция обновления изображения, как правило, состоит из трех шагов:

1. очистка буферов OpenGL;
2. установка положения наблюдателя;
3. преобразование и рисование геометрических объектов.

OpenGL содержит внутри себя несколько различных буферов. Среди них **фрейм-буфер** (куда производится построение изображения), **z-буфер**, служащий для удаления невидимых поверхностей, **буфер трафарета** и **аккумулирующий буфер** (рис. 15.3.)



Рис. 15.3

Для очистки окна (экрана, внутренних буферов) служит команда **void glClear(GLbitfield buf)**, которая очищает буферы, заданные переменной **buf**. Параметр **buf** определяет комбинацию констант, соответствующую буферам, которые нужно очистить. Его значения заданы в табл. 15.5.

Таблица 15.5

Значения параметра **buf**

Значение переменной buf	Комментарий
GL_COLOR_BUFFER_BIT	Очистить буфер изображения (фрейм - буфер)

GL_DEPTH_BUFFER_BIT	Очистить z - буфер
GL_ACCUM_BUFFER_BIT	Очистить аккумулирующий буфер
GL_STENCIL_BUFFER_BIT	Очистить буфер трафарета

Типичная программа вызывает команду для очистки буферов цвета и глубины:

glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT).

При этом цвет, которым очищается буфер изображения, задается командой **void glClearColor (GLclampf red, GLclampf green, GLclampf blue, GLclampf alpha).**

Команда **glClearColor** устанавливает цвет, которым будет заполнен буфер кадра. Первые три параметра команды задают R,G и B компоненты цвета и должны принадлежать отрезку [0,1]. Четвертый параметр задает так называемую альфа-компоненту. Как правило, он равен 1. По умолчанию цвет – черный (0,0,0,1).

Установка положения наблюдателя и преобразования трехмерных объектов (поворот, сдвиг и т. д.) контролируются с помощью задания **матриц преобразования**. Преобразования объектов и настройка положения виртуальной камеры рассмотрены ниже.

Рассмотрим вопрос о том, как передать в OpenGL описания объектов, находящихся в сцене. Каждый объект является набором примитивов OpenGL.

15.7.2.Вершины и примитивы

Вершина является атомарным графическим примитивом OpenGL и определяет точку, конец отрезка, угол многоугольника и т. д. Все остальные примитивы формируются с помощью задания вершин, входящих в данный примитив. Например, отрезок определяется двумя вершинами, являющимися концами отрезка. С каждой вершиной ассоциируются ее атрибуты. В число основных атрибутов входят **положение** вершины в пространстве, **цвет** вершины и **вектор** нормали.

Положение вершины в пространстве

Положение вершины определяется заданием ее координат в двух-, трех-, или четырехмерном пространстве (однородные координаты). Это реализуется с помощью нескольких вариантов команды **glVertex***:

void glVertex[2 3 4][s i f d] (typex, ...),

void glVertex[2 3 4][s i f d]v (type *v).

Каждая команда задает четыре координаты вершины: **x**, **y**, **z**, **w**. Команда **glVertex2*** получает значения **x** и **y**. Координата **z** в таком случае устанавливается по умолчанию равной 0, координата **w** – равной 1. **Vertex3*** получает координаты **x**, **y**, **z** и заносит в координату **w** значение 1. **Vertex4*** позволяет задать все четыре координаты. Например

```
glVertex2s(1,2)
glVertex3f(2.3, 1.5, 0,2)
GLdoublevect[]={1.0, 2.0, 3.0, 4.0}
glVertex4dv (vect)
```

Для ассоциации с вершинами цветов, нормалей и текстурных координат используются текущие значения соответствующих данных, что отвечает организации OpenGL как конечного автомата. Эти значения могут быть изменены в любой моментс помощью вызова соответствующих команд.

Цвет вершины

Для задания текущего цвета вершины используются команды :

void glColor[3 4][b s i f d ub us ui] (type r, type g, type b, type a),
void glColor[3 4][b s i f d ub us ui]v (const type*v).

Первые три параметра задают R, G, B компоненты цвета, а последний параметр определяет коэффициент непрозрачности (так называемая альфа-компонента). Если в названии команды указан тип «f» (float), то значения всех параметров должны принадлежать отрезку [0,1], при этом по умолчанию значение альфа-компоненты устанавливается равным 1.0, что соответствует полной непрозрачности. Тип «ub» (unsignedbyte) подразумевает, что значения должны лежать в отрезке [0,255].

Вершинам можно назначать различные цвета, и, если включен соответствующий режим, то будет проводиться линейная интерполяция цветов по поверхности примитива.

Для управления режимом интерполяции используется команда

void glShadeModel (GLenum mode),вызов которой с параметром GL_SMOOTH включает интерполяцию (установка по умолчанию), а с GL_FLAT – отключает.

Нормаль

Вектор нормали применяется для расчетов освещения и затенения. Нормаль в OpenGL является атрибутом вершины. В общем случае, когда речь идет о грани трехмерного объекта, вершины одной грани имеют разные нормали, так как каждая вершина принадлежит нескольким граням, лежащим в разных плоскостях. Тогда нормаль к вершине можно определить как сумму нормалей к граням, содержащим рассматриваемую вершину, $\vec{N} = \sum_{i=1}^K \vec{N}_i$ или как среднюю нормаль

$\vec{N} = \frac{1}{K} \sum_{i=1}^K \vec{N}_i$, где N_i – вектор нормали к i -й грани, K – число граней, образующих вершину. Для стандартных объектов OpenGLрассчитывает нормали автоматически. При отображении объектов, создаваемых пользователем, нормали должны быть рассчитаны самостоятельно.

После определения координат нормали она должна быть задана командами
void glNormal3[b s i f d] (GLtype nx, GL type ny, GLtype nz),
void glNormal3[b s i f d]v (GLtype *v),

где **nx**, **ny**, **nz** – координаты новой текущей нормали, **v** – массив координат вектора нормали.

Нормали задаются к каждой выводимой вершине.

Для правильного расчета освещения необходимо, чтобы вектор нормали имел единичную длину. Командой **glEnable(GL_NORMALIZE)** можно включить специальный режим, при котором задаваемые нормали будут нормироваться автоматически. По умолчанию нормализация не выполняется.

Режим автоматической нормализации должен быть включен, если приложение использует модельные преобразования растяжения/сжатия, так как в этом случае длина нормалей изменяется при умножении на модельно-видовую матрицу.

Однако применение этого режима уменьшает скорость работы механизма визуализации OpenGL, так как нормализация векторов имеет заметную вычислительную сложность (взятие квадратного корня и т.п.). Поэтому лучше сразу задавать единичные нормали.

Отметим, что команды

void glEnable (GLenum mode),
void glDisable (GLenum mode),

где параметр **mode** определяет вид режима, производят включение и отключение того или иного режима работы конвейера OpenGL. Эти команды применяются достаточно часто, и их возможные параметры будут рассматриваться в каждом конкретном случае.

15.7.3. Операторные скобки glBegin / glEnd

Выше было рассмотрено задание атрибутов одной вершины. Однако, чтобы задать атрибуты графического примитива, одних координат вершин недостаточно. Эти вершины надо объединить в одно целое, определив необходимые свойства. Для этого в OpenGL используются так называемые операторные скобки, являющиеся вызовами специальных команд OpenGL. Определение примитива или последовательности примитивов происходит между вызовами команд

void glBegin (GLenum mode)
void glEnd ()

Параметр **mode** определяет тип примитива, который задается внутри и может принимать значения, представленные на рис. 15.4 и в табл. 15.6.

Таблица 15.6

Значения параметра mode

Значение mode	Комментарий
---------------	-------------

GL_POINTS	Каждая вершина задает координаты некоторой точки.
GL_LINES	Каждая отдельная пара вершин определяет отрезок; если задано нечетное число вершин, то последняя вершина игнорируется $V_0V_1, V_2V_3...$
GL_LINE_STRIP	Каждая следующая вершина задает отрезок вместе с предыдущей – незамкнутая ломаная $V_0V_1V_2V_3...V_n$
GL_LINE_LOOP	Замкнутая ломаная $V_0V_1V_2V_3...V_nV_0$
GL_TRIANGLES	Каждые отдельные три вершины определяют треугольник. Если задано не кратное трем число вершин, то последние вершины игнорируются $V_0V_1V_2, V_3V_4V_5...$
GL_TRIANGLE_STRIP	Каждая следующая вершина задает треугольник вместе с двумя предыдущими – связанная полоса треугольников $V_0V_1V_2, V_2V_1V_3, V_2V_3V_4...$
GL_TRIANGLE_FAN	Треугольники задаются первой вершиной и каждой следующей парой вершин (пары не пересекаются) – веер треугольников $V_0V_1V_2, V_0V_2V_3, V_0V_3V_4...$
GL_QUADS	Каждая отдельная четверка вершин определяет четырехугольник; если задано не кратное четырем число вершин, то последние вершины игнорируются $V_0V_1V_2V_3, V_4V_5V_6V_7, ...$
GL_QUAD_STRIP	Полоса четырехугольников $V_0V_1V_3V_2, V_2V_3V_5V_4 ...$
GL_POLYGON	Последовательно задаются вершины выпуклого многоугольника.

Например, чтобы нарисовать треугольник с разными цветами в вершинах, достаточно написать:

```
GLfloat BlueCol[3] = {0,0,1};
glBegin(GL_TRIANGLES);
glColor3f(1.0, 0.0, 0.0);    //Красный
glVertex3f(0.0, 0.0, 0.0);
glColor3ub(0,255,0);         //Зеленый
glVertex3f(1.0, 0.0, 0.0);
glColor3fv(BlueCol);         //Синий
glVertex3f(1.0, 1.0, 0.0);
glEnd();
```

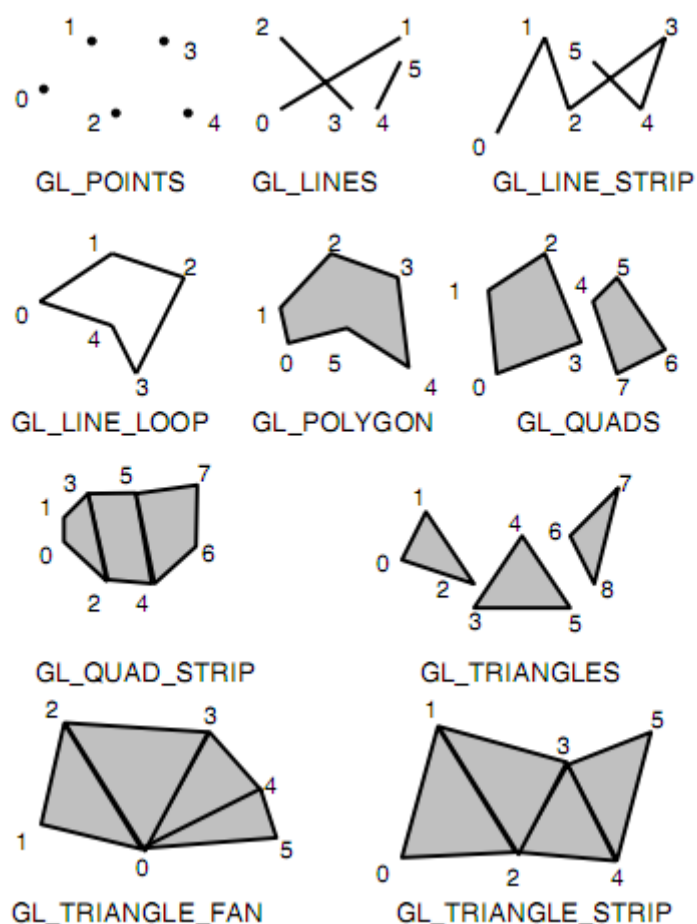


Рис. 15.4

Как правило, разные типы примитивов имеют различную скорость визуализации на разных платформах. Для увеличения производительности предпочтительнее использовать примитивы, требующие меньшее количество информации для передачи на сервер, такие как `GL_TRIANGLE_STRIP`, `GL_QUAD_STRIP`, `GL_TRIANGLE_FAN`. Кроме задания самих многоугольников, можно определить метод их отображения на экране. Для этого необходимо определить понятие **лицевых** и **обратных** граней.

Под **гранью** (многоугольником) в OpenGL подразумевается замкнутый выпуклый многоугольник с несамопересекающейся границей.

По умолчанию лицевой считается та сторона, вершины которой обходятся **против** часовой стрелки. Направление обхода вершин лицевых граней можно изменить вызовом команды

void glFrontFace (GLenum mode) со значением параметра **mode** равным `GL_CW` (clockwise), а вернуть значение по умолчанию можно, указав `GL_CCW` (counter-clockwise). Чтобы изменить метод отображения многоугольника используется команда

voidgl PolygonMode (GLenum face, GLenum mode).

Параметр **mode** определяет, **как** будут отображаться многоугольники, а параметр **face** устанавливает **тип** многоугольников, к которым будет применяться эта команда и может принимать следующие значения:

GL_FRONT – для лицевых граней,

GL_BACK – для обратных граней,

GL_FRONT_AND_BACK – для всех граней

Параметр **mode** может быть равен:

GL_POINT – отображение только вершин многоугольников,

GL_LINE – многоугольники будут представляться набором отрезков,

GL_FILL – многоугольники будут закрашиваться текущим цветом с учетом освещения, и этот режим установлен по умолчанию.

Также можно указывать, какой тип граней отображать на экране. Для этого сначала надо установить соответствующий режим вызовом команды **glEnable (GL_CULL_FACE)**, а затем выбрать тип отображаемых граней с помощью команды **void glCullFace (GLenum mode)**.

Вызов с параметром GL_FRONT приводит к удалению из изображения всех лицевых граней, а с параметром GL_BACK – обратных (установка по умолчанию).

Кроме рассмотренных стандартных примитивов в библиотеках GLU и GLUT описаны более сложные фигуры, такие как сфера, цилиндр, диск (в GLU) и сфера, куб, конус, тор, тетраэдр, додекаэдр, икосаэдр, октаэдр и чайник (в GLUT). Автоматическое наложение текстуры предусмотрено только для фигур из библиотеки GLU.

При изображении объектов в OpenGL можно изменять размер точек и толщину линий. Для этой цели служат следующие команды.

void glPointSize(GLfloat size) – устанавливает диаметр (size) в пикселях точек, изображаемых с помощью примитива GL_POINTS. По умолчанию значение параметра size равно 1.0. В любой реализации OpenGL гарантирована поддержка размера точки равного 1.0. В типичных реализациях MicrosoftWindows поддерживается размер точки от 0.5 до 10 с шагом 0.125.

void glLineWidth(GLfloat size) – устанавливает ширину (size) в пикселях линий, изображаемых с помощью примитивов GL_LINES, GL_LINE_STRIP или GL_LINE_LOOP. По умолчанию значение параметра size равно 1.0. В любой реализации OpenGL гарантирована поддержка размера точки равного 1.0. В типичных реализациях MicrosoftWindows поддерживается размер точки от 0.5 до 10 с шагом 0.125.

15.7.4. Дисплейные списки

В традиционных языках программирования существуют функции и процедуры – т. е. можно выделить определенный набор команд, запомнить его в неко-

тором одном определенном месте и вызывать каждый раз, когда возникает потребность в соответствующей последовательности команд. Подобная возможность существует и в OpenGL – набор команд OpenGL можно запомнить в так называемый дисплейный список (*display list*), при этом все команды и данные переводятся в некоторое внутреннее представление, наиболее удобное для данной реализации OpenGL, и затем вызвать при помощи всего одной команды.

Каждому дисплейному списку соответствует некоторое целое число, идентифицирующее этот список.

Для создания нового дисплейного списка необходимо поместить все команды, которые должны в него войти, между следующими операторными скобками:

```
void glNewList (GLuint list, GLenum mode),  
void glEndList (),
```

где параметр **list** задает номер списка, а параметр **mode** определяет режим обработки команд, входящих в список и может принимать следующие значения:

GL_COMPILE – команды записываются в список без выполнения,

GL_COMPILE_AND_EXECUTE – команды сначала выполняются, а затем записываются в список.

Следует иметь в виду, что если в качестве параметра **list** списка используется уже существующий номер, то новый список не создается, а существующий список заменяется на новый.

Узнать, занят ли данный номер **list** каким-либо дисплейным списком, можно при помощи функции

```
GLboolean glIsList (GLuint list).
```

Зарезервировать **range** свободных подряд идущих номеров для идентификации дисплейных списков можно при помощи функции

```
GLuint glGenLists (GLsizei range).
```

Эта функция возвращает первый свободный номер из блока зарезервированных номеров данной командой.

После того, как список создан, его можно вызвать командой **void glCallList (GLuint list)**, указав в параметре **list** идентификатор нужного списка. Для вызова сразу нескольких списков, можно воспользоваться командой **void glCallLists (GLsizei n, GLenum type, const GLvoid *lists)**, вызывающей **n** списков с идентификаторами из массива **lists**, тип элементов которого указывается в параметре **type**. Это могут быть типы GL_BYTE, GL_UNSIGNED_BYTE, GL_SHORT, GL_INT, GL_UNSIGNED_INT и некоторые другие. Для удаления списков используется команда

void glDeleteLists (GLuint list, GLsizei range), которая удаляет списки с идентификаторами **ID** из диапазона $list \leq ID \leq list + range - 1$.

Пример:

```
glNewList(1, GL_COMPILE);
```

```

glBegin(GL_TRIANGLES);
    glVertex3f(1.0f, 1.0f, 1.0f);
    glVertex3f(10.0f, 1.0f, 1.0f);
    glVertex3f(10.0f, 10.0f, 1.0f);
glEnd();
glEndList();
...
glCallList(1);

```

Замечание. Дисплейные списки нельзя изменять. Список можно уничтожить или создать заново. В дисплейном списке команды запоминаются вместе со своими аргументами на момент передачи, так что в следующем примере последний оператор присваивания дисплейный список не изменяет.

```

GLfloat color [] = {0.0, 0.0, 0.0};
glNewList (1, GL_COMPILE);
glColor3fv (color);
glEndList ();
color [2] = 1.0; //Неизменяет дисплейный список.

```

Не все команды OpenGL могут быть записаны в дисплейный список. К ним относятся такие команды, как **glDeleteLists()**, **glFinish()**, **glGenLists()** и некоторые другие.

Дисплейные списки в оптимальном, скомпилированном виде хранятся в памяти сервера, что позволяет рисовать примитивы в такой форме максимально быстро. В то же время большие объемы данных занимают много памяти, что влечет, в свою очередь, падение производительности. Такие большие объемы (больше нескольких десятков тысяч примитивов) лучше рисовать с помощью массивов вершин.

15.7.5. Массивы вершин

Если вершин много, то, чтобы не вызывать для каждой команду **glVertex*()**, удобно объединять вершины в массивы, используя команду

void glVertexPointer (GLint size, GLenum type, GLsizei stride, void* ptr), которая определяет способ хранения и координаты вершин. При этом **size** определяет число координат вершины (может быть равен 2, 3, 4), **type** определяет тип данных (может быть равен **GL_SHORT**, **GL_INT**, **GL_FLOAT**, **GL_DOUBLE**). Иногда удобно хранить в одном массиве другие атрибуты вершины, тогда параметр **stride** задает смещение от координат одной вершины до координат следующей; если **stride = 0**, это значит, что координаты расположены последовательно. В параметре **ptr** указывается адрес, где находятся данные.

Аналогично можно определить массив нормалей, цветов и некоторых других атрибутов вершины, используя команды:

```

void glNormalPointer (GLenum type, GLsizei stride, void *pointer),
void glColorPointer (GLint size, GLenum type, GLsizei stride, void *pointer).

```

Для того, чтобы эти массивы можно было использовать в дальнейшем, надо вызвать команду

void glEnableClientState (GLenum array), где параметр **array** может принимать значения **GL_VERTEX_ARRAY**, **GL_NORMAL_ARRAY** и **GL_COLOR_ARRAY** соответственно. После окончания работы с массивом желательно вызвать команду **void glDisableClientState (GLenum array)** с соответствующим значением параметра **array**.

Для отображения содержимого массивов используется команда **void glVertexElement (GLint index)**, которая передает OpenGL атрибуты вершины, используя элементы массива с номером **index**. Это аналогично последовательному применению команд вида **glColor*(...)**, **glNormal*(...)**, **glVertex*(...)** с соответствующими параметрами. Однако вместо нее обычно вызывается команда

void glDrawArrays (GLenum mode, GLint first, GLsizei count), рисующая **count** примитивов, определяемых параметром **mode**, используя элементы из массивов с индексами от **first** до **first+count-1**. Это эквивалентно вызову последовательности команд **glVertexElement()** с соответствующими индексами. Параметр **mode** определяет тип примитива и может принимать значения: **GL_POINTS**, **GL_LINE**, **GL_LINE_LOOP**, **GL_LINE_STRIP**, **GL_TRIANGLES**, **GL_TRIANGLE_STRIP**, **GL_TRIANGLE_FAN**, **GL_QUADS**, **GL_QUAD_STRIP** и **GL_POLYGON**.

В случае, если одна вершина входит в несколько примитивов, то вместо дублирования ее координат в массиве удобно использовать ее индекс. Для этого надо вызвать команду

void glDrawElements (GLenum mode, GLsizei count, GLenum type, void *indices), где **indices** – это массив номеров вершин, которые надо использовать для построения примитивов, **type** определяет тип элементов этого массива: **GL_UNSIGNED_BYTE**, **GL_UNSIGNED_SHORT**, **GL_UNSIGNED_INT**, а **count** задает их количество.

Важно отметить, что использование массивов вершин позволяет оптимизировать передачу данных на сервер OpenGL, и, как следствие, повысить скорость рисования трехмерной сцены. Такой метод определения примитивов является одним из самых быстрых и хорошо подходит для визуализации больших объемов данных.

Помимо рассмотренных примитивов, определяемых внутри командных скобок, в OpenGL предусмотрены команды

void glRect[d f i s](GLtype x1, GLtype y1, GLtype x2, GLtype y2)

и

void glRect[d f i s] v (GLtype* v1, GLtype* v2),

которые позволяют достаточно эффективно задавать прямоугольник в плоскости **z = 0**, определяя два его противоположных угла.

Параметры **x1** и **y1** задают координаты одной из вершин прямоугольника, а **x2** и **y2** – координаты его противоположной вершины. Параметры **v1** и **v2** определяют указатели на соответствующие двухточечные массивы вершин.

Если прямоугольник изображается с помощью функции **glRect**, то стороны многоугольника прокладываются между вершинами в следующем порядке: (**x1**, **y1**), (**x2**, **y1**), (**x2**, **y2**), (**x1**, **y2**), а затем обратно к первой вершине.

Их действие аналогично, например, следующей последовательности команд:

```
glBegin(GL_POLYGON) ;  
glVertex2*(x1, y1); // * заменяется соответствующим символом,  
glVertex2*(x2, y1); // зависящим от типа используемых  
glVertex2*(x2, y2); // данных  
glVertex2*(x1, y2);  
glEnd();
```

15.7.6. Вывод текста

Существуют два типа шрифтов – **штриховой (эскизный)** и **растровый**. Штриховой шрифт формируется по тому же принципу, что и прочие графические примитивы. Начертание символа определяется вершинами соответствующих прямолинейных и криволинейных отрезков. Если символ формируется из замкнутых контуров, то его внутреннюю область можно залить. Преимущество использования штриховых шрифтов заключается в том, что с ними можно обращаться в графической системе точно так же, как с любыми другими графическими объектами. Несомненным достоинством штриховых шрифтов является и сохранение начертания во всех деталях при выполнении геометрических преобразований масштабирования или повороте. Поэтому определение символов набора выполняется только для одного, базового, размера, а все прочие получают после выполнения стандартных геометрических преобразований – масштабирования и поворота. Однако визуализируются штриховые шрифты медленнее, чем растровые.

Растровый шрифт определяется значительно проще, а отображается быстрее (рис. 15.5). Символы шрифта определяются на прямоугольной области и представляют собой блоки битов. Каждый блок задает определенный символ в виде образа из нулей и единиц, которые соответствуют засвеченным и незасвеченным точкам раstra. При отображении определенный таким способом символ помещается в буфер кадра с помощью операции побитового переноса, которая выполняется очень быстро.

Увеличить размер символов растрового шрифта можно только дублированием пикселей, что при большом увеличении приводит к формированию символов довольно «грубой» формы. Выполнять какие-либо геометрические преобразования растрового шрифта бессмысленно. Кроме того, поскольку растровые

шрифты, как правило, хранятся в постоянной памяти, они не переносятся с компьютера на компьютер.

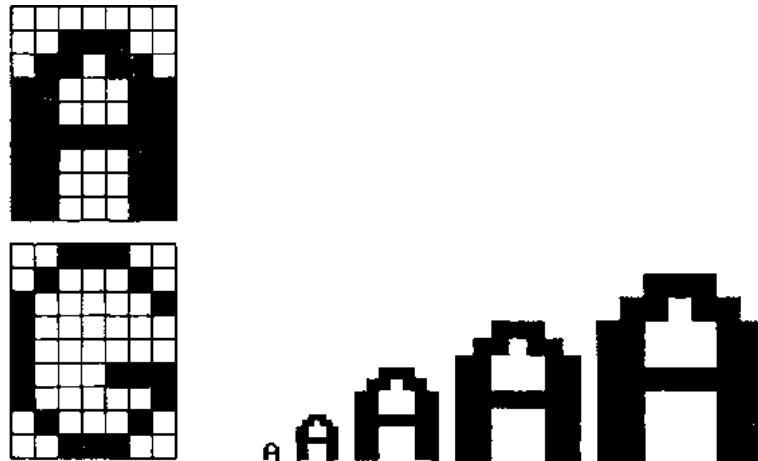


Рис. 15.5

Символы как штриховых, так и растровых шрифтов формируются из других примитивов, а потому в составе основной библиотеки OpenGL нет специального примитива для формирования текста. Но в составе дополнительной библиотеки GLUT имеется несколько наборов символов (как штриховых, так и растровых), которые определены программно, а, следовательно, являются переносимыми.

Растровый символ GLUT можно получить с помощью функции **glutBitmapCharacter (font, character)**.

Параметр **font** определяет начертание шрифта и может принимать значение одной из символьных констант GLUT:

```
GLUT_BITMAP_8_BY_13;  
GLUT_BITMAP_9_BY_15;  
GLUT_BITMAP_TIMES_ROMAN_10;  
GLUT_BITMAP_HELVETICA_12;  
GLUT_BITMAP_HELVETICA_18.
```

Параметру **character** присваивается либо код ASCII, либо отдельный символ, который мы хотим изобразить. Таким образом, чтобы изобразить прописную букву «А», можно воспользоваться либо значением 65 кода ASCII, либо обозначением 'A'.

Каждый символ, созданный с помощью функции **glutBitmapCharacter**, изображается так, что начало координат (нижний левый угол) битового массива находится в текущем растровом положении. После того как битовый массив символа загружается в буфер регенерации, к координате *x* текущего растрового положения добавляется смещение, равное ширине символа.

Изменение значения текущей позиции раstra выполняется с помощью разных модификаций функции **glRasterPos*** (например, **glRasterPos2i(GLintx,**

GLinty), glRasterPos3i(GLintx, GLinty, z GLint).

При установке растрового положения текущий цвет растровых изображений устанавливается равным текущему цвету. В качестве примера изобразим известную текстовую строку «Hellow Word!!!».

```
MyText[]="HellowWord!!!";
glColor3ub(255,0,0); //Цвет текста
glRasterPos2i(-10, 3); //Координаты начала вывода
len=strnlen(MyText,100);
for (int i=0;i<len;i++)
    glutBitmapCharacter(GLUT_BITMAP_9_BY_15, MyText[i]);
```

Эскизный символ изображается путем вызова функции

glutStrokeCharacter (font, character).

Для этой функции параметру **font** можно присвоить значение GLUT_STROKE_ROMAN, чтобы изобразить пропорциональный шрифт, или GLUT_STROKE_MONO_ROMAN, чтобы представить моноширинный шрифт. Размер и положение этих символов можно контролировать с помощью операций аффинных преобразований, которые вызываются перед выполнением процедуры **glutStrokeCharacter**.

15.7.7.Работа с z-буфером

Как уже было сказано, OpenGL поддерживает использование аппаратного буфера для удаления невидимых поверхностей. Каждое поступающее значение глубины фрагмента сравнивается с имеющимся в буфере значением глубины и выводится на экран (или нет) в зависимости от результатов выполнения этого теста. Более того, результат этого теста влияет на содержимое буфера трафарета, что в совокупности представляет поразительно мощное средство создания изображений практически любой степени сложности.

Исходно тест глубины заблокирован и может быть разрешен при помощи команды **glEnable (GL_DEPTH_TEST)**.

Выключить эту проверку можно командой

glDisable (GL_DEPTH_TEST).

Функция сравнения, используемая в тесте, задается командой

void glDepthFunc(GLenum func).

Вызов этой команды позволяет определить ту функцию, которая будет использоваться для сравнения каждого поступающего **z**-значения с тем, которое хранится в буфере глубины. Функцию, используемую для сравнения глубин, определяет параметр **func**, который может принимать одно из следующих значений, указанных в табл. 15.7.

Таблица 15.7

Значения параметра **func**

Константа	Тест завершается положительно...
GL_NEVER	Никогда
GL_LESS	Если поступающее значение меньше, чем хранящееся в буфере
GL_EQUAL	Если поступающее z-значение равно хранящемуся в буфере
GL_LEQUAL	Если поступающее z-значение меньше или равно, чем хранящееся в буфере
GL_GREATER	Если поступающее значение больше, чем хранящееся в буфере
GL_NOTEQUAL	Если поступающее значение не равно хранящемуся в буфере
GL_GEQUAL	Если поступающее значение больше или равно хранящемуся в буфере
GL_ALWAYS	Всегда

По умолчанию используется константа GL_LESS. Если буфера глубины нет, то считается, что тест всегда завершается положительно.

15.7.8. Преобразования координат объектов

15.7.8.1. Системы координат

В OpenGL используются как основные три системы координат: **левосторонняя**, **правосторонняя** и **оконная**. Первые две системы являются трехмерными и отличаются друг от друга направлением оси Z: в правосторонней она направлена на наблюдателя, в левосторонней – в глубину экрана. Ось X направлена вправо относительно наблюдателя, ось Y – вверх.

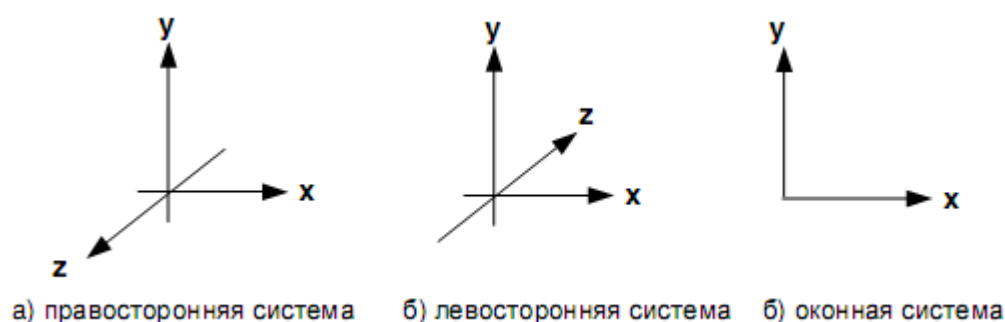


Рис. 15.6

Рассматривая какой-либо трехмерный объект, мы всегда определяем его положение и размеры относительно некоторой привычной и удобной в настоящий момент системы координат, связанной с реальным миром. Такая исходная система координат в компьютерной графике является **правосторонней** и называется **мировой системой координат (МСК)**. В ней над объектами выполняются заданные действия, например, аффинные преобразования. Для того, чтобы можно было изобразить объект на экране, его необходимо предварительно перевести (или преобразовать) в другую систему координат, которая связана с точкой наблюдения и носит название **видовой системы координат** или **системы координат наблюдателя (СКН)**. Эта система координат является левосторонней. В

ней рассчитываются цвета объектов и выполняются заданные отсечения. Далее, координаты СКН переводятся в нормированные (нормализованные) координаты (НК), которые лежат в диапазоне $[-1, 1]$. И, наконец, любое трехмерное изображение мы всегда рисуем на двумерном экране, который имеет свою **экранную (оконную) систему координат (ОСК)**. Взаимное расположение мировой (XYZ) и видовой ($X_eY_eZ_e$) систем координат, а также экранная система координат ($xу$) показаны на рис. 15.7 и рис. 15.8 соответственно.

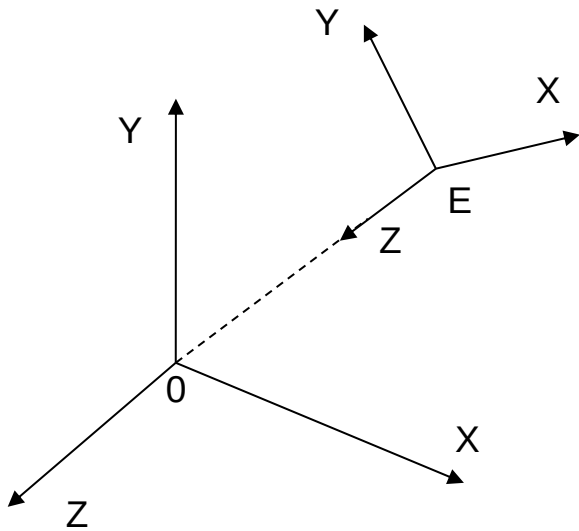


Рис. 15.7

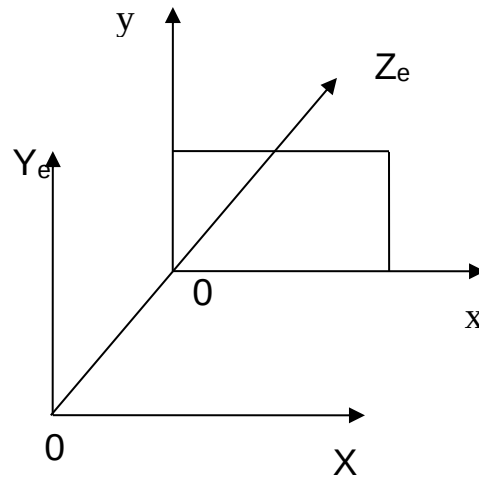


Рис. 15.8

Таким образом, основная задача, которую необходимо решить, заключается в том, что объекты, описанные в мировых координатах, необходимо изобразить на плоской области вывода экрана. То есть необходимо проделать некоторую последовательность преобразований и осуществить проекцию, переводящую трехмерные объекты на двумерную проекционную плоскость. Эту последовательность действий удобно выполнять в несколько этапов, как это и реализовано в OpenGL (рис.5.9).



Координаты отображаемых объектов (примитивов) задаются в МСК. В ней же под воздействием **видовой матрицы** выполняются заданные действия, например аффинные преобразования. В результате воздействия видовой матрицы координаты вершин объектов пересчитываются в СКН. В ней рассчитываются цвета объектов и выполняются заданные отсечения. Далее под воздействием **матрицы проецирования** координаты СКН пересчитываются в нормированные координаты. Диапазон их изменения – $[-1.0, 1.0]$. Непосредственно перед выводом в окно нормированные координаты преобразуются в оконные.

15.7.8.2. Матрицы

В OpenGL предусмотрены **три** типа матриц – **видовая, проекций и текстуры**, все они имеют размер (4×4) .

Для того чтобы с какой-либо матрицей можно было работать, ее необходимо сделать текущей, для чего предусмотрена команда **void glMatrixMode(GLenum mode)**.

Параметр **mode** определяет, с каким набором матриц будут выполняться следующие операции, и может принимать одно из трех значений:

GL_MODELVIEW – последовательность операций над матрицами применяется к видовой матрице,

GL_PROJECTION – последовательность операций над матрицами применяется к матрице проекции,

GL_TEXTURE – последовательность операций над матрицами применяется к матрице текстуры.

После того, как установлена текущая матрица, необходимо определить все элементы. Для этой цели можно использовать команду

void glLoadMatrix[fd](GLtype* m).

Параметр **m** определяет указатель на матрицу 4×4 , хранящуюся по порядку расположения столбцов как 16 последовательных вещественных значений одинарной (f) или двойной (d) точности.

$$m = \begin{pmatrix} a_1 & a_5 & a_9 & a_{13} \\ a_2 & a_6 & a_{10} & a_{14} \\ a_3 & a_7 & a_{11} & a_{15} \\ a_4 & a_8 & a_{12} & a_{16} \end{pmatrix}$$

Эта команда заменяет текущую матрицу на ту, которая определена в **m**. Текущей является одна из матриц – видовой, проекций или текстуры, в зависимости от текущего режима.

Команда **void glLoadIdentity()** заменяет текущую матрицу на единичную (матрицу идентичности).

$$m = I = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Семантически эта команда эквивалентна **glLoadMatrix** с параметром, определяющим единичную матрицу, но выполняется более эффективно.

Часто бывает необходимо сохранить содержимое текущей матрицы для дальнейшего использования, для чего применяются команды

void glPushMatrix (void),

void glPopMatrix (void).

Они записывают и восстанавливают текущую матрицу из стека, причем для каждого типа матриц стек свой. Для модельно-видовых матриц его глубина равна как минимум 32, для остальных – как минимум 2.

При выполнении преобразований объектов, таких как перенос, вращение, масштабирование, могут понадобиться соответствующие операции над матрицами. OpenGL предоставляет набор команд для решения этой задачи. Фундаментальной является команда перемножения матриц

void glMultMatrix[fd](GLtype* m), где параметр **m** определяет указатель на матрицу размером 4x4, хранящуюся по порядку расположения столбцов. Эта команда умножает матрицу, заданную параметром **m** слева, на текущую. Если обозначить текущую матрицу за **M**, передаваемую матрицу за **T**, то в результате выполнения команды **glMultMatrix** текущей становится матрица **M·T** (**T ← M·T**). Однако обычно для изменения матрицы того или иного типа удобно использовать специальные команды, которые по значениям своих параметров создают нужную матрицу и умножают ее на текущую.

Однако к использованию этой следует прибегать только в самом крайнем случае, т. к. для перемножения требуются значительные вычислительные затраты.

В целом, для отображения трехмерных объектов сцены в окно приложения используется последовательность, показанная на рис.15.10.

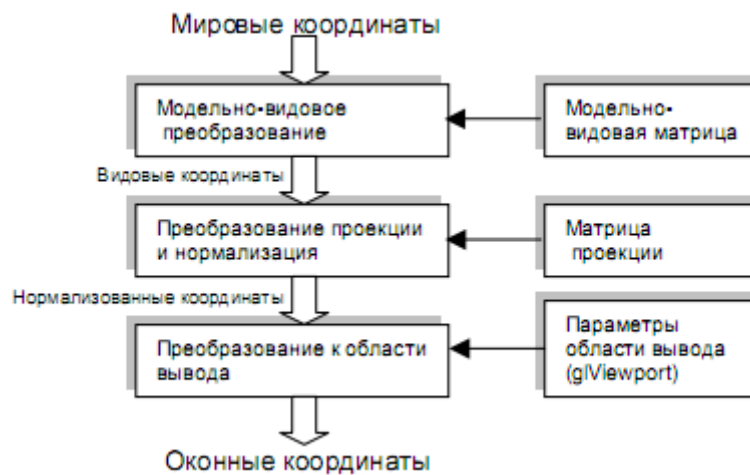


Рис. 15.10

Таким образом, все преобразования объектов и камеры в OpenGL производятся с помощью умножения векторов координат на матрицы. Причем умножение происходит на текущую матрицу в момент определения координаты командой **glVertex*** и некоторыми другими.

15.7.8.3. Модельно-видовые преобразования

К модельно-видовым преобразованиям будем относить перенос, поворот и изменение масштаба вдоль координатных осей. Для проведения этих операций достаточно умножить на соответствующую матрицу каждую вершину объекта и получить измененные координаты этой вершины:

$$(x_e \ y_e \ z_e \ 1)^T = M \cdot (x \ y \ z \ 1)^T$$

где M – матрица модельно-видового преобразования. Перспективное преобразование и проектирование производится аналогично. Сама матрица может быть создана с помощью следующих команд:

```

void glTranslate[f d] (GLtype x, GLtype y, GLtype z),
void glRotate[f d] (GLtype angle, GLtype x, GLtype y, GLtype z) ,
void glScale[f d] (GLtype x, GLtype y, GLtype z),
  
```

где

glTranlsate*() –производит перенос объекта, прибавляя к координатам его вершин значения своих параметров,

glRotate*() –производит поворот объекта против часовой стрелки на угол **angle**(измеряется в градусах) вокруг вектора (x, y, z),

glScale*() –производит масштабирование объекта (сжатие или растяжение) вдоль вектора (x, y, z), умножая соответствующие координаты его вершин на значения своих параметров.

Все эти преобразования изменяют текущую матрицу, а поэтому применяются к примитивам, которые определяются позже. В случае, если надо, например, повернуть один объект сцены, а другой оставить неподвижным, удобно сначала сохранить текущую видовую матрицу в стеке командой **glPushMatrix()**, затем вызвать **glRotate()** с нужными параметрами, описать примитивы, из которых состоит этот объект, а затем восстановить текущую матрицу командой **glPopMatrix()**.

Кроме изменения положения самого объекта часто бывает необходимо изменить положение наблюдателя, что также приводит к изменению модельно-видовой матрицы. Это можно сделать с помощью команды

```
void gluLookAt (GLdouble ex, GLdouble ey, GLdouble ez,  
GLdouble center_x, GLdouble center_y, GLdouble center_z,  
GLdouble up_x, GLdouble up_y, GLdouble up_z),
```

где точка **(ex,ey,ez)** определяет точку наблюдения в МСК, точка **(center_x, center_y, center_z)** задает центр сцены, который будет проектироваться в центр области вывода, а вектор **(up_x, up_y, up_z)** задает направление вектора, вдоль которого в видовом порте (см. ниже) ориентируется ось Y, определяя поворот камеры. Причем вектор **(up_x, up_y, up_z)** не должен быть параллелен линии, проведенной от точки наблюдения **(ex,ey,ez)** до наблюдаемой точки **(center_x, center_y, center_z)**.

Если, например, камеру не надо поворачивать, то задается значение **(0,1,0)**, а со значением **(0,-1,0)** сцена будет перевернута.

Строго говоря, эта команда совершает перенос и поворот объектов сцены, но в таком виде задавать параметры бывает удобнее. Следует отметить, что вызывать команду **gluLookAt()** имеет смысл перед определением преобразований объектов, когда модельно-видовая матрица равна единичной.

Замечание. В общем случае матричные преобразования в OpenGL нужно записывать в обратном порядке. Например, если необходимо сначала повернуть объект, а затем передвинуть его, то сначала надо вызвать команду **glTranslate()**, а только потом – **glRotate()**. После этого можно определить сам объект.

15.7.9. Проекции

Несоответствие между пространственными объектами и плоскими изображениями устраняется путем введения **проекций**, которые отображают объекты на двумерной проекционной картинной плоскости. В зависимости от расстояния между наблюдателем и картинной плоскостью различают два основных класса проекций: **параллельные** и **центральные (перспективные)**. Если это расстояние бесконечно, то проекция будет параллельной, а если конечно, то центральной. При описании центральной проекции необходимо явно задавать **центр проекции**, а для определения параллельной проекции необходимо определить направление проецирования.

OpenGL поддерживает два вида проекций – **ортографическую (параллельную)** и **перспективную**.

В OpenGL существуют стандартные команды для задания ортографической (параллельной) и перспективной проекций, выполнение которых устанавливает в системе координат наблюдателя область видимости графических данных. Первый тип проекции может быть задан командами

void glOrtho (GLdouble x_left, GLdouble x_right, GLdouble y_bottom, GLdouble y_top, GLdouble z_near, GLdouble z_far) и

void gluOrtho2D (GLdouble x_left, GLdouble x_right, GLdouble y_bottom, GLdouble y_top)

Первая команда создает матрицу проекции в усеченный объем видимости (**параллелепипед видимости**) в левосторонней системе координат. Параметры команды задают точки (**x_left, y_bottom, z_near**) и (**x_right, y_top, z_far**), которые отвечают левому нижнему и правому верхнему углам окна вывода. Параметры **z_near** и **z_far** задают расстояние до ближней и дальней плоскостей отсечения по удалению от точки **(0,0,0)** и могут быть отрицательными.

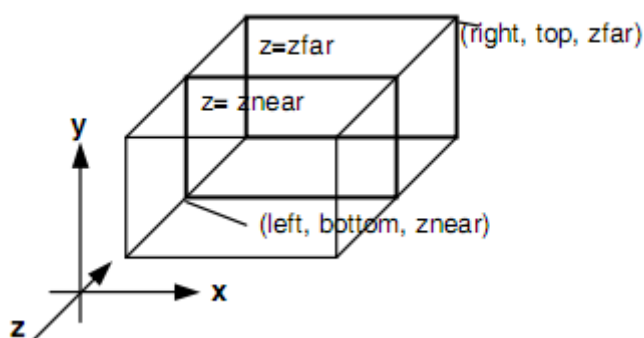


Рис. 15.11

Во второй команде, в отличие от первой, значения **znear** и **zfar** устанавливаются равными **-1** и **1** соответственно. Это удобно, если OpenGL используется для рисования двумерных объектов. В этом случае положение вершин можно задавать, используя команды **glVertex2*()**.

После преобразования ортографического проецирования координаты любых точек внутри преобразованной трехмерной области видимости находятся в диапазоне от **-1** до **1**. Таким образом, преобразованная область видимости представляет собой куб со стороной равной двум. Напомним, что координаты, полученные после преобразования, называются **нормированными (нормализованными)**.

При использовании перспективной проекции в качестве области видимости необходимо определить не параллелепипед, а **усеченную пирамиду** (рис.15.10). Для этого в OpenGL реализованы две команды.

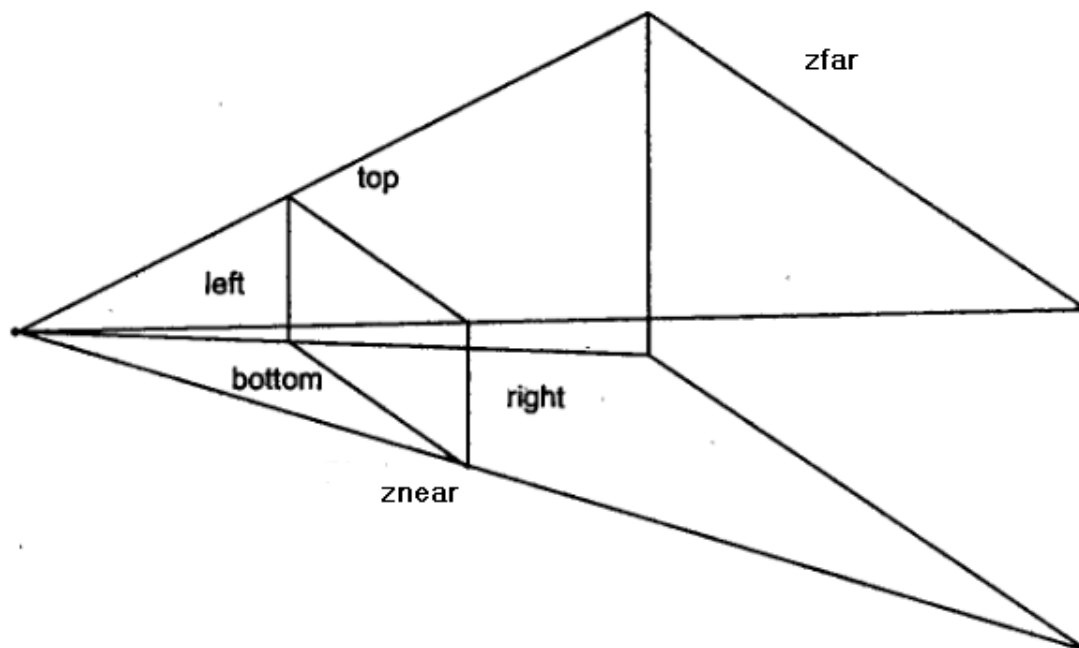


Рис. 15.12

Первая команда

void glFrustrum(GLdouble left, GLdouble right, GLdouble top, GLdouble bottom, GLdouble znear, GLdouble zfar) создает матрицу перспективы. Параметры **left** и **right** определяют координаты левой и правой вертикальных, а **bottom** и **top** – нижней и верхней горизонтальных плоскостей отсечения. Параметры **znear** и **zfar** – расстояния до ближней и дальней плоскостей отсечения по глубине, причем **оба значения** должны быть положительными. Считается, что наблюдатель располагается в точке с координатами (0, 0, 0).

Вторая команда **void gluPerspective (GLdouble angley, GLdouble aspect, GLdouble znear, GLdouble zfar)** задает усеченную пирамиду видимости в левосторонней (видовой) системе координат (рис. 15.13).

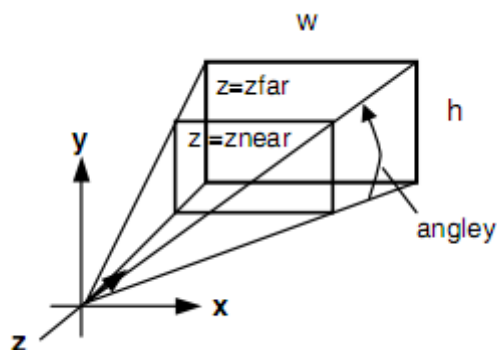


Рис. 15.13

Параметр **angley** определяет угол видимости в градусах по оси Y, $\text{tg}(\text{angley}/2) = \text{top}/\text{near}$ и должен находиться в диапазоне от 0 до 180. Угол видимости вдоль оси X задается параметром **aspect**, который обычно задается как отношение ширины (**width**) и высоты (**height**) сторон области вывода (как правило, размеров окна). Величина отношения **aspect** должна соответствовать заданному видовому порту (см. ниже). Например, если **aspect=2**, то отношение **width/ height** (x/y) также следует задать равным двум.

Параметры **zfar** и **znear** задают расстояние от наблюдателя до плоскостей отсечения по глубине и должны быть положительными. Действие этой команды эквивалентно вызову **glFrustum** при **left = -right, bottom = -top, $\text{tg}(\text{angley}/2) = \text{top}/\text{near}$, aspect = right/top**.

Чем больше отношение **zfar/znear**, тем хуже в буфере глубины будут различаться расположенные рядом поверхности, так как по умолчанию в него будет записываться «сжатая» глубина в диапазоне от 0 до 1 (см. далее).

Также как и для случая ортогографической проекции, усеченный конус видимости отображается на куб со стороной равной двум.

Для задания матрицы проекций необходимо включить режим работы с нужной матрицей командой **glMatrixMode(GL_PROJECTION)** и сбросить текущую, вызвав **glLoadIdentity()**.

Пример:

```
//ортогографическая проекция
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glOrtho(0, w, 0, h, -1.0, 1.0);
```

15.7.10. Область вывода

Область вывода представляет собой прямоугольник в оконной системе координат, размеры которого задаются командой:

void glViewport (GLint x, GLint y, GLint width, GLint height).

Значения всех параметров задаются в пикселах. Параметры **x** и **y** определяют координаты левого нижнего угла прямоугольника вывода в оконной системе координат (видовой порт) и по умолчанию принимаются равными нулю. Размеры оконной системы координат определяются текущими размерами окна приложения, точка (0,0) находится в левом нижнем углу окна.

Команда **glViewport** задает аффинные преобразования области вывода, переводящие нормированные координаты (**x_n**, **y_n**) в оконные координаты (**x_w**, **y_w**).

Преобразование выполняется по формулам:

$$x_w = (x_n + 1.0) \left[\frac{\text{width}}{2} \right] + x, \quad y_w = (y_n + 1.0) \left[\frac{\text{height}}{2} \right] + y,$$

где $[\bullet]$ – целая часть числа.

Используя параметры команды **glViewport()**, OpenGL вычисляет оконные координаты центра области вывода (o_x, o_y) по формулам

$$o_x = x + \frac{width}{2}, \quad o_y = y + \frac{height}{2}.$$

При этом точка с координатами **(0, 0, 0)** располагается в *центре* области вывода.

Пусть $p_x = width$, $p_y = height$.

Тогда оконные координаты вершин можно представить в виде

$$x_w = \frac{p_x}{2} x_n + o_x, \quad y_w = \frac{p_y}{2} y_n + o_y.$$

Команда **glViewport()** обычно используется в функции, зарегистрированной с помощью команды **glutReshapeFunc()**, которая вызывается, если пользователь изменяет размеры окна приложения.

15.8. Создание анимации в OpenGL

Для создания анимации в OpenGL необходимо выполнить следующие действия.

Установить режим двойной буферизации

glutInitDisplayMode(GLUT_RGB|GLUT_DOUBLE).

Зарегистрировать и создать функцию обратного вызова (например, функцию с именем **IdleFunction**), которая вызывается системой когда нет других событий.

Внутри функции **IdleFunction** описываются действия, которые влияют на вид изображения после перерисовки окна. Например, изменяется угол поворота объекта.

```
void IdleFunction( )
{
    //Необходимые действия
    glutPostRedisplay( ); //Обновить окно
}
```

В функции рисования **OnDraw()** использовать команду переключения буферов – **glutSwapBuffers()**.

Для управления процессом запуска и остановки анимации можно использовать команды:

glutIdleFunc(IdleFunction) – разрешает выполнение функции IdleFunction;

glutIdleFunc(NULL) – блокирует выполнение функции IdleFunction.

15.9. Материалы и освещение

Для создания реалистичных изображений необходимо определить как свойства самого объекта, так и свойства среды, в которой он находится. Первая группа свойств включает в себя параметры материала, из которого сделан объект, способы нанесения текстуры на его поверхность, степень прозрачности объекта. Ко второй группе можно отнести количество и свойства источников света, уровень прозрачности среды, а также модель освещения. Все эти свойства можно задавать, вызывая соответствующие команды OpenGL.

15.9.1. Модель освещения

В OpenGL используется модель освещения, в соответствии с которой цвет точки определяется несколькими факторами: **свойствами материала и текстуры, величиной нормали в этой точке, а также положением источника света и наблюдателя**. Для корректного расчета освещенности в точке надо использовать единичные нормали, однако команды типа **glScale*()** могут изменять длину нормалей. Чтобы это учесть, необходимо использовать режим нормализации векторов нормалей, который включается вызовом команды **glEnable(GL_NORMALIZE)**.

Для задания глобальных параметров освещения используются команды

void glLightModel[i f] (GLenum pname, GLenum param),

void glLightModel[i f]v (GLenum pname, const GLtype *params).

Аргумент **pname** определяет, какой параметр модели освещения будет настраиваться. Возможные значения аргументов **pname** и **param** приведены в табл. 15.8.

Таблица 15.8

Значения аргументов *pname* и *param*

Значение pname	Значение param
GL_LIGHT_MODEL_LOCAL_VIEWER	Параметр param должен быть <i>булевым</i> и задает положение наблюдателя. Если он равен GL_FALSE, то направление обзора считается параллельным оси Z вне зависимости от положения в видовых координатах. Если же он равен GL_TRUE, то наблюдатель находится в начале видовой системы координат. Это может улучшить качество освещения, но усложняет его расчет. Значение <i>по умолчанию</i> : GL_FALSE.
GL_LIGHT_MODEL_TWO_SIDE	Параметр param должен быть

	<i>булевым</i> и управляет режимом расчета освещенности, как для лицевых, так и для обратных граней. Если он равен GL_FALSE, то освещенность рассчитывается только для лицевых граней. Если же он равен GL_TRUE, расчет проводится и для обратных граней. Значение <i>по умолчанию</i> : GL_FALSE.
GL_LIGHT_MODEL_AMBIENT	Параметр param должен содержать четыре целых или вещественных числа, которые определяют <i>цвет фонового освещения</i> даже в случае <i>отсутствия</i> определенных источников света. Значение <i>по умолчанию</i> : (0.2, 0.2, 0.2, 1.0).

15.9.2. Спецификация материалов

Для задания параметров текущего материала используются команды
void glMaterial[i f] (GLenum face, GLenum pname, GLtype param),
void glMaterial[i f]v (GLenum face, GLenum pname, GLtype *params).

С их помощью можно определить **рассеянный, диффузный** и **зеркальный** цвета материала, а также степень зеркального отражения и интенсивность излучения света, если объект должен светиться. Какой именно параметр будет определяться значением **param**, зависит от значения **pname**.

Возможные значения аргументов **pname** и **param** приведены в табл. 15.9.

Таблица 15.9

Значения аргументов *pname* и *param*

Значение pname	Значение param
GL_AMBIENT	Параметр params должен содержать четыре целых или вещественных значения цветов RGBA, которые определяют <i>рассеянный цвет</i> материала (цвет материала в тени). Значение <i>по умолчанию</i> : (0.2, 0.2, 0.2, 1.0)
GL_DIFFUSE	Параметр params должен содержать четыре целых или вещественных значения цветов RGBA, которые определяют <i>диффузный цвет</i> материала. Значение <i>по умолчанию</i> : (0.8, 0.8, 0.8, 1.0).
GL_SPECULAR	Параметр params должен содержать четыре целых или вещественных значения цветов RGBA, которые определяют <i>зеркальный цвет</i> материала. Значение <i>по умолчанию</i> : (0.0, 0.0, 0.0, 1.0).
GL_SHININESS	Параметр params должен содержать одно целое или вещественное значение в диапазоне

	от 0 до 128 , которое определяет <i>степень зеркального отражения</i> материала. Значение <i>по умолчанию:0</i> .
GL_EMISSION	Параметр params должен содержать четыре целых или вещественных значения цветов RGBA, которые определяют <i>интенсивность излучаемого света</i> материала. Значение <i>по умолчанию:(0.0, 0.0, 0.0, 1.0)</i> .
GL_AMBIENT_AND_DIFFUSE	Это эквивалентно двум вызовам команды glMaterial*() со значением pname GL_AMBIENT и GL_DIFFUSE и одинаковыми значениями params .

Из таблицы следует, что вызов команды **glMaterial[i f]()** возможен только для установки степени зеркального отражения материала (*shininess*). Команда **glMaterial[i f]v()** используется для задания остальных параметров.

Параметр **face** определяет тип граней, для которых задается этот материал и может принимать значения: GL_FRONT, GL_BACK или GL_FRONT_AND_BACK.

Если в сцене материалы объектов различаются лишь одним параметром, рекомендуется сначала установить нужный режим, вызвав **glEnable()** с параметром GL_COLOR_MATERIAL, а затем использовать команду **void glColorMaterial (GLenum face, GLenum pname)**,

где параметр **face** имеет аналогичный смысл, а параметр **pname** может принимать все перечисленные в таблице значения. После этого значения выбранного с помощью **pname** свойства материала для конкретного объекта (или вершины) устанавливаются вызовом команды **glColor*()**, что позволяет избежать вызовов более ресурсоемкой команды **glMaterial*()** и повышает эффективность программы.

Пример определения свойств материала:

```
float mat_dif[]={0.8,0.8,0.8};
float mat_amb[] = {0.2, 0.2, 0.2};
float mat_spec[] = {0.6, 0.6, 0.6};
float shininess = 0.7 * 128;
...
glMaterialfv (GL_FRONT_AND_BACK, GL_AMBIENT, mat_amb);
glMaterialfv (GL_FRONT_AND_BACK, GL_DIFFUSE, mat_dif);
glMaterialfv (GL_FRONT_AND_BACK, GL_SPECULAR, mat_spec);
glMaterialf (GL_FRONT, GL_SHININESS, shininess);
```

15.9.3. Описание источников света

Определение свойств материала объекта имеет смысл, только если в сцене есть источники света. Иначе все объекты будут черными (или, строго говоря,

иметь цвет, равный рассеянному цвету материала, умноженному на интенсивность глобального фонового освещения, см. команду **glLightModel**). Добавить в сцену источник света можно с помощью команд

void glLight[if] (GLenum light, GLenum pname, GLfloat param),

void glLight[i f]v (GLenum light, GLenum pname, GLfloat *params).

Параметр **light** однозначно определяет источник света. Он выбирается из набора специальных символических имен вида **GL_LIGHTi**, где **i** должно лежать в диапазоне от 0 до константы **GL_MAX_LIGHT**, которая обычно не превосходит восьми.

Параметры **pname** и **params** имеют смысл, аналогичный команде **glMaterial**. Возможные взаимные значения аргументов **pname** и **param** приведены в табл.15.10.

Таблица 15.10

Значения аргументов **pname** и **param**

Значение pname	Значение param
GL_SPOT_EXPONENT	Параметр param должен содержать целое или вещественное число от 0 до 128 , задающее распределение интенсивности света. Этот параметр описывает уровень сфокусированности источника света. Значение <i>по умолчанию</i> : 0 (рассеянный свет).
GL_SPOT_CUTOFF	Параметр param должен содержать целое или вещественное число между 0 и 90 или равное 180 , которое определяет максимальный угол разброса света. Значение этого параметра есть половина угла в вершине конусовидного светового потока, создаваемого источником. Значение <i>по умолчанию</i> : 180 (рассеянный свет).
GL_AMBIENT	Параметр params должен содержать четыре целых или вещественных значения цветов RGBA, которые определяют цвет фонового освещения. Значение <i>по умолчанию</i> : (0.0, 0.0, 0.0, 1.0) .
GL_DIFFUSE	Параметр params должен содержать четыре целых или вещественных значения цветов RGBA, которые определяют цвет диффузного освещения. Значение <i>по умолчанию</i> : (1.0, 1.0, 1.0, 1.0) для GL_LIGHT0 и (0.0, 0.0, 0.0, 1.0) для остальных.
GL_SPECULAR	Параметр params должен содержать четыре целых или вещественных значения цветов RGBA, которые определяют цвет зер-

	кального отражения. Значение <i>по умолчанию</i> : (1.0, 1.0, 1.0, 1.0) для GL_LIGHT0 и (0.0, 0.0, 0.0, 1.0) для остальных.
GL_POSITION	Параметр params должен содержать четыре целых или вещественных числа, которые определяют положение источника света. Если значение компоненты w = 0.0, то источник считается бесконечно удаленным и при расчете освещенности учитывается только <i>направлена точку</i> (x, y, z), в противном случае считается, что источник расположен <i>в точке</i> (x, y, z, w). В первом случае ослабления света при удалении от источника не происходит, т.е. источник считается бесконечно удаленным. Значение <i>по умолчанию</i> : (0.0, 0.0, 1.0, 0.0).
GL_SPOT_DIRECTION	Параметр params должен содержать четыре целых или вещественных числа, которые определяют направление света. Значение <i>по умолчанию</i> : (0.0, 0.0, -1.0, 1.0). Эта характеристика источника имеет смысл, если значение GL_SPOT_CUTOFF <i>отлично</i> от 180 (которое, задано <i>по умолчанию</i>).
GL_CONSTANT_ATTENUATION, GL_LINEAR_ATTENUATION, GL_QUADRATIC_ATTENUATION	Параметр param задает значение одного из трех приведенных коэффициентов, определяющих ослабление интенсивности света при удалении от источника. Допускаются только неотрицательные значения. Если источник не является направленным (см. GL_POSITION), то ослабление обратно пропорционально сумме: $att_{constant} + att_{linear}d + att_{quadratic}d^2$, где d – расстояние между источником света и освещаемой им вершиной, $att_{constant}$, att_{linear} и $att_{quadratic}$ равны параметрам, заданным с помощью констант GL_CONSTANT_ATTENUATION, GL_LINEAR_ATTENUATION и GL_QUADRATIC_ATTENUATION соответственно. <i>По умолчанию</i> эти параметры задаются тройкой (1, 0, 0), и фактически ослабления не происходит.

При изменении положения источника света следует учитывать следующий факт: в OpenGL источники света являются объектами, во многом такими же, как многоугольники и точки. На них распространяется основное правило обработки

координат в OpenGL –параметры, описывающее положение в пространстве, преобразуются текущей модельно-видовой матрицей в момент формирования объекта, т.е. в момент вызова соответствующих команд OpenGL. Таким образом, формируя источник света одновременно с объектом сцены или камерой, его можно привязать к этому объекту. Или, наоборот, сформировать стационарный источник света, который будет оставаться на месте, пока другие объекты перемещаются.

Если **тип** и **положение** источника света не задаются, параметр `GL_POSITION` принимает значения по умолчанию **(0.0, 0.0, 1.0, 0.0)**, что указывает на удаленный источник, лучи света от которого распространяются по отрицательному направлению оси **z**.

Положение источника света включается в описание сцены и вместе с положениями объектов преобразовывается в координаты наблюдения с помощью матриц геометрического преобразования и преобразования точки наблюдения. Следовательно, если требуется зафиксировать источник света относительно объектов на сцене, его положение задается после указания в программе геометрических преобразований и преобразований точки наблюдения. Однако, если требуется, чтобы источник света двигался при изменении точки наблюдения, его положение задается перед спецификацией преобразования точки наблюдения. Кроме того, если необходимо, чтобы источник света двигался вокруг стационарной сцены, к его положению можно применить преобразование трансляции или вращения.

Для использования освещения сначала надо установить соответствующий режим вызовом команды **glEnable(GL_LIGHTNING)**, а затем включить нужный источник командой **glEnable(GL_LIGHTi)**. Еще раз обратим внимание на то, что при выключенном освещении цвет вершины равен текущему цвету, который задается командами **glColor*()**. При включенном освещении цвет вершины вычисляется исходя из информации о материале, нормалях и источниках света. При выключении освещения визуализация происходит быстрее, однако в таком случае приложение должно само рассчитывать цвета вершин.

В Приложении 2 и Приложении 3 приведены примеры создания приложений с использованием библиотеки OpenGL.